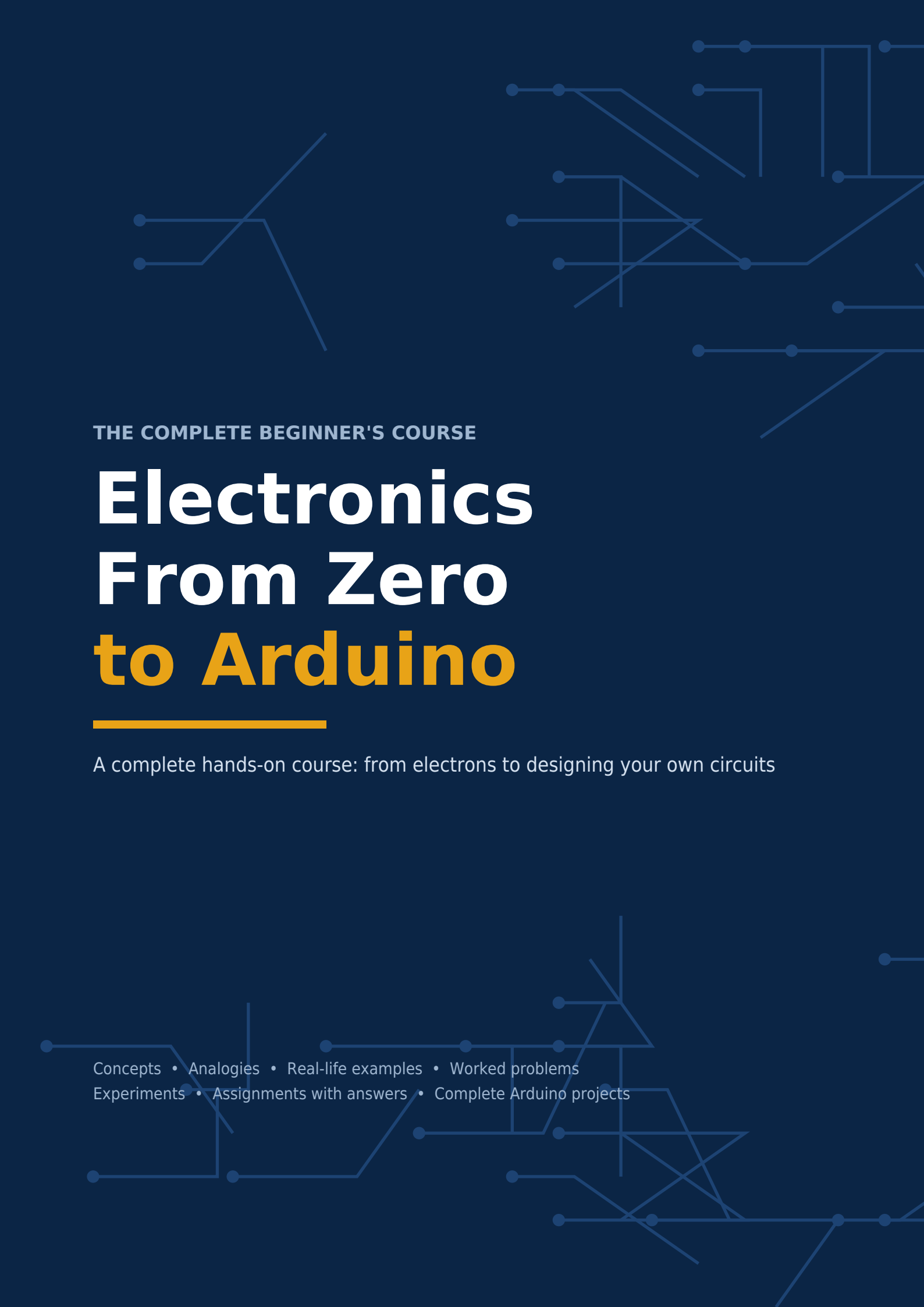




THE COMPLETE BEGINNER'S COURSE

Electronics From Zero to **Arduino**

A complete hands-on course: from electrons to designing your own circuits

Concepts • Analogies • Real-life examples • Worked problems
Experiments • Assignments with answers • Complete Arduino projects

Table of Contents

- Start here: Welcome 8**
- PREFACE Welcome, and how this book works 9**
 - The three parts of the book 9
 - How each chapter is organised 9
 - A twelve-week study plan 10
 - A promise about mathematics 11
 - Your kit and shopping list 11
 - Safety first: ten rules for a lifetime of experiments 12
- Part I: Foundations 13**
- 1 What is electricity? 14**
 - 1.1 Everything is made of atoms 14
 - 1.2 Conductors, insulators and semiconductors 15
 - 1.3 Voltage: the push 16
 - 1.4 Current: the flow 17
 - 1.5 Resistance: the opposition 19
 - 1.6 The complete circuit 20
 - 1.7 Direct current (DC) and alternating current (AC) 21
 - 1.8 Bringing the water analogy together 22
- 2 Ohm’s law, power and energy 24**
 - 2.1 Writing numbers the way engineers do 24
 - 2.2 Ohm’s law 25
 - 2.3 Electrical power 27
 - 2.4 Energy, kilowatt-hours and batteries 28
 - 2.5 A method for solving circuit problems 29
 - 2.6 Cheat sheet: all twelve Ohm and power relations 30
- 3 Your workshop: tools, breadboard, multimeter and schematics 32**
 - 3.1 The beginner’s toolkit 32
 - 3.2 The solderless breadboard 33
 - 3.3 The multimeter 34
 - 3.4 Reading schematic diagrams 37
 - 3.5 Good habits from day one 39
- 4 Resistors, series and parallel circuits 41**
 - 4.1 What a resistor does and how it is built 41
 - 4.2 The specifications that matter 42
 - 4.3 Reading the resistor colour code 43
 - 4.4 Resistors in series 44

4.5 Resistors in parallel	45
4.6 Combining series and parallel	47
4.7 Making the value you need	48
4.8 The potentiometer	48

5 Kirchhoff's laws and voltage dividers 50

5.1 Kirchhoff's current law (KCL)	50
5.2 Kirchhoff's voltage law (KVL)	51
5.3 The voltage divider	52
5.4 The loading problem	54
5.5 Real sources are not perfect: internal resistance	54
5.6 The current divider	55
5.7 Simplifying circuits: a first look at Thevenin's theorem	56
5.8 Bringing it together: the Wheatstone bridge (optional)	56

Part II: Components 59

6 Capacitors 60

6.1 What is a capacitor?	60
6.2 Capacitance and its unit	61
6.3 Energy stored in a capacitor	62
6.4 Charging and discharging: the RC time constant	62
6.5 Capacitors in series and parallel	64
6.6 Capacitor types and how to choose	64
6.7 What capacitors are used for	65

7 Inductors, magnetism and relays 69

7.1 Electricity and magnetism are two sides of one thing	69
7.2 The inductor	70
7.3 The inductive kick: why coils are dangerous to your electronics	71
7.4 Transformers	73
7.5 Relays: switches operated by electricity	73

8 Diodes and LEDs 77

8.1 Semiconductors in two paragraphs	77
8.2 The diode: a one-way valve	77
8.3 What diodes are used for	79
8.4 The LED	81
8.5 Calculating the LED series resistor	82

9 Transistors 87

9.1 The idea: a small signal controls a big flow	87
9.2 The bipolar junction transistor (BJT)	88
9.3 The BJT as a switch: how to design it	89
9.4 The BJT as an amplifier	91

9.5 The MOSFET	91
10 Switches, sensors, motors and actuators	96
10.1 Switches	96
10.2 Sensors: from the physical world to voltages	98
10.3 Actuators: making things happen	99
11 Op-amps and the 555 timer	103
11.1 The operational amplifier	103
11.2 The basic circuits	104
11.3 The 555 timer	107
12 Digital electronics and logic	112
12.1 Analog versus digital	112
12.2 Binary numbers	113
12.3 Logic levels and noise margin	114
12.4 Logic gates	115
12.5 Memory and counting: flip-flops	116
12.6 Bridging analog and digital: ADC and DAC	117
13 Batteries and power supplies	120
13.1 Batteries	120
13.2 Powering the Arduino Uno	122
13.3 Linear voltage regulators	123
13.4 From the wall to DC: a linear power supply	124
13.5 Power design for real projects	125
Part III: Arduino	128
14 Meet the Arduino	129
14.1 What is a microcontroller?	129
14.2 The Arduino Uno R3 in detail	130
14.3 Installing the Arduino IDE and uploading your first sketch	132
14.4 The anatomy of a sketch	133
14.5 Electrical limits: keeping your board alive	135
14.6 When things go wrong: upload and IDE problems	135
15 Programming basics: C++ for Arduino	138
15.1 How a program works	138
15.2 Variables and data types	138
15.3 Operators	140
15.4 Making decisions	141
15.5 Repeating things: loops	141
15.6 Functions	142
15.7 Arrays	143

15.8 Text: char arrays and String	144
15.9 Handy built-in functions	144
15.10 Talking to your computer: the Serial Monitor	144
15.11 Style, comments and structure	145
15.12 Libraries	146
16 Digital inputs and outputs	148
16.1 The three pin modes	148
16.2 Reading a button	149
16.3 Edge detection: reacting to the moment of change	150
16.4 Debouncing	150
16.5 The delay() problem, and millis()	151
16.6 State machines	153
16.7 Many outputs: patterns with arrays	154
17 Analog inputs and sensors	157
17.1 The analogRead() function	157
17.2 Potentiometers: converting readings	158
17.3 Resistive sensors and the voltage divider	159
17.4 Voltage-output sensors	161
17.5 Noise and filtering	162
17.6 Calibration and thresholds	163
18 PWM: analog-style outputs, servos and sound	166
18.1 What is PWM?	166
18.2 analogWrite() on the Uno	167
18.3 PWM for power: motors and LED strips	169
18.4 A real analog voltage from PWM: the RC filter	170
18.5 Servo motors	170
18.6 Making sound	172
19 A tour of common sensors	175
19.1 Ultrasonic distance: HC-SR04	175
19.2 Motion detection: the PIR sensor	176
19.3 Temperature and humidity: DHT11 and DHT22	177
19.4 Sound: the microphone module	178
19.5 Gas and air quality	179
19.6 Rotary encoder	179
19.7 Other common sensors	180
19.8 Working with libraries and datasheets	180
20 Displays: 7-segment, shift registers and LCD	182
20.1 The seven-segment display	182
20.2 The 74HC595 shift register	184
20.3 Multiplexing multi-digit displays	186

20.4 The 16×2 character LCD with I2C	186
20.5 Other displays: how to choose	188
21 Driving motors	190
21.1 Which motor for which job?	190
21.2 Reversing a DC motor: the H-bridge	190
21.3 Stepper motors	193
21.4 A small robot: obstacle avoidance	194
22 Communication: Serial, I2C and SPI	197
22.1 Serial and parallel; synchronous and asynchronous	197
22.2 UART: the Serial port	198
22.3 I2C: two wires, many devices	199
22.4 SPI: fast and simple	202
22.5 Two Arduinos talking	203
23 Interrupts and timers	205
23.1 Polling vs interrupts	205
23.2 External interrupts on the Uno	205
23.3 The rules of ISRs	206
23.4 Measuring a frequency or RPM	207
23.5 Hardware timers	208
23.6 The watchdog timer	210
23.7 Sleep and low power	210
24 Designing your own circuits	213
24.1 The design process	213
24.2 Interface recipes	215
24.3 Reading a datasheet	216
24.4 Worked design: a greenhouse monitor	216
24.5 Design patterns worth knowing	219
25 Debugging and troubleshooting	221
25.1 The debugging mindset	221
25.2 A general procedure	222
25.3 The top 12 faults	222
25.4 Hardware debugging tools and techniques	223
25.5 Power and noise problems	224
25.6 Software bugs and how to catch them	224
25.7 Testing components	225
25.8 Asking for help the right way	225
26 Capstone projects	227
26.1 Project 1: Temperature-controlled fan	227
26.2 Project 2: “Simon Says” memory game	230
26.3 Project 3: Security alarm with entry delay and event log	232

26.4 Your turn: project ideas 234

26.5 The “Can I design by myself?” checklist 234

27 Soldering, PCBs and where to go next 236

27.1 Soldering 236

27.2 From breadboard to permanent circuit 238

27.3 Designing a printed circuit board 238

27.4 Connectors, wiring and enclosures 239

27.5 Where to go next 239

Appendices: Reference 242

APPENDIX A Answers and hints to all assignments 243

APPENDIX B Formula cheat sheet 259

APPENDIX C Standard values, codes and tables 261

APPENDIX D Arduino quick reference 263

APPENDIX E Component pinouts 266

APPENDIX F Glossary 268

START HERE

Welcome

Read this short section first. It explains how the book works, what you need to buy, and the safety habits that will keep you and your equipment safe for life.

IN THIS PART

Preface and how to use this book

Your kit and shopping list

Safety rules

Welcome, and how this book works

This book has one job: to take you from **knowing nothing about electronics** to being able to **design, build, program and debug your own circuits with an Arduino**, without needing anybody to hold your hand. It does not assume any prior knowledge of physics, mathematics beyond school arithmetic, or programming.

Most beginner books fall into one of two traps. Some are “copy this wiring diagram and paste this code” cookbooks: you finish a project, but you cannot make a single change because you never learned *why* it worked. Others are theory textbooks: you learn equations but never touch a wire. This book deliberately mixes the two. Every idea is explained in words first, then with an everyday analogy, then with a real-world example, then with a calculation, and finally you build it and test it on a breadboard.

The three parts of the book

Part	What it covers	What you can do afterwards
I. Foundations (Chapters 1–5)	Charge, current, voltage, resistance, Ohm’s law, power, tools, breadboards, multimeters, reading schematics, series and parallel circuits, Kirchhoff’s laws, voltage dividers.	Analyse any simple resistor circuit on paper, predict what a meter will read, and build it on a breadboard.
II. Components (Chapters 6–13)	Capacitors, inductors, diodes and LEDs, transistors, switches, relays, motors, sensors, op-amps, the 555 timer, digital logic, batteries and power supplies.	Choose the right part for a job, calculate its supporting components, and understand every part in a datasheet.
III. Arduino (Chapters 14–27)	The Arduino board, C++ programming, digital and analog I/O, PWM, sensors, displays, motors, communication buses, interrupts, design method, debugging, projects, soldering and PCBs.	Take an idea, turn it into requirements, choose parts, draw a schematic, write firmware, build, test and fix it: entirely by yourself.

How each chapter is organised

Every chapter follows the same rhythm so you always know where you are. The coloured boxes have fixed meanings:

WHAT YOU WILL LEARN

- Opens each chapter and tells you the goals. Come back to it at the end and check that you can do every item.

ANALOGY

A picture from daily life that makes an invisible idea easy to imagine. Analogies are never perfect, so each one also tells you where it stops working.

REAL-LIFE EXAMPLE | EXAMPLE

A real device or situation (a phone charger, a bicycle light, a car) where the idea is used.

WORKED EXAMPLE | WITH NUMBERS

A fully solved calculation, step by step, with units. Cover the solution, try it yourself, then compare.

HANDS-ON EXPERIMENT | TRY IT

An experiment you do on a breadboard. Doing is the part of learning that reading cannot replace.

TIP

A shortcut or professional habit.

COMMON MISTAKE

An error almost every beginner makes, and how to avoid it.

SAFETY WARNING

Something that can hurt you or destroy a component. Never skip these.

ASSIGNMENTS (sample) | CHAPTER X

X.1 [Calculate] Example assignment: what current flows through a $100\ \Omega$ resistor connected to 5 V?

X.2 [Hands-on] Example: build the circuit and measure the current with a multimeter.

Every assignment is tagged **Calculate**, **Practice**, **Hands-on**, **Think** or **Design**. Answers and hints for every one of them are in Appendix A at the end of the book. Please try each assignment honestly before looking: struggling with a problem for five minutes teaches you more than reading its answer for one.

A twelve-week study plan

You can go faster or slower, but a steady pace beats bursts of enthusiasm. If you can spend about five hours a week (two hours reading, three hours building), this plan works well. If you are already comfortable with a topic, skim the chapter and do only the assignments.

Week	Chapters	Milestone
1	1, 2	You can calculate voltage, current, resistance and power. You know what a volt actually is.
2	3, 4	You can use a multimeter and a breadboard; you can read resistor colours and analyse series/parallel circuits.
3	5, 6	You can design a voltage divider; you understand time constants.
4	7, 8	You can choose an LED resistor and protect against inductive kick.
5	9, 10	You can drive a motor or a relay from a small signal with a transistor.
6	11, 12, 13	You understand amplifiers, logic levels and how to power a project properly.
7	14, 15	Your first sketches run; you can write C++ with variables, loops and functions.
8	16, 17, 18	Buttons, sensors and PWM outputs work reliably (debounced, smoothed).

Week	Chapters	Milestone
9	19, 20	You can read many sensors and drive LCDs, seven-segment displays and shift registers.
10	21, 22, 23	Motors, I2C, SPI, serial, interrupts and timers no longer scare you.
11	24, 25	You can plan a new design from a blank page and debug it systematically.
12	26, 27	You finish the capstone projects and design one of your own.

A promise about mathematics

The only maths you need is addition, subtraction, multiplication, division, and the ability to rearrange a simple equation such as $V = I \times R$ to find I or R . When a formula appears, we show where it comes from and work through several numerical examples. Powers of ten (such as $10^3 = 1000$) are explained in Chapter 2, because engineers use them constantly.

Your kit and shopping list

Electronics is cheap to start. A basic set of parts costs roughly the same as one restaurant meal. Buy an “Arduino starter kit” that contains most of the items below, and add the rest as needed. The list is grouped by when you first need each item. Do not buy everything at once: buy Group A for Part I and II, Group B for Part III, and Group C when you reach the chapter that needs it.

Group	Item	Quantity	Used in
A: basics	Digital multimeter (with continuity beeper)	1	Ch. 3 onward
A	Solderless breadboard (830 tie points) + jumper wires (male-male)	1 + 65	Ch. 3 onward
A	Resistor assortment ¼ W, E12 series, 10 Ω to 1 MΩ	~10 each of common values	Ch. 4 onward
A	LEDs: 5 mm red, green, yellow, blue; 1 RGB (common cathode)	10 + 1	Ch. 8 onward
A	Capacitors: ceramic 100 nF ×10; electrolytic 10 µF, 100 µF, 470 µF, 1000 µF	assorted	Ch. 6 onward
A	Diodes: 1N4148 (signal), 1N4007 (power), 5.1 V Zener	5 each	Ch. 8
A	Transistors: 2N2222 or BC547 (NPN), 2N7000 or IRLZ44N (N-MOSFET)	5 each	Ch. 9
A	9 V battery + clip, 4×AA holder + cells	1 each	Ch. 13
A	Push buttons (tactile), slide switch, 10 kΩ potentiometer ×2	5 / 1 / 2	Ch. 10 onward
A	Wire strippers, small pliers, side cutters	1 each	Ch. 3
B: Arduino	Arduino Uno R3 (or compatible) + USB cable	1	Ch. 14 onward
B	LDR (photoresistor), NTC thermistor 10 kΩ, TMP36 or LM35 sensor	1 each	Ch. 17, 19
B	Active buzzer, passive buzzer/speaker	1 each	Ch. 18
B	SG90 micro servo; small 5–6 V DC motor	1 each	Ch. 18, 21
B	Ultrasonic sensor HC-SR04; DHT11 or DHT22; PIR sensor HC-SR501	1 each	Ch. 19
C: later	16×2 LCD with I2C backpack; single-digit 7-segment display; 74HC595	1 / 1 / 2	Ch. 20
C	L293D or L298N motor driver; 28BYJ-48 stepper with ULN2003 board	1 each	Ch. 21
C	5 V relay module; small 12 V DC fan or lamp (for the relay project)	1	Ch. 10, 26
C	I2C real-time clock (DS3231); 8×8 LED matrix or OLED (optional)	1	Ch. 22

Group	Item	Quantity	Used in
C	Soldering iron (temperature controlled), solder 60/40 or lead-free, flux, perfboard	1 set	Ch. 27

GOOD TO KNOW

All quantities are suggestions. It is normal to lose or damage parts as you learn, so buy extra resistors, LEDs and transistors: they cost pennies. If your kit uses a different but equivalent part (for instance a BC547 instead of 2N2222) the circuits in this book still work; each time it matters, the text says how to adjust.

Safety first: ten rules for a lifetime of experiments

SAFETY WARNING | TEN RULES

Everything in this book runs on **low voltage (5 V to 12 V from USB, batteries or a small adapter)**. At these levels, electricity is essentially harmless to your body. The dangers are elsewhere: heat, short circuits, batteries, soldering, and, above all, **mains electricity** (the 110 V to 240 V from the wall socket).

- **Never connect your projects directly to mains.** Mains voltage can kill. Use only certified power adapters, or a certified, enclosed relay/SSR module, and only after you fully understand isolation and have an experienced person check your work.
- **Never open or repair a plugged-in device** (phone charger, TV, microwave). Capacitors in them can hold a lethal charge even when unplugged.
- **Never short-circuit a battery.** A wire across a lithium or NiMH cell can become red-hot and cause a fire. Never charge a non-rechargeable battery. Handle lithium-polymer (LiPo) cells with great care.
- **Power off before changing wiring.** Unplug the USB cable or battery before moving wires on your breadboard. This protects your Arduino and your parts.
- **Double-check polarity** before power-up: electrolytic capacitors, diodes, LEDs and ICs are damaged (and electrolytics can burst) if connected backwards.
- **Wear eye protection when cutting leads or soldering.** Snipped wire ends fly off at surprising speed, and hot solder can splash.
- **Solder in a ventilated area;** never breathe the fumes. Wash your hands after soldering, especially with leaded solder, and never eat while working.
- **Never touch a soldering iron tip** (about 350 °C, hot enough to burn skin instantly). Always return it to its stand.
- **Keep liquids away from your workbench,** and keep a small fire extinguisher or a bowl of sand in your room if you work with batteries.
- **Ask for help** when unsure. Nobody was born knowing this; every expert has burned a component or two.

PART I

Foundations

Before touching any component you need a clear mental picture of what electricity is and how it behaves. Part I builds that picture, gives you the maths to predict circuits, and teaches you the tools of the trade.

IN THIS PART

- 1 What is electricity?
- 2 Ohm's law, power and energy
- 3 Your workshop: tools, breadboard, multimeter, schematics
- 4 Resistors, series and parallel circuits
- 5 Kirchhoff's laws and voltage dividers

What is electricity?

Electronics is the art of controlling the flow of electricity to make useful things happen: to light an LED, spin a motor, read a temperature or play a sound. Before you can control something, you must understand it. In this first chapter we start from the very beginning: what is “electricity”, physically? Take your time here. Every later chapter, from resistors to Arduino programming, rests on these ideas.

WHAT YOU WILL LEARN IN THIS CHAPTER

- What atoms, charge and electrons are, and why some materials conduct while others insulate
- What **voltage**, **current** and **resistance** really mean, with units and typical values
- Why a circuit must be a complete loop, and what a short circuit is
- The difference between direct current (DC) and alternating current (AC)
- How to picture all of this with water flowing in pipes (and where that picture breaks down)

1.1 Everything is made of atoms

All the matter around you (air, water, copper wire, your own body) is made of tiny building blocks called **atoms**. Atoms are so small that a single grain of sand contains more than a billion billion of them. Although atoms are tiny, they have an inner structure, and it is that structure that gives us electricity.

An atom has a dense centre called the **nucleus**, containing two kinds of particles: **protons** and **neutrons**. Around the nucleus, a cloud of much lighter particles called **electrons** whizzes about. We often draw electrons as little balls on circular orbits (this is the simple “Bohr model”, not quite how nature works, but perfect for our purposes).

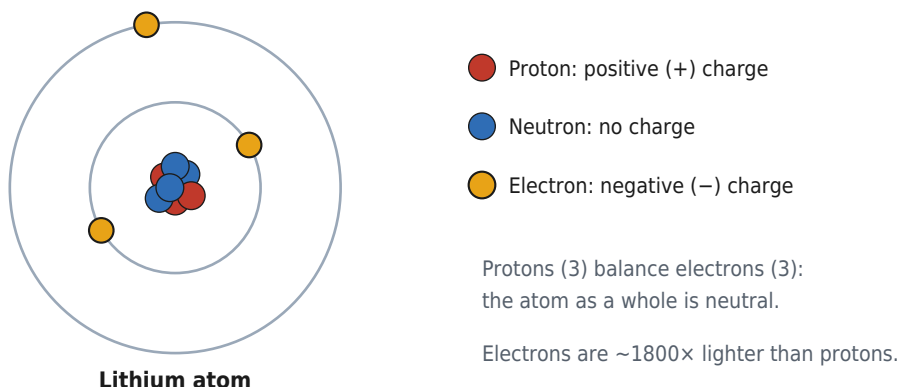


Figure 1.1 A simplified atom. Protons and neutrons sit in the nucleus; electrons orbit around it.

Electric charge

Protons and electrons carry a property called **electric charge**. Protons carry **positive** charge (+); electrons carry **negative** charge (–). Neutrons carry none. The amount of charge on a proton and an electron is exactly equal in size but opposite in sign, so an atom with the same number of protons and electrons is **electrically neutral**: the pluses and minuses cancel out.

The most important rule in all of electricity is short enough to memorise now:

Opposite charges attract. Like charges repel.

A positive charge pulls on a negative charge, and two positive charges (or two negative charges) push each other apart. This push and pull, acting across a distance, is called the **electric force**, and it is the reason anything electrical happens at all.

REAL-LIFE EXAMPLE | STATIC ELECTRICITY

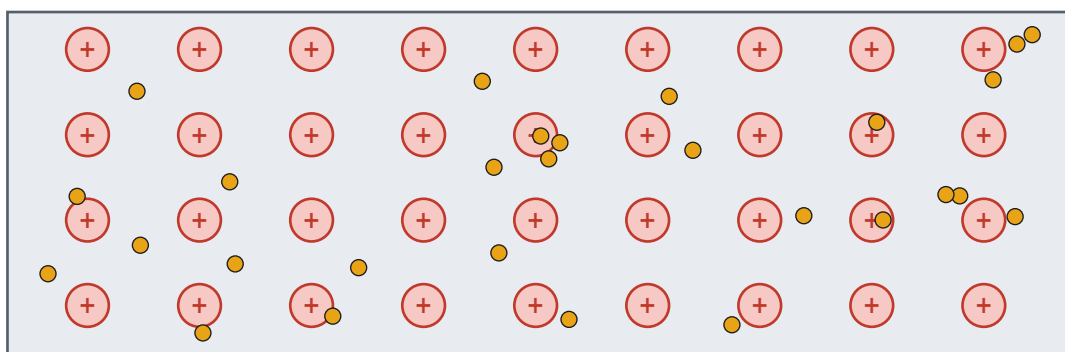
Rub a balloon on your hair. Electrons are rubbed off your hair onto the balloon. The balloon now has extra electrons (negative) and your hair is missing some (positive). Opposite charges attract, so your hair stands up towards the balloon. The little spark you feel when touching a door handle after walking on a carpet is those extra electrons jumping across the gap all at once, at a voltage of several thousand volts, but with so few electrons that it is harmless. Sparks like this are a real enemy of electronics: they can destroy sensitive chips, which is why technicians wear anti-static wrist straps.

The unit of charge: the coulomb

Charge is measured in **coulombs**, abbreviated **C** (named after the French physicist Charles-Augustin de Coulomb). One electron carries only a tiny charge, so a coulomb is a huge number of them: **one coulomb is the charge of about 6.24×10^{18} electrons** (6,240,000,000,000,000,000). You will rarely count coulombs directly, but the idea of counting charge leads straight to the idea of current.

1.2 Conductors, insulators and semiconductors

In most atoms, the inner electrons are held tightly by the nucleus. But the outermost electrons can be loosely bound. In metals such as copper, silver, gold and aluminium, the outer electrons are so loosely held that they detach from their atoms and wander freely through the material. The atoms (now positive ions because they lost an electron) stay in place in a regular lattice, like trees in an orchard, while a “sea” of **free electrons** drifts among them.



Copper: fixed positive ions (red) in a lattice, with a sea of free electrons (yellow) between them

Figure 1.2 Inside a metal wire. The free electrons are what carries current.

A material with many free electrons is a **conductor**: charge moves through it easily. A material whose electrons are all tightly bound is an **insulator**: charge barely moves. Between the two are **semiconductors** (silicon, germanium), whose behaviour we can control; they are the basis of every diode, transistor and microchip, including the one inside your Arduino.

Class	Examples	Free electrons?	Used for
Conductor	Copper, silver, gold, aluminium, steel, graphite, salty water, the human body	Many	Wires, PCB tracks, connectors, contacts
Insulator	Rubber, plastic, glass, dry wood, air, ceramic, PCB fibreglass (FR-4)	Almost none	Wire coating, casings, gloves, PCB base
Semiconductor	Silicon, germanium, gallium arsenide	Few, and controllable	Diodes, LEDs, transistors, integrated circuits

GOOD TO KNOW

Air is an insulator, but not a perfect one. If the voltage between two points is high enough (thousands of volts per millimetre), air breaks down and a spark forms. Lightning is the same effect at enormous scale: about 100 million volts.

1.3 Voltage: the push

Free electrons in a wire wander randomly, so they do not go anywhere in particular. To get a **flow**, you need something that pushes them in one direction. That push is called **voltage**.

ANALOGY | VOLTAGE IS HEIGHT DIFFERENCE

Imagine two water tanks connected by a pipe. If both are filled to the same level, nothing flows. If one tank is higher than the other, water flows from high to low. The greater the **height difference**, the stronger the push on the water.

Voltage is the “height difference” for electric charge. A battery is like a pump that keeps lifting water (charge) from the low tank to the high one, so there is always a push available.

Where the analogy stops: water only flows downhill because of gravity, while electric charge is pushed by electric force. And voltage, unlike height, is always measured **between two points**, never at one point alone.

More precisely, **voltage** (also called **potential difference**) is the amount of energy given to each coulomb of charge as it moves between two points. It is measured in **volts (V)**, named after Alessandro Volta, inventor of the first battery. The formal definition is:

1 volt = 1 joule of energy per coulomb of charge

$$1 \text{ V} = 1 \text{ J} / \text{C}$$

So a 9 V battery gives 9 joules of energy to every coulomb that passes through it. The symbol for voltage in equations is **V** (sometimes **U** or **E** in other countries), and the volt unit is also written **V**. Do not let this confuse you: V in italics/equations is the quantity; V after a number is the unit.

Source or place	Typical voltage
Nerve signal in your body	0.07 V (70 mV)
Silicon diode “forward drop”	0.6 – 0.7 V

Source or place	Typical voltage
A red LED while lit	$\approx 2\text{ V}$
Alkaline AA/AAA cell	1.5 V (1.0 V when flat)
Lithium-ion phone cell	3.7 V (3.0 – 4.2 V range)
Arduino Uno logic level	5 V (ESP32 / Arduino Due: 3.3 V)
USB port	5 V
Car battery	12 V (14 V when charging)
Household mains outlet	230 V (India, Europe, UK), 120 V (USA)
Overhead power transmission lines	11 kV – 765 kV
Lightning bolt	$\sim 100,000,000\text{ V}$

Voltage is always “between two points”

Saying “the voltage is 5 V” is incomplete, like saying “the hill is 50 metres”. You must say **between which two points**: “5 V between the red wire and the black wire”. In circuits we usually choose one point as the **reference** and call it **ground (GND)** or **0 V**. All other voltages are then measured relative to it, just as the height of every hill is measured relative to sea level. Ground is not magic: it is simply the point we agreed to call zero.

1.4 Current: the flow

When a voltage is applied across a conductor, the free electrons start to drift in one direction. This drift of charge is called **electric current**. Current is measured in **amperes (A)**, often shortened to “amps”, named after André-Marie Ampère. The formal definition is:

1 ampere = 1 coulomb of charge passing a point every second

$$I = Q / t$$

Here **I** is the current in amperes, **Q** is the charge in coulombs and **t** is the time in seconds. So 1 A means about 6.24 billion billion electrons passing a point in the wire every second.

ANALOGY | CURRENT IS FLOW RATE

Current is like the **flow rate of water** in a pipe: litres per second. A wide, fast river carries a large current of water, while a dripping tap carries a tiny one. Notice that flow rate measures **how much passes a point per second**, not how fast individual water molecules move.

The same is true in wires: electrons themselves creep along at less than a millimetre per second. Why then does a lamp light instantly when you flip the switch? Because a wire is already full of electrons, like a pipe already full of water. Push in one end and water immediately comes out of the other. The “push” travels near the speed of light; the electrons themselves are slow.

WORKED EXAMPLE | CHARGE FROM CURRENT

A phone charger supplies 2 A. How much charge flows in 10 seconds?

Rearrange $I = Q / t$ to $Q = I \times t$.

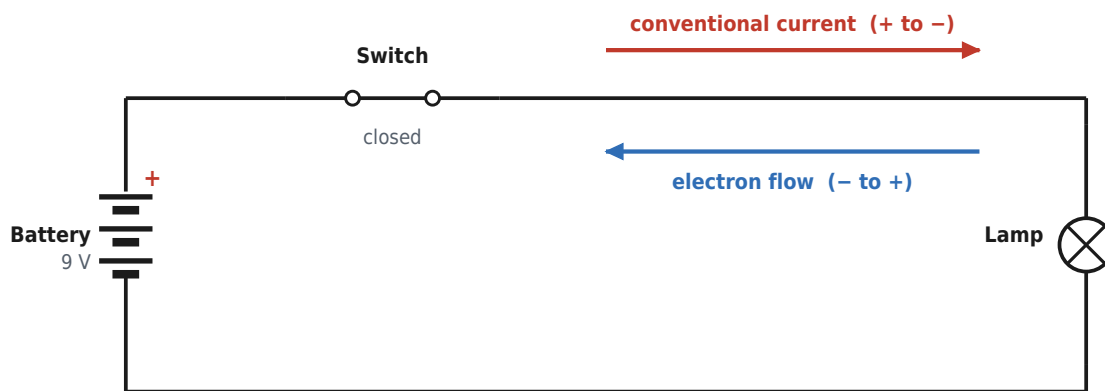
$Q = 2 \text{ A} \times 10 \text{ s} = \mathbf{20 \text{ C}}$.

That is $20 \times 6.24 \times 10^{18} \approx 1.25 \times 10^{20}$ electrons: more than a hundred billion billion.

Conventional current versus electron flow

There is a historical quirk you must know about. When scientists first defined current (long before electrons were discovered), they guessed that current flowed from the **positive** terminal of a battery, through the circuit, to the **negative** terminal. They guessed wrong: in metals, the moving particles are electrons, which are negative, so they actually travel from negative to positive.

By the time this was discovered, all textbooks, diagrams and formulas used the original direction. So we keep it, and call it **conventional current**: it flows from + to –, as if positive charges were moving. Mathematically it makes no difference. **In this book and in every circuit diagram, arrows for current point in the conventional direction (from + toward –).** Just remember the electrons are actually moving the opposite way.



The bottom wire carries the same current, in the same directions, back to the battery.

Figure 1.3 A complete circuit. Conventional current (red) and electron flow (blue) run in opposite directions.

Typical currents

Device	Typical current
Ultra-low-power microcontroller in sleep mode	1 μA (0.000001 A)
A single LED with its resistor	5 – 20 mA (0.005 – 0.02 A)
Arduino Uno board running by itself	~45 mA
Maximum recommended from ONE Arduino pin	20 mA (absolute maximum 40 mA)
Small hobby servo while moving	100 – 500 mA
USB phone charger	1 – 3 A
Laptop	2 – 5 A (from a 19 V supply)
Electric kettle on mains	~9 A at 230 V
Car starter motor	150 – 400 A

SAFETY WARNING

Current is what does the damage to people, not voltage alone. As a rough guide, through the body: about **1 mA** is barely felt, **10 - 20 mA** makes muscles clamp (you cannot let go), and **~100 mA** passing near the heart can be fatal. High voltage is dangerous because it can push a dangerous current through the skin. Low voltages such as 5 V or 9 V cannot push enough current through dry skin to hurt you, which is why our projects are safe to handle.

1.5 Resistance: the opposition

Even with a push, the water flow depends on the pipe. A wide clear pipe lets much water through; a narrow pipe, or one packed with gravel, lets little through. In electricity, this opposition to the flow of current is called **resistance**, measured in **ohms**, written with the Greek capital letter omega, **Ω**, after Georg Ohm. The symbol in equations is **R**.

ANALOGY | RESISTANCE IS A NARROW, GRAVELLY PIPE

Think of a pipe filled with pebbles. Water can squeeze through but only slowly, and the pebbles get a bit warm from the friction. A narrower or longer gravel-filled pipe resists more; a shorter or wider one resists less.

In a wire, electrons collide with the vibrating atoms of the lattice as they drift. Each collision wastes a bit of their energy as **heat**. This is why resistors get warm, why a toaster glows, and why a phone charger cable gets warm at high current.

Four things decide the resistance of a piece of wire:

- **Material.** Copper resists little; nichrome (an alloy used in heaters) resists roughly 65 times more.
- **Length.** A longer wire has more collisions: resistance is proportional to length.
- **Cross-sectional area.** A thicker wire has more room for electrons: resistance is inversely proportional to area.
- **Temperature.** For most metals, resistance rises as they heat up.

$$R = \rho \times L / A$$

ρ (rho) = resistivity of the material in $\Omega \cdot m$, L = length in m , A = cross-section area in m^2

Material	Resistivity ρ ($\Omega \cdot m$) at 20 °C	Type
Silver	1.59×10^{-8}	Best conductor
Copper	1.68×10^{-8}	Standard wire material
Gold	2.44×10^{-8}	Corrosion-free contacts
Aluminium	2.82×10^{-8}	Power lines, lightweight
Iron / Steel	$\sim 1.0 \times 10^{-7}$	Poorer conductor
Nichrome	1.10×10^{-6}	Heater elements
Graphite (carbon)	$\sim 3.5 \times 10^{-5}$	Carbon resistors
Pure silicon	~ 640	Semiconductor
Glass	$10^{10} - 10^{14}$	Insulator
Rubber	$\sim 10^{13}$	Insulator

WORKED EXAMPLE | RESISTANCE OF AN EXTENSION CORD

A copper wire is 10 m long with a cross-section of 0.5 mm^2 (that is $0.5 \times 10^{-6} \text{ m}^2$). What is its resistance?

$$R = \rho L / A = (1.68 \times 10^{-8} \times 10) / (0.5 \times 10^{-6})$$

$$R = 1.68 \times 10^{-7} / 5 \times 10^{-7} = \mathbf{0.336 \, \Omega}$$
 (about a third of an ohm).

That is tiny compared with most parts, but at 10 A the wire would waste $10 \text{ A} \times 0.336 \, \Omega = 3.4 \text{ V}$ (32 W as heat!), which is why heavy appliances need thicker cables.

REAL-LIFE EXAMPLE | WHY THIN JUMPER WIRES AND LONG CABLES MISBEHAVE

Beginners are often surprised when a motor connected to a long, thin wire runs slowly. Thin wire means small A, so larger R, so a larger share of the battery voltage is lost in the wire itself. Cable choice matters.

1.6 The complete circuit

Voltage pushes, resistance opposes, and current is what results, but there is one further requirement: current can only flow in a **closed loop**. Charge cannot pile up in one place; for every electron that leaves the battery, another must return to it. A path from one battery terminal, through your parts, and back to the other terminal is called a **circuit** (from the word “circle”).

Circuit state	What it means	Effect
Closed circuit	Unbroken conductive path from source, through the load, back to source	Current flows; the device works
Open circuit	A break in the path (switch off, cut wire, loose connection)	No current flows; device is off. Voltage is present across the break
Short circuit	A very-low-resistance path that bypasses the load (for instance, a wire directly across the battery)	Huge current flows; wires and batteries overheat, fuses blow, components are destroyed

Every working circuit contains three fundamental parts:

1. A **source** that provides voltage (battery, USB port, power adapter, solar cell).
2. A **load**: something that uses the energy to do useful work (LED, motor, speaker, microcontroller).
3. **Conductors** that connect them in a loop (wires, breadboard strips, PCB tracks).

COMMON MISTAKE

Electricity is **not “used up”** in a circuit. The current leaving the load is exactly the same as the current entering it. What gets used up is **energy**: the charge arrives at the load with energy and leaves without it (that energy became light, heat, motion or sound). Think of water passing through a water wheel: the same water comes out, but it has given up its energy to turn the wheel.

SAFETY WARNING

A short circuit across a battery or power supply is the single most common way beginners damage equipment. If your Arduino suddenly turns off, if a wire feels hot or if your USB port resets, **disconnect power at once** and hunt for the short. Chapter 25 explains how.

1.7 Direct current (DC) and alternating current (AC)

So far we have imagined current flowing in one direction all the time. This is called **direct current (DC)**: a battery or USB port supplies DC. But there is another kind. In **alternating current (AC)**, the voltage swings back and forth, changing direction many times each second. The electrons just jiggle to and fro instead of flowing steadily forward. Wall sockets provide AC because AC is easy to transform between voltages and cheap to transmit across long distances.

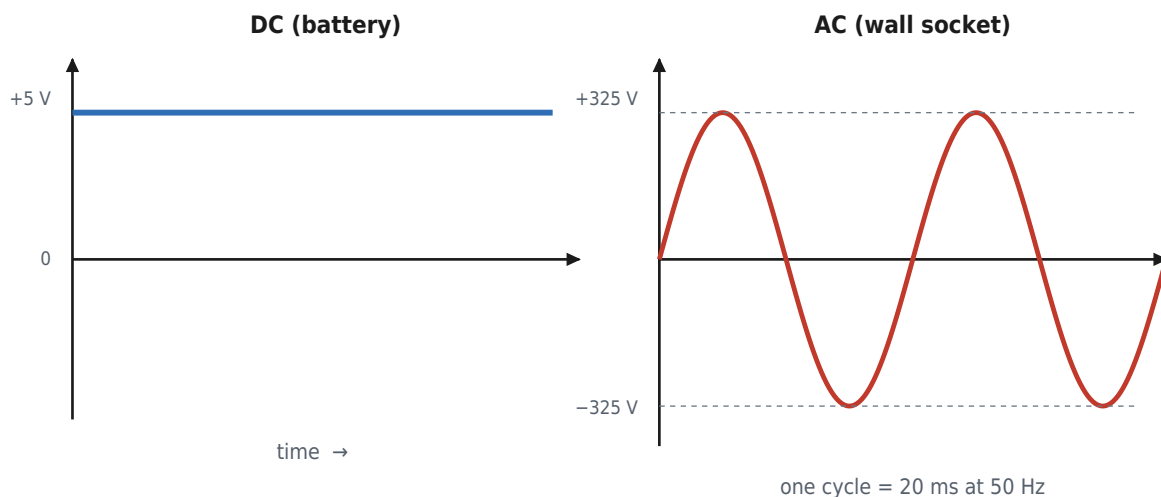


Figure 1.4 DC stays constant. AC swings positive and negative (here two cycles are shown; mains has 50 or 60 every second).

Frequency and period

AC is described by its **frequency**, the number of complete back-and-forth cycles per second, measured in **hertz (Hz)**. Mains electricity is 50 Hz in India, Europe and most countries, and 60 Hz in the USA. The time for one complete cycle is the **period**, $T = 1 / f$. At 50 Hz, $T = 1 / 50 = 0.02 \text{ s} = 20 \text{ ms}$.

$$f = 1 / T$$

frequency (Hz) = $1 \div$ period (seconds)

The 230 V of a mains socket is the **RMS** (“root mean square”) value: the DC voltage that would deliver the same heating power. The actual peak of the swinging voltage is higher, $230 \times \sqrt{2} \approx \mathbf{325 \text{ V}}$, as Figure 1.4 shows. You will meet RMS again when you study power supplies.

GOOD TO KNOW

Almost all electronics inside your gadgets uses DC. That is why phone chargers, laptop bricks and USB adapters exist: they take AC from the wall and convert it to low-voltage DC. Everything in this book, including Arduino projects, uses DC. AC will appear only when we discuss how power adapters work.

1.8 Bringing the water analogy together

We have now met the three fundamental quantities. Here they are side by side with the water picture. Hold on to this table: it is the core mental model of electronics.

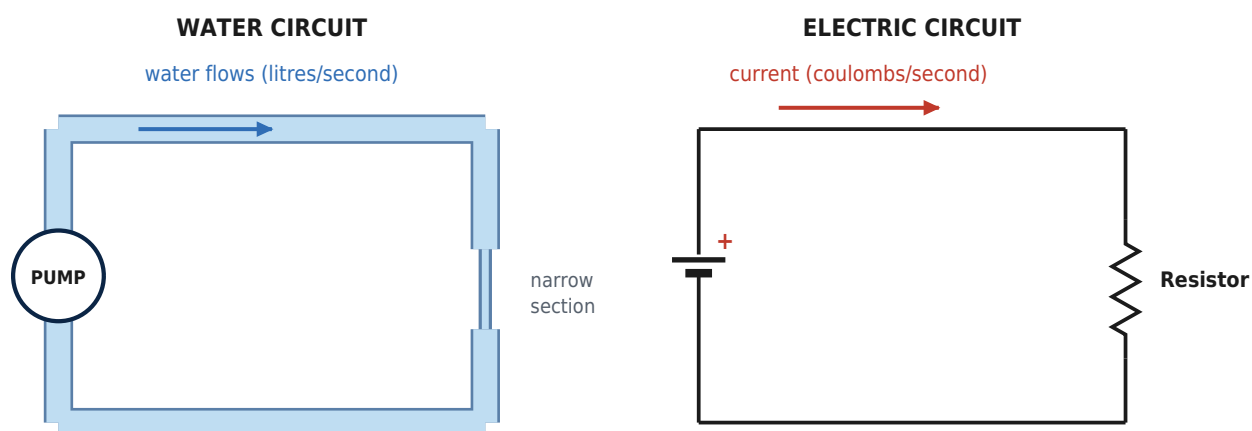


Figure 1.5 The same loop in water form (left) and electrical form (right).

Electrical quantity	Symbol	Unit	Water analogy	Everyday hint
Charge	Q	coulomb (C)	Amount of water (litres)	How many electrons
Voltage	V	volt (V)	Pressure / height difference	The push. Battery rating (9 V)
Current	I	ampere (A)	Flow rate (litres per second)	How fast charge flows. Charger rating (2 A)
Resistance	R	ohm (Ω)	Narrowness of the pipe	Opposition to flow. Colour bands on resistors
Source	—	—	Pump	Battery, USB port, adapter
Load	—	—	Water wheel or turbine	LED, motor, speaker, microcontroller
Switch	—	—	Tap or valve	Button, toggle

You can already reason qualitatively: **more voltage** → **more current** (higher pressure pushes more water), **more resistance** → **less current** (narrower pipe passes less water). In the next chapter these two sentences become one clean equation, Ohm's law, which is the single most useful tool in electronics.

HANDS-ON EXPERIMENT | FEEL THE DIFFERENCE BETWEEN CONDUCTORS AND INSULATORS (NO POWER NEEDED)

Take your multimeter (if you have one, otherwise skip to step 4) and set it to the **continuity / resistance** mode. Touch the two probes together: it beeps or reads close to 0 Ω .

1. Touch the probes to the two ends of a piece of wire. Reading? (Near 0 Ω : a good conductor.)
2. Touch them to opposite ends of a plastic ruler. Reading? (Out of range / "OL": an insulator.)
3. Hold one probe in each hand (dry fingers). Now moisten your fingertips and repeat. Watch the reading change: skin resistance depends on moisture, and that is exactly why wet hands and electricity are a dangerous combination.
4. Without a meter: rub a plastic pen on wool or your hair, then bring it near small pieces of paper. They jump to the pen: static charge attracts neutral objects.

CHAPTER SUMMARY

- Matter is made of atoms with positive protons, neutral neutrons and negative electrons; opposite charges attract, like charges repel.
- Conductors (metals) have free electrons; insulators do not; semiconductors are in between and form the basis of modern electronics.
- **Voltage (V, volts)** is the push, energy per unit charge, measured between two points. **Current (I, amperes)** is the flow of charge, coulombs per second. **Resistance (R, ohms)** is the opposition to flow.
- Current flows only around a complete loop. An open circuit stops current; a short circuit gives dangerously large current.
- Conventional current flows from + to –, opposite to the actual electron motion. All diagrams in this book use conventional current.
- DC flows in one direction (batteries, USB); AC alternates (mains, 50 or 60 Hz). Electronics mostly uses DC.
- Electricity is not used up; energy is transferred from the source to the load.

ASSIGNMENTS | CHAPTER 1

1.1 [Calculate] A wire carries a constant current of 0.5 A for 2 minutes. How much charge passes a point in the wire in that time?

1.2 [Calculate] A 1.5 V cell moves 200 C of charge through a circuit. How much energy does it deliver? (Use $V = 1 \text{ J/C}$, so $E = V \times Q$.)

1.3 [Calculate] How many electrons pass a point each second when the current is 1 mA? (One coulomb $\approx 6.24 \times 10^{18}$ electrons.)

1.4 [Calculate] Find the resistance of 2 m of copper wire with cross-sectional area 1 mm^2 ($1 \times 10^{-6} \text{ m}^2$). Use $\rho = 1.68 \times 10^{-8} \Omega \cdot \text{m}$.

1.5 [Calculate] If the wire in the previous question were replaced with aluminium ($\rho = 2.82 \times 10^{-8} \Omega \cdot \text{m}$) of identical size, what would its resistance be? By what factor is it bigger?

1.6 [Calculate] Mains in India has $f = 50 \text{ Hz}$. What is the period of one cycle? How long do 10 cycles take?

1.7 [Think] A friend says: “The battery uses up the electrons, so eventually it runs out of electrons.” Explain in two or three sentences what is wrong with this statement.

1.8 [Think] Why does a lamp light almost instantly when you flip a switch even though electrons drift only about a millimetre per second?

1.9 [Think] In the water analogy, what corresponds to (a) a battery, (b) a switch, (c) an open circuit, (d) a short circuit?

1.10 [Hands-on] Use a multimeter’s resistance mode to measure five household objects (e.g. a coin, a pencil lead, a plastic pen, a piece of paper, your dry and wet skin). Record your readings in a table and classify each object as conductor, semiconductor-ish or insulator.

Ohm's law, power and energy

Chapter 1 gave you the vocabulary: voltage, current and resistance. In this chapter they get connected by the most famous equation in electronics. Ohm's law lets you predict, with a pencil, what a circuit will do before you build it. Together with the power formulas, it answers practical questions: which resistor should I use? Will it overheat? How long will my battery last? Before the equations, though, we need to learn how engineers write big and small numbers.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How to read and write numbers with prefixes such as k, M, m, μ , n and p
- **Ohm's law** ($V = I \times R$) and how to rearrange it to find any of the three quantities
- What electrical **power** is, how to calculate it, and why parts get hot
- **Energy**, kilowatt-hours, and how to estimate how long a battery will last
- A step-by-step method for solving any basic circuit problem

2.1 Writing numbers the way engineers do

In electronics, values span an enormous range. A modern microcontroller may run at 16,000,000 cycles per second, while a tiny sensor may draw 0.000002 A. Writing all those zeros would be tiring and error-prone, so we use **SI prefixes**: short letters placed before the unit that mean “multiply by”. You must know these by heart, because every resistor, capacitor and datasheet uses them.

Prefix	Symbol	Multiplier	Power of ten	Typical use
giga	G	1,000,000,000	10^9	Processor speed (GHz)
mega	M	1,000,000	10^6	Large resistors (M Ω), clock (MHz)
kilo	k	1,000	10^3	Resistors (k Ω), distance (km)
(none)	—	1	10^0	Volts, amps, ohms, watts
milli	m	0.001	10^{-3}	Small currents (mA), time (ms)
micro	μ	0.000001	10^{-6}	Capacitors (μ F), tiny currents (μ A), time (μ s)
nano	n	0.000000001	10^{-9}	Small capacitors (nF), time (ns)
pico	p	0.000000000001	10^{-12}	Tiny capacitors (pF)

The rule for converting is simple: **each step down the table multiplies the number by 1000 (or divides, going up)**. Two examples:

- $4.7 \text{ k}\Omega = 4.7 \times 1000 \Omega = \mathbf{4700 \Omega}$.
- $250 \text{ mA} = 250 \div 1000 \text{ A} = \mathbf{0.25 \text{ A}}$.
- $100 \text{ nF} = 100 \div 1000 \mu\text{F} = \mathbf{0.1 \mu\text{F}}$.
- $2.2 \text{ M}\Omega = 2.2 \times 1000 \text{ k}\Omega = \mathbf{2200 \text{ k}\Omega}$.

COMMON MISTAKE

Forgetting to convert prefixes before calculating. If you compute $I = V / R$ with $V = 5 \text{ V}$ and $R = 2 \text{ k}\Omega$, you must write $R = 2000 \Omega$, not 2. Otherwise your answer is out by a factor of 1000. A safe habit: **convert everything to base units (volts, amps, ohms) first, calculate, and only then convert the answer to a convenient prefix.**

Another trap: “m” means milli (10^{-3}) but “M” means mega (10^6). A $5 \text{ m}\Omega$ resistor and a $5 \text{ M}\Omega$ resistor differ by a factor of a billion. Capitalisation matters!

TIP

On resistors and in some schematics, the prefix letter replaces the decimal point to avoid smudged dots: **4k7** means $4.7 \text{ k}\Omega$, **2M2** means $2.2 \text{ M}\Omega$ and **0R47** (or **R47**) means 0.47Ω . You will see this often.

2.2 Ohm’s law

In 1827, the German teacher Georg Simon Ohm discovered something remarkable. For many materials, especially metals and carbon, if you double the voltage across them the current doubles; triple the voltage, triple the current. In other words, **voltage and current are proportional**, and the ratio between them is constant. That constant is the resistance. Written as an equation:

$$V = I \times R$$

Voltage (volts) = Current (amperes) $\times$ Resistance (ohms)

This is **Ohm’s law**. It is worth understanding in words: for a given resistor, more voltage produces more current; and for a given voltage, more resistance allows less current. One ohm is defined as the resistance that lets **1 ampere flow when 1 volt is applied**: $1 \Omega = 1 \text{ V} / 1 \text{ A}$.

ANALOGY | PRESSURE, FLOW AND NARROWNESS

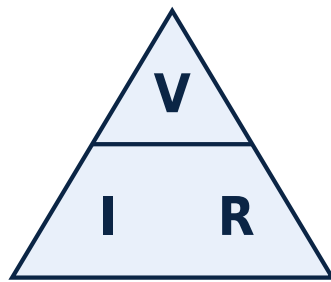
In the water world, the flow through a pipe equals pressure divided by how narrow the pipe is: double the pressure and the flow doubles; make the pipe half as wide (double the resistance) and the flow halves. Ohm’s law is precisely this statement for electricity.

A garden hose makes it real: opening the tap wider raises the pressure (V), the water gushes out (I). Squeezing the hose with your thumb raises the resistance (R), and the flow falls, even though the tap has not moved.

Three forms of one equation

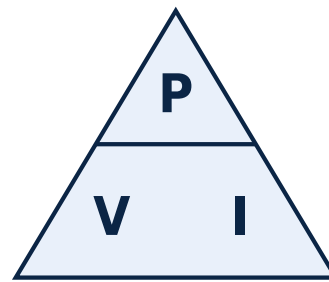
Ohm’s law can be rearranged to solve for whichever quantity you do not know. All three forms say the same thing:

To find...	Use	Say it in words
Voltage V	$V = I \times R$	Voltage equals current times resistance
Current I	$I = V \div R$	Current equals voltage divided by resistance
Resistance R	$R = V \div I$	Resistance equals voltage divided by current



Ohm's law triangle

Cover the quantity you want:



Power triangle

Cover P, V or I in the same way.

Figure 2.1 Memory aids. Cover the unknown: if the remaining two are side by side, multiply; if one sits over the other, divide. (Cover V: $I \times R$. Cover I: $V \div R$. Cover R: $V \div I$.)

WORKED EXAMPLE | FINDING CURRENT

An LED circuit has a $330\ \Omega$ resistor and a 5 V supply across it. How much current flows? (We ignore the LED for now.)

Known: $V = 5\text{ V}$, $R = 330\ \Omega$. **Unknown:** I .

$$I = V \div R = 5 \div 330 = 0.01515\text{ A} = \mathbf{15.2\text{ mA}}$$

Sanity check: a few hundred ohms with a few volts gives a few tens of milliamps. Reasonable.

WORKED EXAMPLE | FINDING VOLTAGE

A current of 20 mA flows through a $150\ \Omega$ resistor. What voltage appears across the resistor?

First convert: $20\text{ mA} = 0.020\text{ A}$.

$$V = I \times R = 0.020 \times 150 = \mathbf{3\text{ V}}$$

This idea (“the voltage across a resistor is $I \times R$ ”) is called a **voltage drop**, and we will use it constantly.

WORKED EXAMPLE | FINDING RESISTANCE

You measure 9 V across an unknown resistor, and a meter in series shows 30 mA. What is R ?

$$R = V \div I = 9 \div 0.030 = \mathbf{300\ \Omega}$$

WORKED EXAMPLE | THE UNIT-PREFIX SHORTCUT

Because $V = I \times R$, if the current is in **mA** and resistance is in **k Ω** , the voltage comes out directly in **volts**: since $10^{-3} \times 10^3 = 1$.

Try: $I = 2\text{ mA}$ through $R = 4.7\text{ k}\Omega$ gives $V = 2 \times 4.7 = \mathbf{9.4\text{ V}}$. No conversions needed. Similarly, V in volts divided by R in $\text{k}\Omega$ gives current in mA.

Ohmic and non-ohmic parts

Ohm's law is not really a “law of nature”. It is a description of how **some** materials behave. If you plot current against voltage for a resistor, you get a straight line through zero: the current is exactly proportional to the voltage. Parts that behave like this are called **ohmic**. Many other parts do not: a diode or an LED passes almost no current until the voltage reaches a threshold (about 0.6 V for silicon, about 2 V for a red LED), then current soars. The resistance of such **non-ohmic** parts changes with voltage. Ohm's law can still be applied to them at any single operating point, but the ratio V/I is no longer constant.

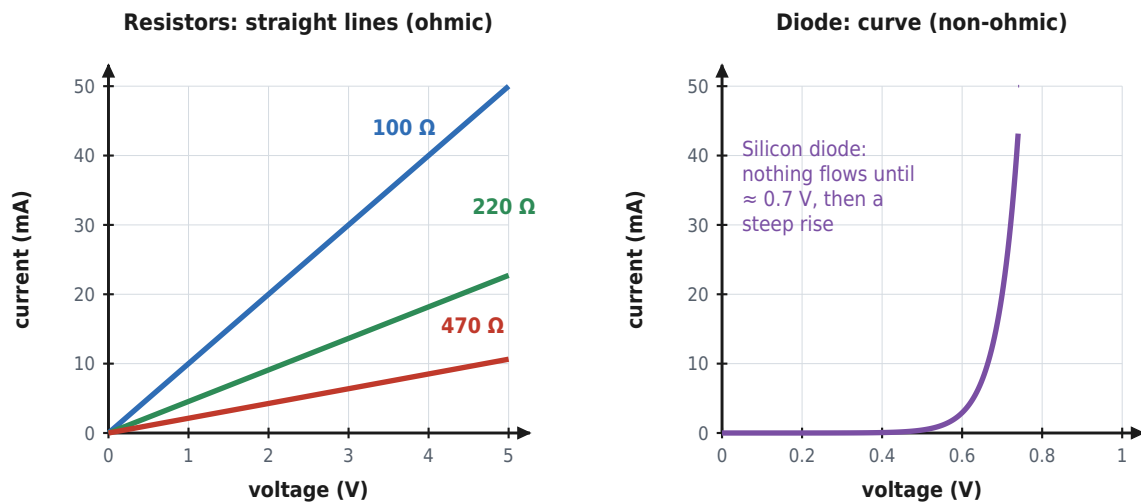


Figure 2.2 Current versus voltage. The steeper the line, the smaller the resistance.

REAL-LIFE EXAMPLE | A PHONE CHARGING CABLE

Your phone charger may supply 5 V. If the cable has a total resistance of 0.5 Ω (both wires together) and the phone draws 2 A, Ohm's law says the cable drops $V = 2 \times 0.5 = 1 \text{ V}$. The phone then receives only 4 V, and charges slowly or refuses to fast-charge. Thick short cables (low R) charge faster than thin long ones. The same idea explains why thin jumper wires can cause trouble with motors.

2.3 Electrical power

Voltage and current together tell us how quickly **energy** is being converted. A tiny LED and a powerful electric heater may both run on the same 230 V, but the heater draws much more current and therefore does much more work per second. The rate of doing work is called **power**, measured in **watts (W)**, named after James Watt. One watt is one joule of energy per second.

$$P = V \times I$$

Power (watts) = Voltage (volts) × Current (amperes)

ANALOGY | POWER IS HEIGHT TIMES FLOW

Consider a waterfall. Its power depends on two things: how far the water falls (height, i.e. voltage) and how much water goes over per second (flow, i.e. current). A tall, thin trickle and a short, wide cascade can deliver the same power. Multiply height by flow and you get power. That is $P = V \times I$.

Because $V = I \times R$, we can substitute it into $P = V \times I$ to get two other very useful versions. Which one you use depends on which two quantities you know:

Formula	Use it when you know...	Derivation
$P = V \times I$	voltage and current	the basic definition
$P = I^2 \times R$	current and resistance	replace V by $I \times R$
$P = V^2 \div R$	voltage and resistance	replace I by $V \div R$

The formula $P = I^2 \times R$ has a lesson hidden in it: the heat wasted in a wire or resistor grows with the **square** of the current. Doubling the current gives four times the heating. That is why high-current cables must be much thicker, and why an overloaded wire can start a fire.

Where does the power go?

In a resistor, all the electrical power is turned into **heat**. In an LED it becomes mostly light plus some heat; in a motor it becomes motion plus heat; in a speaker, sound plus heat. In every device, some energy ends up as heat, and that heat must be able to escape or the part will overheat and fail.

Every resistor has a **power rating** (how much heat it can safely shed). Common through-hole resistors are rated $\frac{1}{4}$ W (**0.25 W**); larger ones 1/2 W, 1 W, 2 W, 5 W and above. **Always ensure the power the resistor will dissipate is comfortably below its rating**: engineers aim for at most half of it.

WORKED EXAMPLE | CHECKING A RESISTOR'S POWER

A 330 Ω resistor has 5 V across it. Will a $\frac{1}{4}$ W resistor survive?

$P = V^2 \div R = 5^2 \div 330 = 25 \div 330 = 0.0758 \text{ W} = \mathbf{76 \text{ mW}}$.

76 mW is less than a third of 250 mW. The resistor stays cool. ✓

WORKED EXAMPLE | A RESISTOR THAT WOULD BURN

A 100 Ω resistor is connected across a 12 V supply. What is the power? Is $\frac{1}{4}$ W enough?

$P = V^2 \div R = 144 \div 100 = \mathbf{1.44 \text{ W}}$. This is almost six times the $\frac{1}{4}$ W rating.

The resistor would become too hot to touch within seconds and could smoke. You would need at least a 3 W part (and preferably 5 W for safety margin).

SAFETY WARNING

A resistor that is too small in wattage will get very hot, discolour, smell of burnt varnish and eventually fail, sometimes catching fire. If anything on your breadboard is too hot to touch, **disconnect the power immediately** and recheck the numbers.

REAL-LIFE EXAMPLE | THE ELECTRIC KETTLE

A kettle is a resistor. A 2000 W kettle on 230 V draws $I = P \div V = 2000 \div 230 = 8.7 \text{ A}$. Its heating element resistance is $R = V^2 \div P = 230^2 \div 2000 = \mathbf{26.5 \Omega}$. All 2000 W becomes heat in the element, which is precisely what we want from a kettle, and exactly what we do not want from the wires leading to it, which is why the cable is much thicker than an LED lead.

2.4 Energy, kilowatt-hours and batteries

Power is a rate; **energy** is the total amount. If you know how many watts something uses and for how long, you know the energy:

$$E = P \times t$$

Energy (joules) = Power (watts) $\times$ Time (seconds)

One watt for one second is one joule. Because joules are small, the electricity meter in your home uses a larger unit, the **kilowatt-hour (kWh)**: the energy used by a 1000 W device running for one hour. One kWh

equals 3.6 million joules. Your bill is “units” × price per unit, where one “unit” is one kWh.

WORKED EXAMPLE | AN ELECTRICITY BILL

A 1500 W room heater runs for 4 hours every day for 30 days. If a unit (kWh) costs ₹7 (an assumed price; check your tariff), what is the cost?

Energy per day = $1.5 \text{ kW} \times 4 \text{ h} = 6 \text{ kWh}$. For 30 days = 180 kWh.

Cost = $180 \times ₹7 = ₹1,260$.

By comparison an LED bulb (9 W) running 4 h a day uses $9 \times 4 \times 30 / 1000 = 1.08 \text{ kWh}$, only ₹7.6. This is why LED lighting saves money.

Battery capacity: mAh and Wh

Small batteries are rated in **milliamp-hours (mAh)**: a 2000 mAh battery can, in principle, supply 2000 mA for one hour, or 200 mA for 10 hours, or 20 mA for 100 hours. Since charge equals current × time, **mAh is a measure of stored charge** (1 mAh = 3.6 coulombs). To compare batteries with different voltages, convert to **watt-hours (Wh)** by multiplying by voltage:

$$\text{Energy (Wh)} = \text{Capacity (Ah)} \times \text{Voltage (V)}$$

$$\text{Runtime (hours)} \approx \text{Capacity (mAh)} \div \text{Load current (mA)}$$

WORKED EXAMPLE | HOW LONG WILL MY ARDUINO RUN?

An Arduino Uno draws about 50 mA. It is powered from a battery pack rated 2000 mAh. Estimate the run time.

Runtime $\approx 2000 \text{ mAh} \div 50 \text{ mA} = \mathbf{40 \text{ hours}}$ in the ideal case.

In practice, batteries deliver less capacity at higher currents and lose some through voltage regulators, so a realistic figure is 70 – 80 % of that: about 28 – 32 hours.

WORKED EXAMPLE | BATTERY ENERGY

A phone battery is labelled 3.7 V, 4000 mAh. How much energy is that?

$E = 3.7 \text{ V} \times 4 \text{ Ah} = \mathbf{14.8 \text{ Wh}}$. Compare a laptop: 50 Wh. A car battery: $12 \text{ V} \times 50 \text{ Ah} = 600 \text{ Wh}$.

2.5 A method for solving circuit problems

Beginners often stare at a circuit problem and freeze. Professionals follow a routine. Adopt the following seven steps every time; after a dozen problems they become automatic.

1. **Draw** the circuit (or read the schematic). Label each part with its value.
2. **List what you know** and what you are asked for, with units.
3. **Convert** every value into base units (V, A, Ω, W): remove all prefixes.
4. **Choose** the formula that links what you know to what you want (Ohm’s law, power formulas).
5. **Rearrange** it so the unknown is alone on one side.
6. **Calculate**, then convert the result into a convenient prefix.
7. **Sanity-check**. Is the answer the right order of magnitude? Does a bigger resistance give a smaller current? Are units consistent? Would the part survive the power?

2.6 Cheat sheet: all twelve Ohm and power relations

If you know any two of V, I, R, P you can find the other two. Here are all twelve formulas in one table. You do not need to memorise them: understand $V = I R$ and $P = V I$ and derive the others when needed.

Unknown ↓ Known →	V and I	V and R	I and R	P and one other
Voltage V	—	—	$V = I R$	$V = P / I$ or $V = \sqrt{P R}$
Current I	—	$I = V / R$	—	$I = P / V$ or $I = \sqrt{P / R}$
Resistance R	$R = V / I$	—	—	$R = V^2 / P$ or $R = P / I^2$
Power P	$P = V I$	$P = V^2 / R$	$P = I^2 R$	—

CHAPTER SUMMARY

- Prefixes: G (10^9), M (10^6), k (10^3), m (10^{-3}), μ (10^{-6}), n (10^{-9}), p (10^{-12}). Always convert to base units before calculating.
- **Ohm's law:** $V = I \times R$. Rearranged: $I = V / R$ and $R = V / I$. It holds for ohmic parts such as resistors; diodes and LEDs are non-ohmic.
- **Power:** $P = V \times I = I^2 R = V^2 / R$, measured in watts. In a resistor it all becomes heat, so check the resistor's power rating ($\frac{1}{4}$ W is common).
- **Energy:** $E = P \times t$. 1 kWh = 1000 W for 1 hour. Battery runtime $\approx$ capacity (mAh) $\div$ current (mA).
- Use the seven-step method: draw, list knowns, convert, choose formula, rearrange, calculate, sanity-check.

ASSIGNMENTS | CHAPTER 2

2.1 [Practice] Convert: (a) 4.7 k Ω to Ω , (b) 330 mA to A, (c) 2200 μ F to mF, (d) 47 nF to μ F, (e) 1.2 M Ω to k Ω , (f) 15 mV to V.

2.2 [Calculate] A 470 Ω resistor is connected across a 12 V supply. What current flows?

2.3 [Calculate] A current of 3 mA flows through a 2.2 k Ω resistor. What is the voltage across it?

2.4 [Calculate] A resistor has 9 V across it and carries 30 mA. What is its resistance?

2.5 [Calculate] A 150 Ω resistor carries 20 mA. What power does it dissipate? Is a $\frac{1}{4}$ W resistor adequate?

2.6 [Calculate] A 100 Ω resistor is placed across a 9 V battery. What power is dissipated, and what is the minimum standard rating you would choose ($\frac{1}{4}$, $\frac{1}{2}$, 1, 2 W)?

2.7 [Calculate] A 1500 W heater runs from 230 V. Find (a) its current, (b) its resistance.

2.8 [Calculate] A 3.7 V, 2500 mAh lithium cell powers a circuit that draws 125 mA. Estimate the runtime, and the energy stored in the cell.

2.9 [Calculate] A 60 W bulb is on for 5 hours daily for 30 days. How many kWh does it use? How much would replacing it with a 9 W LED bulb save (in kWh)?

2.10 [Practice] You measure an unknown component: (1 V, 4.7 mA), (2 V, 9.5 mA), (4 V, 18.9 mA), (8 V, 38 mA). Calculate V/I for each. Is the part ohmic? Estimate its resistance and suggest a standard value.

2.11 [Think] If you double the voltage across a fixed resistor, what happens to (a) the current, (b) the power? Explain.

2.12 [Think] Why does a phone charger cable get warm when charging at 3 A, but stays cool at 0.3 A? Use a formula.

2.13 [Design] You have a 5 V supply and want exactly 10 mA to flow through a resistor. Calculate the resistance needed and the power rating required. Choose from the E12 values: 330, 390, 470, 560, 680 Ω .

Your workshop: tools, breadboard, multimeter and schematics

Theory is essential, but electronics is a hands-on craft. In this chapter you set up your workbench, learn the three skills every electronics person uses daily (building on a breadboard, measuring with a multimeter, and reading schematics), and then verify Ohm's law with your own hands.

WHAT YOU WILL LEARN IN THIS CHAPTER

- The essential hand tools and what each is used for
- How a **solderless breadboard** is wired inside, and how to build tidy, reliable circuits on it
- How to use a **multimeter** to measure voltage, current, resistance and continuity: safely and correctly
- How to read a **schematic diagram**: symbols, reference designators, nets and grounds
- How to turn a schematic into a real circuit on a breadboard

3.1 The beginner's toolkit

You do not need an expensive lab. The following tools cover almost everything in this book.

Tool	What it is used for	Buying advice
Digital multimeter (DMM)	Measures voltage, current, resistance, continuity, diodes	Get one with a continuity buzzer and an auto-off. A €/₹ mid-range meter is plenty; avoid the very cheapest that lack fuse protection.
Solderless breadboard	Build circuits with no soldering; reusable	830 tie-point size gives room for projects; buy 2 or 3.
Jumper wires	Connect points on a breadboard	Male-to-male (65-piece pack). Male-to-female wires later for sensor modules.
Wire strippers	Remove insulation without cutting the copper	Self-adjusting strippers are easiest.
Side cutters (flush)	Trim component leads	Small "flush cut" type.
Needle-nose pliers	Bend leads, place parts, retrieve dropped wires	Small size with smooth jaws.
Tweezers	Handle tiny parts and pull ICs off breadboards	Fine, non-magnetic.
Safety glasses	Protect eyes from flying lead trimmings and solder splashes	Wear them; they are cheap insurance.
Soldering iron + stand (Ch. 27)	Make permanent joints	Temperature-controlled, 30–60 W.
Bench power supply (later)	Adjustable voltage with current limit	Optional but extremely useful as you advance.
Oscilloscope (later)	Shows how voltages change over time	Optional. A cheap USB unit is adequate for a hobbyist.

TIP

Keep a **notebook** (paper or digital). Draw each circuit before you build it, write the measurements you took and note what surprised you. Professional engineers do exactly this. When something breaks three weeks later, the notebook is how you find out what changed.

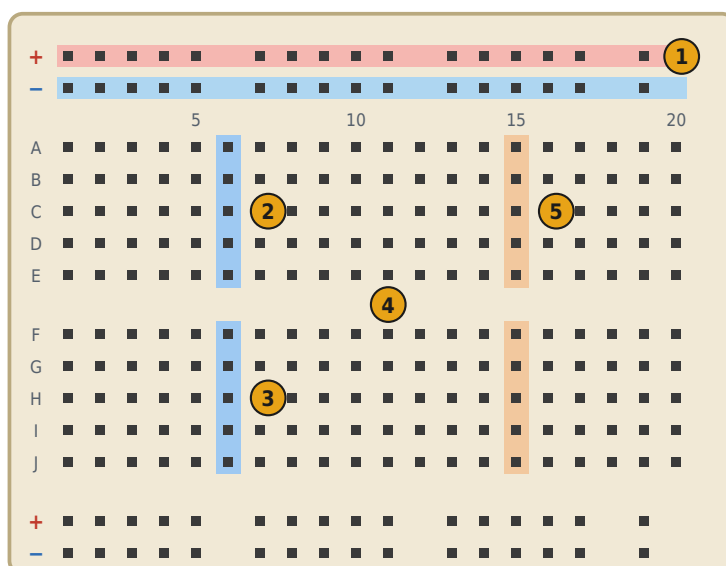
3.2 The solderless breadboard

A breadboard is a plastic board with hundreds of small spring-loaded holes. Push a component lead or a jumper wire into a hole and a metal clip inside grips it, making electrical contact **without solder**. You can build a circuit in minutes, change your mind, pull everything out and start again. It is the single most important prototyping tool for a beginner.

How the holes are connected inside

The key to using a breadboard is knowing which holes are connected underneath. There are two kinds of areas:

- **Terminal strips (the main area).** Each column of five holes (for example column 12, holes A-E) is joined by a metal strip underneath. The holes F-J of the same column are joined together as well, but **the top half (A-E) and the bottom half (F-J) are NOT connected to each other**. The gap in the middle separates them; it is spaced so that a standard chip (IC) can straddle it, with the pins on each side landing in separate, unconnected strips.
- **Power rails (the long lines at the edges).** These are two long strips running the entire length of the board, usually marked with red (+) and blue (-). All the holes in one rail are joined together along the length. Use them to distribute the supply voltage (+) and ground (-) to every part of your circuit.
Caution: on some larger boards, the rails are split in the middle: if your board has a gap in the printed line, bridge it with a jumper wire.



- 1 Power rail: every hole along the red (+) strip is connected. Same for the blue (-) strip.
- 2 Column 6, holes A-E: joined together.
- 3 Column 6, holes F-J: joined to each other, but NOT to A-E above.
- 4 Centre gap: the two halves are isolated. A chip straddles this gap.
- 5 Different columns (6 and 15) are never connected unless you add a wire.

Figure 3.1 A breadboard. Coloured areas show holes joined together beneath the surface: each column-half (5 holes) and each long power rail.

Rules for tidy breadboard building

1. **Plan from the schematic first.** Put the power rails at the top and bottom and connect them to your supply before anything else. Use **red wires for +** and **black or blue for ground** consistently.
2. **Place parts so that each connection shares a column.** To connect the end of a resistor to the start of an LED, put both leads in the same column strip (say A and B of column 12).
3. **Keep leads short.** Trim resistor and LED leads (or bend them neatly) so parts sit flat against the board. Long floppy leads are a source of mistakes and short circuits.
4. **Use different colours for signal wires**, e.g. yellow for a signal from Arduino pin 9, green for pin 10. It makes debugging much easier.
5. **Never force thick leads or wires into the holes.** Standard component wires (0.6 mm) and 22-AWG solid jumpers fit. Bent or wide leads (like some power connectors, or thick LED leads) stretch the contacts and ruin the board.
6. **Turn the power off** before you add, remove or move any wire.
7. **Add the power last** and connect the power rails with jumper wires from the Arduino's 5 V and GND pins.

COMMON MISTAKE

Beginners often assume the whole column of ten holes (A-J) is connected. It is not: the centre gap divides it. Another classic: plugging **two component leads into the same hole**, or placing both ends of one part in the same 5-hole strip, which short-circuits the part. If a circuit does nothing, the first thing to check is which holes are actually connected.

3.3 The multimeter

A multimeter is the electronic equivalent of a doctor's stethoscope: it lets you see what is happening inside a circuit. One instrument, several jobs: it measures **voltage, current and resistance**, and usually also tests continuity, diodes and sometimes capacitance and temperature.

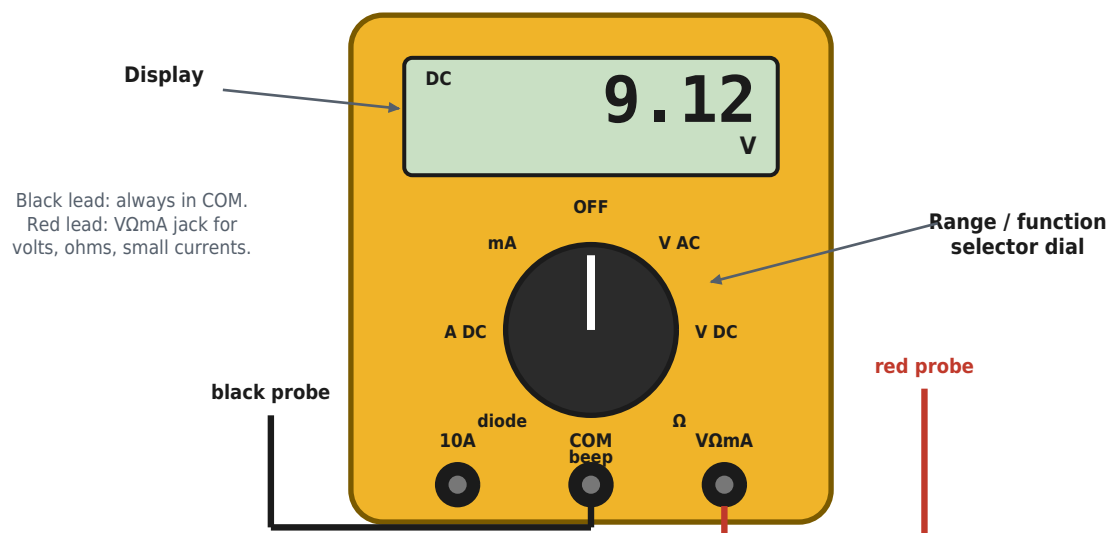


Figure 3.2 A typical digital multimeter. The dial chooses what is measured; the probes plug into the jacks at the bottom.

Measuring voltage: connect in PARALLEL

To measure the voltage across a component, touch the probes to **both sides of it**, without breaking the circuit. The meter is placed **in parallel** (side by side) with the part. A voltmeter has very high internal resistance (about 10 M Ω), so it barely disturbs the circuit. Choose “V with a straight line” (DC) for batteries and USB; “V with a wave” (AC) only for mains-type signals.

Measuring current: connect in SERIES

Current is the flow **through** a wire, so the meter has to become part of the path: you must **break the circuit and insert the meter in the gap**, so that all the current flows through the meter. An ammeter has almost zero internal resistance, so it acts like a piece of wire. Because of this, connecting it across a voltage source (in parallel) creates a short circuit and blows the meter’s internal fuse.

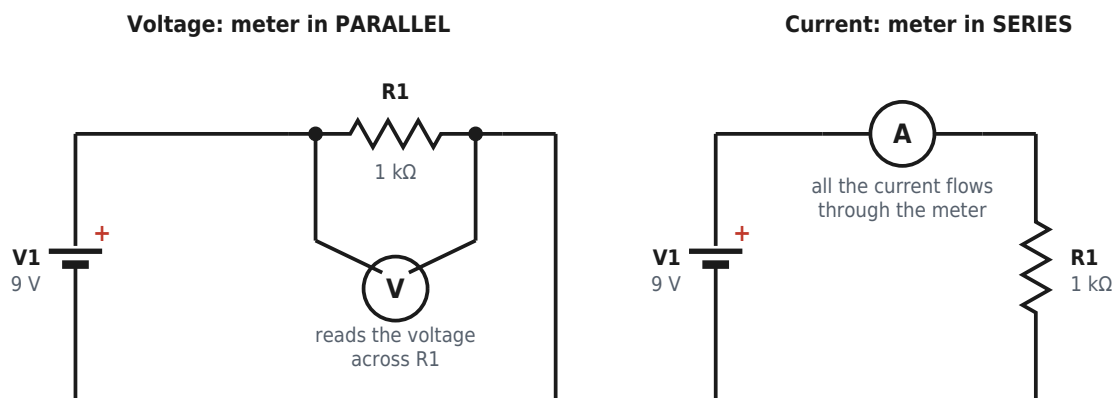


Figure 3.3 Voltmeter across the part (parallel); ammeter in the path (series).

SAFETY WARNING | FUSES AND CURRENT MODE

Never connect a meter set to “current” across a battery or power supply. The ammeter is nearly a short circuit; it will blow its internal fuse (best case) or overheat and damage the meter (worst case). Before you measure current, make sure the red lead is in the correct jack (mA or 10A), and the dial is set to current. After measuring, **return the red lead to the V Ω jack and the dial to voltage**: most meter accidents occur because someone left the meter set for current and then tried to measure voltage.

Measuring resistance: power OFF and part isolated

Set the dial to Ω , touch the probes to the two leads of the resistor. The meter sends its own tiny current through the part and calculates $R = V / I$. Three rules: **(1)** the circuit must be **unpowered**; **(2)** for accurate results at least one lead of the part should be **out of the circuit** (otherwise other parts in parallel confuse the reading); **(3)** do not hold both metal leads with your fingers, or you add your own body resistance (tens of k Ω) in parallel.

Continuity test

The continuity mode (a speaker-like symbol) beeps if the resistance between the probes is very low (less than about 50 Ω). It is the fastest way to find a broken wire, check if two breadboard holes are connected or hunt down a short circuit. **Always use it on an unpowered circuit.**

Diode test

The diode symbol (a triangle with a line) sends a small current through a diode or LED and shows its forward voltage: about 0.5 – 0.7 V for silicon diodes, 1.6 – 3 V for LEDs (a red LED often lights faintly during the test). Reversed, the meter shows “OL”. It also identifies the leads of transistors (Chapter 9).

Task	Dial setting	Red lead jack	How to connect	Circuit powered?
DC voltage (battery, Arduino pin)	V DC (straight line)	VΩmA	In parallel across the part	Yes
AC voltage (mains-type; be careful)	V AC	VΩmA	In parallel	Yes
Small DC current (up to ~200 mA)	mA DC (or A)	mA jack	In series (break the circuit)	Yes
Large DC current (up to 10 A)	A DC	10A jack	In series (break the circuit)	Yes
Resistance	Ω	VΩmA	Across the isolated part	No
Continuity	beep symbol	VΩmA	Across the wire or joint	No
Diode / LED	diode symbol	VΩmA	Red on anode (+), black on cathode (–)	No

TIP

If you do not know the size of the quantity, start at the highest range (or use auto-range) and work down. If the display shows “OL” or “1.”, the value is above the selected range or the circuit is open.

HANDS-ON EXPERIMENT | EXPERIMENT 1: VERIFY OHM’S LAW WITH A BATTERY AND A RESISTOR

You need: 9 V battery with clip, breadboard, one 1 kΩ resistor (brown-black-red), multimeter, two jumper wires.

Step 1 (prediction). From Ohm’s law, predict the current: $I = 9\text{ V} \div 1000\ \Omega = 9\text{ mA}$. Write it in your notebook.

Step 2 (resistance). Before connecting anything, measure the resistor with the meter in Ω mode. Is it exactly 1000 Ω? (Most are within 5 %: between 950 and 1050 Ω.)

Step 3 (voltage). Put the resistor into two different columns of the breadboard. Connect the battery’s red lead to one end and black to the other. Measure the voltage across the battery, then across the resistor (parallel). They should be nearly equal, about 9 V (a fresh 9 V battery reads up to 9.6 V).

Step 4 (current). Disconnect the red battery lead from the resistor. Set the meter to mA, red lead in the mA jack. Put the meter between the battery’s red lead and the resistor (series). You should read approximately 9 mA.

Step 5 (compare). Calculate $V \div I$ with your measured values. It should match the measured resistance to a few percent. Differences arise from resistor tolerance, meter accuracy and the battery’s own small internal resistance.

Step 6 (repeat). Replace the resistor with 470 Ω, 2.2 kΩ, and 10 kΩ. Record V, I, and $R = V \div I$ in a table. Plot I against R: what shape do you see?

3.4 Reading schematic diagrams

A **schematic** is the map of an electrical circuit. It shows **what is connected to what**, not where parts physically sit. Every electronics book, datasheet and online project uses schematics, so reading them fluently is as important as reading music for a musician. Fortunately, there are only a couple of dozen symbols to learn, and you will already recognise most.

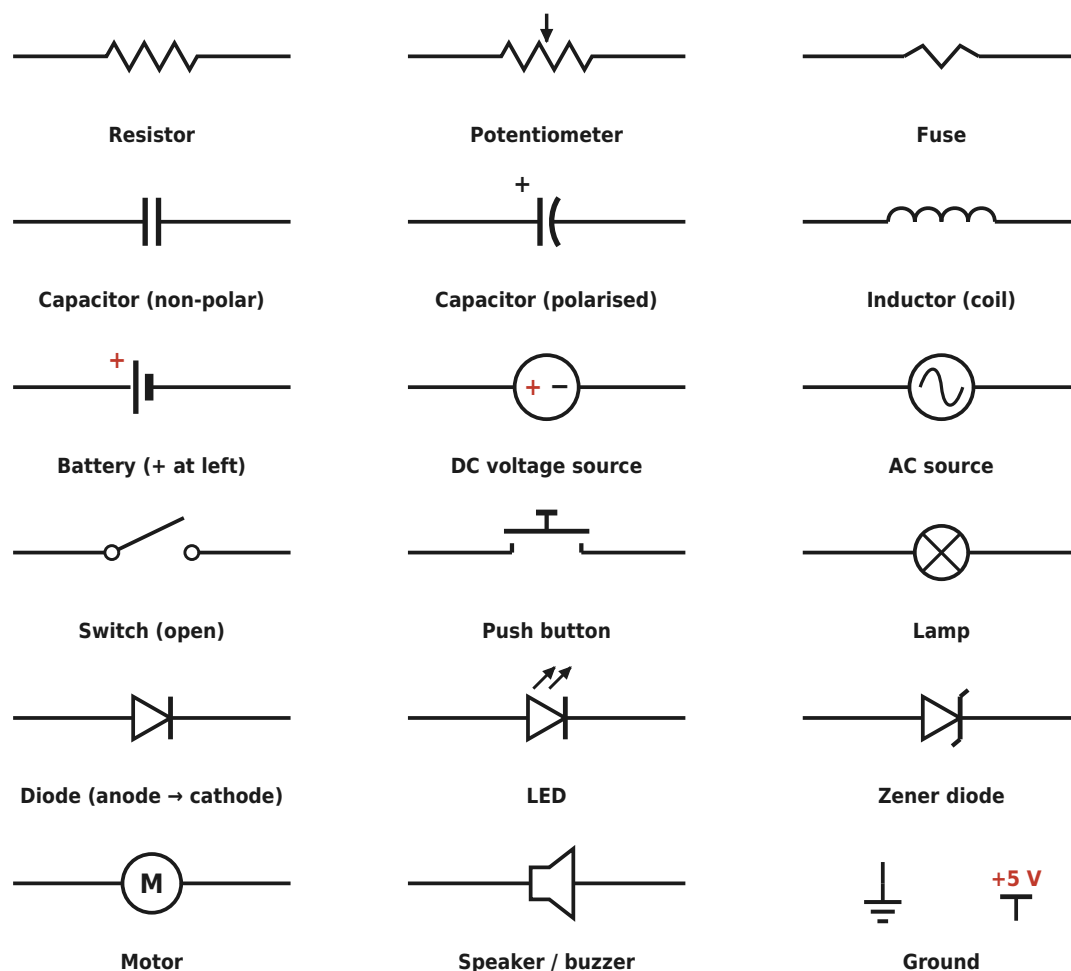


Figure 3.4 The symbols you will meet most often. (Transistors, op-amps and chips appear in later chapters.)

Reference designators and values

Every part on a schematic carries a **name** (its **reference designator**) and often a **value**. The first letter tells you the kind of part; the number just tells you which one it is. So “R3” is the third resistor and “C1” is the first capacitor. Standard letters:

Letter	Part	Letter	Part
R	Resistor	Q	Transistor
C	Capacitor	U (or IC)	Integrated circuit (chip)
L	Inductor / coil	SW or S	Switch, button
D	Diode (LEDs are D or LED)	J or P	Connector, jack, header
F	Fuse	B or BT	Battery
K	Relay	M	Motor

Letter	Part	Letter	Part
LS or BZ	Speaker / buzzer	X or Y	Crystal / oscillator

Wires, junctions and crossings

Lines are wires. Where two wires cross, the meaning depends on the dot: **a dot means connected; no dot means the wires simply pass over each other without touching**. Some older diagrams show a small “hop” arc for non-connected crossings. To keep drawings tidy, wires are often replaced by **net labels**: two points marked with the same name (for example “SDA” or “+5V”) are connected even without a drawn wire. Similarly, every ground symbol on a schematic is the same connection.

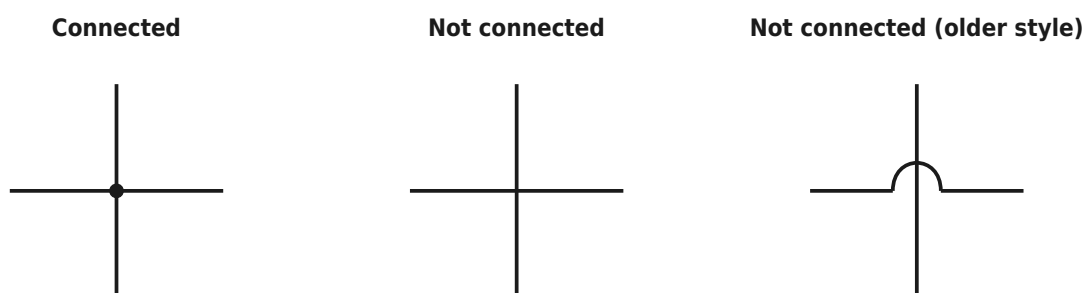


Figure 3.5 Junction dot = connected; plain crossing = not connected.

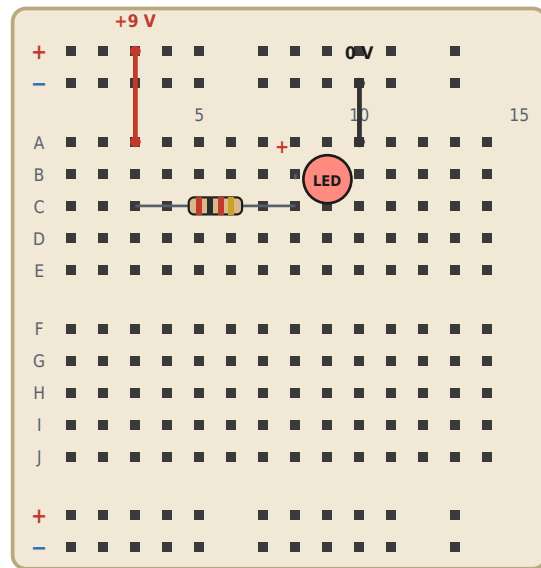
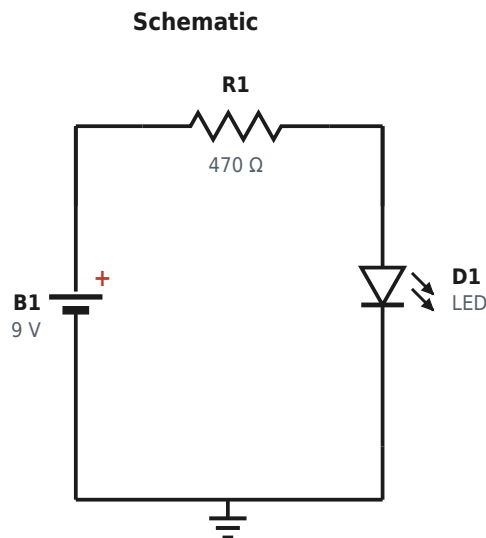
The reading order

By convention, schematics are drawn with the **positive supply at the top, ground at the bottom** and signals flowing **from left (inputs) to right (outputs)**. This is not a law, but following it makes your own diagrams easier for others (and future you) to read. The flow of a signal, not the physical layout of the parts, is what you want to see at a glance.

From schematic to breadboard

A recipe for turning any schematic into a working breadboard circuit:

1. Identify the **nets**: sets of points connected by wires. In the schematic below, one net is “battery + → resistor”, another is “resistor → LED anode”.
2. Give each net a column (or rail) on the breadboard: all part leads that belong to the same net go in holes that are connected.
3. Place the parts one at a time, tick them off on the printed schematic with a pencil.
4. Add jumper wires only where two nets must connect but are too far apart.
5. Recheck every connection against the schematic **before** applying power.



Breadboard (battery clip connects to the two top rails)

Figure 3.6 The same circuit as a schematic and as breadboard wiring. Red wire brings +9 V to column 3; resistor bridges columns 3 and 8; LED bridges 8 and 10; black wire returns to ground.

3.5 Good habits from day one

- **Test in stages.** Build a small part of the circuit, test it, then add the next. A 30-part circuit built all at once and failing gives you 30 suspects; a stage-by-stage build gives you one.
- **Measure before you assume.** If a voltage should be 5 V, put the meter on it. A meter reading beats a guess.
- **Take photographs** of working breadboards. When your project breaks after you move it, the photo is your reference.
- **Label your parts.** Keep resistors in labelled compartments; loose resistors look identical until you decode the colour bands.
- **Keep the workbench uncluttered and dry.** Static electricity, spilled drinks and stray wire clippings cause more failures than bad design.
- **Use a common ground.** Connect every ground together in one system. Half the mysteries in electronics come from grounds that are not actually connected.

CHAPTER SUMMARY

- A breadboard connects holes in 5-hole column halves and along the long power rails; the centre gap isolates top from bottom.
- Use consistent colours (red +, black –), keep leads short, plan from the schematic, and power off before rewiring.
- A multimeter measures **voltage in parallel, current in series**, and **resistance/continuity with the circuit unpowered**. Never place an ammeter across a source.
- Schematics show connections, not physical layout. A dot at a crossing means connected; each part has a letter-number name and often a value.
- To build from a schematic, identify the nets, map each net to a breadboard strip and verify every connection before applying power.

ASSIGNMENTS | CHAPTER 3

3.1 [Practice] On a breadboard, which of these holes are connected: (a) A5 and E5, (b) A5 and F5, (c) A5 and A6, (d) F12 and J12, (e) two holes on the top power rail 20 holes apart?

3.2 [Practice] You want to measure the current through an LED circuit powered by a 5 V supply. Describe exactly where the meter goes and how you connect it, and name one thing that will go wrong if you connect it across the supply instead.

3.3 [Practice] Your meter displays “OL” when measuring the resistance of a component. Give two possible explanations.

3.4 [Practice] Explain the difference between measuring resistance in a powered and an unpowered circuit, and why the second is correct.

3.5 [Practice] Draw the schematic of a 9 V battery in series with a 1 k Ω resistor and a switch, with a voltmeter across the resistor and an ammeter in the loop. Label all parts with reference designators.

3.6 [Calculate] In Experiment 1 you measured 8.9 V across a resistor and 8.6 mA through it. What is the resistor’s actual resistance? If it is marked 1 k Ω , what is its percentage error?

3.7 [Hands-on] Carry out Experiment 1 for four resistor values and fill in a table with columns R (measured), V, I (measured), V/I. State whether Ohm’s law is confirmed.

3.8 [Hands-on] Use the continuity mode to trace the connections on your own breadboard: confirm that columns are connected as in Figure 3.1, and find out whether your power rails are split in the middle.

3.9 [Think] Why is it a bad idea to build a big circuit all at once and then power it up? Give at least two reasons.

Resistors, series and parallel circuits

The resistor is the most common component in electronics: a typical circuit board holds dozens. It is also the ideal first component to study, because everything it does follows directly from Ohm's law. In this chapter we meet the resistor in detail and then discover what happens when several resistors are connected together. These simple rules (series adds, parallel shares) are the backbone of circuit analysis.

WHAT YOU WILL LEARN IN THIS CHAPTER

- What resistors are made of, what their key specifications mean, and how they are used
- How to read the **colour code** and the standard **E12/E24** value series
- How resistors behave when connected **in series** and **in parallel**
- How to reduce a mixed network to a single equivalent resistance, and find every voltage and current in it
- How to make a value you do not have out of the parts you do have

4.1 What a resistor does and how it is built

A resistor is a component designed to have a specific, stable resistance. It has two leads and no polarity: you can insert it either way round. It does three jobs in circuits:

- **Limit current:** for example, protecting an LED from destroying itself (Chapter 8).
- **Divide voltage:** making a lower voltage from a higher one (Chapter 5).
- **Set timing and bias:** together with capacitors (Chapter 6) and transistors (Chapter 9).

The common through-hole resistor is a small cylinder with coloured bands. Inside, a thin film of carbon or metal is deposited on a ceramic rod. A helical groove is cut to lengthen the path and raise resistance to the desired value. Metal caps with leads are pressed on the ends and the body is coated in lacquer.

Type	How it is made	Accuracy	Where used
Carbon film	Carbon layer on ceramic rod	$\pm 5\%$ (gold band)	General purpose, cheapest; found in kits
Metal film	Metal alloy layer on ceramic rod	$\pm 1\%$ (brown band) or better	Precision, low noise, stable with temperature; preferred for design
Wire-wound	Resistance wire (e.g. nichrome) wound on a core	$\pm 1-5\%$	High power (5 W – hundreds of W): heaters, power supplies, brake resistors
Thick/thin film SMD	Tiny rectangle soldered on board surface	$\pm 1-5\%$	Almost all modern products, e.g. phones, Arduino boards
Carbon composition (old)	Carbon powder + binder	$\pm 10-20\%$	Vintage equipment

Variable and special resistors

- **Potentiometer (“pot”)**: a resistor with a sliding contact (the wiper). Three terminals. Turn the knob to change the resistance, e.g. volume controls. Explained in section 4.8 and Chapter 5.
- **Trimmer**: a tiny pot adjusted with a screwdriver, set once during calibration.
- **LDR (light-dependent resistor)**: resistance falls as light increases (Chapter 17).
- **Thermistor (NTC/PTC)**: resistance changes with temperature (Chapter 17).
- **Varistor, fuse-resistor and others** exist, but we do not need them yet.

4.2 The specifications that matter

Resistance value

Measured in ohms (Ω), kilohms (k Ω) and megohms (M Ω): a 4.7 k Ω resistor is 4700 Ω . The value printed on a resistor is its **nominal** value. The actual value differs slightly within the tolerance.

Tolerance

Tolerance is how far the true value may be from the marked value, in percent. A 1 k Ω , $\pm 5\%$ resistor can be anywhere between 950 Ω and 1050 Ω . For most jobs (LED resistors, pull-ups) this hardly matters. For precision circuits (voltage dividers for accurate measurement), use $\pm 1\%$.

Power rating

The maximum power the resistor can turn into heat without damage, as we calculated in Chapter 2. Kit resistors are $\frac{1}{4}$ W (0.25 W). Keep the actual dissipation below half the rating for a safety margin.

Preferred values: the E-series

Why do resistors come in odd values like 47, 68, 82 rather than 50, 60, 70, 80? Because of **tolerance**. If parts are $\pm 10\%$, then a 100 Ω part covers 90–110 Ω . The next value that just touches the top of that range is 120 Ω (108–132 Ω)... and so on. Choosing values spaced by a constant **ratio** means the whole range from 10 to 100 is covered without gaps or big overlaps. These sets are called **E-series**, named by the number of values per decade: **E6**, **E12**, **E24**, **E48**, **E96**, **E192**. E12 ($\pm 10\%$) and E24 ($\pm 5\%$) are the ones you will use.

Series	Tolerance	Values per decade	The values (multiply by 1, 10, 100, 1k, 10k, 100k, 1M)
E12	$\pm 10\%$	12	10, 12, 15, 18, 22, 27, 33, 39, 47, 56, 68, 82
E24	$\pm 5\%$	24	10, 11, 12, 13, 15, 16, 18, 20, 22, 24, 27, 30, 33, 36, 39, 43, 47, 51, 56, 62, 68, 75, 82, 91
E96	$\pm 1\%$	96	Finer steps (10, 10.2, 10.5, 10.7, 11.0 ...); metal film parts

TIP

If a calculation gives you 500 Ω , you can not buy a 500 Ω resistor from the E12 series. Choose the nearest standard value (470 Ω or 560 Ω) and check that the result is still acceptable. Circuit designers do this constantly. **Rounding up or down is a design decision**: for an LED resistor round **up** (less current, safer), for a pull-up resistor either is usually fine.

4.3 Reading the resistor colour code

Resistors are too small to print numbers on, so values are painted as coloured bands, readable from any angle. Learning this code takes an hour and saves you many hours of meter measuring. Here is the code:

Colour	Digit	Multiplier	Tolerance
 Black	0	×1	—
 Brown	1	×10	±1 %
 Red	2	×100	±2 %
 Orange	3	×1 k	—
 Yellow	4	×10 k	—
 Green	5	×100 k	±0.5 %
 Blue	6	×1 M	±0.25 %
 Violet	7	×10 M	±0.1 %
 Grey	8	×100 M	±0.05 %
 White	9	×1 G	—
 Gold	—	×0.1	±5 %
 Silver	—	×0.01	±10 %

Figure 4.1 The resistor colour code. Digits use black to white; gold and silver mark tolerance or small multipliers.

Most resistors have **four bands**. Precision (metal film) resistors often have **five bands**. Hold the resistor so that the **tolerance band (gold, silver, or a gap-separated band) is on the right**. Read from the left:

Bands	Band 1	Band 2	Band 3	Band 4	Band 5
4-band	1st digit	2nd digit	Multiplier (10^n)	Tolerance	—
5-band	1st digit	2nd digit	3rd digit	Multiplier (10^n)	Tolerance



brown · black · red · gold
 $1\ 0 \times 100 \pm 5\% = 1\ \text{k}\Omega$



yellow · violet · orange · gold
 $4\ 7 \times 1000 \pm 5\% = 47\ \text{k}\Omega$



brown · black · black · red · brown
 $1\ 0\ 0 \times 100 \pm 1\% = 10\ \text{k}\Omega$

The last band (tolerance) is set slightly apart from the others.

Figure 4.2 Worked colour-code examples. The first two are 4-band, the last is a 5-band (1 % metal film) resistor.

WORKED EXAMPLE | DECODING A RESISTOR

Bands: **orange, orange, brown, gold**.

Orange = 3, orange = 3 → digits “33”. Brown = $\times 10 \rightarrow 330 \Omega$. Gold = $\pm 5 \%$.

Answer: **$330 \Omega \pm 5 \%$** (between 313.5Ω and 346.5Ω).

WORKED EXAMPLE | CHOOSING THE COLOURS FOR A VALUE

You need a **2.2 k Ω , 5 %** resistor.

2.2 k Ω = 2200Ω = 22×100 , so digits “2”, “2” and multiplier $\times 100$.

Colours: **red, red, red, gold**.

TIP

Memory aid. Many people use a rhyme for the order: **Bad Boys Race Our Young Girls But Violet Generally Wins** (Black, Brown, Red, Orange, Yellow, Green, Blue, Violet, Grey, White). Or just remember that the rainbow order (red, orange, yellow, green, blue, violet) is 2, 3, 4, 5, 6, 7, with black-brown at the start (0, 1) and grey-white at the end (8, 9).

Confused about direction? The gold or silver band is always last. If your resistor has no gold/silver, the tolerance band is the one set slightly apart from the rest. When in doubt, **measure it with the multimeter**.

Surface-mount resistor codes

Tiny SMD resistors print numbers instead of bands. Three digits: the first two are digits, the third is the number of zeros: **103** = 10 followed by 3 zeros = $10,000 \Omega$ = 10 k Ω ; **471** = 470Ω ; **222** = 2.2 k Ω . Four-digit codes (1 % parts) use three digits plus zeros: **1002** = 10.0 k Ω . An “R” marks a decimal point: **4R7** = 4.7Ω . **000** is a zero-ohm link (a jumper). Those on your Arduino board are of this kind.

4.4 Resistors in series

Two components are **in series** when they are connected end to end in a single chain, so that **the same current must pass through both**. There is no branch between them where current could escape.

$$R_{\text{total}} = R_1 + R_2 + R_3 + \dots$$

In series, resistances simply add.

ANALOGY | PIPES IN A ROW

Water flows through one pipe, then a second narrower pipe, then a third. All the water must pass through each section, so the flow is the same everywhere. Each narrow section adds resistance to the flow, and the total resistance is the sum: making the path longer never helps.

Think of queuing for airport check-in, then security, then passport control. The total waiting time is the sum of the three waits.

Three consequences of series connection:

1. **Same current** in every part: I is the same through R_1 , R_2 and R_3 .
2. **Voltages add**: the supply voltage is shared among the resistors, $V_{\text{source}} = V_1 + V_2 + V_3$. The bigger the resistor, the bigger its share ($V = I R$ with the same I).

3. **Total resistance is bigger** than any single resistor.

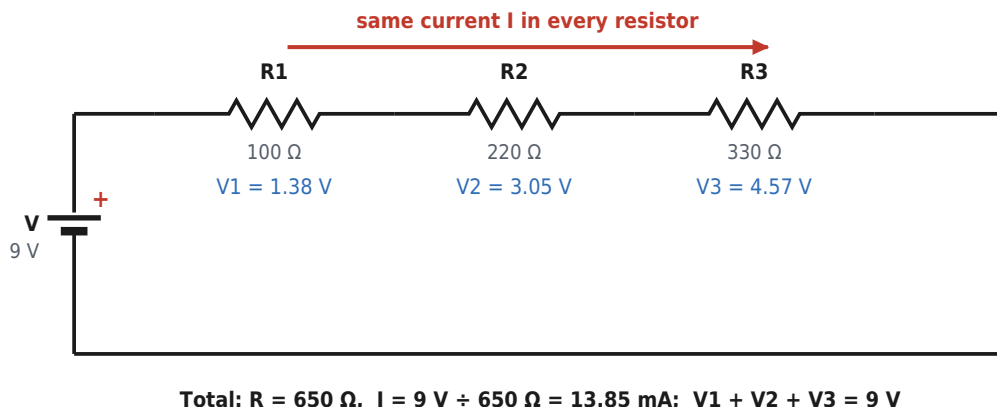


Figure 4.3 Three resistors in series across a 9 V battery. The battery voltage divides in proportion to the resistances.

WORKED EXAMPLE | SERIES CALCULATION

$R_1 = 100\ \Omega$, $R_2 = 220\ \Omega$, $R_3 = 330\ \Omega$ in series across a 9 V battery (Figure 4.3).

Total resistance: $100 + 220 + 330 = 650\ \Omega$.

Current: $I = V \div R = 9 \div 650 = 0.01385\ \text{A} = 13.85\ \text{mA}$ (the same everywhere).

Voltage across each: $V_1 = 0.01385 \times 100 = 1.385\ \text{V}$; $V_2 = 0.01385 \times 220 = 3.046\ \text{V}$; $V_3 = 0.01385 \times 330 = 4.569\ \text{V}$.

Check (voltages add up to the supply): $1.385 + 3.046 + 4.569 = 9.000\ \text{V} \checkmark$.

Power in each: $P_1 = I^2 R_1 = 0.0192\ \text{W}$, $P_2 = 0.0422\ \text{W}$, $P_3 = 0.0633\ \text{W}$; total = $0.1246\ \text{W} = V \times I = 9 \times 0.01385 \checkmark$.

REAL-LIFE EXAMPLE | OLD-FASHIONED STRING LIGHTS

Traditional Christmas lights and some cheap LED strips are wired in series. The mains voltage is shared between all the bulbs; each gets a small share. The weakness is that if **one bulb fails and breaks the chain, all go out**, just like breaking one link in a chain. Modern lights add a shunt that bypasses a failed bulb, or wire bulbs in parallel.

REAL-LIFE EXAMPLE | BATTERIES IN SERIES

Two 1.5 V AA cells in series give 3 V; four give 6 V. Voltages add, just like resistances do. That is why a TV remote uses two cells in a row: 3 V is what its chip needs. The same current flows through both cells, and the pair lasts as long as one cell would.

4.5 Resistors in parallel

Two components are **in parallel** when both of their ends are joined together, so that **the same voltage appears across each**. Current has a choice of routes: it splits between the branches and rejoins afterwards.

$$1 / R_{\text{total}} = 1 / R_1 + 1 / R_2 + 1 / R_3 + \dots$$

In parallel, the reciprocals add.

For two resistors there is a handy shortcut (“product over sum”), and for identical resistors an even easier one:

$$R_{\text{total}} = (R_1 \times R_2) \div (R_1 + R_2)$$

Two resistors in parallel: product over sum

$$R_{\text{total}} = R \div n$$

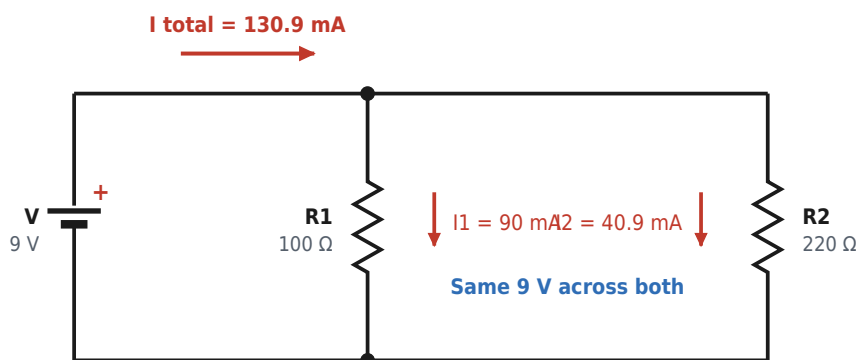
n identical resistors in parallel

ANALOGY | MULTIPLE PIPES SIDE BY SIDE

Imagine three pipes side by side between two water tanks. Each extra pipe gives the water another path, so **more total water flows** for the same pressure. The overall resistance goes **down**, not up. Or think of supermarket checkouts: opening more lanes reduces the total queuing resistance, and each lane serves shoppers independently.

Consequences of parallel connection:

1. **Same voltage** across every branch: V is the same across R_1 , R_2 , R_3 .
2. **Currents add:** $I_{\text{total}} = I_1 + I_2 + I_3$. Smaller resistances take bigger currents ($I = V / R$).
3. **Total resistance is smaller** than the smallest single resistor.



$$R = 100 \times 220 \div 320 = 68.75 \Omega \text{ (less than the smaller resistor)}$$

Figure 4.4 Two resistors in parallel across a 9 V battery. The current divides between the branches; the voltage is the same across each.

WORKED EXAMPLE | PARALLEL CALCULATION

$R_1 = 100 \Omega$ and $R_2 = 220 \Omega$ in parallel across 9 V (Figure 4.4).

Total resistance: $R = (100 \times 220) \div (100 + 220) = 22000 \div 320 = 68.75 \Omega$.

Branch currents: $I_1 = 9 \div 100 = 90 \text{ mA}$; $I_2 = 9 \div 220 = 40.9 \text{ mA}$.

Total current: $90 + 40.9 = 130.9 \text{ mA}$. Check: $9 \div 68.75 = 130.9 \text{ mA}$ ✓.

WORKED EXAMPLE | THREE RESISTORS IN PARALLEL

1 k Ω , 1 k Ω and 500 Ω in parallel.

$$1/R = 1/1000 + 1/1000 + 1/500 = 0.001 + 0.001 + 0.002 = 0.004 \text{ S.}$$

$$R = 1 \div 0.004 = \mathbf{250 \Omega}.$$

Sanity check: less than the smallest (500 Ω). ✓

REAL-LIFE EXAMPLE | HOUSEHOLD WIRING AND USB HUBS

Every socket and lamp in a house is connected **in parallel** across the mains, so each gets the full voltage and one can be switched off without affecting the others. Adding more appliances adds more current from the same wires. That is why overloading a socket with too many appliances (each drawing several amps) can overheat the cable: the total current is the sum. A fuse or circuit breaker protects against that.

COMMON MISTAKE

Adding resistances in parallel. Beginners often add parallel resistors like series ones, getting a value larger than either. Always check: parallel total must be **smaller than the smallest branch**; series total **larger than the largest**.

4.6 Combining series and parallel

Real circuits are usually a mix. The method is to **reduce step by step**: find a group that is clearly in series or clearly in parallel, replace it with its single equivalent resistor, and repeat until only one resistor remains. Then work backwards to find each voltage and current.

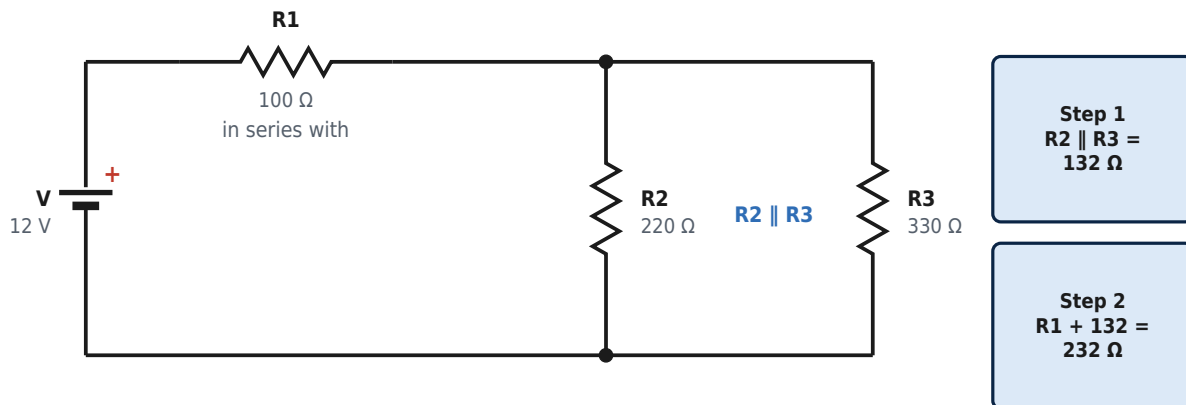


Figure 4.5 A series-parallel network. R2 and R3 are in parallel with each other; that pair is in series with R1.

WORKED EXAMPLE | SOLVING A SERIES-PARALLEL CIRCUIT

12 V supply, $R_1 = 100 \Omega$ in series with $R_2 = 220 \Omega \parallel R_3 = 330 \Omega$ (Figure 4.5). Find every voltage and current.

Step 1: parallel pair. $R_{23} = (220 \times 330) \div (220 + 330) = 72600 \div 550 = \mathbf{132 \Omega}$.

Step 2: series. $R_{\text{total}} = 100 + 132 = \mathbf{232 \Omega}$.

Step 3: total current. $I = 12 \div 232 = \mathbf{51.7 \text{ mA}}$. This all passes through R_1 .

Step 4: voltage across R_1 . $V_1 = 0.0517 \times 100 = \mathbf{5.17\text{ V}}$. The remaining voltage falls across the parallel pair: $12 - 5.17 = \mathbf{6.83\text{ V}}$ (check: $0.0517 \times 132 = 6.83\text{ ✓}$).

Step 5: branch currents. $I_2 = 6.83 \div 220 = \mathbf{31.0\text{ mA}}$; $I_3 = 6.83 \div 330 = \mathbf{20.7\text{ mA}}$. Sum = 51.7 mA ✓ .

4.7 Making the value you need

You can build almost any value from standard parts. If you need a value between two standards, or you have run out of one, combine resistors:

- **In series:** $100\ \Omega + 47\ \Omega = 147\ \Omega$ (close to $150\ \Omega$). Series adds; you can only make a larger value.
- **In parallel:** two $300\ \Omega$ resistors give $150\ \Omega$ exactly; two equal resistors always give half. Parallel makes a smaller value.
- **Trim for precision:** $1\text{ k}\Omega$ in parallel with $10\text{ k}\Omega$ gives $909\ \Omega$ (a 10 % reduction), useful for fine-tuning.
- **Extra power capacity:** Two $100\ \Omega\ \frac{1}{4}\text{ W}$ resistors in series make $200\ \Omega$, $\frac{1}{2}\text{ W}$. The power is shared. Two in parallel make $50\ \Omega$, $\frac{1}{2}\text{ W}$. Sharing works both ways.

4.8 The potentiometer

A potentiometer is a resistor with a third, movable contact called the **wiper**. The fixed resistive track between terminals 1 and 3 has constant resistance (for example $10\text{ k}\Omega$). Turning the knob moves the wiper along the track: the resistance between the wiper (terminal 2) and either end changes, always adding up to the total. Two ways to use it:

- **As a variable resistor (rheostat):** use the wiper and one end. Turning the knob changes R from 0 to the full value.
- **As a variable voltage divider:** connect the two ends across a voltage and take the wiper as the output; it delivers any voltage between 0 and the supply. This is by far the most common use (volume knob, Arduino analog input). Chapter 5 explains why.

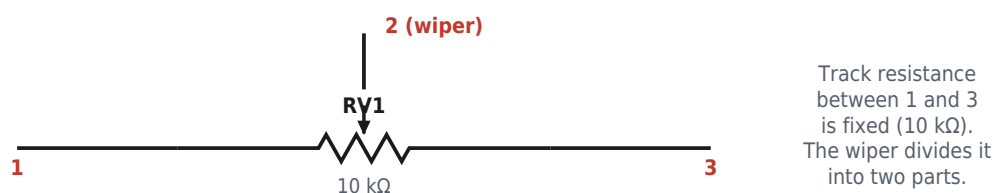


Figure 4.6 Potentiometer symbol: a resistor with an arrow (the wiper) pointing at the track.

HANDS-ON EXPERIMENT | EXPERIMENT 2: SERIES AND PARALLEL WITH A MULTIMETER

You need: three resistors (for example $1\text{ k}\Omega$, $2.2\text{ k}\Omega$, $4.7\text{ k}\Omega$), multimeter, breadboard.

1. Decode each resistor's colours and write the nominal value. Then measure it with the meter in Ω mode. Compute the percentage difference.
2. **Series:** put the three resistors in a chain on the breadboard (each junction in its own column). Measure across the whole chain. Compare with $R1 + R2 + R3$.
3. **Parallel:** connect all three resistors between two adjacent rails or two columns. Measure the combined resistance. Compare with the parallel formula. (Prediction for 1 k , 2.2 k , 4.7 k : $1/R = 0.001 + 0.000455 + 0.000213 = 0.001668$, so $R \approx 600\ \Omega$.)

4. Take the series circuit from step 2, connect a 9 V battery across the chain and **measure the voltage across each resistor**. Do the voltages add up to the battery voltage? Are they in proportion to the resistances?
5. Try one “mixed” circuit: 1 k Ω in series with the parallel pair (2.2 k Ω || 4.7 k Ω). Predict, then measure.

CHAPTER SUMMARY

- A resistor limits current, divides voltage and sets timing. Key specs: **value, tolerance, power rating**. Values come from the E12/E24 series.
- **Colour code:** digits 0–9 = black, brown, red, orange, yellow, green, blue, violet, grey, white. 4-band: digit, digit, multiplier, tolerance; 5-band: digit, digit, digit, multiplier, tolerance.
- **Series:** same current; voltages add; $R_{\text{total}} = R_1 + R_2 + \dots$
- **Parallel:** same voltage; currents add; $1/R_{\text{total}} = 1/R_1 + 1/R_2 + \dots$; two resistors: $(R_1 R_2) / (R_1 + R_2)$; n equal: R / n .
- Reduce mixed networks step by step, then work back to find every voltage and current. Always sanity-check: series total > largest; parallel total < smallest.

ASSIGNMENTS | CHAPTER 4

4.1 [Practice] Decode these 4-band resistors: (a) red-red-brown-gold, (b) orange-orange-yellow-gold, (c) brown-black-orange-gold, (d) green-blue-red-gold, (e) blue-grey-brown-gold.

4.2 [Practice] Decode the 5-band resistor yellow-violet-black-red-brown.

4.3 [Practice] Give the 4-band colours (with a gold tolerance band) for: 4.7 k Ω , 100 k Ω , 1 M Ω , 330 Ω , 2.2 k Ω .

4.4 [Practice] What do these SMD codes mean: 103, 471, 4R7, 1002?

4.5 [Calculate] A 1 k Ω resistor has a tolerance of 5 %. What is the range of values it might actually have?

4.6 [Calculate] Find the total resistance of 47 Ω , 100 Ω and 220 Ω in series, and the current when connected to a 5 V supply.

4.7 [Calculate] Find the voltage across each resistor of the previous question.

4.8 [Calculate] Find the equivalent of 330 Ω in parallel with 330 Ω , and of 1 k Ω in parallel with 4.7 k Ω .

4.9 [Calculate] Three 1 k Ω resistors are wired in parallel across a 5 V supply. Find the total resistance, total current and the current in each branch.

4.10 [Calculate] A 12 V battery supplies $R_1 = 220 \Omega$ in series with ($R_2 = 470 \Omega$ || $R_3 = 470 \Omega$). Find total resistance, total current, and the voltage across the parallel pair.

4.11 [Design] You need 150 Ω but only have 100 Ω , 47 Ω , 330 Ω and 300 Ω resistors. Give two different ways to make it (approximately or exactly).

4.12 [Design] You need a resistor of about 1.5 k Ω and only have 1 k Ω and 470 Ω parts in $\frac{1}{4}$ W. What combination gives roughly 1.5 k Ω ?

4.13 [Think] A string of ten identical bulbs in series stops working when one bulb burns out, but the bulbs in your house continue to work when one fails. Explain the difference in terms of series/parallel.

4.14 [Hands-on] Carry out Experiment 2 and produce a table of nominal, measured, predicted and measured-combination values. Comment on the differences.

Kirchhoff's laws and voltage dividers

Series and parallel rules are enough for simple networks, but some circuits cannot be reduced that way. Two laws found by Gustav Kirchhoff in 1845 work on every circuit, however tangled. They are really just statements that charge and energy are conserved. Once you know them, you also understand the voltage divider, which you will use in almost every Arduino project that reads a sensor.

WHAT YOU WILL LEARN IN THIS CHAPTER

- **Kirchhoff's current law (KCL):** what flows into a junction flows out
- **Kirchhoff's voltage law (KVL):** the voltages around any loop add to zero
- The **voltage divider**: the most important two-resistor circuit in electronics
- How a **load** changes a divider's output, and why real batteries have internal resistance
- The **current divider**, and a first look at Thevenin's idea of simplifying circuits

5.1 Kirchhoff's current law (KCL)

A **node** (or junction) is a point where two or more wires meet. Charge cannot be created or destroyed, and it cannot pile up at a junction, so whatever current flows **into** a node must flow **out** of it.

$$\Sigma I_{\text{in}} = \Sigma I_{\text{out}}$$

The sum of currents entering a node equals the sum of currents leaving it.

ANALOGY | PIPE JUNCTION

A water pipe splits into three at a T-junction. If 10 litres per minute arrive, then $4 + 6 = 10$ litres per minute must leave in the two branches. Not a drop is lost or gained at the joint (a leaking joint would be a fault, just as a stray current path would be in a circuit).



$$\text{KCL at the node: } 10 \text{ mA} = 4 \text{ mA} + I_3 \rightarrow I_3 = 6 \text{ mA}$$

Figure 5.1 KCL at a junction. Currents in equal currents out.

A useful sign convention: count currents **entering** the node as positive and **leaving** as negative, then the algebraic sum is zero. KCL also explains the parallel-resistor result you used in Chapter 4: the total current equals the sum of the branch currents.

5.2 Kirchhoff's voltage law (KVL)

Walk once around any closed loop in a circuit, adding up the voltage rises (across sources) and the voltage drops (across resistors and other loads). You must arrive back where you started at the same voltage, so the total change is zero.

$$\Sigma V_{\text{around a loop}} = 0$$

Voltage rises = voltage drops in every closed loop.

ANALOGY | A ROUND-TRIP HIKE

Hike around a circular mountain trail that returns to your starting point. You go up and down many times, but when you finish you are back at the same height as at the start: total climb = total descent. A battery is a chair-lift that lifts you a fixed height; each resistor is a slope where you descend.

A recipe for using KVL:

1. Choose a loop and a direction (clockwise is customary).
2. Assign a **current direction** in each branch (a guess is fine; if it comes out negative, the real current runs the other way).
3. Going around: a source traversed from $-$ to $+$ is a **rise** ($+V$). A resistor traversed **with** the current is a **drop** ($-IR$); against the current, a rise ($+IR$).
4. Write $\Sigma V = 0$ and solve.

WORKED EXAMPLE | KVL IN A SINGLE LOOP

A 12 V battery drives $R_1 = 100 \Omega$ and $R_2 = 220 \Omega$ in series (Figure 5.2). Find I .

Around the loop: $+12 - I \times 100 - I \times 220 = 0 \rightarrow 12 = I \times 320 \rightarrow I = 12 \div 320 = \mathbf{37.5 \text{ mA}}$.

Voltage drops: $V_1 = 3.75 \text{ V}$, $V_2 = 8.25 \text{ V}$; and $3.75 + 8.25 = 12 \checkmark$. This is the same result as the series rule, which is just KVL in disguise.

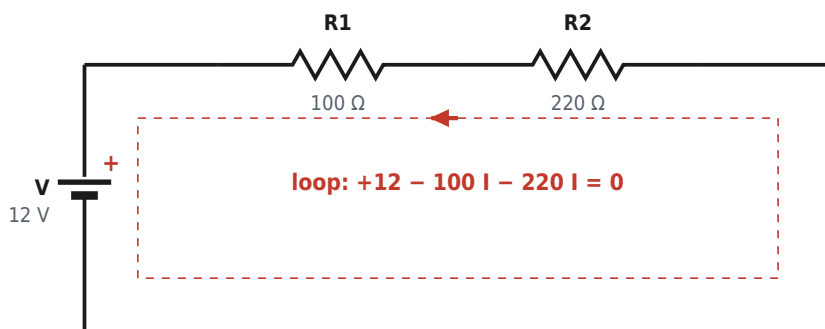
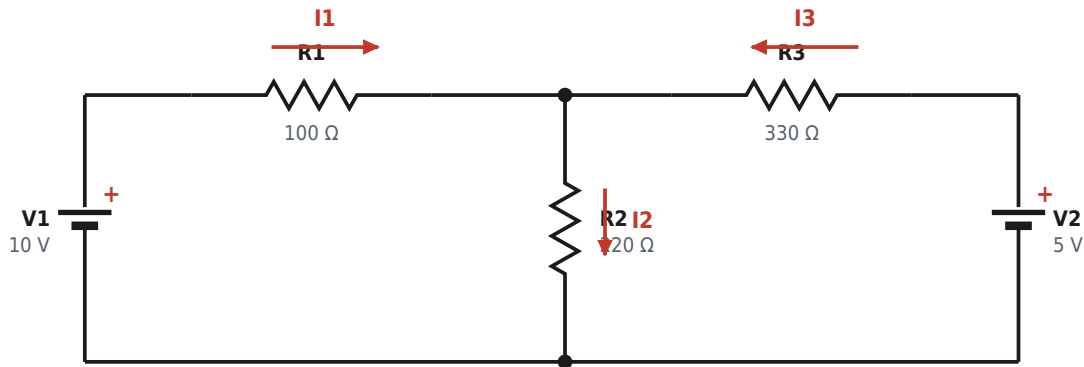


Figure 5.2 One loop, one current. KVL gives the same answer as adding the series resistances.

A circuit that series/parallel cannot solve

Figure 5.3 has two batteries and three resistors. No pair of resistors is purely in series or in parallel, so the reduction method of Chapter 4 fails. KCL and KVL solve it in three steps: choose unknown currents, write one KCL and two KVL equations, solve.



Unknowns: I_1 , I_2 , I_3 (arrows show assumed directions)

Figure 5.3 A two-source network solved with Kirchhoff's laws.

WORKED EXAMPLE | SOLVING FIGURE 5.3

KCL at the top node: $I_1 + I_3 = I_2$ (currents into the node from R_1 and R_3 both leave through R_2).

Left loop (V_1 , R_1 , R_2): $10 - 100 I_1 - 220 I_2 = 0$.

Right loop (V_2 , R_3 , R_2): $5 - 330 I_3 - 220 I_2 = 0$. (V_2 's + terminal is on the R_3 side, so it pushes current toward R_2 too.)

Substitute $I_2 = I_1 + I_3$: (a) $100 I_1 + 220 (I_1 + I_3) = 10 \rightarrow 320 I_1 + 220 I_3 = 10$; (b) $330 I_3 + 220 (I_1 + I_3) = 5 \rightarrow 220 I_1 + 550 I_3 = 5$.

From (a): $I_1 = (10 - 220 I_3) \div 320$. Insert in (b): $220 \times (10 - 220 I_3) \div 320 + 550 I_3 = 5 \rightarrow 6.875 - 151.25 I_3 + 550 I_3 = 5 \rightarrow 398.75 I_3 = -1.875 \rightarrow I_3 = -4.70 \text{ mA}$.

The negative sign means I_3 actually flows the other way (V_1 overpowers V_2 , pushing current back into it). Then $I_1 = (10 + 220 \times 0.0047) \div 320 = 34.5 \text{ mA}$ and $I_2 = 34.5 - 4.7 = 29.8 \text{ mA}$.

Check with the right loop: $5 - 330 \times (-0.0047) - 220 \times 0.0298 = 5 + 1.55 - 6.56 \approx 0 \checkmark$.

GOOD TO KNOW

You will rarely need to solve such systems by hand once you start using Arduino, but the skill of writing KCL/KVL equations is what lets you understand any circuit diagram. Programs like the free simulators (Falstad's circuit simulator, Tinkercad Circuits, LTspice) do this arithmetic for you: use them to check your answers.

5.3 The voltage divider

Two resistors in series across a voltage source form a **voltage divider**. The current through both is the same, $I = V_{in} \div (R_1 + R_2)$. The output voltage is taken across the lower resistor R_2 :

$$V_{out} = V_{in} \times R_2 \div (R_1 + R_2)$$

The output is the input scaled by the fraction $R_2 / (R_1 + R_2)$.

Derivation, in three lines: $I = V_{in} \div (R_1 + R_2)$; $V_{out} = I \times R_2$; therefore $V_{out} = V_{in} \times R_2 \div (R_1 + R_2)$. The output is always **less than the input**, and it is a **fraction** set purely by the ratio of the resistors.

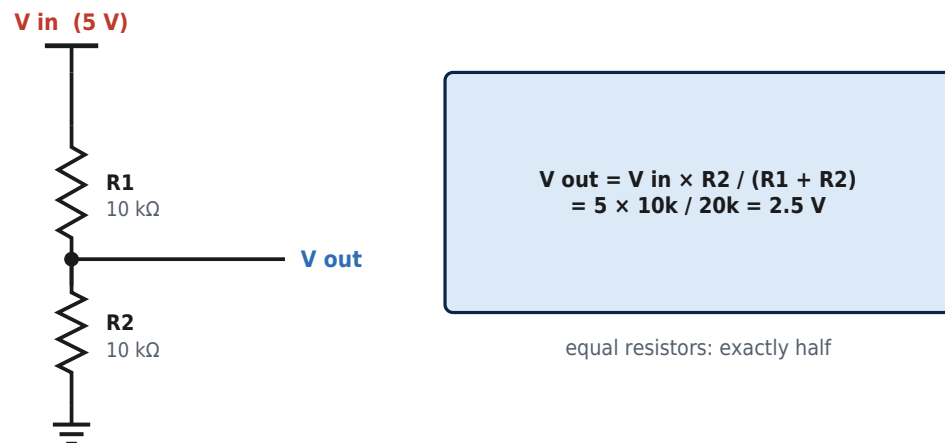


Figure 5.4 The voltage divider. The output is the fraction $R_2/(R_1+R_2)$ of the input.

WORKED EXAMPLE | DESIGNING A DIVIDER

A sensor puts out up to 12 V, but your Arduino input tolerates only 5 V. You need a divider that scales 12 V down to at most 4.8 V.

Ratio needed: $V_{out}/V_{in} = 4.8 / 12 = 0.4 = R_2 \div (R_1 + R_2)$.

So $R_2 = 0.4 (R_1 + R_2) \rightarrow 0.6 R_2 = 0.4 R_1 \rightarrow R_2 = R_1 \times 0.667$. Choose $R_1 = 15 \text{ k}\Omega$ and $R_2 = 10 \text{ k}\Omega$ (ratio 0.4 exactly).

Check: $V_{out} = 12 \times 10 \div 25 = 4.8 \text{ V} \checkmark$. **Current drawn:** $12 \div 25 \text{ k}\Omega = 0.48 \text{ mA}$, tiny, so the divider wastes little energy.

Practical rule: choose the absolute values so that the divider current is small (to save power) but much larger than the current the load draws (to keep the output steady). 1 k Ω to 100 k Ω is the usual range.

REAL-LIFE EXAMPLE | EVERYDAY DIVIDERS

A volume control is a potentiometer wired as a divider: the wiper picks a fraction of the signal voltage.

A light sensor (LDR) in series with a fixed resistor is a divider whose ratio changes with the light level (Chapter 17).

A battery indicator divides the battery voltage to a level a microcontroller can read.

Level shifting for 5 V to 3.3 V signals often uses two resistors (Chapter 22).

The potentiometer as a variable divider

Connect a potentiometer's outer terminals to +5 V and GND. Its wiper output moves smoothly between 0 V and 5 V as you turn the knob, because the wiper is the junction between two variable resistances whose sum is always constant. At mid-position, the output is exactly half. This is the classic way to give an Arduino a knob (Chapter 17).

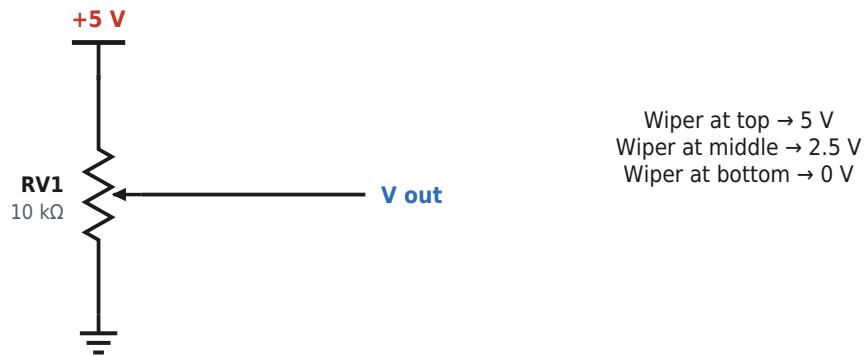


Figure 5.5 A potentiometer as an adjustable voltage divider. The wiper delivers any voltage between 0 V and the supply.

5.4 The loading problem

The divider formula assumes that **nothing is connected to V_{out}** (or that whatever is connected draws negligible current). When you connect a load resistor R_L to the output, R_L is in **parallel with R_2** , lowering the effective lower resistance and pulling V_{out} down.

WORKED EXAMPLE | LOADED DIVIDER

$R_1 = R_2 = 10\text{ k}\Omega$ from 5 V; unloaded output = 2.5 V.

Load $R_L = 10\text{ k}\Omega$: $R_2 \parallel R_L = 5\text{ k}\Omega$. $V_{out} = 5 \times 5 \div (10 + 5) = \mathbf{1.67\text{ V}}$ (a 33 % drop!).

Load $R_L = 1\text{ M}\Omega$ (like a good input): $R_2 \parallel R_L = 9.9\text{ k}\Omega$. $V_{out} = 5 \times 9.9 \div 19.9 = \mathbf{2.49\text{ V}}$ (almost unaffected).

Load $R_L = 100\text{ }\Omega$ (like a small LED path): $R_2 \parallel R_L = 99\text{ }\Omega$. $V_{out} = 5 \times 99 \div 10099 = \mathbf{0.05\text{ V}}$. The divider is useless for driving it.

TIP

Rule of thumb: the load resistance should be at least 10 times larger than R_2 (the lower resistor of the divider) for less than about 10 % error. Arduino analog inputs have a very high input resistance ($\sim 100\text{ M}\Omega$ for the pin itself, though the ADC's sampling circuit prefers a source below about $10\text{ k}\Omega$), so a divider of $10\text{ k}\Omega$ or less is happy. But **never use a divider to power a device** such as a motor, an LED or a servo: the output collapses under load. Use it only for signals (tiny currents), or use a regulator (Chapter 13).

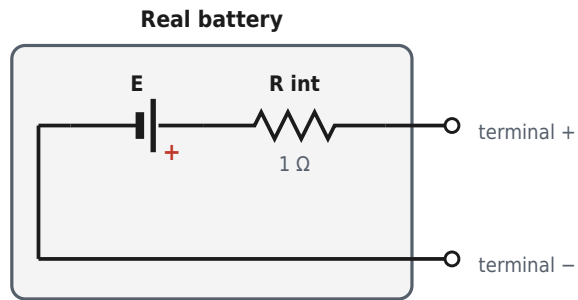
COMMON MISTAKE

Treating a voltage divider as a small power supply. The output voltage is only correct when almost no current is drawn from it.

5.5 Real sources are not perfect: internal resistance

So far we treated a battery as an ideal source that delivers exactly its rated voltage whatever current you draw. A real battery behaves as an ideal source **in series with a small internal resistance R_{int}** . When current flows, some voltage is lost across R_{int} ($V = I R$), so the terminal voltage sags under load.

$$V_{\text{terminal}} = V_{\text{open circuit}} - I \times R_{\text{int}}$$



Load draws 100 mA: loss = 0.1 V
Load draws 1 A: loss = 1 V

Old/cheap batteries: R_{int} is much larger (10 Ω or more)

Figure 5.6 A real battery = ideal source + internal resistance. Under load the terminal voltage drops.

WORKED EXAMPLE | A WEAK 9 V BATTERY

A used 9 V battery has an open-circuit voltage of 8.4 V and $R_{int} = 30 \Omega$. It is loaded by a 100 Ω motor.

$I = 8.4 \div (30 + 100) = 64.6 \text{ mA}$. The motor sees $V = 64.6 \text{ mA} \times 100 \Omega = \mathbf{6.46 \text{ V}}$, not 8.4 V; the missing 1.94 V dropped inside the battery.

This is why a “9 V” battery measured with a multimeter (very high resistance, so tiny current) shows 9 V but cannot run a motor. **Always measure a battery under load!**

REAL-LIFE EXAMPLE | WHY PHONES GET SLOWER WHEN THE BATTERY IS COLD OR OLD

Internal resistance of lithium cells rises as they age and get cold. Under a heavy load (camera, game), the terminal voltage sags and can dip below the phone’s minimum, causing an unexpected shutdown, even though the battery indicator showed 30 %. The same happens with a weak battery driving a servo on your Arduino: the sagging voltage resets the microcontroller.

5.6 The current divider

The parallel companion of the voltage divider. When current arrives at two parallel resistors, it divides in **inverse** proportion to the resistances (more current takes the easier path):

$$I_1 = I_{\text{total}} \times R_2 \div (R_1 + R_2)$$

The current in a branch is the total times the OTHER resistor over the sum.

WORKED EXAMPLE | SPLITTING A CURRENT

A 30 mA current enters $R_1 = 100 \Omega$ in parallel with $R_2 = 200 \Omega$.

$I_1 = 30 \times 200 \div 300 = \mathbf{20 \text{ mA}}$ (the lower resistor gets the larger share); $I_2 = 30 \times 100 \div 300 = \mathbf{10 \text{ mA}}$. Sum = 30 mA ✓.

Check via voltage: $20 \text{ mA} \times 100 \Omega = 2 \text{ V}$; $10 \text{ mA} \times 200 \Omega = 2 \text{ V}$. Same voltage across both, as parallel demands ✓.

REAL-LIFE EXAMPLE | AMMETER SHUNTS

A basic meter movement can only take a tiny current (say 50 μA). To measure large currents, a very small **shunt resistor** is placed in parallel; almost all the current goes through the shunt and the meter sees a known small fraction. The multimeter's 10 A range is exactly this.

5.7 Simplifying circuits: a first look at Thevenin's theorem

Any network of resistors and sources, however complicated, with two output terminals behaves as a **single voltage source V_{th} in series with a single resistance R_{th}** as seen from those terminals. This is **Thevenin's theorem**. It is extremely useful: you can replace a whole circuit by two numbers and immediately calculate what any load will see.

- V_{th} = the open-circuit output voltage (nothing connected).
- R_{th} = the resistance seen looking back into the terminals with the sources “turned off” (voltage sources replaced by wires).

WORKED EXAMPLE | THEVENIN EQUIVALENT OF A VOLTAGE DIVIDER

Divider: $V_{\text{in}} = 5\text{ V}$, $R_1 = R_2 = 10\text{ k}\Omega$.

V_{th} = open-circuit output = **2.5 V**.

$R_{\text{th}} = R_1 \parallel R_2$ (with the 5 V source shorted, both resistors go to ground) = **5 k Ω** .

So the divider is a 2.5 V source with 5 k Ω internal resistance. Any load R_L then gives $V = 2.5 \times R_L \div (5\text{ k} + R_L)$: for 10 k Ω , $V = 1.67\text{ V}$ ✓ (matches the loaded divider above).

This explains the loading problem elegantly: the divider is a **weak source** with high output resistance, exactly like a nearly flat battery.

TIP

Whenever you connect the output of one thing to the input of another (a sensor to an ADC, an Arduino pin to an LED), ask: **what is the output resistance of the first, and what is the input resistance of the second?** A good design has low output resistance driving high input resistance. This one rule prevents most signal problems.

5.8 Bringing it together: the Wheatstone bridge (optional)

Two voltage dividers side by side form a **Wheatstone bridge**. If both dividers have the same ratio, the two midpoints are at the same voltage and a meter between them reads zero: the bridge is “balanced”. When one resistor changes slightly (a strain gauge on a bridge, or a thermistor), the balance tips and a small voltage appears. This is how many precision sensors detect tiny changes. Kitchen scales use it.

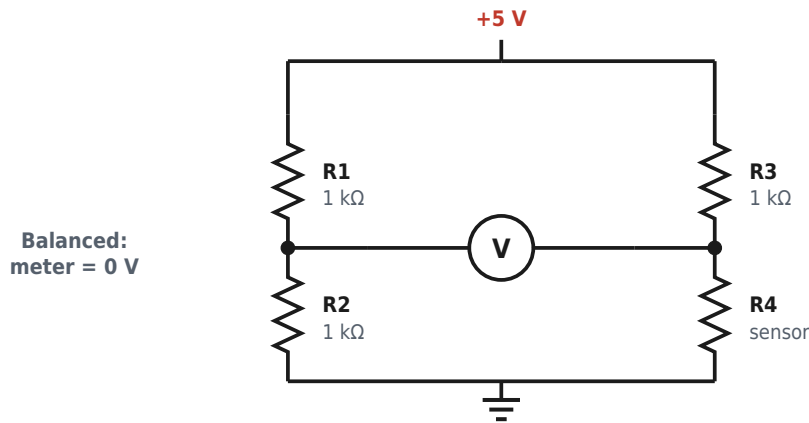


Figure 5.7 A Wheatstone bridge. The meter measures the difference between two dividers.

CHAPTER SUMMARY

- **KCL:** currents entering a node equal currents leaving it. **KVL:** the voltages around any closed loop sum to zero.
- Write unknown currents, one KCL and enough KVL equations, and solve; a negative answer means the real current is opposite to your assumed direction.
- **Voltage divider:** $V_{\text{out}} = V_{\text{in}} \times R_2 / (R_1 + R_2)$. A potentiometer is an adjustable divider.
- A divider only works for **small loads**: keep R_{load} at least $10 \times R_2$. Never power devices from a divider.
- Real batteries have **internal resistance**: the terminal voltage falls as more current is drawn. Test batteries under load.
- **Current divider:** the branch with the smaller resistance gets the bigger share: $I_1 = I \times R_2 / (R_1 + R_2)$.
- **Thevenin:** any two-terminal network equals a source V_{th} in series with R_{th} . Look for low source resistance driving high load resistance.

ASSIGNMENTS | CHAPTER 5

5.1 [Calculate] At a node, 8 mA flows in through one wire and 3 mA flows in through another. Two more wires carry current out, one of 5 mA. What is the current in the last wire?

5.2 [Calculate] A 9 V battery is connected across three resistors in series: 1 kΩ, 2 kΩ and 6 kΩ. Use KVL to find the current and the voltage across each resistor.

5.3 [Calculate] A divider uses $R_1 = 22 \text{ k}\Omega$ (top) and $R_2 = 10 \text{ k}\Omega$ (bottom) from 9 V. Find V_{out} .

5.4 [Calculate] You want to make 3.3 V from a 5 V source with a divider. Choose R_1 and R_2 from the E12 series and compute the resulting V_{out} .

5.5 [Calculate] A 10 kΩ potentiometer is wired between 5 V and GND. The wiper is 30 % of the way from the GND end. What is V_{out} (no load)? What if a 10 kΩ load is connected to the wiper?

5.6 [Calculate] A divider of 100 kΩ (top) and 100 kΩ (bottom) from 5 V is connected to a load of 100 kΩ. What is the loaded output voltage? What if the divider were made from 1 kΩ + 1 kΩ instead?

5.7 [Calculate] A battery has an open-circuit voltage of 6.0 V. When it drives a 20 Ω load, the terminal voltage measures 5.4 V. Find the internal resistance.

5.8 [Calculate] 60 mA arrives at a parallel pair of 47 Ω and 120 Ω. Find the current in each.

5.9 [Calculate] Find the Thevenin equivalent (V_{th} and R_{th}) of a 12 V source feeding a divider of $R_1 = 2 \text{ k}\Omega$ and $R_2 = 4 \text{ k}\Omega$.

ASSIGNMENTS | CHAPTER 5

5.10 [Calculate] Refer to Figure 5.3. Suppose V2 is removed and replaced with a wire (short circuit). Find the current through R2.

5.11 [Think] Why can't you power a 100 Ω motor from a voltage divider made of two 1 k Ω resistors? What would you use instead?

5.12 [Think] A student measures an almost-flat 9 V battery with a multimeter and gets 8.7 V, but the battery cannot light a small bulb. Explain.

5.13 [Hands-on] Build the divider $R1 = R2 = 10\text{ k}\Omega$ from 5 V (Arduino 5 V pin or a battery pack, 9 V is also fine if you predict 4.5 V). Measure V_{out} unloaded, then loaded with 10 k Ω , 1 k Ω , and 100 k Ω . Compare with the loaded-divider formula.

5.14 [Hands-on] Wire a 10 k Ω potentiometer as a divider on 5 V. With a multimeter on the wiper, rotate from one end to the other and record the voltage at five positions. Is the relationship linear?

PART II

Components

Resistors are only the beginning. Part II introduces the parts that store energy, control the direction of current, amplify signals and switch loads: capacitors, inductors, diodes, LEDs, transistors, relays, motors, sensors, op-amps, the 555 timer, logic gates and power supplies.

IN THIS PART

- 6 Capacitors
- 7 Inductors, magnetism and relays
- 8 Diodes and LEDs
- 9 Transistors
- 10 Switches, sensors, motors and actuators
- 11 Op-amps and the 555 timer
- 12 Digital electronics and logic
- 13 Batteries and power supplies

Capacitors

A resistor limits how fast charge flows. A capacitor does something different: it **stores** charge, like a tiny rechargeable battery that charges and discharges in a fraction of a second. Capacitors are found in every circuit board ever made. They smooth power supplies, remove noise, set timing and pass signals. Understanding them is the key to understanding why circuits behave differently over time.

WHAT YOU WILL LEARN IN THIS CHAPTER

- What a capacitor is, how it stores energy, and the unit **farad**
- How capacitors charge and discharge, and the **RC time constant**
- Series and parallel capacitors; energy stored
- How to choose between ceramic, electrolytic, film and other types, and read their markings
- The jobs capacitors do: decoupling, smoothing, filtering, timing and coupling

6.1 What is a capacitor?

A capacitor consists of two conducting plates separated by a thin insulator called the **dielectric** (air, ceramic, plastic film, oxide). No current can flow **through** the insulator. Yet when you connect a battery across a capacitor, current does flow briefly: electrons pile up on one plate (making it negative) and are drawn off the other plate (making it positive). Charge builds up until the capacitor's voltage equals the battery's, then current stops. The capacitor is now **charged**, and holds energy in the electric field between its plates.

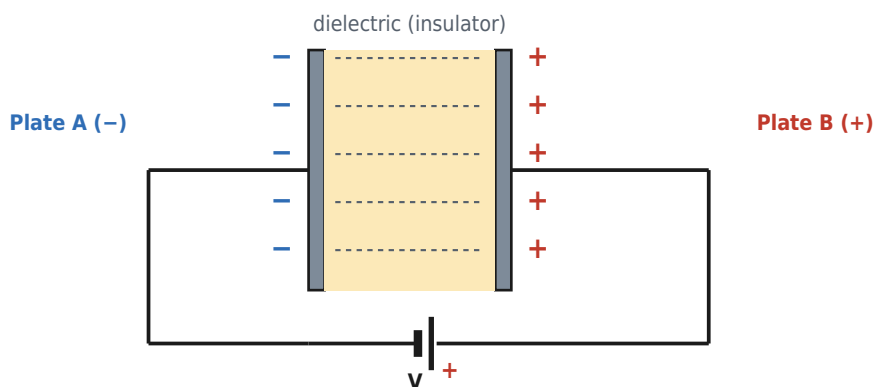


Figure 6.1 A capacitor: two plates with an insulator between them. Applying a voltage moves electrons from one plate to the other.

ANALOGY | THE RUBBER MEMBRANE IN A PIPE

Picture a **rubber membrane stretched across the middle of a water pipe**, blocking it. Water cannot pass through the membrane, but when you push water in from one side, the membrane stretches, and water flows briefly on both sides. Push hard enough for long enough and the membrane is stretched to its limit; then flow stops. Release the pressure and the stretched membrane pushes the water back, so the flow reverses.

The membrane is the dielectric; its stretchiness is the **capacitance**; the stretch is the stored charge. A big, floppy membrane (large capacitance) absorbs a lot of water before it becomes tight.

Where it stops working: real capacitors also have a maximum voltage; if the membrane over-stretches, it bursts (the dielectric breaks down).

6.2 Capacitance and its unit

Capacitance (C) tells you how much charge a capacitor stores per volt of voltage. It is measured in **farads (F)**, named after Michael Faraday:

$$Q = C \times V$$

Charge (coulombs) = Capacitance (farads) × Voltage (volts)

One farad is enormous: a capacitor that stores one coulomb at one volt. Practical capacitors are much smaller, so we use small prefixes:

Unit	Symbol	Value	Typical parts
millifarad	mF	10^{-3} F	Rare; big power-supply capacitors sometimes quoted as 4.7 mF
microfarad	μF (or uF)	10^{-6} F	Electrolytic capacitors for power smoothing: 1 μF – 10,000 μF
nanofarad	nF	10^{-9} F	Ceramic/film: 1 nF – 1000 nF (1 μF)
picofarad	pF	10^{-12} F	Small ceramic capacitors: oscillators, RF (10 pF – 1000 pF)
farad	F	1 F	“Supercapacitors”: 0.1 F – 3000 F, used as memory backup and energy storage

Conversions: **1 μF = 1000 nF = 1,000,000 pF**. So 100 nF = 0.1 μF and 4700 pF = 4.7 nF. What decides the capacitance? Three factors:

- **Plate area A:** bigger plates store more charge ($C \propto A$).
- **Distance d** between the plates: thinner insulator means a stronger effect ($C \propto 1/d$).
- **Dielectric material:** some insulators (ceramic, oxides) boost capacitance many times over air.

$$C = \epsilon \times A \div d$$

ε (epsilon) = permittivity of the dielectric

This explains construction tricks: to fit a large capacitance in a small package, manufacturers roll up long thin foils (film capacitors) or etch the aluminium surface to enlarge its area (electrolytics), and use very thin, high-permittivity dielectrics.

WORKED EXAMPLE | CHARGE STORED

A 1000 μF capacitor is charged to 9 V. How much charge does it hold?

$$Q = C V = 1000 \times 10^{-6} \text{ F} \times 9 \text{ V} = 9 \times 10^{-3} \text{ C} = \mathbf{9 \text{ mC}}.$$

For comparison, a 1000 mAh battery holds 3600 C; the capacitor stores roughly 0.25 % of that at a much lower energy per coulomb, but it delivers it in milliseconds rather than hours.

6.3 Energy stored in a capacitor

Charging a capacitor takes work; that energy is stored and can be returned. The stored energy is:

$$E = \frac{1}{2} \times C \times V^2$$

Energy (joules) = one half $\times$ capacitance $\times$ voltage squared

Note the V squared: doubling the voltage quadruples the energy. That is why camera flash units charge capacitors to hundreds of volts.

WORKED EXAMPLE | ENERGY COMPARISONS

Smoothing capacitor: 1000 μF at 9 V $\rightarrow E = 0.5 \times 0.001 \times 81 = \mathbf{0.0405 \text{ J}}$. Small.

Camera flash: 100 μF at 300 V $\rightarrow E = 0.5 \times 100 \times 10^{-6} \times 90000 = \mathbf{4.5 \text{ J}}$ released in about a millisecond, a power of several kilowatts.

A 3.7 V, 1000 mAh lithium cell: $3.7 \times 1 \text{ Ah} \times 3600 = 13,320 \text{ J}$. Batteries store thousands of times more energy than ordinary capacitors, but capacitors can release theirs much faster.

SAFETY WARNING

A charged capacitor can hold a dangerous energy even after the power is off. High-voltage capacitors (in camera flashes, microwave ovens, old CRT monitors and power supplies) can deliver a **lethal shock** days after unplugging. Never open these devices. On low-voltage circuits in this book (5 – 12 V), the energy is harmless, but a large capacitor can produce a brief spark and heat if shorted across, so discharge it through a resistor rather than a screwdriver.

6.4 Charging and discharging: the RC time constant

When you connect a capacitor to a supply through a resistor, it does not charge instantly. At first the capacitor is empty, has 0 V across it, so the full supply voltage appears across the resistor and a large current flows ($I = V / R$). As the capacitor fills, its voltage rises, the voltage across the resistor falls, the current shrinks, and the charging slows down. The voltage climbs quickly at first, then gradually levels off, in the shape of an **exponential curve**.

$$V_C(t) = V_S \times (1 - e^{-t / RC})$$

Charging: capacitor voltage after time t

$$V_C(t) = V_0 \times e^{-t / RC}$$

Discharging through a resistor

Here $e \approx 2.718$ is a mathematical constant; you need not manipulate it, only use the results. The quantity $\tau = R \times C$ (tau) is called the **time constant**. It has units of seconds when R is in ohms and C in farads ($10 \text{ k}\Omega \times 100 \mu\text{F} = 10^4 \times 10^{-4} = 1 \text{ s}$). After certain multiples of τ , the voltage reaches these fractions:

Time	Charging: % of final voltage	Discharging: % of starting voltage
1 τ	63.2 %	36.8 %
2 τ	86.5 %	13.5 %

Time	Charging: % of final voltage	Discharging: % of starting voltage
3τ	95.0 %	5.0 %
4τ	98.2 %	1.8 %
5τ	99.3 %	0.7 % (considered “fully charged/discharged”)

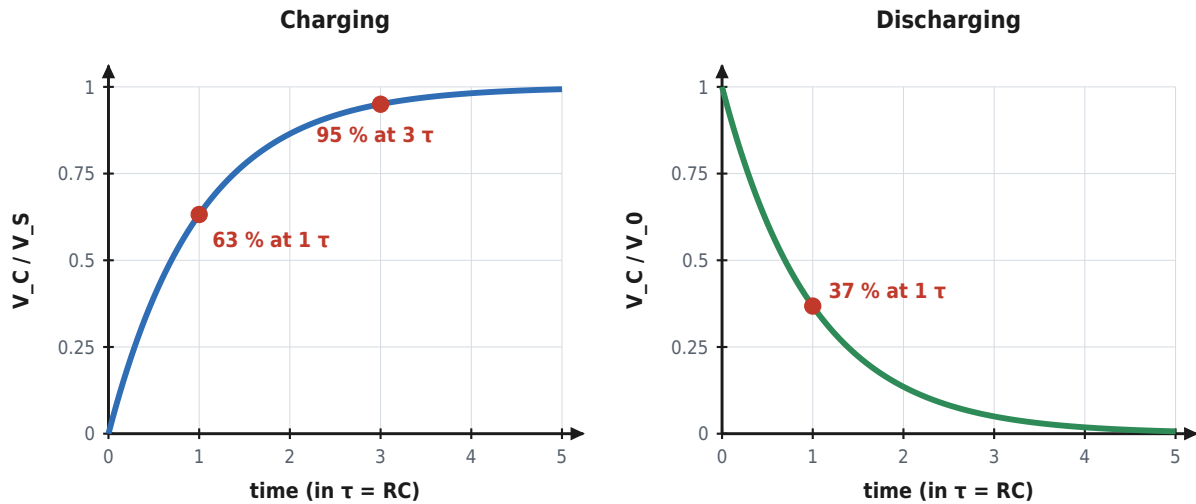


Figure 6.2 Charge and discharge curves. After one time constant a charging capacitor has reached 63 % of its final value.

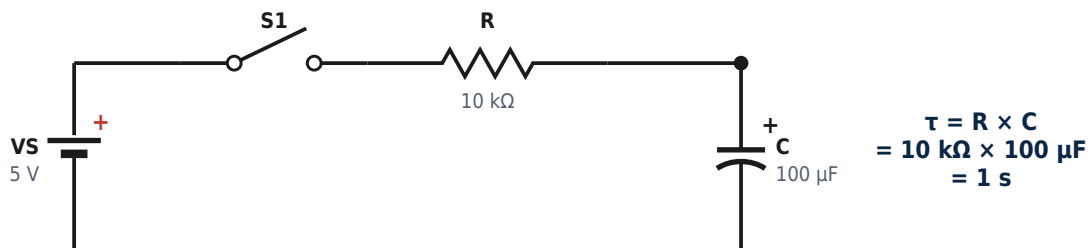


Figure 6.3 A basic RC charging circuit. Close S1 and the capacitor voltage rises to 5 V with time constant $\tau = 1 \text{ s}$.

WORKED EXAMPLE | TIME TO REACH A THRESHOLD

Using Figure 6.3, how long until the capacitor reaches 3.0 V (60 % of 5 V)?

Solve $3 = 5 (1 - e^{-t/\tau}) \rightarrow e^{-t/\tau} = 0.4 \rightarrow t/\tau = \ln(1/0.4) = 0.916 \rightarrow t = \mathbf{0.916 \text{ s}}$.

With a microcontroller pin as a threshold detector, this is exactly how simple timers work: choose R and C to set the delay.

ANALOGY | A BUCKET FILLED THROUGH A HOSE

Filling a bucket through a narrow hose (R). At the start the empty bucket accepts water quickly because the pressure difference is greatest. As it fills, the water level pushes back and the inflow slows. It takes ages to top up the last drop. A bigger bucket (C) or narrower hose (R) both make the filling take longer: $\tau = R \times C$.

REAL-LIFE EXAMPLE | THE CAMERA FLASH AND THE “PING” OF A KEYBOARD

A flash unit slowly charges its capacitor from a small battery through a resistor-like circuit (taking several seconds, which is the “charging” whine you hear). Pressing the shutter discharges it in a millisecond through the flash tube. Slow charge, fast discharge. The same idea powers the defibrillator.

6.5 Capacitors in series and parallel

Capacitors combine in the **opposite** way to resistors:

$$\text{Parallel: } C_{\text{total}} = C_1 + C_2 + C_3 + \dots$$

Plates add: more area, more capacitance.

$$\text{Series: } 1 / C_{\text{total}} = 1 / C_1 + 1 / C_2 + \dots$$

Plates separated further: less capacitance.

WORKED EXAMPLE | COMBINATIONS

Parallel: 100 μF and 47 μF $\rightarrow$ 147 μF . Use it to make up a value or spread current.

Series: 100 μF and 100 μF $\rightarrow$ 50 μF ; two capacitors in series can also share a higher voltage (two 16 V parts safely hold about 32 V if they are equal and have balancing resistors).

Series (unequal): 10 μF and 40 μF : $(10 \times 40) / (10 + 40) = 8 \mu\text{F}$.

COMMON MISTAKE

Do not mix up the rules: resistors in **series** add, capacitors in **parallel** add.

6.6 Capacitor types and how to choose

Type	Typical values	Polarised ?	Best for	Watch out for
Ceramic disc / MLCC	1 pF – 100 μF (10 nF – 100 nF typical)	No	Decoupling (100 nF), filters, timing, high frequencies	Value changes with voltage and temperature (cheap X7R/Y5V); tiny microphonic noise
Aluminium electrolytic	0.47 μF – 47,000 μF	YES	Bulk energy storage, power-supply smoothing	Wrong polarity = heats, vents, or explodes; tolerance –20/+80 %; ages and dries
Tantalum	0.1 μF – 1000 μF	YES	Compact, stable, low leakage	Can fail short and burn if reverse-connected or surged
Film (polyester, polypropylene)	1 nF – 10 μF	No	Precision timing, audio, filters, AC	Larger and more expensive
Supercapacitor	0.1 F – 3000 F	Yes	Memory backup, energy harvesting	Very low voltage rating (2.7 V per cell); big leakage

Polarity: the long lead and the stripe

Electrolytic and tantalum capacitors are **polarised**: one lead must always be more positive than the other. On an electrolytic can, a **stripe with minus signs** (–) marks the negative lead, which is also the **shorter lead**; the longer lead is positive. For tantalum, a stripe or bar marks the **positive** lead. Connecting them backwards damages them. Electrolytics can vent hot gas or burst with a bang, hence the safety note at the start of the book.

Voltage rating

Every capacitor has a maximum working voltage printed on it (6.3 V, 10 V, 16 V, 25 V, 50 V ...). **Never exceed it**. A safe rule is to choose a rating at least **1.5 to 2 times** the highest voltage the capacitor will see. For a 5 V rail, a 10 V or 16 V capacitor is ideal; for a 12 V rail, at least 25 V.

Reading capacitor codes

Small capacitors carry a three-digit code, the same as SMD resistors: two digits and a multiplier, in **picofarads**. Examples:

Marking	Meaning	Value
104	10 followed by 4 zeros pF = 100,000 pF	100 nF = 0.1 μ F (the most common one you will use!)
103	10,000 pF	10 nF
102	1000 pF	1 nF
471	470 pF	470 pF
22	22 pF	22 pF (used with crystals)
224	220,000 pF	220 nF

Large electrolytics simply print “470 μ F 25 V”. Letters after the number (J = ± 5 %, K = ± 10 %, M = ± 20 %) give tolerance, e.g. 104K = 100 nF ± 10 %.

6.7 What capacitors are used for

1. Decoupling (bypass) capacitors

When a chip switches, it demands a sudden gulp of current for a few nanoseconds. The wires from the power supply have small inductance and resistance, so they cannot respond that fast, and the chip’s supply voltage momentarily dips, creating noise and glitches. A small capacitor (usually **100 nF ceramic**) placed **right next to the chip’s power pins** acts as a local reservoir: it supplies the fast current spikes while the supply catches up. Every integrated circuit on every board has one. Your Arduino has several already.

REAL-LIFE EXAMPLE | THE ARDUINO BOARD

Look at the underside of your Arduino Uno near the USB connector and microcontroller: tiny 100 nF (marked 104) capacitors sit next to the chip pins. A 47 μ F (or larger) electrolytic and a 100 nF also sit on the supply rail. Remove or omit them and the board can behave erratically.

2. Smoothing (reservoir) capacitors

After a rectifier turns AC into pulsating DC, a large electrolytic capacitor charges to the peak voltage and then supplies current between the peaks, greatly reducing the ripple. The bigger the capacitor, the smaller the ripple. This is why a phone charger contains one or more electrolytic capacitors. (Chapter 13 treats this in detail.)

3. Filtering

A capacitor passes fast changes easily but blocks steady voltages; resistors do neither. Combined, they make **filters**. A **low-pass RC filter** lets slow signals through and cuts fast noise; its **cut-off frequency** is where the output has dropped to 70.7 % (−3 dB):

$$f_c = 1 / (2 \pi R C)$$

Cut-off frequency of an RC filter, in hertz

WORKED EXAMPLE | SENSOR NOISE FILTER

A sensor is plagued by 1 kHz-and-above noise but its signal changes only a few times a second. Use $R = 10 \text{ k}\Omega$ and $C = 1 \text{ }\mu\text{F}$.

$$f_c = 1 \div (2\pi \times 10,000 \times 0.000001) = 1 \div 0.0628 = \mathbf{15.9 \text{ Hz.}}$$

Signals slower than about 16 Hz pass; noise at 1 kHz is reduced by a factor of about 60. Placing this filter between a sensor and an Arduino analog input gives a much steadier reading.

4. Timing and delays

Because charging takes time $\tau = RC$, a resistor and capacitor make an adjustable delay: LED fade, power-on reset, button debounce, 555 timer oscillators (Chapter 11).

5. Coupling (blocking DC, passing AC)

In an audio circuit, a capacitor in series with the signal path passes the varying audio but blocks the steady DC bias. That is why the wire from your phone's audio jack to an amplifier has a coupling capacitor. The frequency response is determined by the same RC formula.

6. Energy storage and power ride-through

A large capacitor keeps a circuit alive for a moment during a power interruption; supercapacitors keep clocks running in some devices for days.

Capacitive reactance: a capacitor's "resistance" to AC

For a sinusoidal signal of frequency f , a capacitor's opposition to current is its **reactance** (measured in ohms):

$$X_C = 1 / (2 \pi f C)$$

High frequency or big $C \rightarrow$ low opposition. DC ($f = 0$) $\rightarrow$ infinite: it blocks.

WORKED EXAMPLE | REACTANCE EXAMPLES

100 nF at 1 kHz: $X_C = 1 \div (2\pi \times 1000 \times 10^{-7}) = \mathbf{1.59\text{ k}\Omega}$.

100 nF at 10 MHz: $X_C = \mathbf{0.16\text{ }\Omega}$, almost a short circuit. This is why a 100 nF capacitor across a chip's supply pins shorts out high-frequency noise.

100 μF at 50 Hz: $X_C = \mathbf{31.8\text{ }\Omega}$.

HANDS-ON EXPERIMENT | EXPERIMENT 3: CHARGE AND DISCHARGE A CAPACITOR AND WATCH IT WITH A MULTIMETER AND AN LED

You need: 9 V battery (or 5 V from your Arduino), breadboard, 1000 μF electrolytic capacitor ($\geq 16\text{ V}$), 10 k Ω resistor, a 470 Ω resistor, a red LED, multimeter.

1. **Slow charging.** Connect the 10 k Ω resistor in series with the capacitor (mind the polarity: stripe to ground!) across the supply. Time constant $\tau = 10\text{ k}\Omega \times 1000\text{ }\mu\text{F} = \mathbf{10\text{ s}}$. Set your meter to DC volts across the capacitor. Connect the supply, and note the voltage after 10 s, 20 s, 30 s, 50 s. Expected: 63 %, 86 %, 95 %, 99 % of the supply.

2. **Discharge.** Remove the supply and connect a 10 k Ω resistor across the capacitor. Note the voltage every 10 s. It should fall to 37 % after 10 s, 14 % after 20 s.

3. **The LED test.** Charge the capacitor **directly** from the supply through a 470 Ω resistor ($\tau = 0.47\text{ s}$). Disconnect the supply. Connect an LED with a 470 Ω resistor across the capacitor. The LED lights and fades away over a couple of seconds as the stored charge drains: the capacitor is a temporary battery.

4. **Change the values.** Try 100 μF instead: the times become ten times shorter. Predict then confirm.

5. **Series/parallel.** Put two 1000 μF in parallel (expect 2000 μF : twice as long a decay) and in series (500 μF : half as long).

CHAPTER SUMMARY

- A capacitor stores charge on two plates separated by an insulator: $Q = C V$. Unit: farad; typical values are pF, nF, μF .
- Stored energy is $E = \frac{1}{2} C V^2$; big capacitors can deliver a lot of energy quickly and can stay charged after power-off.
- Through a resistor, a capacitor charges/discharges exponentially with time constant $\tau = R C$: 63 % after 1 τ , 95 % after 3 τ , 99 % after 5 τ .
- Capacitors add in **parallel**, combine like resistors in series in **series**.
- Ceramic capacitors are non-polar and great for decoupling (100 nF = “104”); electrolytics are polarised and have big values; always respect polarity and voltage ratings.
- Uses: decoupling, smoothing, filtering ($f_c = 1 / 2\pi RC$), timing, coupling, energy storage. Reactance $X_C = 1 / 2\pi f C$ shrinks as frequency rises.

ASSIGNMENTS | CHAPTER 6

6.1 [Calculate] Convert: (a) 0.1 μF to nF, (b) 2200 nF to μF , (c) 470 pF to nF, (d) 4.7 μF to pF.

6.2 [Calculate] How much charge does a 470 μF capacitor hold at 12 V? How much energy?

6.3 [Calculate] Find the time constant of $R = 4.7\text{ k}\Omega$ and $C = 220\text{ }\mu\text{F}$. How long until the capacitor is essentially fully charged (5 τ)?

6.4 [Calculate] A 100 μF capacitor charges through 22 k Ω from 5 V. What is its voltage after 2.2 s? ($\tau = 2.2\text{ s}$)

6.5 [Calculate] Using the same circuit, how long does it take to reach 4 V? (Use $t = -\tau \ln(1 - V/V_s)$.)

ASSIGNMENTS | CHAPTER 6

6.6 [Calculate] Find the equivalent capacitance of 22 μF and 47 μF in (a) parallel and (b) series.

6.7 [Calculate] Design an RC low-pass filter with a cut-off of about 100 Hz. Choose $R = 10\text{ k}\Omega$ and find C . Choose the nearest standard capacitor value.

6.8 [Calculate] What is the reactance of a 100 nF capacitor at (a) 50 Hz, (b) 5 kHz, (c) 1 MHz?

6.9 [Practice] Decode: (a) 104, (b) 473, (c) 221, (d) a small disc capacitor marked “22”.

6.10 [Practice] A capacitor is marked “470 μF 10 V” and you plan to use it on a 9 V battery line. Is that a good choice? What would you choose?

6.11 [Think] Why do we place a 100 nF capacitor right next to the supply pins of a chip rather than at the power connector?

6.12 [Think] A capacitor is described as “blocking DC and passing AC”. Use the reactance formula to explain.

6.13 [Hands-on] Do Experiment 3. Plot the measured voltage against time on graph paper and compare with the predicted curve.

Inductors, magnetism and relays

A capacitor stores energy in an electric field. An inductor stores energy in a **magnetic field**. Capacitors resist changes in **voltage**; inductors resist changes in **current**. Inductors are less common than resistors and capacitors in beginner circuits, but the ideas in this chapter matter a lot in practice because every motor, relay, solenoid, speaker, buzzer and transformer is built on the same magnetic principle. Ignoring it is the number one way to destroy an Arduino output pin.

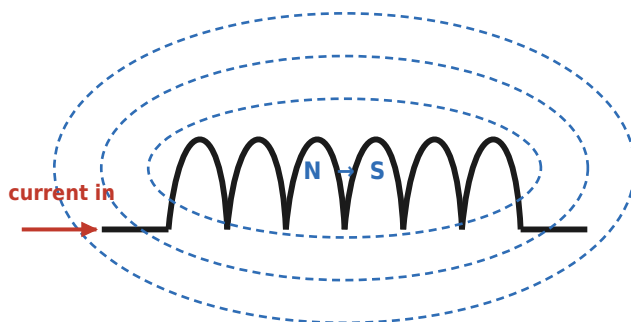
WHAT YOU WILL LEARN IN THIS CHAPTER

- How electricity and magnetism are connected, and what an **electromagnet** is
- What an **inductor** does, the unit **henry**, and why inductors resist changes in current
- The **inductive kick** (back-EMF): the reason motors and relays can destroy transistors
- How **transformers** change voltage, and how **relays** and solenoids work
- How to switch relays safely from a small signal

7.1 Electricity and magnetism are two sides of one thing

In 1820 Hans Christian Ørsted noticed that a compass needle twitched when he switched a current on in a nearby wire. He had discovered that **a moving charge (a current) creates a magnetic field** around it. Eleven years later, Michael Faraday found the reverse: **a changing magnetic field creates a voltage** in a nearby wire. Together these two facts underpin electric motors, generators, transformers, speakers, microphones, wireless charging and radio.

- **Current makes a magnetic field.** The field circles the wire. Coil the wire into a spiral (a **solenoid**) and the fields from each turn add up, creating a strong field through the middle, just like a bar magnet.
- **Add an iron core** and the field multiplies hundreds of times. That is an **electromagnet**: magnetic only while current flows, and stronger with more turns, more current or an iron core.
- **A changing field creates voltage.** Move a magnet in and out of a coil (or change the coil's current) and a voltage appears across the coil. This is **electromagnetic induction**.



Field lines loop through the coil and back around outside, like a bar magnet.

More turns, more current, or an iron core → stronger magnet.

Figure 7.1 A coil (solenoid) carrying current creates a magnetic field like a bar magnet; the field is strongest inside.

REAL-LIFE EXAMPLE | EVERYWHERE YOU LOOK

Loudspeaker: the audio current in a coil moves a cone back and forth against a fixed magnet. **Microphone:** sound moves a coil in a magnet and makes a voltage (the reverse). **Electric motor:** current in a coil in a magnetic field feels a force and rotates. **Generator/dynamo:** rotating a coil in a field makes electricity. **Induction cooker, wireless phone charger:** a changing field in one coil induces current in another.

7.2 The inductor

An **inductor** (also called a coil, choke or reactor) is simply a wire wound into a coil, often around a core of iron or ferrite. Its symbol is a row of loops. Its property is called **inductance (L)**, measured in **henries (H)**, named after Joseph Henry. Common values are microhenries (μH), millihenries (mH) and, for big power inductors, henries.

What does an inductor do? Whenever the current through it **changes**, its magnetic field changes, and by Faraday's law that changing field induces a voltage across the coil which **opposes the change** (this is Lenz's law). So an inductor fights any attempt to change its current: it wants the current to stay as it is.

$$V_L = L \times (\Delta I / \Delta t)$$

Voltage across an inductor = inductance $\times$ rate of change of current

Read it: if the current is **steady** ($\Delta I = 0$), there is **no voltage** across an ideal inductor: it looks like a plain wire to DC. If the current changes **fast**, a **large** voltage appears. A 1 H inductor produces 1 V when the current is changing at 1 A per second.

ANALOGY | A HEAVY FLYWHEEL

Think of a **heavy flywheel** on a water wheel (or a bus full of people). It is hard to get it started, and once moving it is hard to stop: it has **inertia**. Push the water and the flywheel resists the change in flow; stop the water and the spinning flywheel keeps the flow going for a moment. Current in an inductor behaves the same way: it has "electrical inertia".

Compare with a capacitor, which behaves like a spring or membrane: it resists changes in **pressure** (voltage). Inductor: current inertia. Capacitor: voltage elasticity.

	Capacitor	Inductor
Stores energy in	Electric field	Magnetic field
Resists changes in	Voltage	Current
At DC (steady state)	Acts like an open circuit (blocks)	Acts like a wire (passes)
At high frequency	Passes (low reactance)	Blocks (high reactance)
Energy	$\frac{1}{2} C V^2$	$\frac{1}{2} L I^2$
Time constant with R	$\tau = R C$	$\tau = L / R$
Reactance	$X_C = 1 / (2\pi f C)$	$X_L = 2\pi f L$

Energy stored, time constant, and reactance

$$E = \frac{1}{2} \times L \times I^2$$

Energy stored in an inductor (joules)

When you connect an inductor L through a resistor R to a DC supply, the current does not jump immediately to V/R ; it grows exponentially with time constant $\tau = L / R$, reaching 63 % of its final value after one τ , and about 99 % after 5 τ . It is the mirror image of the capacitor.

WORKED EXAMPLE | INDUCTOR TIME CONSTANT

A relay coil has $L = 100 \text{ mH}$ and a resistance of 100Ω . Its time constant is $\tau = 0.1 \div 100 = \mathbf{1 \text{ ms}}$. So the current takes about 5 ms to reach full value: the relay contacts close after a few milliseconds (also due to mechanical movement).

WORKED EXAMPLE | ENERGY IN A COIL

A motor winding has $L = 10 \text{ mH}$ and carries 0.5 A . $E = \frac{1}{2} \times 0.01 \times 0.25 = \mathbf{1.25 \text{ mJ}}$. That looks tiny, but if the current is cut in a microsecond, the resulting power (E/t) is 1.25 kW momentarily, which is why switching off an inductive load produces a dangerous spike.

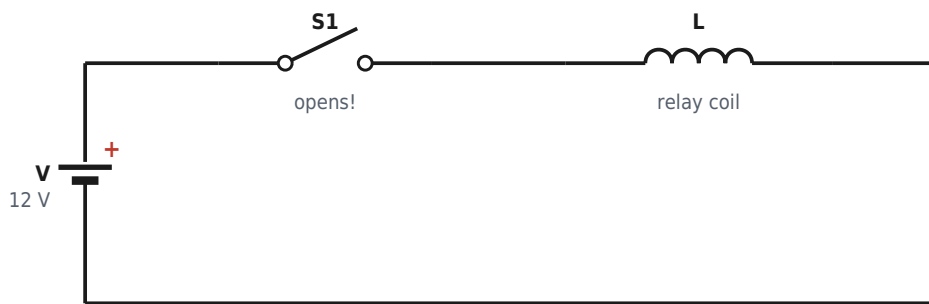
For AC, an inductor's opposition is its **inductive reactance** $X_L = 2\pi f L$ (in ohms), which **rises with frequency**: the opposite of a capacitor. A 10 mH inductor at 50 Hz has $X_L = 3.14 \Omega$, while at 100 kHz it has $6.28 \text{ k}\Omega$. That makes inductors useful to block high-frequency noise while allowing DC through (chokes and ferrite beads).

Inductors in series and parallel

If they are far enough apart not to couple magnetically, inductors combine **like resistors**: series adds, $L_{\text{total}} = L_1 + L_2$; parallel: $1/L_{\text{total}} = 1/L_1 + 1/L_2$.

7.3 The inductive kick: why coils are dangerous to your electronics

Because an inductor opposes changes in current, what happens when you suddenly open a switch in series with a coil? The current wants to keep flowing but the circuit is broken, so $\Delta I / \Delta t$ becomes enormous (the current falls to zero in almost no time) and the coil generates a **huge voltage spike** in an attempt to keep the current moving. This spike, sometimes called **back-EMF, inductive kick or flyback voltage**, is of opposite polarity to the supply and can reach **tens or hundreds of volts**, easily enough to blow up a transistor or an Arduino pin, or to create a visible spark across the switch contacts.



The moment S1 opens, the coil forces current to continue: voltage at the switch spikes.

Figure 7.2 Opening a switch in series with a coil creates a large voltage spike. Without protection it can spark across the contacts or destroy a transistor.

The cure: the flyback (freewheeling) diode

The standard fix is to put a **diode across the coil, reverse-biased in normal operation**. When the supply is on, the diode does nothing (it is blocked). When the switch turns off and the coil tries to push current in the same direction, the diode provides a **safe path** for that current to circulate through the coil and diode until the energy has dissipated as a little heat. The diode clamps the spike to about 0.7 V above the supply.

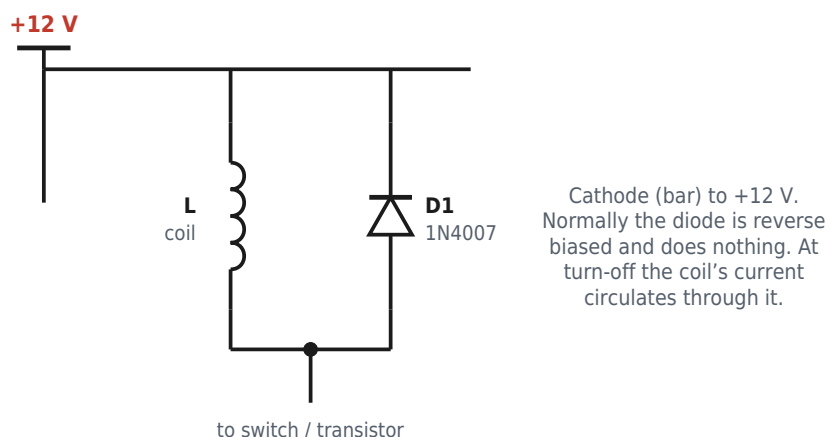


Figure 7.3 Flyback diode across an inductive load. Always include one across every relay coil, motor and solenoid that you switch electronically.

SAFETY WARNING

Every relay coil, DC motor, solenoid and buzzer coil switched by a transistor or Arduino pin must have a flyback diode across it. The diode must be connected **backwards** (cathode/stripe toward the positive supply). Forgetting it is one of the most common causes of a mysteriously reset or dead Arduino. Use a 1N4007 (general purpose) or 1N4148 (small coils). Relay modules for Arduino already contain this diode; bare relays do not.

7.4 Transformers

A **transformer** is two (or more) coils wound on the same iron core. AC in the **primary** coil creates a changing magnetic field in the core; that field passes through the **secondary** coil and induces an AC voltage there. The voltages are in the ratio of the number of turns:

$$V_s / V_p = N_s / N_p$$

Secondary voltage / primary voltage = secondary turns / primary turns

A transformer with 1000 turns on the primary and 50 on the secondary divides the voltage by 20: 230 V becomes 11.5 V. Power stays (almost) the same, so if the voltage drops the **current rises** in proportion. Transformers work **only with AC**: a steady DC produces no changing field and no output.

WORKED EXAMPLE | A STEP-DOWN TRANSFORMER

A mains transformer has 2300 turns on the primary (230 V) and 115 turns on the secondary. Secondary voltage = $230 \times 115 / 2300 = \mathbf{11.5\text{ V}}$ (RMS). If it delivers 1 A to a load, the power is 11.5 W, so the primary draws about $11.5 \div 230 = \mathbf{50\text{ mA}}$ (neglecting losses).

REAL-LIFE EXAMPLE | YOUR PHONE CHARGER AND THE POLE OUTSIDE YOUR HOUSE

The heavy black brick of an old mains adapter is a transformer followed by a rectifier. Modern chargers use tiny high-frequency transformers inside a switching supply. The grey cylinder on the electric pole steps 11 kV down to 230 V for the homes nearby. A transformer also provides **electrical isolation**: there is no metal connection between primary and secondary, an important safety feature.

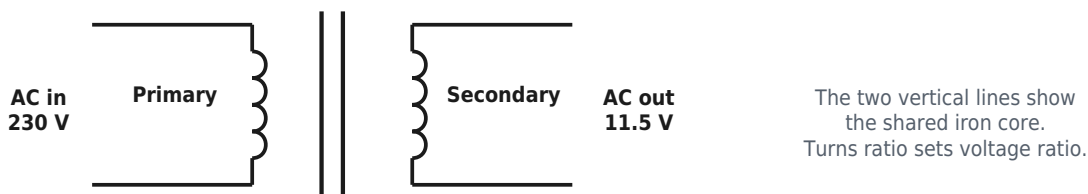


Figure 7.4 Transformer symbol: two coils and a core. Only AC passes through.

7.5 Relays: switches operated by electricity

A **relay** is an electrically operated switch: a small current in a coil (an electromagnet) pulls a metal armature and moves contacts that can switch a much larger current or voltage, completely **isolated** from the control side. This lets a tiny 5 V, 20 mA signal control a 230 V lamp or a heavy motor.

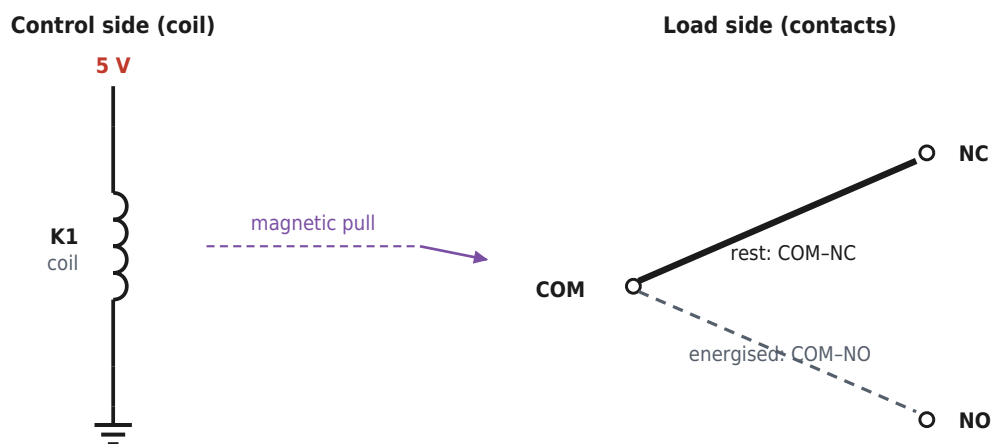


Figure 7.5 A relay. The coil (left) magnetically moves the contacts (right), which are electrically separate from the coil circuit. At rest COM is joined to NC; when the coil is energised, COM connects to NO.

Term	Meaning
Coil voltage	Voltage needed to operate the relay: 5 V, 12 V, 24 V. Use the value stated, and supply enough current.
Coil resistance / current	A 5 V relay with 70 Ω coil draws $5/70 = 71$ mA: too much for an Arduino pin (max 20 mA), so a transistor is needed.
COM (common)	The moving terminal.
NO (normally open)	Not connected to COM at rest; connects when the coil is energised. Use for “turn ON when triggered”.
NC (normally closed)	Connected to COM at rest; disconnects when energised. Use for “fail-safe ON”.
Contact rating	Maximum current and voltage the contacts can safely switch, e.g. 10 A / 250 V AC, 10 A / 30 V DC. DC ratings are lower than AC because DC arcs persist.
SPDT / DPDT	Single-pole double-throw (one COM, NO and NC) or double-pole double-throw (two independent sets).

Driving a relay from an Arduino

A bare relay coil needs more current than an Arduino pin can supply (typical relay: 40 – 90 mA at 5 V; pin limit: 20 mA continuous), so it needs a **transistor driver** and a **flyback diode** (Chapter 9 builds it). For simplicity, buy **relay modules**, small boards that include the transistor, diode and LED indicator. You connect: **VCC** to 5 V, **GND** to GND, **IN** to an Arduino pin, and the load to COM/NO. Many modules are **active-LOW** (relay turns on when the IN pin goes LOW); check the module’s documentation.

SAFETY WARNING

Relays that switch **mains voltage (110 - 240 V)** are not for beginners. Bare terminals, poor insulation and exposed traces can be lethal. **Do not switch mains loads until you have a properly enclosed, certified module, correct fuses and guidance from an experienced person.** All experiments in this book use low-voltage loads (LEDs, small 12 V lamps or fans) on the relay contacts.

Other coil devices

- **Solenoid:** a coil with a plunger that pulls in when energised: door locks, valves. Needs a diode and a driver.
- **Solid-state relay (SSR):** switches using semiconductors, no moving parts, silent, fast; input side is an LED. Good for AC loads.
- **Buzzer:** an “active” buzzer contains an oscillator: apply DC and it sounds. A “passive” buzzer or speaker needs an AC signal (from `tone()`, Chapter 18).
- **Motor:** a rotating electromagnet, treated fully in Chapters 10 and 21.

TIP

Hearing a click is a diagnostic. When a relay module works you can hear the click of the contacts and see the LED on the module. If it clicks but the load doesn't work, the problem is on the load side; if there is no click, the problem is the control side (signal, power or transistor).

CHAPTER SUMMARY

- A current creates a magnetic field; a changing magnetic field creates a voltage. A coil with a core is an electromagnet.
- An inductor (unit: henry) resists changes in current: $V = L \Delta I / \Delta t$. It stores $E = \frac{1}{2} L I^2$, has time constant $\tau = L / R$, and reactance $X_L = 2\pi f L$ (rises with frequency).
- Suddenly interrupting current in a coil produces a **large voltage spike** (inductive kick). Protect transistors and Arduino pins with a **flyback diode** across every coil: cathode to +.
- A transformer changes AC voltage in proportion to the turns ratio and provides isolation; it does not work with DC.
- A relay is a coil-operated switch with COM, NO and NC contacts, electrically isolated from the control side; drive bare coils with a transistor and a flyback diode; never experiment with mains.

ASSIGNMENTS | CHAPTER 7

7.1 [Calculate] A coil of 22 mH carries 300 mA. How much energy does it store?

7.2 [Calculate] An RL circuit has $L = 50$ mH and $R = 25 \Omega$. Find τ . How long until the current is essentially at its final value?

7.3 [Calculate] What is the reactance of a 10 mH inductor at (a) 100 Hz, (b) 10 kHz?

7.4 [Calculate] A transformer has 500 primary turns and 25 secondary turns and a 230 V input. Find the secondary voltage. If the secondary supplies 2 A, estimate the primary current.

7.5 [Calculate] A 12 V relay has a coil resistance of 240 Ω . What current does the coil draw and what power does it consume? Could an Arduino pin drive it directly?

7.6 [Calculate] Two 10 mH inductors are connected in series, then in parallel. What are the equivalent inductances?

7.7 [Practice] Draw the schematic of a 12 V relay coil, driven by a battery via a switch, with a flyback diode. Mark the polarity of the diode.

7.8 [Practice] A relay module has terminals labelled COM, NO and NC. You want a lamp to be ON only while the relay is energised. Where do you connect the lamp and the supply?

7.9 [Think] Why is a diode across a coil connected “backwards” with respect to the supply?

ASSIGNMENTS | CHAPTER 7

7.10 [Think] Why does a transformer not work on DC? What happens if you connect a mains transformer to a battery?

7.11 [Think] Explain why relay contacts rated 10 A / 250 V AC may be rated for only 10 A / 30 V DC.

7.12 [Hands-on] With a relay module powered from 5 V, use a jumper wire to connect IN to GND (or 5 V, depending on the module) and listen for the click. Use your multimeter in continuity mode to verify that COM-NO connect only when the relay is on. Also check COM-NC. Then determine whether your module is active-LOW or active-HIGH.

Diodes and LEDs

Resistors, capacitors and inductors are “linear” parts: double the voltage and the current doubles. Semiconductor devices break that rule and give electronics its real power. The simplest of them is the diode, a one-way street for current. It leads directly to the light-emitting diode (LED), the first component almost everyone connects to an Arduino. This chapter turns “put a resistor in series with the LED” from a rule you memorise into something you can calculate for any LED and any supply.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How a **diode** works: one-way current flow, forward voltage and reverse breakdown
- Types of diodes: signal, rectifier, Schottky, Zener and photodiode
- Rectifying AC into DC with a **bridge rectifier**; reverse-polarity protection; the flyback diode revisited
- How **LEDs** work, their colours and forward voltages, and how to **calculate the series resistor** every time
- Series/parallel LED wiring rules, RGB LEDs, and what an Arduino pin can safely drive

8.1 Semiconductors in two paragraphs

In Chapter 1 you learned that semiconductors such as silicon lie between conductors and insulators. Pure silicon barely conducts. But if a few atoms of another element are added (a process called **doping**), it becomes useful in two flavours. **N-type** silicon has extra free electrons (negative carriers); **P-type** silicon has “holes”: missing electrons that behave like positive carriers.

Join a piece of P-type to a piece of N-type and you get a **P-N junction**. At the boundary, electrons from the N side fill holes on the P side, forming a thin “depletion region” with no free carriers, like a small insulating wall. This junction is the heart of every diode, and (with more layers) every transistor.

8.2 The diode: a one-way valve

A diode has two terminals: the **anode (A)**, connected to the P side, and the **cathode (K)**, connected to the N side. Its symbol is a triangle pointing in the direction of conventional current flow, with a bar at the cathode end.

- **Forward biased** (anode more positive than cathode by about 0.6 – 0.7 V for silicon): the depletion wall collapses; the diode **conducts**, with only a small voltage drop across it.
- **Reverse biased** (anode more negative than cathode): the wall widens; the diode **blocks**, passing only a negligible leakage current (nanoamps).
- **Reverse breakdown**: at a high enough reverse voltage (50 V, 1000 V... depending on the type) the wall breaks down and a large current flows; a normal diode may be destroyed, but a Zener diode is designed for it.

ANALOGY | THE NON-RETURN VALVE

A diode is a **check valve** (non-return valve) in a water pipe. Water flowing in the right direction pushes the flap open and passes with a little loss of pressure (the 0.7 V drop). Water pushing the wrong way presses the flap shut and nothing gets through, unless the pressure is so high it breaks the valve (breakdown).

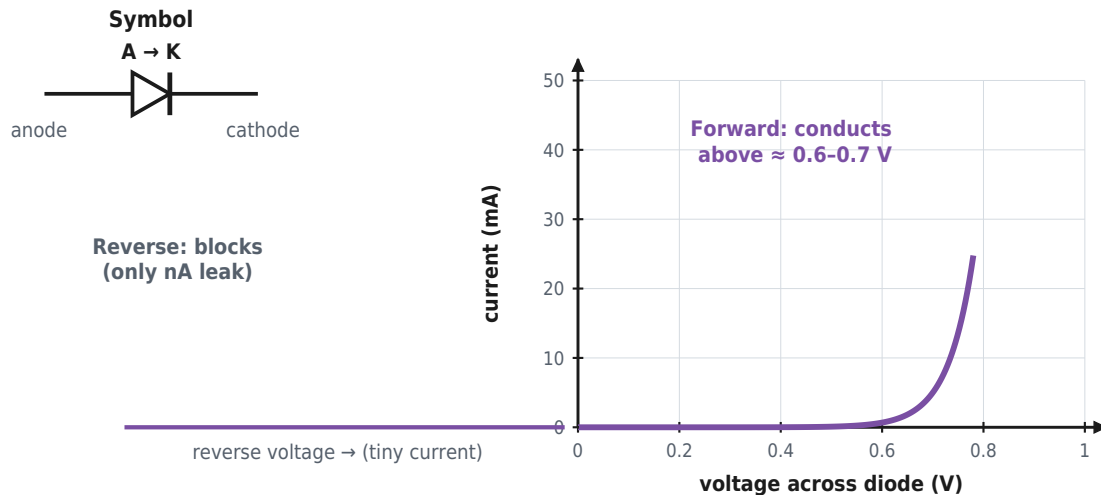


Figure 8.1 The diode's voltage-current curve: almost nothing flows until the forward voltage (~ 0.7 V for silicon) is reached, then current rises steeply. In reverse, essentially nothing flows.

Type	Forward drop	Typical use	Example part
Silicon signal diode	0.6 – 0.7 V	Fast, small currents (≤ 200 mA): logic, protection, signal detection	1N4148
Silicon rectifier	0.7 – 1.0 V	Power supplies, flyback and polarity protection (1 A+)	1N4001 – 1N4007 (1 A), 1N5400 (3 A)
Schottky diode	0.15 – 0.45 V	Low-loss rectification, fast switching, solar and battery circuits	1N5817, 1N5819, SS14
Zener diode	≈ 0.7 V forward; reverse breakdown at a set voltage	Voltage reference and clamping	1N4733 (5.1 V), BZX55C3V3
LED	1.8 – 3.4 V (colour dependent)	Light indicators and lighting	Any 5 mm LED
Photodiode	—	Light sensing: current proportional to light	BPW34

COMMON MISTAKE

The 1N4001–1N4007 all look the same and differ only in **reverse voltage rating** (50 V to 1000 V), so any of them serves for low-voltage jobs. The 1N4148 and 1N4007 are **not** interchangeable for high currents or speed: 4148 is fast but rated only 200 mA; 4007 is robust at 1 A but slower. Look for the stripe: it always marks the **cathode**.

Identifying the leads

On a diode's body a **stripe** (or bar) marks the **cathode**, matching the bar of the symbol. On an LED the cathode is the **shorter lead** and the side of the plastic rim with a **flat edge**. For any diode you can also use the multimeter's **diode test** mode: red probe on anode, black on cathode gives a reading of about 0.5 – 0.7 V (silicon); swapped, it reads "OL".

8.3 What diodes are used for

1. Rectification: AC to DC

A diode passes only the positive half of an AC waveform. Used as a **half-wave rectifier**, it discards the negative half. Four diodes in a diamond form a **bridge rectifier** that flips the negative half over so both halves appear positive: **full-wave rectification**. A large capacitor then fills the gaps to produce nearly steady DC. This is the first stage of nearly every mains power adapter.

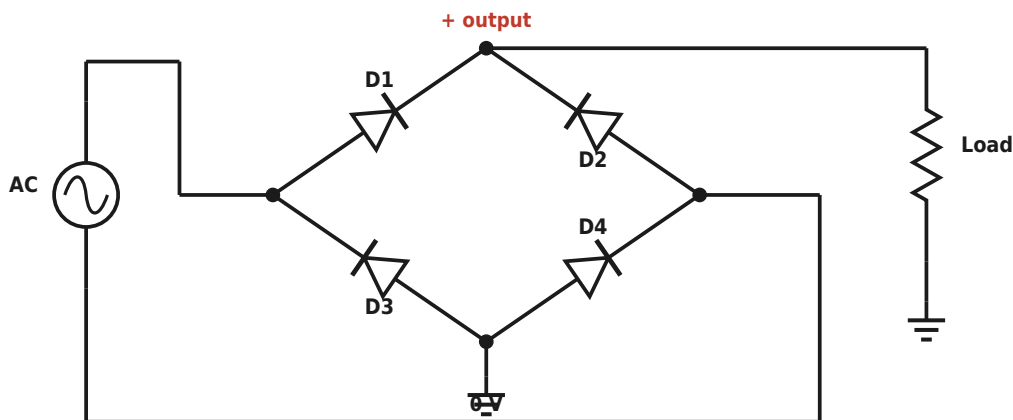


Figure 8.2 A bridge rectifier. On the positive half-cycle D1 and D4 conduct; on the negative half-cycle D2 and D3 conduct. Either way the load sees current in the same direction.

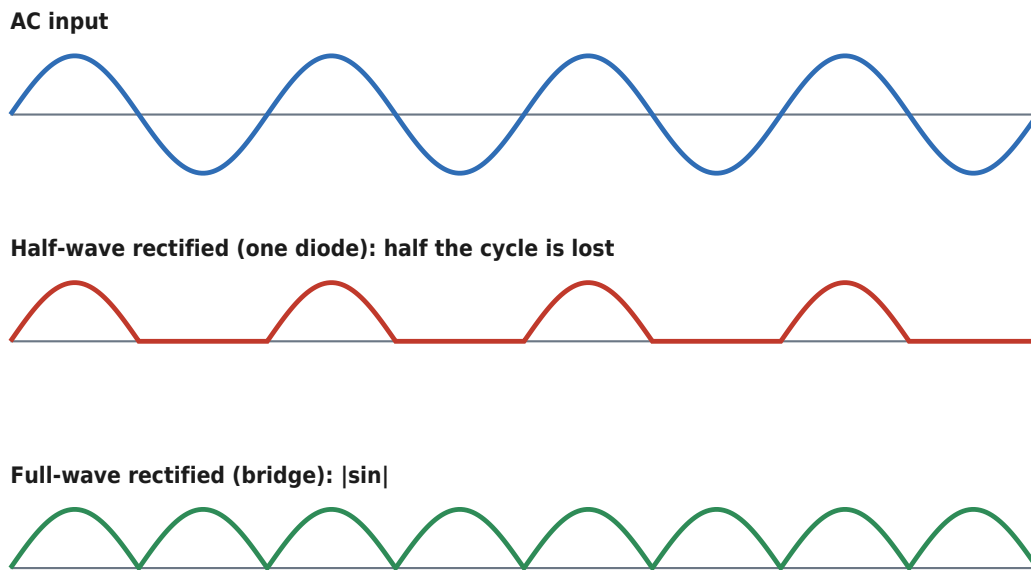


Figure 8.3 AC input (top), half-wave (middle) and full-wave (bottom) rectification. A smoothing capacitor (Chapter 13) turns the humps into almost flat DC.

GOOD TO KNOW

Each diode in the bridge drops about 0.7 V, and two are in the current path at a time, so the DC output peak is the AC peak minus about 1.4 V. A 12 V RMS transformer output peaks at $12 \times \sqrt{2} \approx 17$ V, and gives up to about 15.6 V DC after rectification and smoothing.

2. Reverse-polarity protection

Batteries and power jacks are easily connected backwards. A single series diode passes current only when the supply is connected the right way round, protecting the circuit at a cost of about 0.7 V. A Schottky diode costs less voltage (0.3 V). The barrel jack on an Arduino Uno has exactly this protection diode.

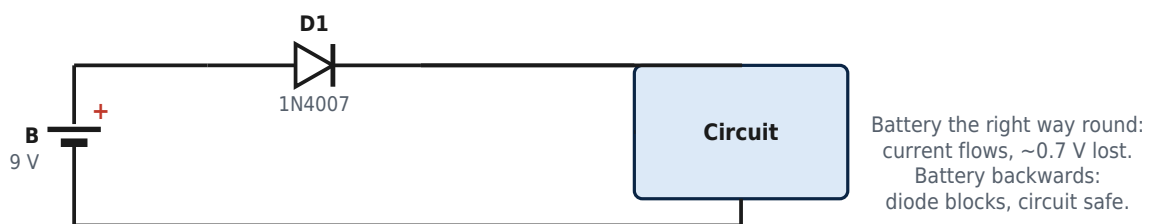


Figure 8.4 A series diode as reverse-polarity protection.

3. Flyback / freewheeling (Chapter 7)

A diode across a relay coil or motor, reverse-biased in normal operation, gives the inductive spike a harmless path.

4. Clamping and protecting inputs

Two diodes from an input pin to the +5 V rail and to ground “clamp” voltage spikes: anything above 5.7 V or below -0.7 V is shunted away. Microcontroller pins contain such diodes inside; they are why an input can survive a small overvoltage, but only for a few mA. Never rely on them for more.

5. Voltage regulation with a Zener diode

A Zener diode is built to conduct in **reverse** at a precise voltage (its “Zener voltage” V_Z : 3.3 V, 5.1 V, 12 V ...) and hold that voltage constant even as the current varies. Used with a series resistor R_S , it makes a simple regulator.

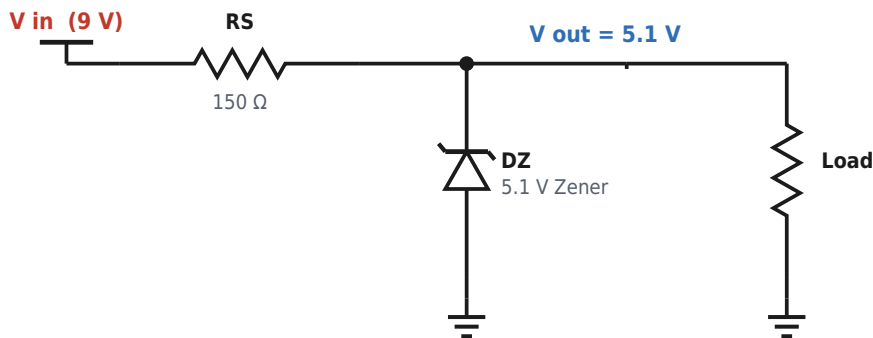


Figure 8.5 A Zener shunt regulator. The Zener sits in reverse (cathode to +) and clamps the output to V_Z . Any extra current passes through the Zener.

WORKED EXAMPLE | DESIGNING A ZENER REGULATOR

You have a 9 V battery and want 5.1 V for a load that draws up to 20 mA. Use a 5.1 V Zener (with about 5 mA needed through it to stay in regulation).

Series resistor: $R_S = (V_{in} - V_Z) \div (I_{load} + I_{Z,min}) = (9 - 5.1) \div (0.020 + 0.005) = 3.9 \div 0.025 = \mathbf{156\ \Omega} \rightarrow$ use **150 Ω** .

Zener power (worst case: no load): $I = 3.9 \div 150 = 26\text{ mA}$, so $P_Z = 5.1 \times 0.026 = 133\text{ mW}$. A 500 mW Zener is fine.

Resistor power: $P = I^2 R = 0.026^2 \times 150 = 0.10\text{ W}$; a $\frac{1}{4}$ W resistor is fine.

Limitations: it wastes at least 26 mA continuously, so it is poor for battery projects; the output droops if the load is too big. For serious work use a linear regulator such as the 7805 (Chapter 13).

8.4 The LED

A **light-emitting diode (LED)** is a diode built from special semiconductor materials so that, when current flows in the forward direction, electrons and holes recombine and release energy as **light**. The colour depends on the material’s energy gap, not on any coloured plastic (although the plastic is often tinted to match). LEDs are efficient (little heat), fast, tiny, long-lived (50,000+ hours) and cheap, so they have replaced filament bulbs for indicators and lighting.

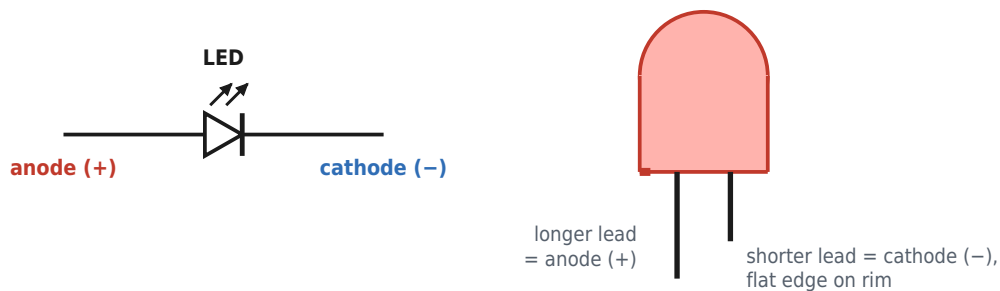


Figure 8.6 LED symbol (two arrows = emitting light) and a 5 mm LED: the longer lead is the anode; the flat side of the rim marks the cathode.

Forward voltage and current

An LED, like any diode, has a **forward voltage** V_F , but it is larger than for a silicon diode and depends on colour. Also, LED current is extremely sensitive to voltage: a tenth of a volt too much can double or triple the current. That is why an LED must **never** be connected directly to a voltage source: it must always have a **current-limiting resistor** in series (or a constant-current driver).

LED colour	Typical forward voltage V_F	Material	Notes
Infrared (940 nm)	1.2 – 1.5 V	GaAs	Remote controls; invisible (a phone camera shows it)
Red	1.8 – 2.2 V	AlGaAs / GaAsP	Cheapest and most efficient; use 2.0 V in calculations
Orange / yellow	2.0 – 2.2 V	GaAsP	
Green (standard)	2.0 – 2.4 V	GaP	Dim
Green (bright/"pure")	3.0 – 3.4 V	InGaN	Same family as blue
Blue	2.8 – 3.4 V	InGaN	Awarded the 2014 Nobel Prize in Physics
White	2.8 – 3.4 V	Blue LED + phosphor	Modern lighting
UV	3.2 – 4.0 V	InGaN / AlGaN	Handle carefully; do not look at it

TIP

Datasheet values are the best source. Failing that, **measure V_F yourself**: use the multimeter's diode test, or put the LED with a resistor on a known supply and measure the voltage across the LED.

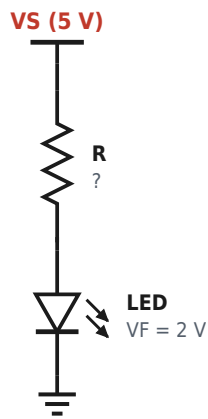
Currents: small indicator LEDs are typically rated at **20 mA maximum**; you normally run them at **5 – 15 mA**, which gives good brightness while extending life. High-power LEDs (1 W, 3 W) need 350 mA or more and a proper driver.

8.5 Calculating the LED series resistor

The current-limiting resistor takes up the difference between the supply voltage and the LED's forward voltage. The LED and the resistor are in series, so the same current flows through both. By KVL (Chapter 5): $V_S = V_R + V_F$, so the resistor voltage is $V_S - V_F$, and by Ohm's law:

$$R = (V_S - V_F) \div I_F$$

The LED resistor formula



$$\text{Voltage across } R = V_S - V_F = 5 - 2 = 3 \text{ V}$$

$$\text{Current } I_F = 10 \text{ mA (chosen for brightness)}$$

$$R = 3 \text{ V} \div 0.010 \text{ A} = 300 \Omega$$

$$\text{nearest E12 value above: } 330 \Omega \rightarrow I = 3 \div 330 = 9.1 \text{ mA}$$

Figure 8.7 LED resistor calculation on a 5 V supply.

Follow these steps every time:

1. **Find the supply voltage V_S** (5 V for an Arduino pin, 3.3 V for a 3.3 V board, 9 V for a battery).
2. **Find the LED's V_F** (from the table or by measurement).
3. **Choose the LED current I_F** (10 mA is a good default; less than 20 mA; less than the pin limit).
4. **Compute** $R = (V_S - V_F) / I_F$.
5. **Round UP** to the next standard resistor value (a slightly lower current is safe; a higher one is not).
6. **Check the resistor power:** $P = (V_S - V_F) \times I_F$; $\frac{1}{4}$ W is plenty for an LED at 5 V.
7. **Check the actual current** with the standard value: $I = (V_S - V_F) \div R_{\text{chosen}}$.

WORKED EXAMPLE | RED LED ON AN ARDUINO PIN

$$V_S = 5 \text{ V}, V_F = 2.0 \text{ V}, I_F = 15 \text{ mA}.$$

$$R = 3 \div 0.015 = 200 \Omega \rightarrow \text{next standard value up in E12: } \mathbf{220 \Omega}.$$

Actual current = $3 \div 220 = 13.6 \text{ mA}$ ✓. Power = $3 \times 0.0136 = 41 \text{ mW}$ ✓.

In practice, almost everyone uses 220 Ω or 330 Ω for 5 V indicator LEDs; 330 Ω gives 9 mA: bright enough and gentle on the pin.

WORKED EXAMPLE | BLUE LED ON 3.3 V

$$V_S = 3.3 \text{ V}, V_F = 3.0 \text{ V}, I_F = 10 \text{ mA}.$$

$$R = 0.3 \div 0.01 = \mathbf{30 \Omega}.$$

The margin is tiny; a small change in V_F or supply changes the current a lot: with $V_F = 3.2 \text{ V}$, current is only 3.3 mA; with $V_F = 2.8 \text{ V}$, it is 17 mA. Blue and white LEDs on 3.3 V need care or a higher supply voltage.

WORKED EXAMPLE | WHITE LED ON A 9 V BATTERY

$$V_S = 9 \text{ V}, V_F = 3.2 \text{ V}, I_F = 15 \text{ mA}.$$

$$R = 5.8 \div 0.015 = 387 \Omega \rightarrow \mathbf{390 \Omega}.$$

Power = $5.8 \times 0.0149 = 86 \text{ mW}$ ($\frac{1}{4}$ W is fine).

WORKED EXAMPLE | SEVERAL LEDs IN SERIES

Three red LEDs ($V_F = 2\text{ V}$ each) in series from 9 V at 15 mA. Total $V_F = 6\text{ V}$.

$R = (9 - 6) \div 0.015 = \mathbf{200\ \Omega}$. The same current flows through all three, so all have equal brightness. The rule: the sum of forward voltages must be less than the supply, with some headroom (at least about 1 V) for the resistor.

LEDs in parallel: one resistor each

It is tempting to connect several LEDs in parallel with one shared resistor. **Don't.** Because the LED's current depends so steeply on voltage and no two LEDs have identical V_F , the LED with the lowest V_F takes more than its share and may burn out, followed by the others in turn. **Give each LED its own resistor**, and connect these LED-plus-resistor branches in parallel.

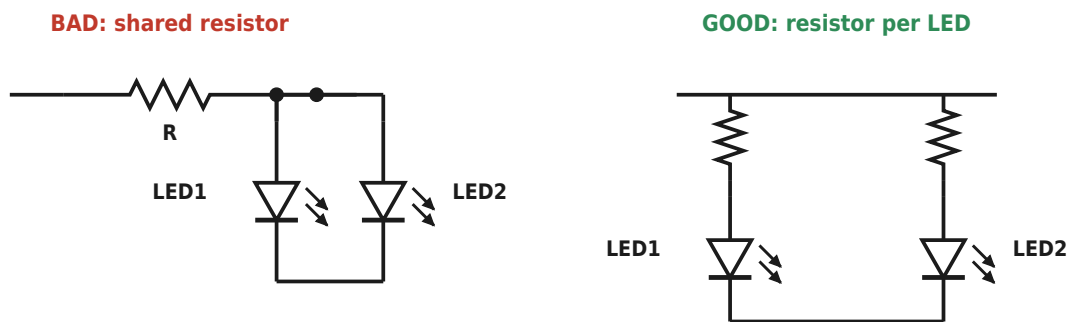


Figure 8.8 Parallel LEDs need individual resistors so each current is set independently.

How much can an Arduino pin drive?

SAFETY WARNING

An Arduino Uno pin can supply about **20 mA continuously** (absolute maximum 40 mA), and the whole chip has a total limit of about **200 mA** across all pins (the 5 V pin you draw from the regulator is separate). So **at most ~10 LEDs at 20 mA**, but 5 – 10 mA per LED is more sensible. For larger loads such as LED strips, high-power LEDs, or many LEDs, drive them through a **transistor** (Chapter 9) or a driver IC.

Common-anode and common-cathode: the RGB LED

An **RGB LED** contains a red, a green and a blue LED die in one package with four leads. In a **common-cathode** part all three cathodes are joined to one lead (the longest), which goes to ground; you feed each colour through its own resistor from an Arduino pin. In a **common-anode** part the shared lead is the anode and goes to +5 V, and colours turn on when their pin goes **LOW**. Mixing red, green and blue in different proportions (using PWM, Chapter 18) creates any colour. Use separate resistors: red needs about 220 Ω , green and blue about 100 – 150 Ω on 5 V because their V_F is higher.

LED die	Connection	Resistor (5 V, ~10 mA)
Red ($V_F \approx 2.0\text{ V}$)	own pin via resistor	$(5 - 2.0)/0.010 = 300\ \Omega \rightarrow 330\ \Omega$
Green ($V_F \approx 3.0\text{ V}$)	own pin via resistor	$(5 - 3.0)/0.010 = 200\ \Omega \rightarrow 220\ \Omega$
Blue ($V_F \approx 3.0\text{ V}$)	own pin via resistor	$(5 - 3.0)/0.010 = 200\ \Omega \rightarrow 220\ \Omega$
Common	longest lead	to GND (common cathode)

Other LED details

- **Brightness** is roughly proportional to current. But the eye responds logarithmically, so a 5 mA LED looks not much dimmer than a 20 mA one. Running at 5 – 10 mA saves power and prolongs life.
- **Reverse voltage:** LEDs tolerate only about 5 V in reverse, so never connect them backwards to a higher voltage; a reverse-parallel diode can protect them on AC.
- **LED arrays** (bar graphs, 7-segment displays, matrices) are just LEDs in convenient packages, each needing its own current limiting.
- **Flashing LEDs** contain a small chip; they need no extra parts but still need a resistor.
- **LED as a sensor:** an LED generates a tiny voltage when illuminated and can be used as a crude light sensor. Interesting, but beyond our scope.

HANDS-ON EXPERIMENT | EXPERIMENT 4: LED, RESISTOR AND THE V_F MEASUREMENT

You need: breadboard, 5 V source (Arduino 5 V pin or 3 AA cells ≈ 4.5 V), red LED, resistors 100 Ω , 330 Ω , 1 k Ω , 10 k Ω , multimeter.

1. **Diode test:** set the meter to diode mode. Red probe on the LED's long lead, black on the short. Read V_F (many red LEDs show 1.7 – 1.9 V and glow faintly). Swap the probes: "OL".
2. **Build** the circuit: +5 V $\rightarrow$ 330 Ω $\rightarrow$ LED anode; LED cathode $\rightarrow$ GND. The LED lights. With the meter (DC volts), measure the voltage across the resistor and across the LED. Confirm that they add up to the supply. Compute the current $I = V_R / R$.
3. **Vary the resistor:** replace 330 Ω with 100 Ω , 1 k Ω and 10 k Ω . For each, measure V_{LED} , V_R , and the current. Notice: the LED voltage barely changes (1.8 – 2.0 V) while the current changes 100-fold and brightness dims dramatically.
4. **Check the polarity behaviour:** reverse the LED. Nothing lights and the current is almost zero. Nothing is damaged (at 5 V).
5. **Try a white/blue LED:** its V_F is about 3 V. With 330 Ω on 5 V compute the current (≈ 6 mA) and measure. **Never** connect an LED to the supply without a resistor. It may work for a second, then die: watch how bright and how hot it gets.

CHAPTER SUMMARY

- A diode conducts only from anode to cathode (about 0.6 – 0.7 V drop for silicon); the stripe marks the cathode. It blocks in reverse until breakdown.
- Uses: **rectification** (bridge rectifier turns AC into pulsating DC), reverse-polarity protection, flyback protection, clamping. A **Zener** in reverse holds a fixed voltage.
- An **LED** is a diode that emits light. V_F depends on colour (red ≈ 2 V, blue/white ≈ 3 V). Long lead = anode.
- **Always use a series resistor:** $R = (V_S - V_F) / I_F$, round up to a standard value, and check power. 5 – 15 mA suits most indicators.
- Parallel LEDs need one resistor each. Series LEDs: V_F values add; supply must exceed the sum.
- An Arduino pin supplies about 20 mA maximum; use transistors for heavier loads.

ASSIGNMENTS | CHAPTER 8

8.1 [Calculate] A red LED ($V_F = 2.0\text{ V}$) must run at 12 mA from a 5 V supply. Compute R and choose the standard E12 value. What is the actual current?

8.2 [Calculate] A green LED ($V_F = 2.2\text{ V}$) is powered from a 9 V battery at 10 mA. Find R, the standard value, and the resistor power.

8.3 [Calculate] A white LED ($V_F = 3.2\text{ V}$) is driven from the 3.3 V pin of an Arduino at about 2 mA. Find R. Is this a reliable design?

8.4 [Calculate] Four red LEDs ($V_F = 2\text{ V}$ each) are placed in series and powered from 12 V at 20 mA. Find R and the resistor's power.

8.5 [Calculate] Ten LEDs each needing 10 mA are wired in parallel with separate resistors on 5 V. What total current is drawn? Can one Arduino pin supply it?

8.6 [Calculate] A 12 V, 1 A power adapter is fed through a bridge rectifier whose diodes each drop 0.7 V, then a smoothing capacitor. Estimate the DC peak voltage if the transformer secondary is 9 V RMS. (Peak = RMS $\times$ 1.414.)

8.7 [Calculate] Design a Zener regulator: input 12 V, Zener 5.1 V, load up to 15 mA, minimum Zener current 5 mA. Find R_S and the power ratings.

8.8 [Practice] You lose the label on a diode; it has a black stripe near one end. Which lead is the cathode? How can you verify with a multimeter?

8.9 [Practice] Draw the schematic of one LED with a resistor on an Arduino pin (pin 8), showing polarity. Mark the current direction.

8.10 [Think] Why can an LED be destroyed almost instantly if connected directly to a 5 V supply, while a resistor of the same nominal size would survive?

8.11 [Think] Two LEDs in parallel share one resistor and, after a few hours, one of them dies. Explain why the other quickly follows.

8.12 [Hands-on] Do Experiment 4 for a red LED and a blue LED and fill in a table of V_F , current, and brightness (subjective, 1-5) at 100 Ω , 330 Ω , 1 k Ω and 10 k Ω .

Transistors

The Arduino's output pins are weak: 20 mA at most. Yet you will want to control things that need hundreds of milliamps or amperes at 12 V: motors, relays, LED strips, solenoids. The device that bridges this gap is the **transistor**: a tiny electrically controlled valve in which a small current or voltage controls a much larger flow. Transistors are also what a microchip is made of; your Arduino's processor contains millions of them. This chapter shows how to use them as **switches** (the most important job for a beginner) and briefly as **amplifiers**.

WHAT YOU WILL LEARN IN THIS CHAPTER

- What a transistor is and why it is called the most important invention of the 20th century
- How the **bipolar junction transistor (BJT)** works: current gain, saturation and cutoff
- How to design a **transistor switch** to drive relays, motors, buzzers and LED strips from a small signal
- How **MOSFETs** work, when to prefer them, and what "logic-level" means
- How to read pinouts, check limits, and test a transistor with a multimeter

9.1 The idea: a small signal controls a big flow

ANALOGY | A TINY LEVER THAT OPENS A HUGE TAP

Consider a huge water main with a tap on it. You can turn the tap with a tiny effort of your fingertip, yet the water flow it controls may be enormous. A transistor is that tap: a small **control input** (a trickle of current into the base, or a small voltage on the gate) opens or closes the path for a large current between two other terminals.

A relay does something similar, using a coil and mechanical contacts. A transistor has **no moving parts**, so it switches millions of times faster, silently, and lasts forever, but unlike a relay it does not provide isolation.

Transistors have two important uses:

- **As a switch:** completely on or completely off. Computers, microcontrollers, relay drivers and motor controllers work this way.
- **As an amplifier:** a small varying signal (from a microphone, sensor or antenna) controls a larger copy of itself. Audio amplifiers, radio receivers and sensor conditioning circuits work this way.

There are two major families: **bipolar junction transistors (BJT)**, controlled by a small **current**, and **field-effect transistors (FET, especially MOSFET)**, controlled by a **voltage** on an insulated gate. Both come in "N" and "P" polarity versions.

9.2 The bipolar junction transistor (BJT)

A BJT has three terminals: the **base (B)** is the control input, the **collector (C)** and **emitter (E)** carry the main current. An **NPN** transistor (the most common) turns on when its base is made more positive than the emitter. A **PNP** transistor is the mirror image and turns on when the base is more negative than the emitter. The symbol's arrow is on the emitter and points in the direction of conventional current: **out** of the emitter for NPN ("**N**ot **P**ointing **i**n"), **in** for PNP.

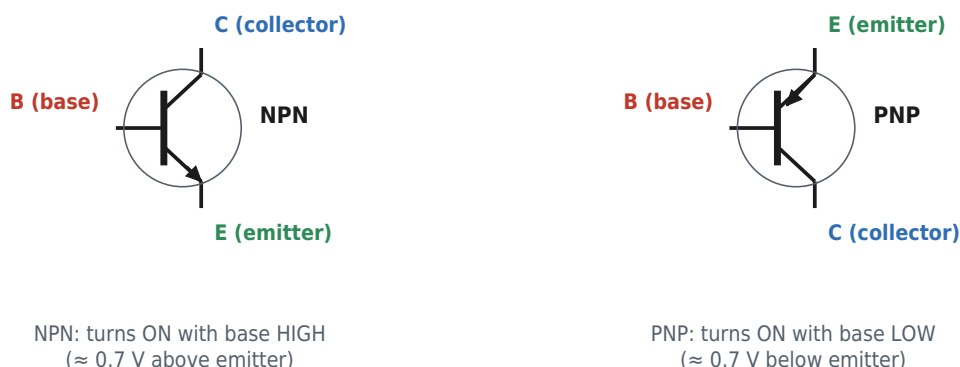


Figure 9.1 Symbols of NPN and PNP bipolar transistors with their terminals.

How it works: current gain

A small current I_B flowing into the base (through the base-emitter junction, which behaves like a diode with about **0.7 V** across it) allows a much larger current I_C to flow from collector to emitter. The ratio is the **current gain**, called β (beta) or h_{FE} :

$$I_C = \beta \times I_B \quad I_E = I_C + I_B$$

β is typically 100 – 300 for small-signal transistors (varies a lot between individual parts).

ANALOGY | SLUICE GATE

The base current is like the small water flow that pushes a lever to open a big sluice gate. Water from the small flow joins the big flow at the exit (emitter): $I_E = I_C + I_B$.

Three operating regions

Region	Condition	What happens	Use
Cutoff	$V_{BE} < \approx 0.6 \text{ V}$ (base current ≈ 0)	$I_C \approx 0$. Transistor is OFF : an open switch.	Switch OFF
Active	$V_{BE} \approx 0.7 \text{ V}$, $V_{CE} > 0.3 \text{ V}$	$I_C = \beta \times I_B$: proportional control.	Amplifier
Saturation	Base current large enough that I_C is limited by the load, not by β	$V_{CE} \approx 0.1 - 0.3 \text{ V}$. Transistor is fully ON : a closed switch.	Switch ON

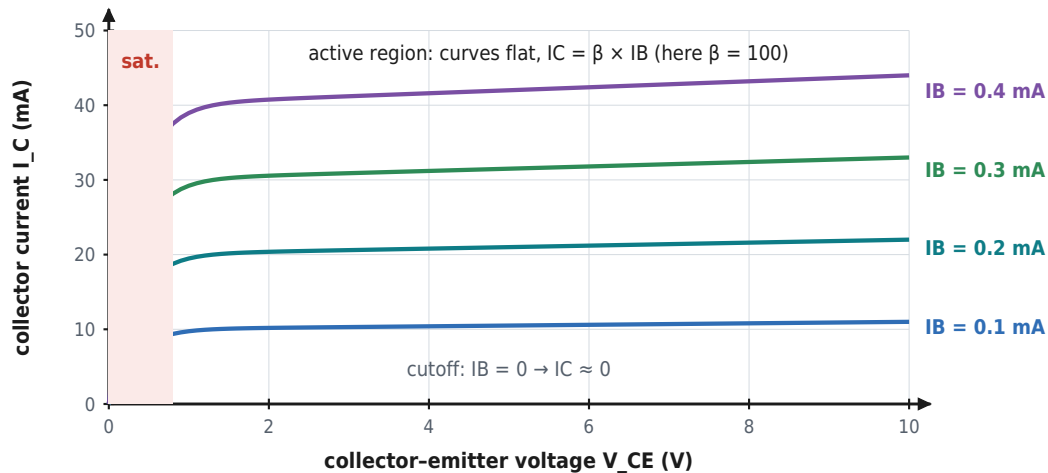


Figure 9.2 BJT output characteristics. For each base current the collector current is roughly constant in the active region; near $V_{CE} \approx 0$ the transistor saturates.

9.3 The BJT as a switch: how to design it

For a switch we want the transistor **fully on or fully off**. To turn it fully on (saturation), supply a base current that is **much more than I_C/β** . The standard trick is **forced beta**: choose the base current as one tenth of the collector current, $I_B = I_C / 10$. That guarantees saturation even for a weak transistor with low β .

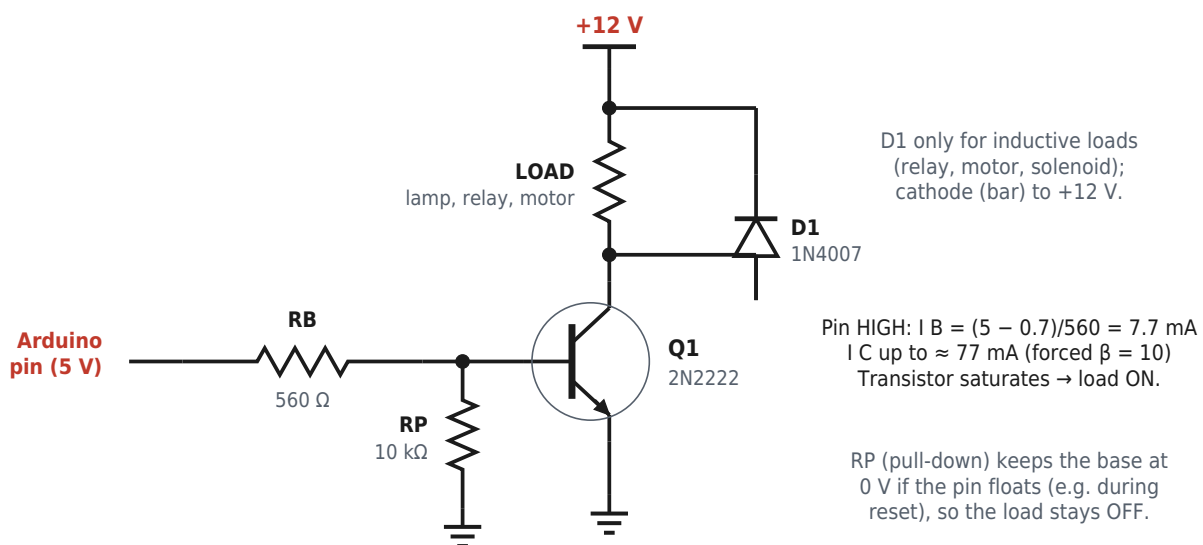


Figure 9.3 NPN low-side switch: the load sits between the supply and the collector; the emitter is grounded. R_B limits base current; R_P is a pull-down.

The design recipe for an NPN switch with a microcontroller pin:

1. **Find the load current I_C** (from the datasheet or from $V \div R$). Check that it is below the transistor's $I_{C(max)}$.
2. **Set the base current $I_B = I_C / 10$** (forced $\beta = 10$). Check that it is below the Arduino pin limit of 20 mA (preferably $\leq 10\text{ mA}$). If I_B would be too large, use a Darlington pair or a MOSFET.
3. **Compute the base resistor:** $R_B = (V_{drive} - 0.7\text{ V}) \div I_B$, where $V_{drive} = 5\text{ V}$ for an Arduino pin. Round **down** slightly to a standard value (more base current is safer).

4. **Add a pull-down** (10 k Ω base to ground) so the transistor stays off when the pin is not driven.
5. **For coils, motors and solenoids, add a flyback diode** across the load (Chapter 7).
6. **Check the transistor power:** $P \approx V_{CE(sat)} \times I_C$ (about 0.2 V $\times$ I_C); tiny. But if the transistor only partly turns on it can get hot.

WORKED EXAMPLE | DRIVING A 5 V RELAY FROM AN ARDUINO PIN

Relay coil: 5 V, 70 $\Omega \rightarrow I_C = 5 \div 70 = \mathbf{71 \text{ mA}}$. Transistor: 2N2222 ($I_{C(max)}$ 600 mA – fine).

$I_B = I_C/10 = 7.1 \text{ mA}$ (below the 20 mA pin limit ✓).

$R_B = (5 - 0.7) \div 0.0071 = 606 \Omega \rightarrow$ choose **560 Ω** $\rightarrow I_B = 4.3/560 = 7.7 \text{ mA}$.

Add: 10 k Ω pull-down from base to ground, and a 1N4007 flyback diode across the coil, cathode to the + side.

Transistor dissipation: 0.2 V $\times$ 71 mA = 14 mW (no heat).

WORKED EXAMPLE | DRIVING A SMALL 12 V DC MOTOR

Motor: 12 V, running current 200 mA (stall current 600 mA). Transistor: TIP120 (Darlington, 5 A) or a MOSFET.

With a plain 2N2222 the forced- β rule gives $I_B = 20 \text{ mA}$, which is exactly the pin's absolute limit: **too risky**. A 2N2222 at 200 mA would need at least 5 – 10 mA base current in practice and the transistor dissipates more.

Options: (a) use a Darlington (TIP120: $\beta \approx 1000$, needs $I_B \approx 2 - 5 \text{ mA}$; but $V_{CE(sat)} \approx 1 \text{ V}$ wastes power); (b) use a logic-level MOSFET (section 9.5), which needs almost no gate current: the best choice.

TIP

Which side to switch? Put the switch (the transistor) on the **ground side** of the load, a “low-side switch” with an NPN or N-channel MOSFET. It is the simplest arrangement and the one you should use with the Arduino. Switching the positive side (“high-side”) needs a PNP or P-channel device and a more complicated drive.

Pinouts: check before you connect

The most common small BJTs come in a **TO-92** plastic package: a half-cylinder with a flat face. Looking at the **flat face with the leads pointing down**:

Transistor	Left $\rightarrow$ right	Type	$I_{C(max)}$	Typical use
2N2222 / PN2222A	E – B – C (PN2222) or C – B – E (some 2N2222 metal-can/TO-18 versions)	NPN	600 mA – 1 A	General switch
BC547 / BC548	C – B – E	NPN	100 mA	Small-signal, switch
2N3904	E – B – C	NPN	200 mA	General small-signal
2N3906 / BC557	E – B – C (2N3906) / C – B – E (BC557)	PNP	200 mA	PNP switch
TIP120	B – C – E (metal tab is C)	NPN Darlington	5 A	Motors, relays, solenoids

SAFETY WARNING

Pinouts differ between transistor models and even between manufacturers of the “same” number.

A wrongly connected transistor usually just does not work, but a wrongly connected one in a power circuit can also be destroyed. Always check the datasheet for the exact part in your kit.

Testing a BJT with a multimeter

In **diode-test mode**, a good NPN transistor shows about 0.6 V from base (red probe) to collector and from base to emitter (red on B, black on E or C). With reversed probes it shows OL. Between collector and emitter it shows OL in both directions. This also identifies which lead is the base and the type (for PNP the readings occur with the black probe on the base). Many multimeters also have an **hFE** socket that reports β directly.

9.4 The BJT as an amplifier

In the active region, small changes in base current produce large, proportional changes in collector current. Add a collector resistor R_C and the varying current becomes a **varying voltage**; a small changing signal at the base becomes a larger, inverted copy at the collector. The **voltage gain** of the standard “common-emitter” circuit is roughly:

$$A_V \approx -R_C \div R_E$$

with an emitter resistor R_E ; a large gain, negative sign means inverted

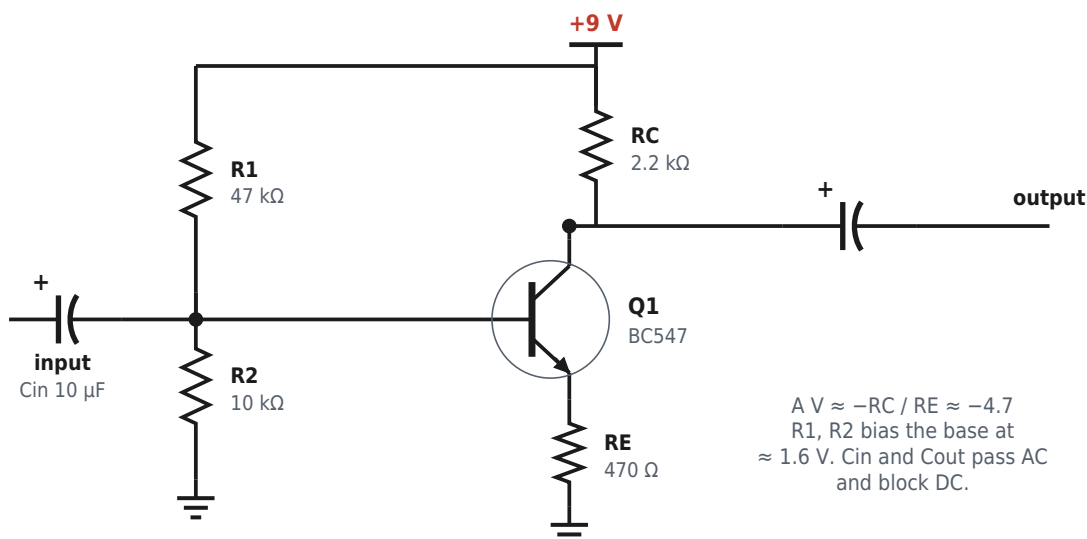


Figure 9.4 A common-emitter amplifier. The resistors R_1 and R_2 set a steady operating point; the capacitors couple the AC signal in and out.

GOOD TO KNOW

Amplifier design (bias points, coupling, frequency response) fills whole books. For Arduino projects you will rarely build a discrete-transistor amplifier; you will use **op-amps** (Chapter 11) or ready-made modules. The key point to remember is that in the active region a transistor **multiplies** small signals.

9.5 The MOSFET

The **MOSFET** (metal-oxide-semiconductor field-effect transistor) is the workhorse of modern electronics: it is the building block of microprocessors and the preferred switch for power. It has three terminals: **gate (G)**, **drain (D)** and **source (S)**. The gate is separated from the channel by a thin **insulating oxide**, so **no steady current flows into the gate**; instead, the **voltage** on the gate creates an electric field that opens (or closes) a conducting channel between drain and source. That is why it is “voltage-controlled”, unlike the current-controlled BJT.

BJT terminal	MOSFET terminal	Role
Base (B)	Gate (G)	Control input
Collector (C)	Drain (D)	Load side (higher voltage for N-type)
Emitter (E)	Source (S)	Ground side (for N-type)

ANALOGY | TRICKLE VS STATIC CHARGE

A BJT is a tap that needs a **continuous trickle of water pushed into its lever** to stay open. A MOSFET is a tap you open by **charging a plate with static electricity**: the plate needs charge only while you change its state, then holds it with practically no further flow. That is why MOSFETs are so efficient, and why they are also sensitive to static electricity.

N-channel MOSFET as a low-side switch

Connect the load between the supply and the **drain**, and the **source to ground**. Apply a gate voltage V_{GS} (relative to the source) above the **threshold voltage** $V_{GS(th)}$ and the channel forms; the MOSFET conducts with a small on-resistance $R_{DS(on)}$. Make V_{GS} 0 V and it turns off.

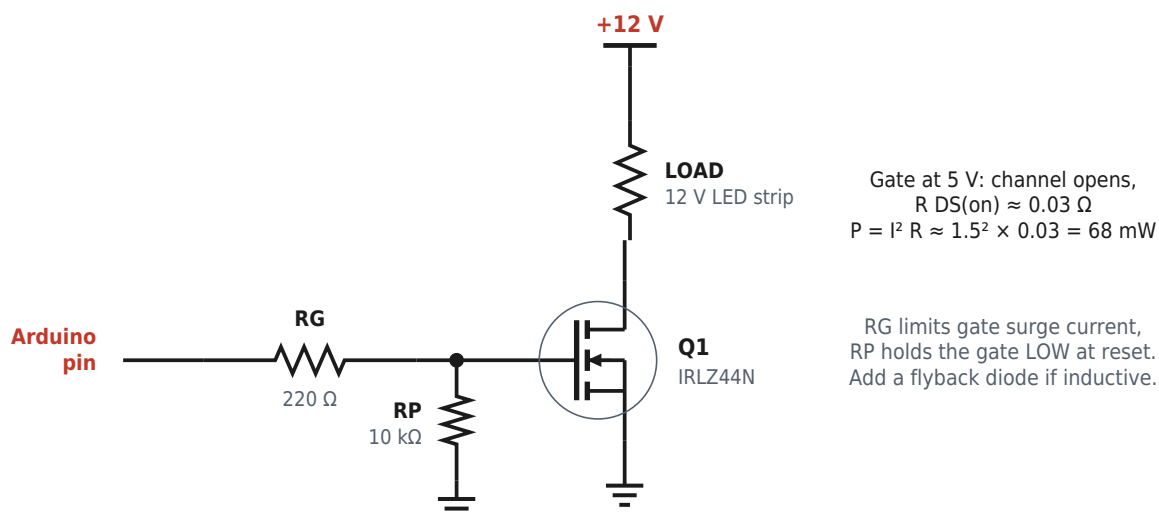


Figure 9.5 N-channel MOSFET low-side switch. The Arduino pin drives the gate through a small resistor; a pull-down keeps the MOSFET off at power-up.

Logic-level MOSFETs: read the datasheet

MOSFETs are not all the same. **Standard** power MOSFETs (such as the IRF540 or IRF3205) need about **10 V** on the gate to turn fully on; with only 5 V from an Arduino they conduct partly, get hot and can be destroyed. You need a **logic-level MOSFET**, guaranteed to be fully on at $V_{GS} = 4.5$ V or even 2.5 V. Search the datasheet for $R_{DS(on)}$ **specified at $V_{GS} = 4.5$ V (or 5 V)**, not at 10 V.

MOSFET	Type	$V_{DS(max)}$	$I_{D(max)}$	$R_{DS(on)}$ at 5 V	Comments
2N7000	N, TO-92	60 V	200 mA	~ 5 Ω	Small loads only; cheap; $V_{GS(th)}$ 0.8 – 3 V, borderline for 3.3 V
BS170	N, TO-92	60 V	500 mA	~ 5 Ω	Similar; small relays, LEDs
IRLZ44N	N, TO-220	55 V	47 A	0.022 – 0.035 Ω	Logic-level; excellent Arduino-friendly power switch
IRLB8721	N, TO-220	30 V	62 A	0.013 Ω	Very low $R_{DS(on)}$; works even at 3.3 V logic

MOSFET	Type	$V_{DS(max)}$	$I_{D(max)}$	$R_{DS(on)}$ at 5 V	Comments
IRF540N	N, TO-220	100 V	33 A	$\approx 0.052 \Omega$ only at 10 V	NOT logic level: needs a gate driver (avoid with Arduino)
IRF9540	P, TO-220	-100 V	-19 A	(10 V gate)	P-channel for high-side; needs level shifting

WORKED EXAMPLE | CHOOSING A MOSFET FOR A 12 V LED STRIP

An LED strip draws 1.5 A at 12 V. Use an IRLZ44N with $R_{DS(on)} \approx 0.03 \Omega$ at 5 V.

Loss in the MOSFET = $I^2 R = 1.5^2 \times 0.03 = \mathbf{0.068 \text{ W}}$: no heatsink needed. Voltage lost = $1.5 \times 0.03 = 0.045 \text{ V}$, negligible.

Compare with a 2N2222 at 1.5 A: it could not even carry it (600 mA max). And a TIP120 Darlington would drop $\sim 1 \text{ V}$ (1.5 W of heat!). The MOSFET is the natural choice.

Practical MOSFET rules

- **Gate resistor (100 - 470 Ω):** the gate acts like a small capacitor (hundreds of pF to nF); the resistor limits the surge current from the Arduino pin when it charges.
- **Pull-down (10 k Ω gate to ground):** the gate is a high-impedance input and will pick up noise and drift; this holds it off.
- **Static:** MOSFET gates are fragile; touch grounded metal before handling, and keep them in the packaging until needed.
- **Body diode:** power MOSFETs contain a parasitic diode from source to drain that conducts if the drain goes below the source. It conveniently helps with inductive loads, but add an external flyback diode for motors and relays.
- **Heat:** the loss is $I^2 \times R_{DS(on)}$. Below roughly 1 W a TO-220 package needs no heatsink.

	BJT (e.g. 2N2222)	MOSFET (e.g. IRLZ44N)
Controlled by	Base current (needs I_B)	Gate voltage (almost no current)
Best for	Small loads up to $\sim 500 \text{ mA}$, amplifiers, low cost	Motors, LED strips, heaters, anything above $\sim 500 \text{ mA}$
When ON	$V_{CE(sat)} \approx 0.2 \text{ V}$, losses = $0.2 \times I_C$	$R_{DS(on)} \approx 0.01 - 0.05 \Omega$, losses = $I^2 R$
Drive from Arduino	Needs base resistor	Needs gate resistor + pull-down; logic-level type
Handling	Robust	Static-sensitive
Speed	Fast	Very fast

Transistor arrays and ready-made drivers

When you need to switch many loads (a stepper motor's coils, a bank of relays), buy a **Darlington array such as the ULN2003 (7 channels, 500 mA each, built-in flyback diodes)**. It contains all the transistors, base resistors and diodes in one chip. Motor driver ICs such as the L293D and L298N (Chapter 21) go further and include full H-bridges. Relay modules include the transistor and diode too.

HANDS-ON EXPERIMENT | EXPERIMENT 5: BUILD A TRANSISTOR SWITCH AND SEE AMPLIFICATION IN ACTION

You need: breadboard, 9 V battery (or 5 V supply), 2N2222 or BC547 NPN, resistors 1 k Ω , 10 k Ω , 470 Ω , a red LED, a 100 k Ω potentiometer (optional), multimeter.

Part A: the LED switch. Wire: battery + $\rightarrow$ LED + 470 Ω (series) $\rightarrow$ collector; emitter $\rightarrow$ battery $-$. Base $\rightarrow$ 10 k Ω $\rightarrow$ battery +. The LED lights. The base current is $(9 - 0.7)/10 \text{ k}\Omega = 0.83 \text{ mA}$ and it turns on an LED at $\sim 15 \text{ mA}$: gain! Remove the base resistor's connection to + and the LED goes out.

Part B: touch switch. Replace the base resistor by two bare wires; touch both with a wet finger. Your finger's resistance (about 100 k Ω – 1 M Ω) passes a tiny base current; the LED lights faintly, showing the current gain of the transistor.

Part C: measure the gain. Insert your meter (mA) in the collector path and in the base path (or measure voltage across resistors and calculate). Compute $\beta = I_C / I_B$ for a few base resistors (100 k Ω , 47 k Ω , 10 k Ω). In the active region you should see $\beta \approx 100 - 300$, and in saturation the ratio drops (the LED resistor limits the collector current).

Part D: the pin-driven relay (after Chapter 14): connect the base resistor to an Arduino pin (560 Ω) and a relay coil with a diode.

CHAPTER SUMMARY

- A transistor lets a small current or voltage control a much larger one. Two main uses: **switch** (fully on/off) and **amplifier** (proportional).
- **BJT**: terminals B, C, E; $I_C = \beta I_B$; $V_{BE} \approx 0.7 \text{ V}$. For a switch, use forced beta: $I_B = I_C / 10$, $R_B = (V_{\text{drive}} - 0.7) / I_B$.
- Always add a **pull-down** at the base and a **flyback diode** across inductive loads. Switch on the **ground side** (low-side) with NPN.
- **MOSFET**: G, D, S; voltage controlled; use **logic-level** parts ($R_{DS(on)}$ specified at 4.5 V) with Arduinos; add gate resistor + pull-down. Power lost $= I^2 R_{DS(on)}$.
- Check the **pinout** and ratings (I_C , V_{CE} , power) in the datasheet before connecting. Use ULN2003 or motor driver ICs for multiple loads.

ASSIGNMENTS | CHAPTER 9

9.1 [Calculate] A BJT has $\beta = 200$. If the base current is 0.05 mA, what is the collector current (active region)? What is the emitter current?

9.2 [Calculate] You want to switch a 12 V buzzer that draws 40 mA using an Arduino pin (5 V). Choose I_B (forced $\beta = 10$) and compute R_B . Select a standard value.

9.3 [Calculate] A 5 V relay has a coil resistance of 125 Ω . Design the driver with a 2N2222: give I_C , I_B , R_B (standard value) and the transistor power.

9.4 [Calculate] A 12 V, 250 mA lamp is switched by a 2N2222 from a 5 V Arduino pin using forced $\beta = 10$. Is that acceptable? What could you do?

9.5 [Calculate] A MOSFET with $R_{DS(on)} = 0.04 \Omega$ carries 3 A. What is the power dissipated? Would it need a heatsink? (assume $\leq 1 \text{ W}$ is fine unheatsinked.)

9.6 [Calculate] The same MOSFET is fed 12 V. If it were replaced by an IRF540 (needs 10 V gate) driven from 5 V and had $R_{DS(on)} = 0.5 \Omega$ at that gate voltage, what would the dissipation be at 3 A?

9.7 [Practice] Draw an Arduino pin controlling a 12 V solenoid (300 mA) via an N-channel logic-level MOSFET with gate resistor, pull-down, and flyback diode. Label the component values.

ASSIGNMENTS | CHAPTER 9

9.8 [Practice] Explain why the emitter of an NPN low-side switch is connected to ground and why the ground of the external 12 V supply must also be joined to the Arduino's ground.

9.9 [Practice] A student tests an NPN with the multimeter diode test. Red on pin 1, black on pin 2: 0.65 V. Red on pin 1, black on pin 3: 0.65 V. Other combinations: OL. Which pin is the base? Is it NPN or PNP?

9.10 [Think] Why must the transistor be driven hard into saturation when used as a switch, rather than left in the active region?

9.11 [Think] What is the purpose of a 10 k Ω resistor from the base (or gate) to ground?

9.12 [Hands-on] Do Experiment 5 parts A-C. Record base current, collector current and β for at least three base resistors, and note when the transistor saturates.

Switches, sensors, motors and actuators

An electronic system has three parts: **inputs** (sensors and switches that tell it about the world), a **brain** (the Arduino) and **outputs** (actuators that change the world: light, sound, motion). So far you have met the brain's language, voltage and current, and a few outputs. This chapter is a guided tour of the parts you will connect to the Arduino for the rest of the book. Think of it as an illustrated parts catalogue with the electronic principles included.

WHAT YOU WILL LEARN IN THIS CHAPTER

- The different kinds of **switches**, and the essential concept of **pull-up** and **pull-down** resistors
- How **sensors** turn the physical world (light, heat, distance, motion, sound) into electrical signals
- The three output signal types of a sensor: analog voltage, digital, and serial data
- How **DC motors**, **servos**, **stepper motors**, buzzers and solenoids work and what each needs to be driven safely
- How to choose the right sensor and actuator for a project

10.1 Switches

A switch opens or closes a circuit by hand. Switches are described by their **poles** (how many separate circuits they control) and **throws** (how many positions each pole connects to):

Abbreviation	Meaning	Example
SPST	Single pole, single throw: one circuit ON/OFF	Simple toggle or slide switch; a push button
SPDT	Single pole, double throw: one common terminal connects to either of two others	Changeover: choose battery A or B; relay contacts (COM/NO/NC)
DPST / DPDT	Two independent poles operated together	Reversing the polarity of a motor (DPDT); mains switches
Momentary (push button)	Closed only while pressed	Doorbell, tactile button, keyboard key
Latching (toggle, slide, rocker)	Stays in the chosen position	Power switch

The tactile push button

The tiny four-legged button in your kit is a momentary SPST. Its four pins are joined **in pairs** inside: the two pins on the same side are permanently connected; pressing the button connects the left pair to the right pair. If you place it across the centre gap of a breadboard so that each side lands in a different pair of columns, you can use one pin from each side. If it seems to be “always pressed”, you have rotated it 90°.

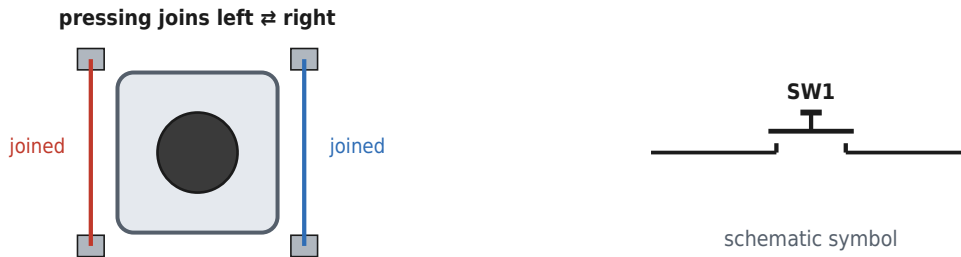


Figure 10.1 A four-pin tactile button. The pins on each side are joined inside. Pressing connects one side to the other.

The floating input problem: pull-up and pull-down resistors

A microcontroller pin configured as an **input** measures voltage: HIGH (near 5 V) or LOW (near 0 V). But what if nothing is connected to it, or a button is connected that is open? The pin is **floating**: it is not tied to any voltage, and it acts like an antenna picking up stray signals from your hand, the mains wiring and nearby circuits. Its reading flips randomly between 0 and 1. A switch by itself therefore cannot make a reliable input.

You must always give the input a defined default state with a **pull-up** or **pull-down** resistor:

- **Pull-down:** a resistor (typically 10 kΩ) from the pin to **ground**. With the button open the pin is pulled to LOW. The button connects the pin to +5 V: pressing gives HIGH.
- **Pull-up:** a resistor (10 kΩ) from the pin to **+5 V**. With the button open the pin is pulled HIGH. The button connects the pin to ground: pressing gives LOW (the logic is **inverted**: pressed = 0).

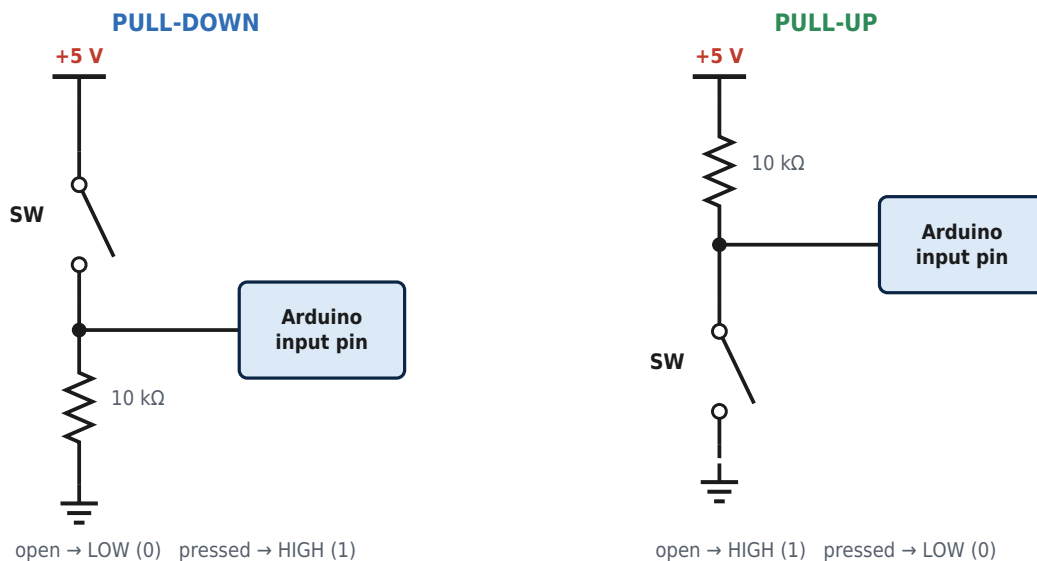


Figure 10.2 Pull-down (left) and pull-up (right) resistors give the input a defined state when the button is open. The resistor also limits the current when the button is pressed: $5\text{ V} \div 10\text{ k}\Omega = 0.5\text{ mA}$.

ANALOGY | SPRING ON A WEATHERVANE

The pin is a **weathervane** that will point wherever the breeze takes it. The pull resistor is a **gentle spring** that returns the vane to a known direction whenever nothing pushes it. A button then pushes it firmly the other way. The spring must be gentle (10 kΩ) so the button can overpower it easily, yet firm enough to beat stray noise.

TIP

The ATmega328P in the Arduino has **built-in pull-up resistors (20-50 k Ω)** that you can enable in code with `pinMode(pin, INPUT_PULLUP)` (Chapter 16). Then you need only connect the button between the pin and GND: no external resistor. It is the standard technique. The logic is inverted (pressed = LOW).

COMMON MISTAKE

A button between the pin and +5 V without any resistor to ground leaves the pin floating when the button is open. Random readings, “ghost presses”. Always add a pull-down or use `INPUT_PULLUP`.

SAFETY WARNING

Never connect a button directly between +5 V and GND (with no resistor in series). Pressing it would short-circuit the supply.

Contact bounce

A mechanical switch is not perfectly clean. When the contacts meet, they **bounce** microscopically for a few milliseconds, making and breaking the connection many times before settling. The Arduino is fast enough to see every bounce and to count one press as several. The solution (**debouncing**) is either a small delay in software (Chapter 16) or a capacitor across the switch (RC filter, Chapter 6). Figure 10.3 shows what an oscilloscope sees.

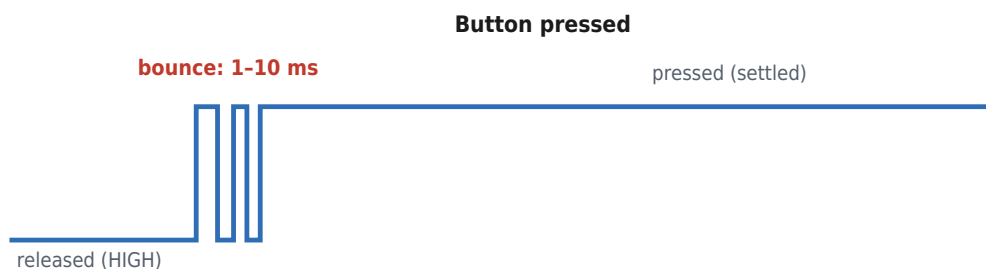


Figure 10.3 Contact bounce: the signal oscillates for a few milliseconds when a switch is pressed.

10.2 Sensors: from the physical world to voltages

A **sensor** (or transducer) converts a physical quantity (light, temperature, distance, force, sound) into an electrical signal the Arduino can read. Most sensors work in one of these ways:

- **Resistive:** the resistance changes with the quantity (LDR for light, thermistor for temperature, flex sensor, force-sensitive resistor). Read them by putting them into a **voltage divider** (Chapter 5) and measuring the divider output with an analog input (Chapter 17).
- **Voltage output:** the sensor produces a voltage proportional to the quantity (LM35, TMP36 temperature sensors; a potentiometer; analog accelerometers).
- **Digital output:** the sensor outputs simply HIGH/LOW (PIR motion sensor, limit switch, comparator modules).
- **Serial output:** a chip inside the sensor does the measurement and sends numbers by a bus: I2C, SPI, 1-Wire (DHT11/22, DS18B20, BMP280, MPU6050, RTC modules).
- **Pulse/timing output:** the sensor’s output timing carries the information (HC-SR04 ultrasonic: pulse width $\propto$ distance).

Quantity	Sensor (example)	Output	Chapter
Light level	LDR (photoresistor); photodiode; phototransistor; BH1750 (I2C lux)	Analog (divider); digital lux	17, 19
Temperature	NTC thermistor 10 k Ω ; LM35 / TMP36; DS18B20; DHT11 / DHT22	Analog; analog; 1-Wire digital; digital	17, 19
Humidity	DHT11 / DHT22; SHT31	Digital	19
Distance	HC-SR04 ultrasonic; VL53L0X laser; Sharp IR	Pulse; I2C; analog	19
Motion / presence	PIR (HC-SR501); microwave RCWL-0516	Digital	19
Acceleration / tilt	ADXL335 (analog); MPU6050 (I2C, also gyro); tilt switch	Analog; I2C; digital	22
Sound	Electret microphone module (KY-038 or MAX4466)	Analog + digital threshold	19
Gas / air quality	MQ-2, MQ-135, MH-Z19 CO ₂	Analog; serial	19
Soil moisture	Capacitive soil sensor	Analog	17
Magnetic field	Hall effect (A3144, SS49E); reed switch	Digital; analog	19
Force / bend	FSR; flex sensor	Resistive $\rightarrow$ analog	17
Position / rotation	Potentiometer; rotary encoder; Hall sensor	Analog; digital pulses	17, 23
Time	DS3231 real-time clock	I2C	22
Touch	Capacitive touch (TTP223), copper pad	Digital	19
Current / voltage	ACS712 (current); resistor divider	Analog	17

Sensor modules

Most sensors are sold as small **breakout modules** on a printed circuit board with the necessary resistors, capacitors and connectors. They typically have three or four pins: **VCC** (5 V or 3.3 V; check!), **GND**, and one or more signal pins (**OUT** or **AO/DO** for analog/digital, **SDA/SCL** for I2C, **TRIG/ECHO** for ultrasonic). Some have a small blue **trimmer potentiometer** for adjusting a threshold, and an LED showing digital status.

TIP

Always check the supply voltage of a module. 3.3 V-only modules (many I2C sensors) can be damaged by 5 V on VCC or on the signal lines; the datasheet or a note on the board tells you. Many breakout boards include a regulator and level shifter and accept 3–5 V; if in doubt look up the exact module.

10.3 Actuators: making things happen

Buzzers and speakers

- **Active buzzer:** contains its own oscillator. Apply DC (5 V) and it beeps at a fixed tone (about 2–4 kHz). Draw ~20–30 mA: connect directly to an Arduino pin (some) or use a transistor. Marked with a “+” on the top.
- **Passive buzzer / speaker:** no oscillator. You must supply an alternating signal: the Arduino `tone(pin, frequency)` function generates any pitch, so you can play melodies (Chapter 18). Use a 100 – 220 Ω series resistor with a small speaker.

DC motors

A DC motor is a coil that rotates in a magnetic field when current flows. Basic facts:

- **Speed is proportional to voltage**, roughly. A 6 V motor at 3 V runs at about half speed. PWM (Chapter 18) varies the average voltage efficiently.
- **Torque (turning force) is proportional to current**. Under load a motor draws more current. When **stalled** (shaft blocked) it draws the **stall current**, often 5 – 10 times the free-running current. Design your driver for stall current!
- **Reversing the polarity reverses the direction**; an **H-bridge** (Chapter 21) does this electronically.
- **Back-EMF**: a spinning motor also acts as a generator, producing a voltage that opposes the supply. That is why the free-running current is low: the net voltage across the winding is $V_{\text{supply}} - V_{\text{back-EMF}}$.
- **Noise**: brushed motors create electrical noise (sparks at the brushes) that can reset an Arduino. Solder a **100 nF ceramic capacitor across the motor terminals**, add a flyback diode, and use a separate supply or large decoupling capacitors.
- **Never power a motor directly from an Arduino pin**. Use a transistor, MOSFET or a motor-driver chip, and a separate motor supply with a common ground.

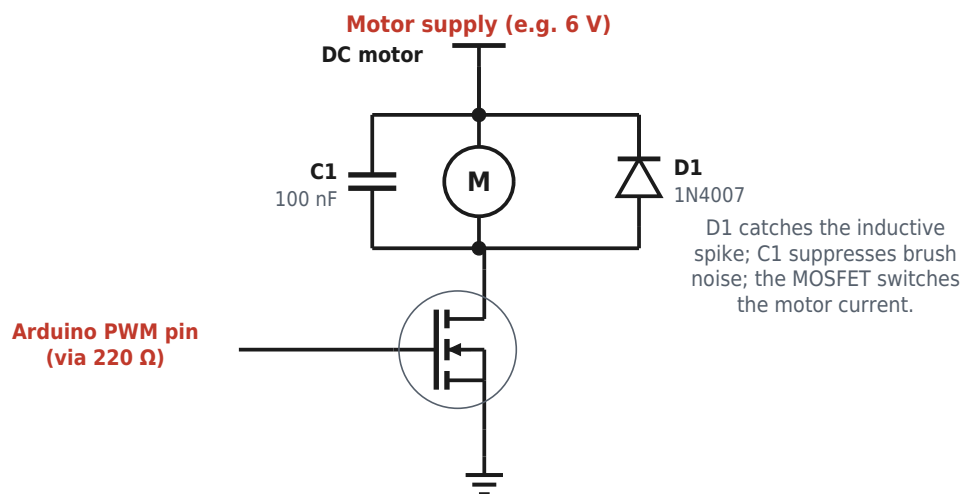


Figure 10.4 A safe way to drive a small DC motor: MOSFET switch, flyback diode across the motor, 100 nF noise capacitor at the terminals.

Servo motors

A **hobby servo** (SG90, MG996R...) is a small geared DC motor, a position sensor (potentiometer) and a control circuit in one case. It has **three wires**: brown/black = GND, red = +5 V, orange/yellow = signal. You tell it the angle you want (0 – 180°) with a pulse and it turns the shaft there and holds it. Timing: a pulse every **20 ms (50 Hz)**; a pulse width of **1 ms = 0°**, **1.5 ms = 90°**, **2 ms = 180°** (the exact limits vary). The Arduino Servo library does this for you (Chapter 18).

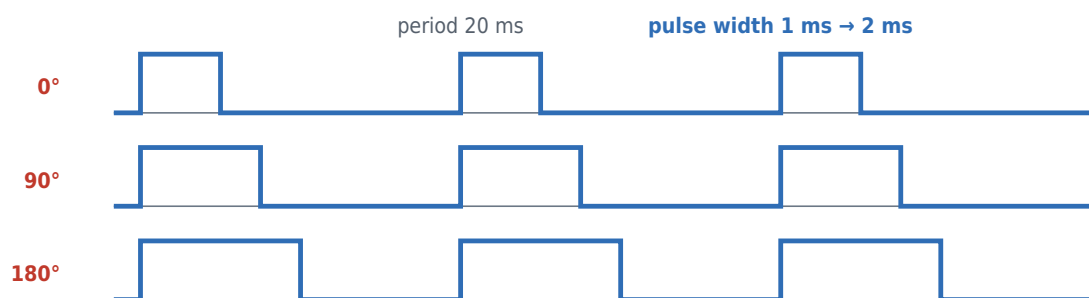


Figure 10.5 Servo control signal: the width of a pulse (1–2 ms) repeated every 20 ms sets the angle.

SAFETY WARNING

A micro-servo draws 100 – 250 mA moving and up to ~700 mA if stalled. **Power servos from a separate 5 V supply** (battery pack or USB adapter) for anything beyond a single SG90, join the grounds, and add a 470 – 1000 μ F electrolytic capacitor across the servo supply. The Arduino's 5 V pin can brown out if a servo pulls too much.

Stepper motors

A **stepper motor** rotates in fixed **steps** (for example $1.8^\circ = 200$ steps per revolution) when its coils are energised in a sequence. It has no feedback sensor, but as long as you do not overload it, it stops exactly where the code says. Ideal for 3D printers, CNC, camera sliders and clocks. The common **28BYJ-48** stepper (5 V, unipolar, 4 coils) with a **ULN2003** driver board gives 2048 steps per revolution (geared). Larger **NEMA 17** bipolar steppers use drivers such as the A4988 or DRV8825.

Actuator	Controls	Typical current	Driver needed	Best for
LED	Light	5 – 20 mA	Resistor	Indicators
Active buzzer	Sound (fixed tone)	20 – 30 mA	Pin or transistor	Alarms, beeps
Passive buzzer / speaker	Sound (any tone)	10 – 50 mA	Pin via resistor / transistor	Melodies, sound effects
DC motor	Speed, direction	100 mA – several A	MOSFET or H-bridge	Wheels, fans, pumps
Servo	Angle (0 – 180°)	100 – 700 mA	Signal pin + external 5 V	Robot arms, steering, flaps
Stepper	Precise angle / speed	200 mA – 2 A	ULN2003 / A4988 driver	Positioning, 3D printing
Relay	Switching big loads	40 – 90 mA coil	Transistor + diode (module)	Lamps, pumps, appliances
Solenoid	Linear push / pull	200 mA – 2 A	MOSFET + diode	Locks, valves

HANDS-ON EXPERIMENT | EXPERIMENT 6: READING A SWITCH PROPERLY (NO ARDUINO NEEDED YET)

You need: breadboard, 5 V source (or 3 AA cells), tactile button, 10 k Ω , 470 Ω , LED, multimeter.

1. Build the **pull-down** circuit of Figure 10.2 (left) with a multimeter connected between the input node and ground. Read the voltage with the button open (about 0 V) and pressed (about 5 V).
2. Now remove the pull-down resistor and repeat. Touch the floating input node with your finger: the meter drifts; it is a floating input.
3. Build the **pull-up** version (right) and confirm the inverted logic.

4. Compute the current in the 10 k Ω pull resistor when the button is pressed ($5\text{ V} / 10\text{ k}\Omega = 0.5\text{ mA}$), and verify with the meter in mA mode. Compare with what a 470 Ω would have drawn (10.6 mA): the larger the resistor, the less power is wasted.

CHAPTER SUMMARY

- Switches: SPST, SPDT, DPDT; momentary versus latching. A tactile button's pins are joined in pairs.
- A switch input must have a defined state: use a **pull-up** or **pull-down** (10 k Ω); or enable the Arduino's internal pull-up (INPUT_PULLUP, pressed = LOW). A floating input reads random values. Buttons also **bounce** for a few milliseconds.
- Sensors output analog voltages, digital levels, timed pulses or serial data; buy modules and check their supply voltage (3.3 V vs 5 V).
- DC motors: speed $\propto$ voltage, torque $\propto$ current; stall current is large; add a flyback diode and a noise capacitor; use a transistor/MOSFET/H-bridge and a separate supply with common ground.
- Servos take a 1 – 2 ms pulse every 20 ms and hold an angle; steppers rotate in exact steps. Give both a proper power source.

ASSIGNMENTS | CHAPTER 10

10.1 [Practice] You connect a button between an Arduino input pin and GND with no other components, and read the pin in code. The reading flickers between HIGH and LOW even when the button is not pressed. Explain and give two fixes.

10.2 [Calculate] A 10 k Ω pull-up is connected to 5 V and a button pulls it to ground. What current flows through the resistor when the button is pressed? What power is dissipated?

10.3 [Calculate] Suppose the pull-up in the previous question were 100 Ω instead. What current and power result? Why is 100 Ω a poor choice?

10.4 [Calculate] A 28BYJ-48 stepper has 2048 steps per revolution. How many steps rotate the shaft by 90°? By 10°?

10.5 [Calculate] A servo expects a pulse from 1 ms (0°) to 2 ms (180°). What pulse width would you send to go to 45°? To 135°?

10.6 [Calculate] A DC motor draws 150 mA free-running and 900 mA stalled at 6 V. You use a MOSFET with $R_{DS(on)} = 0.05\text{ }\Omega$. What power does the MOSFET dissipate at stall?

10.7 [Practice] Identify a suitable sensor and output type for each: (a) is it dark outside? (b) is someone in the room? (c) how far is the wall? (d) what is the room temperature in °C?

10.8 [Think] Why should motors be powered from a supply separate from the Arduino's 5 V pin, and why must the grounds still be connected?

10.9 [Think] Explain why the stall current is so much higher than the running current.

10.10 [Hands-on] Do Experiment 6 and record voltages for the pull-up and pull-down circuits with the button open and pressed. Try touching the floating input and record the maximum reading you see.

Op-amps and the 555 timer

Not everything can (or should) be solved in software. Sensor signals are often too small, too noisy or the wrong shape for the Arduino's input. Sometimes you need a blinking light or a delay with no code at all. Two classic chips solve these problems and have been in production for over fifty years: the operational amplifier and the 555 timer. You will meet them in sensor modules, alarms, audio circuits and countless hobby projects, so it is worth understanding both.

WHAT YOU WILL LEARN IN THIS CHAPTER

- What an **operational amplifier (op-amp)** is and the two golden rules that make it easy to use
- The basic op-amp circuits: **follower, non-inverting amplifier, inverting amplifier and comparator**
- How to condition a sensor signal so an Arduino can read it
- How the classic **555 timer** works in monostable and astable modes, with design formulas
- When to reach for an analog chip instead of code

11.1 The operational amplifier

An **operational amplifier (op-amp)** is a small integrated circuit that amplifies the **difference** between two input voltages by a very large factor (100,000 or more). It has two inputs, the **non-inverting input (+)** and the **inverting input (-)**, one output, and power supply pins. The **LM358** (dual op-amp, works from a single 3 – 32 V supply) and **LM324** (quad) suit Arduino projects; the **741** and **TL072** appear in older designs and audio.

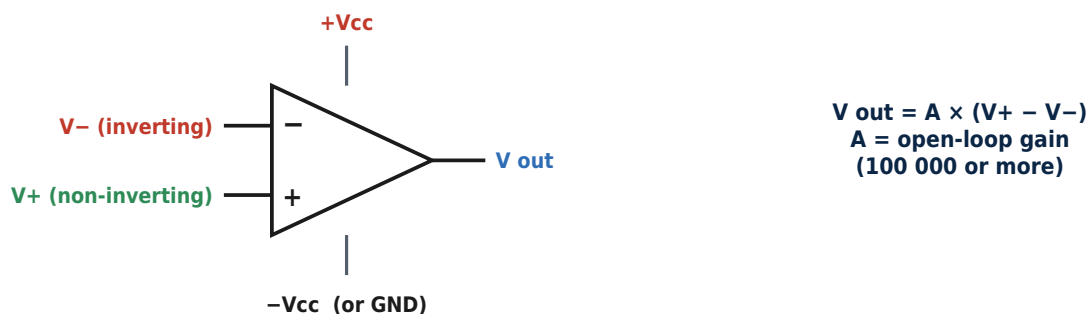


Figure 11.1 The op-amp symbol. The output is the huge gain A times the difference between the $+$ and $-$ inputs, limited by the supply range.

With a gain of 100,000, a difference of only 0.1 mV between the inputs would try to produce 10 V. Used **alone** (no feedback), an op-amp simply swings its output to one supply rail or the other: high if $V_{+} > V_{-}$, low if $V_{+} < V_{-}$. That is a **comparator**. The real power comes with **negative feedback**: connect part of the output back to the inverting input. The op-amp then adjusts its output continuously to make the two inputs equal. That gives the two “golden rules” for negative-feedback circuits:

THE TWO GOLDEN RULES (with negative feedback)

- **Rule 1:** the output does whatever it takes to make $V_+ = V_-$ (the two inputs sit at the same voltage).
- **Rule 2:** the inputs draw **no current** (their input resistance is enormous).

From these two rules alone you can analyse every op-amp amplifier in this chapter without advanced mathematics.

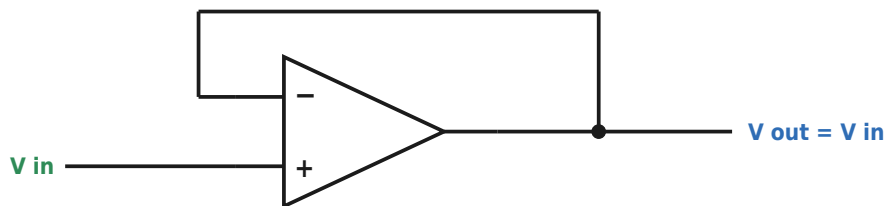
ANALOGY | STEERING WITH FEEDBACK

Imagine steering a car whose wheel is so sensitive that the slightest twist swings the car wildly. Steering blind, you could not drive straight. But if you watch the road and correct continuously (feedback), you can hold the car on any line you choose. The op-amp does the same: it compares where the output is (through the feedback network) with where you want it (the + input), and drives the output until the difference is zero.

11.2 The basic circuits

1. The voltage follower (buffer)

Connect the output directly to the – input. Rule 1 says $V_- = V_+$, so $V_{\text{out}} = V_{\text{in}}$: a gain of exactly 1. Why bother? Because the input draws almost no current, while the output can supply many milliamps. A follower **isolates** a weak source from a load. Remember the voltage divider that collapsed under load (Chapter 5)? Put a buffer after it and the load no longer matters.



Gain = 1, high input resistance, low output resistance

Figure 11.2 Voltage follower (unity-gain buffer).

2. The non-inverting amplifier

A voltage divider (R_1 to ground, R_2 from the output) feeds part of the output back to the – input. Rule 1 forces $V_- = V_{\text{in}}$. The divider gives $V_- = V_{\text{out}} \times R_1 \div (R_1 + R_2)$. Setting them equal and solving:

$$A_V = V_{\text{out}} / V_{\text{in}} = 1 + R_2 / R_1$$

Non-inverting amplifier: gain ≥ 1 , output in phase with input

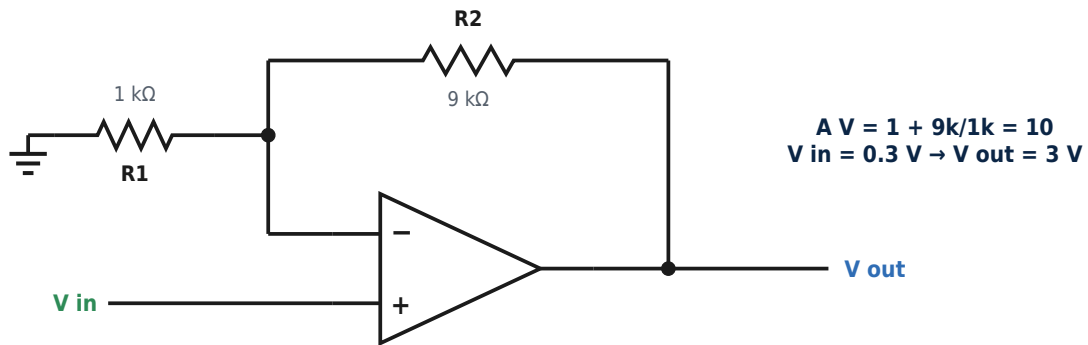


Figure 11.3 Non-inverting amplifier. R_1 and R_2 set the gain. The output can never exceed the supply voltage.

WORKED EXAMPLE | SCALING A SENSOR FOR THE ARDUINO

A sensor outputs 0 – 1 V but you want to use the Arduino’s full 0 – 5 V ADC range (better resolution). Needed gain = 5.

Gain = $1 + R_2/R_1 = 5 \rightarrow R_2/R_1 = 4$. Choose $R_1 = 10\text{ k}\Omega$, $R_2 = 39\text{ k}\Omega$ (E12) $\rightarrow$ gain 4.9 (max output 4.9 V) ✓.

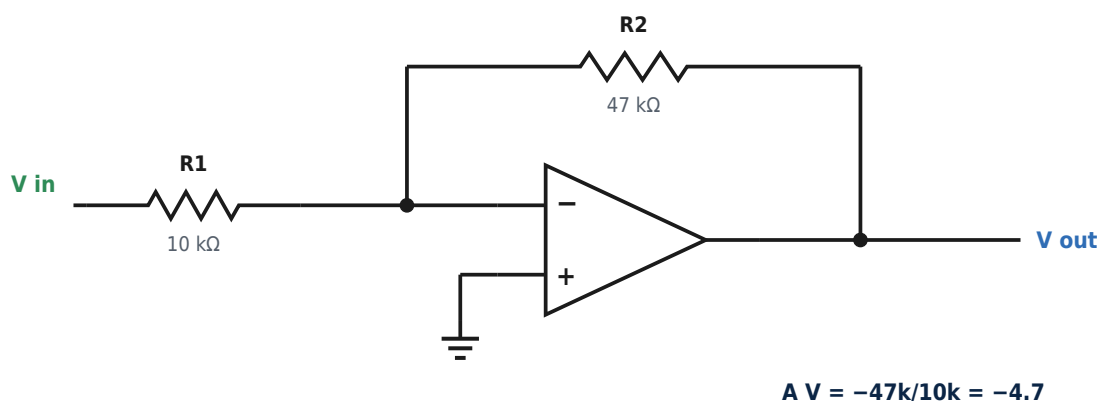
Remember: an LM358 running from 5 V cannot swing all the way to 5 V: its output tops out near **3.5 V** (about 1.5 V below the supply). For full range use a **rail-to-rail** op-amp such as MCP6002 or LMV358.

3. The inverting amplifier

Here the input goes through R_1 to the – input, the + input is grounded, and R_2 feeds the output back to the – input. Rule 1 makes the – input a “virtual ground” (0 V, since $V_+ = 0$). So the input current is V_{in}/R_1 ; by Rule 2 it all flows through R_2 , giving a voltage drop across R_2 of $(V_{in}/R_1) \times R_2$; the output is at 0 minus this:

$$A_V = V_{out} / V_{in} = - R_2 / R_1$$

Inverting amplifier: gain can be any value, and the output is inverted



The – input is a “virtual ground”: it is held at 0 V by the feedback.

Figure 11.4 Inverting amplifier. Gain is set entirely by R_2/R_1 ; the negative sign means the output is upside-down relative to the input.

GOOD TO KNOW

With a single 5 V supply an inverting amplifier cannot produce negative outputs, so it needs a second (negative) supply or a bias voltage at the + input (a “virtual ground” at half the supply, made by two equal resistors and a buffer). Audio circuits do this all the time.

4. The comparator

With **no feedback**, the huge gain saturates the output: if $V_+ > V_-$ the output goes to the positive rail, otherwise to the negative rail (or 0 V). The op-amp thus **compares** two voltages and tells you which is larger with a digital-looking output. Typical uses: light-activated switches (LDR against a reference), over-temperature alarms, battery low detectors. Modules like the “LDR sensor board” and “sound sensor KY-038” include an LM393 comparator plus a trimmer for the threshold.

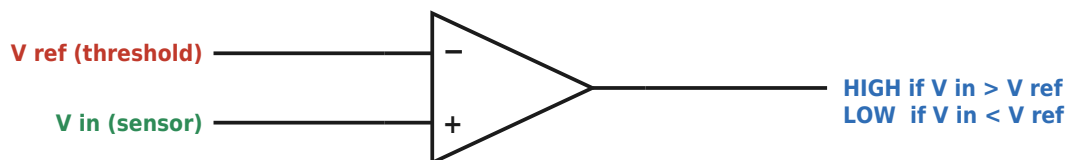


Figure 11.5 An op-amp (or dedicated comparator such as LM393) used as a comparator.

TIP

A plain comparator chatters (rapidly flips) when the input hovers near the threshold. Adding a little positive feedback creates **hysteresis** (a **Schmitt trigger**): the rising and falling thresholds differ, so the output switches cleanly. The Arduino’s own inputs are Schmitt-triggered internally.

Practical op-amp notes

- **Supply and range.** A general op-amp’s output cannot reach the rails. LM358: within ~ 1.5 V of the positive rail, but down to about 0 V. Rail-to-rail types (MCP6002, LMV358, TLV2372) reach within millivolts.
- **Bandwidth.** Gain $\times$ bandwidth is roughly constant (a 1 MHz op-amp with gain 100 gives only 10 kHz). Fine for sensors, limited for fast signals.
- **Decoupling.** Put a 100 nF capacitor between the supply pins and ground, close to the chip.
- **Input range.** Inputs of most op-amps must stay within the supply range. Do not exceed it.
- **Other circuits.** Adders (summing), differential amplifiers, instrumentation amplifiers (for bridge sensors: INA125, HX711 for load cells), active filters, integrators and oscillators are all built from the same two rules.

REAL-LIFE EXAMPLE | YOUR HEADPHONES AND YOUR FITNESS TRACKER

A microphone produces a few millivolts. An op-amp amplifies it thousands of times so a digital converter can read it. A heart-rate sensor's pulse signal is tiny and full of noise; an op-amp stage amplifies and filters it. Pretty much every analog sensor in a product has an op-amp behind it.

11.3 The 555 timer

The **NE555** (introduced in 1971 by Hans Camenzind at Signetics) is probably the most popular integrated circuit ever made, with billions produced. It contains two comparators, a flip-flop (a one-bit memory, Chapter 12), a discharge transistor and a voltage divider of three 5 k Ω resistors (hence the name). With just a resistor or two and a capacitor, it creates precise time delays and oscillations. It works from 4.5 – 15 V (the CMOS versions **TLC555** / **LMC555** / **7555** run from as low as 1.5 V and draw much less current).

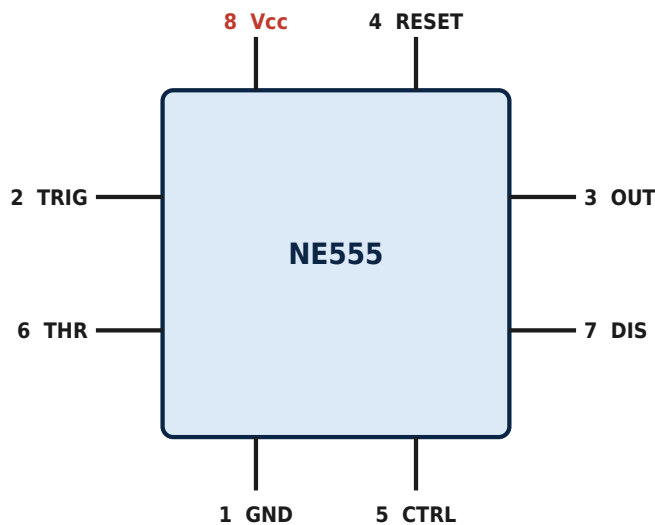


Figure 11.6 The eight pins of the 555 (schematic arrangement). Pin 4 (RESET) is tied to Vcc when unused; pin 5 gets a 10 nF capacitor to ground for noise immunity.

Pin	Name	Function
1	GND	Ground (0 V)
2	TRIGGER	When this input falls below 1/3 Vcc , the output goes HIGH and timing starts
3	OUTPUT	Output; can source or sink up to ~200 mA (NE555). Drives LEDs and small relays directly
4	RESET	Active LOW: pulling it to ground forces the output LOW. Tie to Vcc if unused
5	CONTROL	Access to the internal 2/3 Vcc reference. Normally 10 nF to ground
6	THRESHOLD	When this input rises above 2/3 Vcc , the output goes LOW
7	DISCHARGE	Connects to ground when the output is LOW: discharges the timing capacitor
8	Vcc	Supply +4.5 to +15 V (NE555)

Inside, three equal resistors set two thresholds at **1/3 Vcc** and **2/3 Vcc**. The comparators watch pins 2 and 6 and set or reset the flip-flop. The timing capacitor charges through your resistors between those thresholds, and that is what makes the timing.

Monostable mode: a one-shot pulse (a timer)

Give the 555 a brief trigger pulse (pin 2 pulled below $1/3 V_{cc}$) and the output goes HIGH for a fixed time, then returns LOW by itself, even if the trigger is still present or has gone away. The length of the pulse depends on one resistor R and one capacitor C :

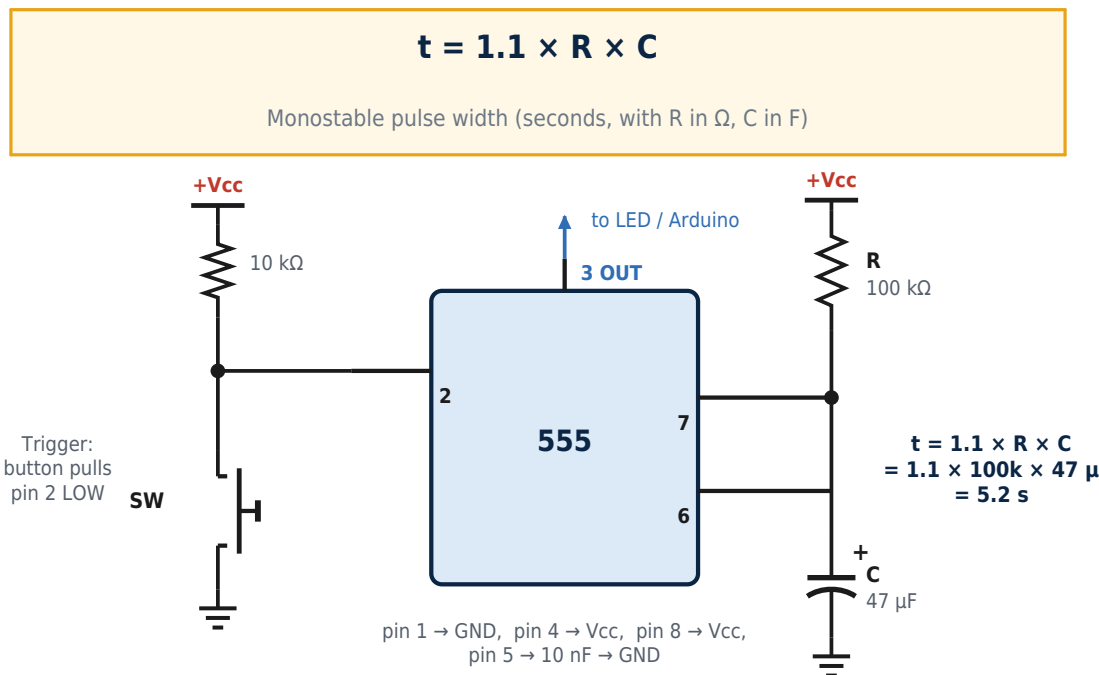


Figure 11.7 Monostable connections (summary diagram): timing resistor R from V_{cc} to pins 6 and 7 joined; timing capacitor C from those pins to ground; trigger on pin 2 with a pull-up and a button to ground.

WORKED EXAMPLE | A 5-SECOND DELAY

Choose $C = 47 \mu\text{F}$, then $R = t \div (1.1 C) = 5 \div (1.1 \times 47 \times 10^{-6}) = 96.7 \text{ k}\Omega$. The nearest standard value is **100 kΩ**, giving $t = 1.1 \times 100,000 \times 47 \times 10^{-6} = \mathbf{5.17 \text{ s}}$.

Electrolytic capacitors have wide tolerance (-20% , $+80\%$), so real timings vary by $\pm 20\%$ or more. For accuracy use a film capacitor or a microcontroller.

REAL-LIFE EXAMPLE | STAIR-LIGHTS AND HAND DRYERS

A hand-dryer runs for a fixed 30 s after you press the button, then stops: a monostable. So does the stair light with a “timer”. In the old days, this was a 555; now it is often a microcontroller.

Astable mode: a free-running oscillator

Wire the 555 with **two resistors and a capacitor** and it oscillates continuously: the capacitor charges through $R_1 + R_2$ to $2/3 V_{cc}$, then discharges through R_2 to $1/3 V_{cc}$, forever. The output is a square wave. Design formulas:

$$f = 1.44 \div ((R_1 + 2 R_2) \times C)$$

Frequency (Hz)

$$t_{\text{HIGH}} = 0.693 \times (R_1 + R_2) \times C$$

Output HIGH time (seconds)

$$t_{\text{LOW}} = 0.693 \times R_2 \times C$$

Output LOW time (seconds)

$$\text{Duty cycle} = (R_1 + R_2) \div (R_1 + 2 R_2)$$

Always more than 50 % (use a diode across R2 to reach 50 %)

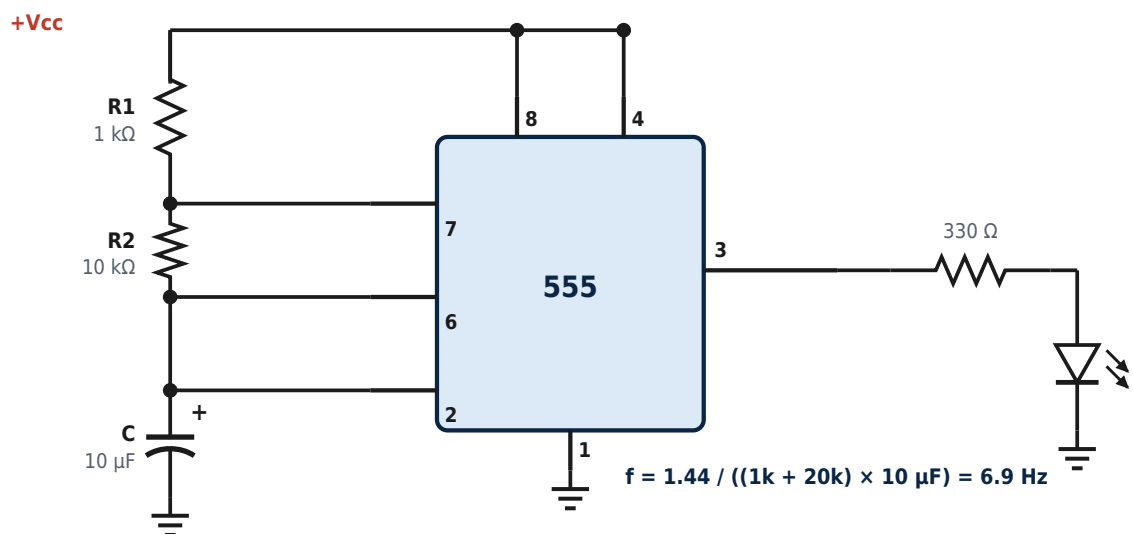


Figure 11.8 Astable 555: a blinker. (Pin 5 should also have 10 nF to ground; omitted for clarity.) The LED blinks about 7 times a second.

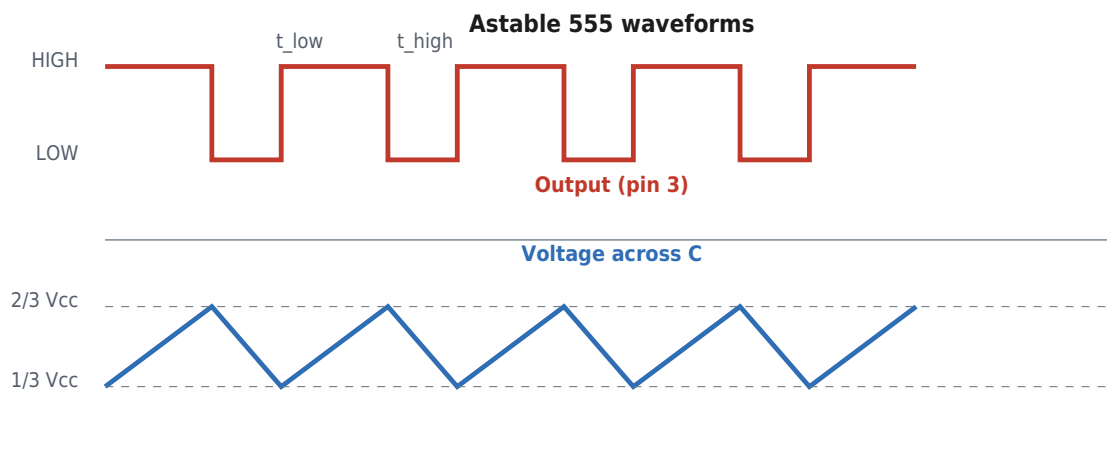


Figure 11.9 The capacitor voltage ramps between 1/3 and 2/3 of the supply; the output toggles each time a threshold is reached.

WORKED EXAMPLE | DESIGNING A 1 KHZ TONE GENERATOR

Choose $C = 10 \text{ nF}$ and $R_1 = 1 \text{ k}\Omega$. Then $R_1 + 2R_2 = 1.44 \div (f C) = 1.44 \div (1000 \times 10^{-8}) = 144 \text{ k}\Omega$. So $R_2 = (144 - 1) \div 2 = 71.5 \text{ k}\Omega \rightarrow \mathbf{68 \text{ k}\Omega}$ (E12) gives $f = 1.44 \div (137 \text{ k}\Omega \times 10 \text{ nF}) = \mathbf{1.05 \text{ kHz}}$.

Connect a small speaker in series with a 100Ω resistor and a $10 \mu\text{F}$ capacitor to the output: you hear a buzz. This was the circuit of countless electronic toys.

WORKED EXAMPLE | A SLOW BLINKER

$R_1 = 1 \text{ k}\Omega$, $R_2 = 10 \text{ k}\Omega$, $C = 10 \mu\text{F}$: $f = 1.44 \div (21 \text{ k}\Omega \times 10 \mu\text{F}) = \mathbf{6.86 \text{ Hz}}$; $t_{\text{HIGH}} = 0.693 \times 11 \text{ k}\Omega \times 10 \mu\text{F} = 76 \text{ ms}$; $t_{\text{LOW}} = 69 \text{ ms}$; duty = $11/21 = 52 \%$.

TIP

Can I do this with an Arduino? Of course, and for most projects the microcontroller's `millis()` and `tone()` functions are far more flexible. The 555 shines when you need a timing function with **no code**, at very low cost, that keeps working even if the microcontroller hangs (a “watchdog”), or to learn how timing works in hardware.

HANDS-ON EXPERIMENT | EXPERIMENT 7: A 555 LED BLINKER

You need: a 555 (NE555 or TLC555) in DIP-8, a 9 V battery (or 5 V), resistors $1 \text{ k}\Omega$, $10 \text{ k}\Omega$, 330Ω , capacitors $10 \mu\text{F}$ (electrolytic) and 10 nF (ceramic 103), red LED, breadboard.

1. Place the IC across the centre gap. Notch/dot marks pin 1; count pins counter-clockwise from there (pins 1-4 down the left side, 5-8 up the right).
2. Wire as in Figure 11.8: pin 8 and pin 4 to +V; pin 1 to ground; R_1 ($1 \text{ k}\Omega$) from +V to pin 7; R_2 ($10 \text{ k}\Omega$) from pin 7 to pin 6; a link between pins 6 and 2; C ($10 \mu\text{F}$, stripe to ground) from pin 2 to ground; 10 nF from pin 5 to ground; output pin 3 $\rightarrow 330 \Omega \rightarrow \text{LED} \rightarrow \text{ground}$.
3. Power up. The LED should blink at about 7 Hz. Measure the voltage across C with the meter: it should wobble between $\approx 1/3$ and $2/3$ of the supply (slow enough to see with a fast meter; try $100 \mu\text{F}$ for a slower blink).
4. Change C to $100 \mu\text{F}$: the blink slows by a factor of ten. Change R_2 to $47 \text{ k}\Omega$: slower again. Verify against the formula.
5. **Challenge:** replace R_2 by a $100 \text{ k}\Omega$ potentiometer in series with $1 \text{ k}\Omega$ to make a blink-rate knob.

CHAPTER SUMMARY

- An op-amp amplifies the difference between its inputs by a huge factor. With negative feedback: **$V_+ = V_-$** and **no input current**.
- **Follower:** gain 1 (buffer). **Non-inverting:** $A = 1 + R_2/R_1$. **Inverting:** $A = -R_2/R_1$. **Comparator:** no feedback; output is HIGH or LOW.
- Watch supply limits: an LM358 output cannot reach the positive rail; use rail-to-rail parts for full-range work.
- **555 monostable:** one pulse of length $t = 1.1 R C$. **555 astable:** $f = 1.44 / ((R_1 + 2R_2) C)$; duty = $(R_1 + R_2) / (R_1 + 2R_2)$.
- Always give the 555 pin 4 $\rightarrow V_{\text{cc}}$ and a 10 nF capacitor on pin 5, and decouple the supply.

ASSIGNMENTS | CHAPTER 11

11.1 [Calculate] A non-inverting amplifier has $R_1 = 2.2 \text{ k}\Omega$ and $R_2 = 22 \text{ k}\Omega$. What is its gain? What is V_{out} for $V_{\text{in}} = 0.25 \text{ V}$?

11.2 [Calculate] Design a non-inverting amplifier with a gain of 6 using $R_1 = 10 \text{ k}\Omega$. Which standard E24 value would you use for R_2 ?

11.3 [Calculate] An inverting amplifier uses $R_1 = 4.7 \text{ k}\Omega$ and $R_2 = 47 \text{ k}\Omega$. Find the gain. If $V_{\text{in}} = -0.3 \text{ V}$, what is V_{out} ? (Assume a dual supply.)

11.4 [Calculate] A thermistor divider gives 0.8 V at 25°C and 1.1 V at 40°C . You want the output to span roughly $0 - 5 \text{ V}$ for that change. Explain what must be done (you need an offset and gain: give the gain required).

11.5 [Calculate] An LM358 runs from 5 V . What is the highest output voltage you can expect? Would a gain-5 amplifier for a $0-1 \text{ V}$ sensor work?

11.6 [Calculate] A 555 in monostable mode has $R = 220 \text{ k}\Omega$ and $C = 4.7 \text{ }\mu\text{F}$. How long is the output pulse?

11.7 [Calculate] You want a 10-second monostable delay using a $100 \text{ }\mu\text{F}$ capacitor. Find R and choose a standard value.

11.8 [Calculate] A 555 astable uses $R_1 = 4.7 \text{ k}\Omega$, $R_2 = 47 \text{ k}\Omega$, $C = 0.1 \text{ }\mu\text{F}$. Find f , t_{HIGH} , t_{LOW} and duty cycle.

11.9 [Calculate] Design a 555 astable that flashes an LED once per second (1 Hz). Use $C = 100 \text{ }\mu\text{F}$ and $R_1 = 1 \text{ k}\Omega$. Find R_2 .

11.10 [Think] Why does an op-amp with no feedback act as a comparator rather than an amplifier?

11.11 [Think] In the inverting amplifier, why is the $-$ input called a “virtual ground”?

11.12 [Hands-on] Build Experiment 7 and record the blink frequency for three different C values. Compare each with the formula. How large is the error and what could cause it?

Digital electronics and logic

Everything so far has dealt with voltages that can take any value: **analog** electronics. Computers, phones and the Arduino, however, work with only two states: on and off. This is **digital electronics**. It is easier to build, extremely resistant to noise, and forms the basis of every microcontroller. In this chapter you learn how digital circuits represent numbers and make decisions, and how the digital and analog worlds are connected. The Arduino's `digitalWrite()`, `digitalRead()` and `analogRead()` functions will then make complete sense.

WHAT YOU WILL LEARN IN THIS CHAPTER

- The difference between **analog** and **digital**, and why digital is so robust
- The **binary** and **hexadecimal** number systems and how to convert between them
- **Logic levels** (HIGH/LOW), thresholds and noise margin; 5 V versus 3.3 V logic
- **Logic gates** (AND, OR, NOT, NAND, NOR, XOR), truth tables and Boolean expressions
- Memory and counting: latches, flip-flops, counters and shift registers
- How **analog-to-digital (ADC)** and **digital-to-analog (DAC)** conversion work, including the Arduino's 10-bit ADC

12.1 Analog versus digital

An **analog** signal can have any value in a continuous range: the voltage from a microphone, the temperature reading of a thermistor, the position of a potentiometer. A **digital** signal has only a finite set of values, in practice two: **HIGH** (logic 1, near the supply voltage) and **LOW** (logic 0, near ground).

ANALOGY | DIMMER VERSUS ON/OFF SWITCH

Compare a **dimmer** switch with an **ordinary** light switch. The dimmer offers infinitely many brightness levels (analog); the switch is either on or off (digital). Or compare a traditional clock with continuously sweeping hands to a digital clock which shows one of 1440 distinct times.

Digital wins on **reliability**: a HIGH signal that has been degraded by noise from 5 V to 4.2 V is still clearly a HIGH, and the receiving chip regenerates a perfect 5 V. An analog signal that has picked up 0.8 V of noise is permanently changed. That is why digital music, photos and communications can be copied endlessly without loss.

	Analog	Digital
Values	Continuous range (0–5 V, any value)	Two levels (0 or 5 V); numbers built from many bits
Noise	Adds directly to the signal	Ignored if small; restored at each stage
Storage / copying	Degrades each copy	Perfect copies
Examples	Microphone, LDR, thermistor, potentiometer, speaker	Button, LED on/off, logic chips, microcontroller pins, I2C/SPI data
Arduino functions	<code>analogRead()</code> , <code>analogWrite()</code> (PWM)	<code>digitalRead()</code> , <code>digitalWrite()</code>

12.2 Binary numbers

Humans count in **decimal** (base 10) with ten digits (0 – 9): each position is worth ten times the one on its right (ones, tens, hundreds...). A digital circuit has only two symbols, so it counts in **binary** (base 2) with the digits **0** and **1**; each position is worth **two times** the one to its right. One binary digit is a **bit**. Eight bits make a **byte**.

Bit position	7	6	5	4	3	2	1	0
Weight (2^n)	128	64	32	16	8	4	2	1
Example bits	1	0	1	1	0	1	0	1

To convert binary to decimal, **add the weights** of the positions that hold a 1. The example above: $128 + 32 + 16 + 4 + 1 = 181$. To convert decimal to binary, repeatedly subtract the biggest weight that fits (or divide by 2 and record the remainders).

WORKED EXAMPLE | DECIMAL ↔ BINARY

Binary 01101100 → decimal: $64 + 32 + 8 + 4 = 108$.

Decimal 200 → binary: $200 - 128 = 72$ (bit 7 = 1); $72 - 64 = 8$ (bit 6 = 1); bit 5 = 0, bit 4 = 0; $8 - 8 = 0$ (bit 3 = 1); the rest 0 → **11001000**.

Range: n bits can represent 2^n different values, from 0 to $2^n - 1$. 8 bits: 0 – 255. 10 bits: 0 – 1023. 16 bits: 0 – 65,535.

Hexadecimal: binary's shorthand

Long strings of bits are hard to read, so programmers group them in fours and write each group as one **hexadecimal** (base 16) digit: 0 – 9 then A = 10, B = 11, C = 12, D = 13, E = 14, F = 15. So one byte = two hex digits. In code, hex numbers are written with a **0x** prefix and binary numbers with **0b**.

Decimal	0	1	2	3	4	5	6	7
Binary	0000	0001	0010	0011	0100	0101	0110	0111
Hex	0	1	2	3	4	5	6	7

Decimal	8	9	10	11	12	13	14	15
Binary	1000	1001	1010	1011	1100	1101	1110	1111
Hex	8	9	A	B	C	D	E	F

WORKED EXAMPLE | HEX EXAMPLES

0xB5 = 1011 0101 = 181 (the same number as above). **0xFF** = 255. **0x3C** = 0011 1100 = 60. **0x20** (a typical I2C address) = 32.

REAL-LIFE EXAMPLE | WHERE YOU WILL MEET BINARY IN THIS BOOK

Arduino port registers (**PORTB = 0b00100000**; sets pin 13 HIGH), I2C addresses (0x27 for an LCD backpack, 0x68 for the MPU6050/RTC), shift registers (sending 8 bits to control 8 LEDs), and the ADC value (a 10-bit number).

12.3 Logic levels and noise margin

A digital input decides HIGH or LOW by comparing its voltage with **thresholds**. Real chips guarantee: any voltage **below** V_{IL} is read as LOW; any voltage **above** V_{IH} is read as HIGH; the region in between is **undefined** (the chip may read either; avoid it). Likewise, outputs guarantee at least V_{OH} when HIGH and at most V_{OL} when LOW. The gaps between output and input limits are the **noise margins**, which allow noise to be tolerated.

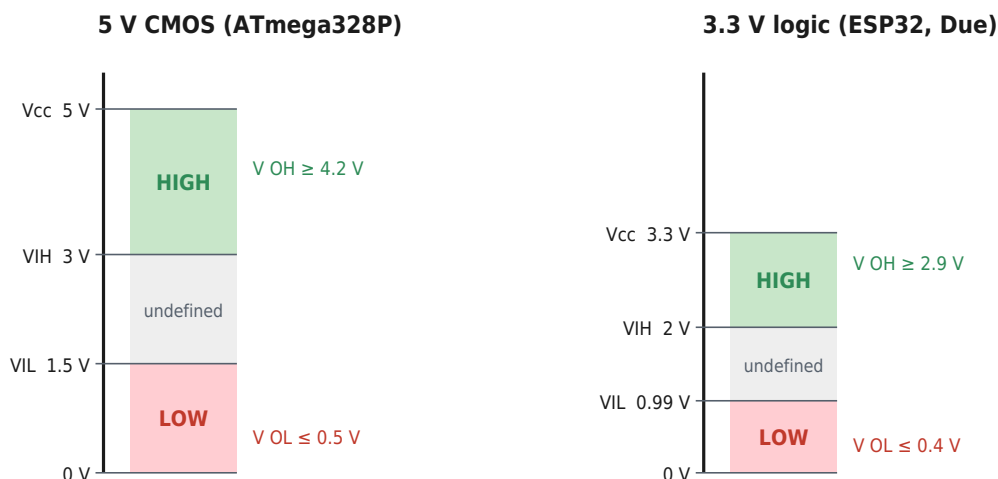


Figure 12.1 Logic thresholds. The Arduino Uno (ATmega328P at 5 V) reads anything above about 3.0 V as HIGH and below 1.5 V as LOW. Values are typical; check the datasheet.

SAFETY WARNING

Never connect a 5 V output directly to a 3.3 V-only input. Sensors, ESP32/ESP8266 boards, Raspberry Pi and many I2C modules are 3.3 V devices. Their input pins may be damaged by 5 V. Use a **level shifter** (bidirectional MOSFET boards for I2C, or a simple two-resistor divider for a one-way signal from 5 V to 3.3 V: 1 k Ω + 2 k Ω gives 3.33 V). Going the other way, a 3.3 V output is usually read as HIGH by a 5 V Arduino input (> 3.0 V), which is fine for most parts, but marginal.

Logic family	Supply	V_{IL} (max LOW)	V_{IH} (min HIGH)	Notes
TTL (74LS, 74xx)	5 V	0.8 V	2.0 V	Old bipolar logic; inputs float HIGH; heavier current draw
CMOS (4000, 74HC)	2 – 6 V (HC)	0.3 Vcc	0.7 Vcc	Low power; very high input resistance; unused inputs must be tied
74HCT	5 V	0.8 V	2.0 V	CMOS with TTL-compatible thresholds; convenient level-shifting up from 3.3 V
ATmega328P (Arduino Uno)	5 V	0.3 Vcc = 1.5 V	0.6 Vcc = 3.0 V	Output: ± 20 mA per pin (40 mA absolute max)

TIP

CMOS inputs are extremely high-impedance and **float** if left unconnected: they pick up noise and can draw excess current (even oscillate). **Always tie unused inputs of a CMOS gate to +V or ground** (or to another used input).

12.4 Logic gates

A **logic gate** is a small circuit that computes a simple decision from one or more digital inputs. Inside a gate are transistors (Chapter 9); a chip like the 74HC00 has four NAND gates in one package. Every gate has a **symbol**, a **truth table** (listing the output for every possible input combination) and a **Boolean expression** (algebra with 0 and 1).

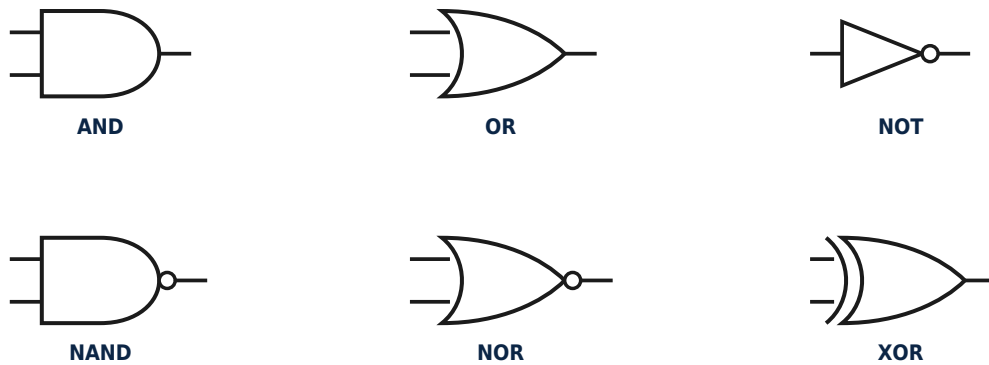


Figure 12.2 The six basic gate symbols. A small circle (“bubble”) on the output means inversion (NOT).

A	B	AND $A \cdot B$	OR $A + B$	NAND	NOR	XOR $A \oplus B$	NOT A
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	1
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	0

- **AND:** output is 1 only when **all** inputs are 1. (“The lamp is on only if the switch AND the door sensor are closed.” Two switches in **series**.)
- **OR:** output is 1 when **any** input is 1. (Two switches in **parallel**: either turns the lamp on.)
- **NOT (inverter):** output is the opposite of the input.
- **NAND = NOT AND; NOR = NOT OR. NAND and NOR are “universal”: any logic function can be built from NAND gates alone.**
- **XOR (exclusive OR):** output is 1 when the inputs are **different**. Used in adders and comparators; also a “toggle” when one input is 1.

In C/C++ (Arduino) you use the same ideas: `&&` (AND), `||` (OR), `!` (NOT) on conditions, and `&`, `|`, `^`, `~` on individual bits.

Boolean algebra: three rules worth knowing

- **A AND 1 = A; A AND 0 = 0; A OR 0 = A; A OR 1 = 1.**
- **De Morgan’s laws:** $\text{NOT}(A \text{ AND } B) = (\text{NOT } A) \text{ OR } (\text{NOT } B)$, and $\text{NOT}(A \text{ OR } B) = (\text{NOT } A) \text{ AND } (\text{NOT } B)$. They let you swap gate types (which is how NAND-only designs are made).
- **Distribution:** $A \cdot (B + C) = A \cdot B + A \cdot C$.

WORKED EXAMPLE | DESIGN A DOOR-ALARM LOGIC

An alarm should sound (output = 1) when the system is **armed** ($A = 1$) **and** either the **door** sensor (D) **or** the **window** sensor (W) is open ($= 1$).

Expression: Alarm = $A \cdot (D + W)$. Build with one OR gate (D, W) feeding one AND gate (with A).

Truth table check: $A = 1, D = 0, W = 1 \rightarrow \text{alarm} = 1 \cdot (0 + 1) = 1 \checkmark$. $A = 0, D = 1, W = 1 \rightarrow 0 \checkmark$ (disarmed).

On the Arduino the same logic is one line: `if (armed && (door || window)) digitalWrite(BUZZER, HIGH);`.

Combinational circuits: gates that calculate

Combinations of gates without memory, whose outputs depend only on present inputs, are **combinational circuits**. The classic example is a **half adder** which adds two bits: $\text{sum} = A \text{ XOR } B$ and $\text{carry} = A \text{ AND } B$. Chain full adders and you can add numbers of any size: that is how a processor's arithmetic unit works. Other examples: **multiplexers** (select one of many inputs), **decoders** (turn a binary code into one active line: driving a 7-segment display) and **comparators**.

A	B	Sum (A XOR B)	Carry (A AND B)	Meaning
0	0	0	0	$0 + 0 = 00$
0	1	1	0	$0 + 1 = 01$
1	0	1	0	$1 + 0 = 01$
1	1	0	1	$1 + 1 = 10$ (binary 2)

12.5 Memory and counting: flip-flops

Gates forget: change the input and the output changes. To remember something (a bit), a circuit needs **feedback**: the output feeds back to help hold its own state. The simplest such circuit is the **latch** (two cross-coupled NOR or NAND gates: the SR latch); improved versions with a clock input are called **flip-flops**. A **D flip-flop** copies its input D to its output Q on each rising edge of a clock signal, and holds it until the next edge. It stores exactly one bit.

ANALOGY | A WALL SWITCH VS A LATCHING LAMP

A gate is like a **light switch on a wall** (the light follows the switch position). A flip-flop is like a **push-button lamp** that stays on after you press once and off after you press again: it has memory of what happened before.

- **Register:** a row of D flip-flops sharing a clock: stores an n-bit number (the Arduino's registers such as PORTB and DDRB are like this).
- **Counter:** flip-flops connected so that each halves the frequency of the previous one: a binary counter (74HC393, 4040). A divide-by-2 chain, $16 \text{ MHz} \rightarrow 8 \text{ MHz} \rightarrow 4 \text{ MHz} \dots$: this is how the Arduino's timers (Chapter 23) work.
- **Shift register:** flip-flops chained so bits move one place each clock pulse. A **serial-in, parallel-out** register such as the 74HC595 turns three Arduino pins into eight outputs (Chapter 20).
- **Microcontroller:** millions of gates and flip-flops, plus memory, in a single chip.

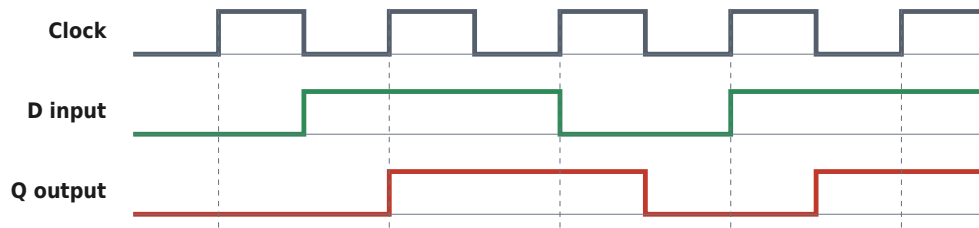


Figure 12.3 A D flip-flop samples D at each rising clock edge (dashed lines) and holds the result at Q until the next edge.

12.6 Bridging analog and digital: ADC and DAC

The real world is analog; the Arduino is digital. An **analog-to-digital converter (ADC)** measures a voltage and produces a number. A **digital-to-analog converter (DAC)** does the reverse. Both are at the heart of music players, phones, sensors and scientific instruments.

How the ADC works

The ADC compares the input voltage with a **reference voltage** V_{ref} (5 V on the Uno by default) and splits that range into 2^n equal steps (n = number of bits, the **resolution**). It outputs the integer number of steps that fit the input:

$$\text{ADC value} = (V_{\text{in}} \div V_{\text{ref}}) \times (2^n - 1)$$

For the Arduino's 10-bit ADC: value = $V_{\text{in}} / 5 \text{ V} \times 1023$ (0 to 1023)

$$\text{Step size (resolution)} = V_{\text{ref}} \div 2^n$$

$5 \text{ V} / 1024 = 4.88 \text{ mV per step}$

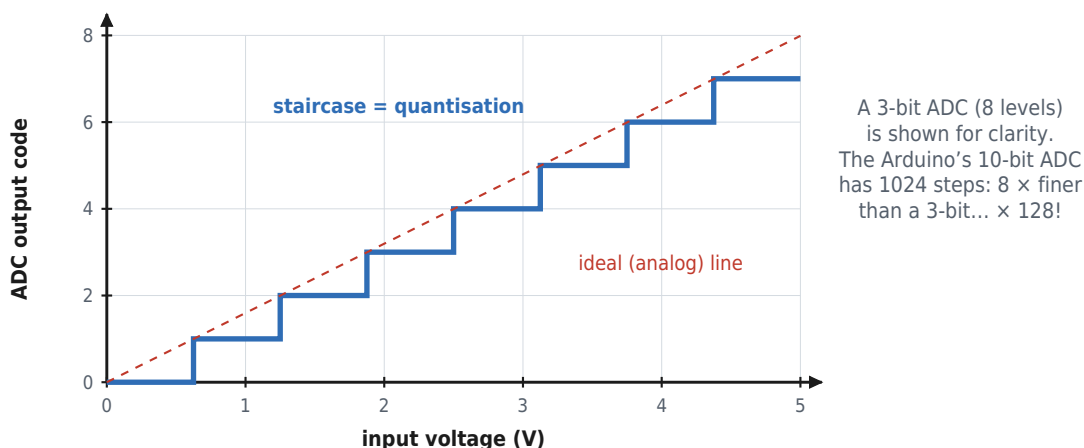


Figure 12.4 An ADC turns a continuous voltage into a staircase of discrete numbers. Quantisation error is up to half a step.

Resolution	Number of levels	Step with $V_{\text{ref}} = 5$ V	Step with $V_{\text{ref}} = 3.3$ V	Typical device
8 bits	256	19.5 mV	12.9 mV	Small microcontrollers, simple DAC
10 bits	1024	4.88 mV	3.22 mV	Arduino Uno/Nano ADC
12 bits	4096	1.22 mV	0.81 mV	ESP32, Arduino Due/Zero, ADS1015
16 bits	65,536	76 μ V	50 μ V	ADS1115 external ADC, audio

WORKED EXAMPLE | READING A POTENTIOMETER

The Arduino reads `analogRead(A0) = 620`. What voltage is that? $V = 620 \times 5 \div 1023 = \mathbf{3.03 \text{ V}}$. (Arduino's own documentation often uses 1024: $620 \times 5/1024 = 3.027 \text{ V}$: the difference is negligible.)

Conversely, a 1.65 V input reads about $1.65/5 \times 1023 = \mathbf{338}$.

Use `map()` (Chapter 15) to convert 0 – 1023 to a more useful range, e.g. 0 – 255 for PWM or 0 – 180 for a servo angle.

Sampling and the Nyquist limit

An ADC also measures at discrete moments in time, at the **sampling rate**. To capture a signal of frequency f without distortion, you must sample at **more than twice f** (the **Nyquist criterion**). Otherwise, high frequencies masquerade as low ones (**aliasing**), like the wagon wheels that seem to turn backwards in old films. The Arduino Uno's `analogRead()` takes about 100 μ s, i.e. about 9,600 samples/s, enough for audio up to a few kHz at best; it is ample for temperature, light, or knobs.

The DAC and PWM

A DAC produces an analog voltage from a digital number (the Arduino Due, Zero, ESP32 have true DACs). The Arduino Uno has none; instead it uses **PWM** (pulse-width modulation): a fast on/off signal whose duty cycle sets the average voltage. `analogWrite(pin, 128)` gives a 50 % duty and an average of about 2.5 V. Add an RC filter (Chapter 6) and you get a real analog voltage. This is the subject of Chapter 18.

The ATmega328P as a digital system

The heart of the Arduino Uno is exactly the collection you have just met: gates and flip-flops (millions of transistors), registers that hold configuration bits for each pin, counters (timers), an ADC, and memory: 32 KB of flash for your program, 2 KB of SRAM for variables and 1 KB of EEPROM for data that survives power-off. Your code manipulates these bits, and the pins are their electrical face.

HANDS-ON EXPERIMENT | EXPERIMENT 8: DISCOVER LOGIC WITH SWITCHES AND LEDs (NO CHIPS NEEDED)

You need: breadboard, 5 V supply, two push buttons, two 330 Ω resistors, one LED, jumper wires.

1. **AND:** wire the two buttons in **series** with the LED and a resistor: the LED lights only when both are pressed.
2. **OR:** wire the two buttons in **parallel** with each other, then in series with LED + resistor: the LED lights when either is pressed.
3. **NAND with a transistor:** if you have a 2N2222 and a 10 k Ω , build the AND circuit to drive a transistor whose collector LED lights when the AND output is LOW. Compare with the truth table.
4. **Optional, with a 74HC08 (quad AND) or 74HC32 (quad OR):** verify the truth table with buttons, LED and 10 k Ω pull-downs on the inputs. Remember: tie unused inputs to ground and add a 100 nF capacitor across the chip's power pins.

CHAPTER SUMMARY

- Analog signals are continuous; digital signals have discrete levels (HIGH/LOW). Digital is noise-resistant and copyable.
- Binary uses bits (0/1); n bits give 2^n values (8 bits: 0 – 255). Hex (0x...) is shorthand for groups of four bits.
- Inputs read HIGH above V_{IH} and LOW below V_{IL} (Uno: 3.0 V / 1.5 V). Never put 5 V on a 3.3 V pin. Tie unused CMOS inputs.
- Gates: AND, OR, NOT, NAND, NOR, XOR each have a truth table; NAND and NOR are universal. In code: `&&`, `||`, `!`.
- Flip-flops store one bit; registers, counters and shift registers are built from them.
- ADC value = $V_{in} / V_{ref} \times 1023$ (10 bit, 4.88 mV per step); sample faster than $2 \times$ the signal frequency. The Uno makes “analog” outputs with PWM.

ASSIGNMENTS | CHAPTER 12

12.1 [Calculate] Convert to decimal: (a) 1010, (b) 11111111, (c) 10000000, (d) 01100110.

12.2 [Calculate] Convert to binary (8 bits): (a) 5, (b) 37, (c) 100, (d) 255.

12.3 [Calculate] Convert: (a) 0x2F to decimal, (b) 0xA0 to binary, (c) 173 to hex, (d) 0b11010110 to hex.

12.4 [Calculate] How many different values can a 12-bit number take? What is the largest?

12.5 [Calculate] An Arduino analog input reads 768. What voltage is applied (5 V reference)? What would the ADC read for 1.0 V?

12.6 [Calculate] What is the voltage resolution (step size) of a 12-bit ADC with a 3.3 V reference? How much better is it than the Uno’s ADC?

12.7 [Calculate] You want to read a signal with frequency components up to 8 kHz. What is the minimum sampling rate? Can an Uno do it with `analogRead()` at ~ 9.6 kS/s?

12.8 [Practice] Complete the truth table for $Y = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$ for all 8 input combinations.

12.9 [Practice] Write the Arduino condition (C++) that is true when a button is pressed (pin 2 reads LOW with `INPUT_PULLUP`) and a sensor value (variable `v`) exceeds 500.

12.10 [Practice] Using De Morgan’s law, rewrite `NOT(A AND B)` using only OR and NOT.

12.11 [Design] Design a logic circuit for a car dome-light: it turns on if any door is open (D1, D2, D3) OR the light switch is on (S), but only if the ignition is off ($I = 0$). Write the Boolean expression and the Arduino condition.

12.12 [Think] Explain why a 5 V Arduino output connected to a 3.3 V sensor input may damage the sensor, while the 3.3 V sensor output into a 5 V Arduino input generally works.

12.13 [Think] Why is it important to tie unused CMOS inputs to +V or ground?

12.14 [Hands-on] Do Experiment 8 and fill in the truth table for the series and the parallel switch circuits, marking the LED state for each combination.

Batteries and power supplies

Every circuit needs energy from somewhere, and power problems cause a huge share of beginner mysteries: random resets, dim displays, motors that stall, sensors that read nonsense. This chapter takes power seriously. You will learn where the energy comes from, how to convert it to the voltage you need, how to count the current your project uses, and how to keep a noisy motor from resetting your microcontroller.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How batteries work, the main chemistries, and how to estimate **run time**
- How to supply an Arduino project: USB, the barrel jack, VIN, the 5 V pin, and battery packs
- How a **linear voltage regulator** (7805, AMS1117) works and how much heat it makes
- What a **switching (buck/boost) converter** is and when it is better
- How to build a simple **mains-to-DC** supply on paper: transformer, rectifier, smoothing, regulator
- Practical power design: current budgets, decoupling, common ground, and safety

13.1 Batteries

A **battery** converts stored chemical energy into electrical energy. Inside each **cell** are two different materials (electrodes) in a chemical mixture (electrolyte). A chemical reaction pushes electrons out of the negative electrode; when a circuit is connected, they flow through it and return at the positive electrode. The voltage of a cell depends only on its chemistry, not on its size; its **capacity** (how long it lasts) depends on its size.

Chemistry	Nominal voltage	Rechargeable?	Typical capacity	Strengths / cautions
Alkaline (AA, AAA, 9 V)	1.5 V per cell (9 V = 6 cells)	No	AA $\approx$ 2000–2800 mAh (low current); 9 V $\approx$ 500 mAh	Cheap, long shelf life. Voltage sags as they drain; poor at high current; can leak
NiMH (AA, AAA)	1.2 V (1.4 V fresh)	Yes (500–1000 cycles)	AA $\approx$ 1900–2500 mAh	Good for hobby packs; low internal resistance; self-discharge (use “low-self-discharge” types)
Lithium-ion / Li-poly (18650, LiPo pouch)	3.7 V (3.0 – 4.2 V)	Yes	18650 $\approx$ 2000–3500 mAh	High energy and current. Need protection circuits; fire risk if punctured, overcharged or short-circuited
LiFePO ₄	3.2 V (2.5 – 3.65 V)	Yes	18650-size $\approx$ 1500 mAh	Very safe, long life
Lead-acid (car, UPS)	2 V per cell (12 V = 6 cells)	Yes	7 Ah – 100 Ah	Cheap, heavy, high surge current. Acid and gas
Coin cell (CR2032)	3 V	No	220 mAh	Tiny, memory backup; only a few mA at most

Capacity, C-rate and internal resistance

Capacity is stated in **mAh** or **Ah** (Chapter 2). Two things reduce what you actually get: (1) **discharge rate**: the faster you draw current, the less total capacity you extract; (2) **internal resistance**, which makes the terminal voltage sag (Chapter 5). A **C-rate** expresses current relative to capacity: for a 2000 mAh battery, $1C = 2\text{ A}$, $0.5C = 1\text{ A}$, $0.1C = 200\text{ mA}$. Batteries are typically specified at $0.2C$ for capacity, and high-power types are rated for continuous currents of $10C$ or more.

WORKED EXAMPLE | ESTIMATING RUN TIME WITH A SAFETY FACTOR

A project draws an average 120 mA. Power: $4 \times \text{AA alkaline}$ (6 V, $\sim 2000\text{ mAh}$ at that current). Estimated runtime = $2000 \div 120 = 16.7\text{ h}$ in theory. Use a 70 % derating for voltage droop and cut-off: $\approx \mathbf{11.7\text{ hours}}$.

Now add a regulator (linear) from 6 V to 5 V: the regulator wastes $(6 - 5) \times 0.12 = 0.12\text{ W}$, but current from the battery is still 120 mA, so the run time is unchanged; the battery gives $6\text{ V} \times 0.12\text{ A} = 0.72\text{ W}$ while the load takes 0.6 W. Efficiency 83 %.

TIP

Battery life budgeting: measure the real average current of your project with a multimeter in series (mA range), or estimate it from datasheets: LED $\sim 10\text{ mA}$ each; servo idle 10 mA and moving 200 mA; ESP8266 WiFi 70–170 mA; Arduino Uno $\sim 45\text{ mA}$ (of which the power LED and USB chip use 10–15 mA). Multiply by the fraction of time each part is active.

SAFETY WARNING

Lithium-ion / LiPo cells are not toys. Never short-circuit them, puncture them, charge them without a proper charger (like a TP4056 module with protection), or discharge below $\sim 3.0\text{ V}$. Use cells with a **protection circuit** (built-in in most 18650 protected cells and LiPo packs sold with Arduino projects). Never leave a charging LiPo unattended; charge on a non-flammable surface. Swollen or hot cells are dangerous: stop using them.

Connecting cells: series and parallel

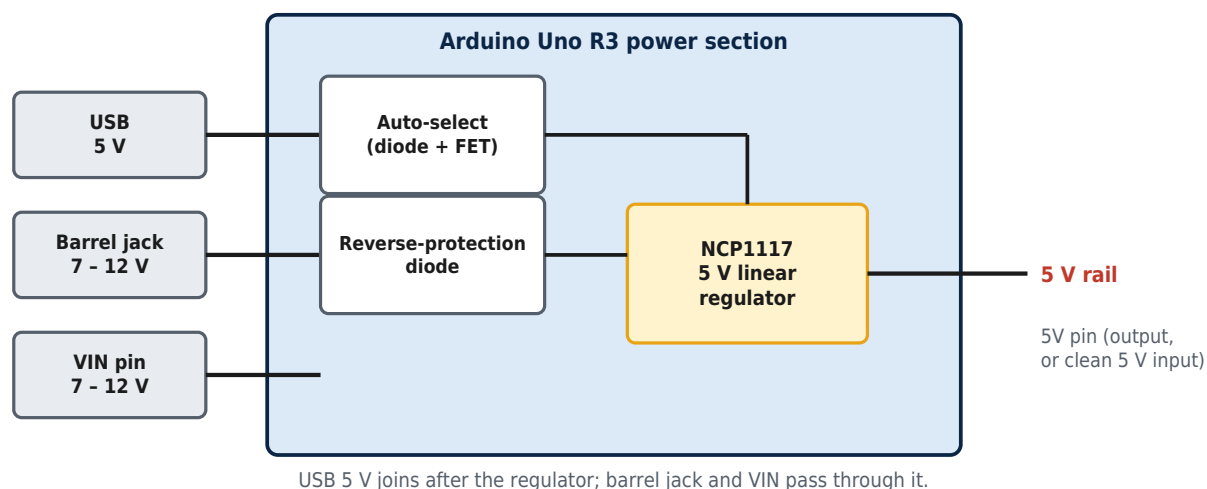
- **In series** (+ to –): voltages **add**, capacity stays the same. Four AA in series: 6 V (alkaline, 1.5 V each) or 4.8 V (NiMH, 1.2 V each).
- **In parallel** (+ to +, – to –): voltage stays the same, capacities **add**. Only connect **identical** cells in parallel, charged to the same voltage, or large currents will flow between them.

REAL-LIFE EXAMPLE | WHY AN ARDUINO SOMETIMES BEHAVES ODDLY ON FOUR NIMH CELLS

Four fresh NiMH cells (1.4 V each) give $\approx 5.6\text{ V}$: fine for the barrel jack (7–12 V recommended, though it usually works from 6–7 V) but somewhat below the regulator's dropout margin at low charge; the Arduino may reset when a motor starts and the battery voltage sags. A 6-cell NiMH pack (7.2 V) or a 2S Li-ion pack (7.4 V) is a better match for the barrel jack.

13.2 Powering the Arduino Uno

The Uno can be powered in four ways. Understanding them prevents damage:



USB 5 V joins after the regulator; barrel jack and VIN pass through it.

Figure 13.1 How power reaches the Uno. USB 5 V bypasses the linear regulator; the barrel jack and VIN go through it.

Method	Voltage	Notes
USB (type-B / USB-C on clones)	5 V	Simplest; up to 500 mA (USB 2.0 port), through a 500 mA resettable fuse. Perfect for development
Barrel jack (2.1 mm, centre-positive)	7 - 12 V (6 - 20 V absolute)	Goes through the on-board 5 V regulator; anything above ~9 V wastes heat in the regulator
VIN pin	7 - 12 V	Same regulator path; use to connect a battery pack directly; no reverse protection
5V pin (as input)	exactly 5 V (4.75 - 5.25)	Bypasses the regulator and the fuse; only use a clean regulated 5 V (e.g. from a buck converter); the board is unprotected
3.3V pin	—	An output (from the USB chip's regulator), max 50 mA. Do not feed power here on the Uno

SAFETY WARNING

Never connect a voltage above 5.5 V to the 5V pin and never connect USB and an external supply to the 5V pin at the same time. With USB plus barrel jack it is safe (auto-select). Reverse polarity at VIN may destroy the board.

How much current can the Arduino supply?

Source	Available current	Notes
5 V pin, powered by USB	≈ 400–450 mA total (USB limit 500 mA minus the board itself)	Includes everything you connect to the 5 V pin
5 V pin, powered from the barrel jack at 9 V	≈ 800–1000 mA as far as the regulator allows, but heat limits it (see section 13.3)	At 12 V input the regulator gets very hot above 250 mA
One I/O pin	20 mA continuous (40 mA absolute max)	Whole chip ≈ 200 mA over all pins
3.3 V pin	50 mA	Small sensor modules only

13.3 Linear voltage regulators

A **voltage regulator** takes a higher, possibly varying voltage and produces a steady lower one. The simplest type is the **linear regulator**, such as the **LM7805** (fixed 5 V, up to 1 A), the **LM7809/7812** (9/12 V), the **LM317** (adjustable) and the low-dropout **AMS1117-3.3** (3.3 V, 800 mA), all common on Arduino modules. A linear regulator behaves like a **variable resistor in series with the load**, adjusted continuously to keep the output constant; the excess input voltage is simply burned off as **heat**.

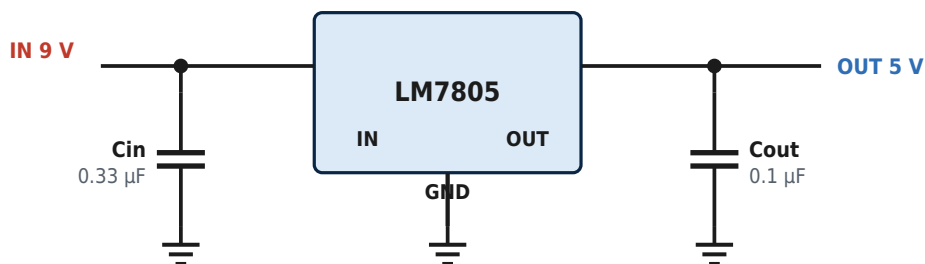


Figure 13.2 The 7805 regulator: input capacitor ($\geq 0.33 \mu\text{F}$), output capacitor ($0.1 \mu\text{F}$; add $10 - 100 \mu\text{F}$ electrolytic for bulk). Pinout (front, left to right): IN – GND – OUT.

Key numbers for a linear regulator:

- **Dropout voltage:** the minimum difference between input and output for regulation to work. **7805: about 2 V** (needs $\geq 7 \text{ V}$ in). **LDO types (AMS1117, MCP1700, LM1117): 0.2 – 1.2 V.**
- **Quiescent current:** what it draws by itself (7805: $\sim 5 \text{ mA}$; MCP1700: $1.6 \mu\text{A}$), important for batteries.
- **Heat dissipation:** the regulator's power loss is $(V_{\text{in}} - V_{\text{out}}) \times I_{\text{load}}$.

$$P_{\text{loss}} = (V_{\text{in}} - V_{\text{out}}) \times I_{\text{load}}$$

All the wasted energy becomes heat in the regulator

WORKED EXAMPLE | HOW HOT DOES THE REGULATOR GET?

Arduino on 12 V: the on-board regulator drops 7 V. At a 300 mA load: $P = 7 \times 0.3 = \mathbf{2.1 \text{ W}}$. A small SOT-223/DPAK regulator with no heatsink safely dissipates about 1 W: the board **overheats** and the regulator shuts down.

Fix 1: power from 7.5–9 V instead: $P = (9 - 5) \times 0.3 = 1.2 \text{ W}$ (borderline). Fix 2: use a **buck converter** module to make 5 V from 12 V and feed it into the 5V pin.

Fix 3: for a TO-220 7805 add a heatsink (thermal resistance 5–10 °C/W; at 2.1 W and 25 °C/W without a heatsink the case rises by 50 °C or more).

Linear versus switching regulators

	Linear (7805, LDO)	Switching (buck, boost, buck-boost)
How it works	Burns off the excess voltage as heat	Rapidly chops the input into pulses, smooths with an inductor and capacitor
Efficiency	$V_{\text{out}} / V_{\text{in}}$ (e.g. $5/12 = 42 \%$)	80 – 95 %
Noise	Very quiet	Some switching noise (kHz – MHz) unless well filtered

	Linear (7805, LDO)	Switching (buck, boost, buck-boost)
Complexity, cost	Simple, tiny, 1 part + 2 capacitors	Needs an inductor and diode/FET; sold as ready modules (LM2596, MP1584, XL6009)
Can step up (boost)?	No, only down	Yes: boost converters step up (3.7 V → 5 V; 5 V → 12 V)
Best for	Small current or small voltage drop; low-noise analog	Large voltage drops, battery projects, motors, high current

ANALOGY | THROTTLED TAP VS. PULSED TAP

A linear regulator is like **pouring water from a high tap into a low bucket by throttling the tap**: you waste the pressure difference as turbulence. A buck converter is like **filling the bucket with quick bursts from a large tap into a buffer tank**: nothing is thrown away, only regulated by how long you open the tap each time (the duty cycle).

TIP

Practical recommendation: for battery projects with a 7–12 V source, a cheap **LM2596 or MP1584 “buck converter” module** set to 5 V (adjust with the screw trimmer and a multimeter *before* connecting the Arduino!) feeds the Arduino’s 5V pin. For small 3.3 V sensor supplies from 5 V, an AMS1117-3.3 or the Arduino’s 3.3 V pin is fine.

13.4 From the wall to DC: a linear power supply

Every mains adapter contains, in one form or another, the following chain. Understanding it lets you use wall-wart adapters wisely, and repair or build a supply.



Figure 13.3 Block diagram of a classic mains adapter (linear supply).

Ripple. After the rectifier, the smoothing capacitor charges to the peak and discharges through the load between peaks, giving a sawtooth **ripple**. For a full-wave rectifier at mains frequency f (50 Hz, ripple at 100 Hz), the peak-to-peak ripple voltage is approximately:

$$V_{\text{ripple}} \approx I_{\text{load}} \div (2 \times f \times C)$$

Ripple (volts) for a full-wave rectifier with smoothing capacitor C

WORKED EXAMPLE | SIZING THE SMOOTHING CAPACITOR

Supply for 500 mA, 50 Hz mains, allow 1 V ripple: $C = I \div (2 f V_{\text{ripple}}) = 0.5 \div (2 \times 50 \times 1) = 0.005 \text{ F} = \mathbf{5000 \mu F}$. Use 4700 μF , voltage rating $\geq 1.5 \times \text{peak}$ (25 V for a 12 V RMS transformer, whose peak is $\approx 17 \text{ V}$).

Why big capacitors? They must store enough energy to supply the load during the 10 ms between rectified peaks. A larger load current needs a larger capacitor for the same ripple.

Check regulator headroom: the DC minimum (valley) must stay above $V_{\text{out}} + \text{dropout}$. For a 7805 (dropout 2 V): valley $\geq 7 \text{ V}$. With 12 V RMS: peak $17 \text{ V} - 1.4 \text{ V}$ (two bridge diodes) = 15.6 V ; minus 1 V ripple = 14.6 V minimum: plenty.

Switch-mode adapters (all modern phone and laptop chargers) work differently: they rectify the mains **first**, chop it at 50–100 kHz through a small ferrite transformer, then rectify and smooth. High frequency means small transformers and capacitors, so they are light and efficient (80–90 %), but they contain lethal voltages on the primary side.

SAFETY WARNING

Do not build or open mains supplies as a beginner. Buy certified plug-in adapters (5 V USB, 9 V/12 V DC barrel). Check the ratings printed on the adapter: voltage, current, and polarity symbol. **Centre-positive** is the usual convention for Arduino: a $\oplus$ symbol with the dot inside means centre = +. Cheap unregulated adapters marked “12 V” can output 16–18 V with no load, dangerously high for small boards; choose **regulated** adapters.

13.5 Power design for real projects

The current budget

Before building anything with several parts, write a **current budget**: list each part and its typical and peak current, add them up, and compare with what the supply can give. Include a margin of 30 – 50 %.

Part	Typical	Peak	Note
Arduino Uno board	45 mA	60 mA	Includes power LED and USB chip
16x2 LCD with backlight	25 mA	30 mA	Backlight is most of it
DHT22 sensor	1 mA	1.5 mA	Measures every 2 s
4 LEDs at 10 mA	20 mA	40 mA	Average with 50 % duty
SG90 servo	10 mA idle	250 mA moving	Stall: 650 mA
Buzzer (active)	0	30 mA	Occasionally
Total	$\approx 100 \text{ mA}$	$\approx 410 \text{ mA}$	USB (500 mA) is just about adequate; use external 5 V if adding more

Good power practice

- **Give motors and servos their own supply**, with the grounds connected (star ground: join the grounds at one point, near the supply).
- **Decouple:** a 100 nF ceramic across every IC's supply pins and a 10 – 100 μF electrolytic across each rail section. Put a 470 – 1000 μF capacitor across servo/motor supplies.
- **Short, thick wires for high current** (0.5 mm² and up for amps); thin breadboard jumpers have noticeable resistance. Remember $V = I R$ (Chapter 2).
- **Keep sensitive analog wiring away from motor wiring**; use twisted pairs for long runs.

- **Fuse the supply** (a resettable polyfuse or a proper fuse) in projects with batteries or supplies that can deliver many amps.
- **Add a power switch and an LED** so you know when the project is on.
- **Test the voltage first:** measure the supply with the multimeter before connecting anything valuable (especially a buck converter that you just adjusted).

COMMON MISTAKE

Different ground references. Two supplies with unconnected grounds produce signals that have no shared reference: the receiving chip cannot tell what is HIGH. Always join grounds of all supplies in a project (at one point).

HANDS-ON EXPERIMENT | EXPERIMENT 9: MEASURE REGULATOR HEAT AND BATTERY SAG

You need: 9 V battery, LM7805, 100 nF and 330 nF capacitors, breadboard, a 100 Ω 2 W resistor (or two 220 Ω $\frac{1}{4}$ W in parallel, $\approx 110 \Omega$), multimeter.

1. Build the circuit of Figure 13.2. Measure the output with no load: 5.0 V.
2. Connect the 100 Ω resistor across the output: the current is $5 \text{ V} \div 100 \Omega = 50 \text{ mA}$. Measure the output voltage; measure the battery voltage while loaded. Compute the regulator loss $(9 - 5) \times 0.05 = 0.2 \text{ W}$. Touch the regulator: barely warm.
3. Try 47 Ω (about 106 mA): loss 0.43 W: the regulator warms noticeably. Always touch briefly and carefully.
4. **Battery sag:** measure the 9 V battery with no load, then with the 47 Ω across it directly (no regulator) — the battery voltage drops (internal resistance) — calculate the internal resistance $R_{\text{int}} = \Delta V \div I$ as in Chapter 5.

CHAPTER SUMMARY

- Batteries: alkaline 1.5 V, NiMH 1.2 V, Li-ion 3.7 V; capacity in mAh; runtime $\approx \text{capacity} \div \text{current} \times 0.7$. Li-ion needs protection.
- The Uno accepts USB 5 V, barrel jack/VIN 7 – 12 V (through a linear regulator) or a clean 5 V on the 5V pin. Pins supply 20 mA; the 3.3 V pin 50 mA.
- **Linear regulators** (7805, AMS1117) waste $(V_{\text{in}} - V_{\text{out}}) \times I$ as heat; **switching regulators** (buck/boost modules) are 80–95 % efficient.
- A mains adapter: transformer $\rightarrow$ rectifier $\rightarrow$ smoothing capacitor (ripple $\approx I / 2fC$) $\rightarrow$ regulator. Use certified adapters; never work on mains.
- Design power with a **current budget**, decouple every IC, separate motor and logic supplies with a common ground.

ASSIGNMENTS | CHAPTER 13

13.1 [Calculate] A project draws 250 mA average. You power it from a 2600 mAh Li-ion cell (3.7 V) through a boost converter to 5 V at 85 % efficiency. Estimate the run time. (Energy method: battery energy $\times$ efficiency $\div$ load power.)

13.2 [Calculate] A 7805 is fed with 9 V and supplies 200 mA. How much power does it dissipate? What temperature rise do you get with a thermal resistance of 65 $^{\circ}\text{C}/\text{W}$ (TO-220 without heatsink)?

13.3 [Calculate] The same 7805 is fed 12 V and supplies 500 mA. Power? Is a heatsink necessary?

13.4 [Calculate] A 12 V, 1 A adapter feeds a buck converter (90 % efficient) that supplies 5 V at 1 A. What current does the buck converter draw from the 12 V supply?

ASSIGNMENTS | CHAPTER 13

13.5 [Calculate] Design the smoothing capacitor for a 300 mA supply from 50 Hz mains (full-wave rectifier) with 0.5 V ripple.

13.6 [Calculate] Total the current budget for: Arduino (45 mA), 3 LEDs (10 mA each), one relay module (70 mA), one PIR sensor (65 mA), one buzzer (30 mA). Can a USB port supply it?

13.7 [Practice] Explain why the Arduino Uno gets hot when powered with 12 V and driving several LEDs and a servo from its 5 V pin.

13.8 [Practice] You measure a “12 V” unregulated adapter with no load and read 17 V. Is it safe for the Arduino’s barrel jack? Explain.

13.9 [Think] Why must a battery pack of two cells connected in parallel use identical cells at equal charge?

13.10 [Think] Your Arduino resets every time a small DC motor connected to the same 5 V supply starts. Give three possible remedies.

13.11 [Hands-on] Carry out Experiment 9. Report the output voltage of the 7805 at three load currents and the regulator’s temperature rise (by touch: cool/warm/hot) each time.

PART III

Arduino

Now everything you learned comes together. In Part III you will learn to program the Arduino, read switches and sensors, drive LEDs, motors and displays, talk to other chips, and finally design complete projects on your own.

IN THIS PART

- 14 Meet the Arduino
- 15 Programming basics: C++ for Arduino
- 16 Digital inputs and outputs
- 17 Analog inputs and sensors
- 18 PWM: analog-style outputs, servos and sound
- 19 A tour of common sensors
- 20 Displays: 7-segment, shift registers and LCD
- 21 Driving motors
- 22 Communication: Serial, I2C and SPI
- 23 Interrupts and timers
- 24 Designing your own circuits
- 25 Debugging and troubleshooting
- 26 Capstone projects
- 27 Soldering, PCBs and where to go next

Meet the Arduino

You now understand voltage, current, resistors, diodes, transistors and logic. It is time to add a brain. The Arduino is a small, inexpensive, open-source computer board that lets you control the physical world with a few lines of code. It was created in 2005 in Ivrea, Italy, for design students who were not engineers, and it works: millions of people, from children to professionals, prototype ideas with it. In this chapter you set up, upload your first program and make an LED blink: the “Hello, world” of hardware.

WHAT YOU WILL LEARN IN THIS CHAPTER

- What a **microcontroller** is and what the Arduino board adds to it
- A guided tour of the **Arduino Uno**: chip, pins, power, USB, reset button and indicators
- How to install the **Arduino IDE**, select board and port, and upload your first program
- The structure of a **sketch**: `setup()` and `loop()`; comments and syntax
- Blink: from the built-in LED to an external LED on a breadboard
- The **absolute maximum ratings** that keep your board alive, and how to fix common upload errors

14.1 What is a microcontroller?

A **microcontroller (MCU)** is a complete tiny computer on a single chip. Unlike the processor in your laptop, which needs separate memory chips, storage, graphics and input/output chips, a microcontroller contains everything on one piece of silicon:

- a **CPU** (the processor that executes instructions),
- **flash memory** (non-volatile: it stores your program, even when power is off),
- **SRAM** (volatile working memory for variables) and **EEPROM** (small permanent storage for settings),
- **general-purpose input/output (GPIO) pins** that you can set HIGH or LOW or read as digital signals,
- **peripherals**: timers, an analog-to-digital converter (ADC), serial communication (UART, I2C, SPI) and more.

A microcontroller is designed to run **one program, forever**, reacting to inputs and controlling outputs: exactly what a washing machine, microwave, remote control, thermostat or toy needs. It runs at megahertz rather than gigahertz, uses milliwatts, and costs from a few cents to a few dollars.

ANALOGY | AN OFFICE VERSUS A WORKSHOP

A laptop’s processor is like a **big office with many specialists** who can do anything but need a large building. A microcontroller is like a **one-person workshop**: one skilled person with all the tools within arm’s reach, dedicated to one job, cheap to run, and always on.

The **Arduino** is not itself a microcontroller. It is a **development board** built around one: the board adds a USB interface, a voltage regulator, a crystal for accurate timing, an LED, a reset button and sockets (headers) that make each pin easy to reach. Just as important, Arduino provides free software (the **IDE**, libraries and a simple programming framework) so you do not need to study the chip’s 400-page datasheet to make an LED blink.

14.2 The Arduino Uno R3 in detail

The Uno is the most popular board and the one this book uses. Its microcontroller is the **ATmega328P** made by Microchip (formerly Atmel).

Specification	Value
Microcontroller	ATmega328P (8-bit AVR)
Operating voltage / logic level	5 V
Input voltage (VIN / barrel jack)	7 – 12 V recommended (6 – 20 V limits)
Clock speed	16 MHz (crystal): one instruction $\approx$ 62.5 ns
Flash memory (program)	32 KB (0.5 KB used by the bootloader)
SRAM (variables)	2 KB
EEPROM (permanent data)	1 KB
Digital I/O pins	14 (D0 – D13); 6 of them can output PWM (D3, D5, D6, D9, D10, D11)
Analog input pins	6 (A0 – A5), 10-bit ADC; can also be used as digital pins
DC current per I/O pin	20 mA recommended (40 mA absolute max); 200 mA total for the chip
Serial (UART)	1 (pins 0 = RX, 1 = TX, shared with USB)
I2C	A4 = SDA, A5 = SCL (also on separate header pins near AREF)
SPI	D10 = SS, D11 = MOSI, D12 = MISO, D13 = SCK
External interrupts	D2 (INT0), D3 (INT1); pin-change interrupts on all pins
Built-in LED	D13 (“L”), also called LED_BUILTIN
USB interface	ATmega16U2 (original) or CH340 (many clones)

GOOD TO KNOW

“5 V logic” means HIGH = about 5 V. Compare with 3.3 V boards (Arduino Due, ESP32, Raspberry Pi Pico, Nano 33 IoT). Always check the logic level of modules you connect (Chapter 12).

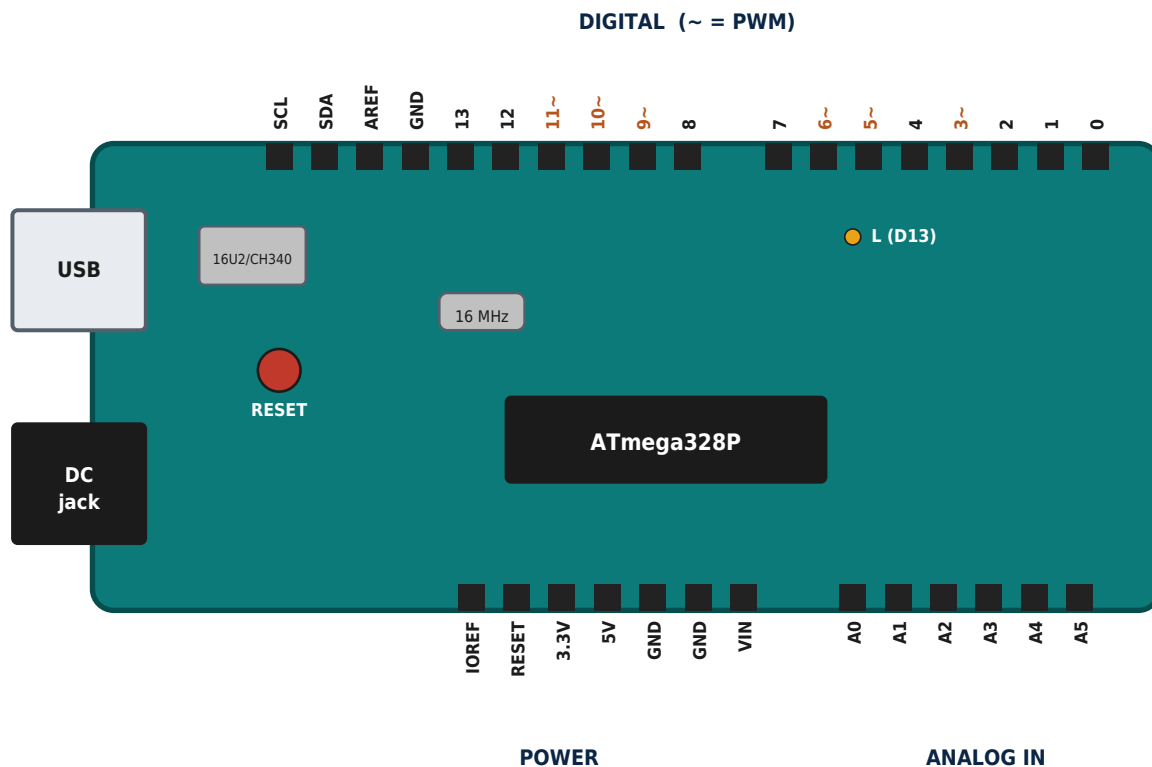


Figure 14.1 Simplified layout of the Arduino Uno R3. Orange labels mark PWM-capable pins (~). Pin 13 has the built-in LED; A4/A5 are also the I2C pins.

Component tour

Part	What it does
ATmega328P (the big black chip)	The microcontroller; runs your program
USB connector (type B)	Powers the board (5 V) and communicates with your computer: uploading sketches and Serial Monitor
ATmega16U2 / CH340	Small chip that converts USB to the serial signal for the main chip
Barrel jack + VIN	External power input (7 – 12 V); goes through the 5 V regulator (Chapter 13)
Reset button	Restarts your program from the beginning (and the bootloader)
16 MHz crystal	Provides the precise clock; without it timing would be unreliable
Power LED (ON)	Shows the board is powered
L LED (pin 13)	Controllable by your code; used by Blink
TX / RX LEDs	Blink when data is sent/received over USB (pins 0 and 1)
ICSP headers	Programming headers for burning the bootloader or programming without USB
Female headers	Sockets along both long edges for jumper wires and shields

The pins in groups

- **Digital pins 0 - 13:** each can be an input or an output. Pins 0 and 1 are shared with USB serial: **avoid using them** for other things while uploading, as they cause upload failures. Pin 13 has the LED. Pins marked ~ (3, 5, 6, 9, 10, 11) can produce PWM.
- **Analog pins A0 - A5:** measure voltages (0 – 5 V) as numbers 0 – 1023. They can also serve as digital pins (referred to as D14 – D19).
- **Power pins:** 5V (regulated 5 V output), 3.3V (50 mA output), GND (three of them), VIN (input from the barrel jack), IOREF (tells shields the logic voltage), RESET (pull LOW to restart), AREF (analog reference)

voltage).

Other Arduino boards

Board	Chip / speed	Pins (digital / analog)	Notes
Uno R3	ATmega328P, 16 MHz, 5 V	14 / 6	Standard; best for learning; shield-compatible
Nano	ATmega328P, 16 MHz, 5 V	14 / 8	Uno in a breadboard-friendly form (mini-USB / USB-C); cheap
Pro Mini	ATmega328P, 8/16 MHz, 3.3/5 V	14 / 6	No USB; tiny; needs a programmer
Mega 2560	ATmega2560, 16 MHz, 5 V	54 / 16	More pins, memory (256 KB flash), 4 serial ports
Leonardo / Micro	ATmega32U4, 16 MHz, 5 V	20 / 12	Native USB: can act as a keyboard or mouse
Due / Zero / MKR	ARM Cortex, 48–84 MHz, 3.3 V	varies	32-bit, true DAC (Due, Zero), 12-bit ADC
ESP32 / ESP8266 (with Arduino core)	Xtensa, 80–240 MHz, 3.3 V	varies	WiFi and Bluetooth; programmed with the same IDE and functions

TIP

Everything in this book works on the Uno and (with small pin changes) the Nano and Mega. Once you master the Uno, moving to other boards is easy since the same `digitalWrite`, `analogRead` and libraries apply.

14.3 Installing the Arduino IDE and uploading your first sketch

You program the Arduino with the **Arduino IDE** (Integrated Development Environment), a free program for Windows, macOS and Linux from arduino.cc/en/software. (There is also a web-based **Arduino Cloud Editor**, which needs no installation but requires a browser plug-in or agent to talk to the board.) The steps are:

1. **Download and install** the Arduino IDE (version 2.x is current). Accept driver installation prompts (on Windows, this installs the serial driver).
2. **Connect the board** with a USB cable: an A-to-B cable for the Uno. Use a **data cable**, not a charge-only cable (a very common trap). The green ON LED lights.
3. **Select the board:** menu **Tools ▶ Board ▶ Arduino AVR Boards ▶ Arduino Uno** (or pick it from the drop-down in IDE 2). For a Nano, choose “Arduino Nano” and processor “ATmega328P (Old Bootloader)” if uploads fail.
4. **Select the port:** **Tools ▶ Port** and choose the one labelled “COMx (Arduino Uno)” on Windows or “/dev/cu.usbmodem...” / “/dev/ttyACM0” on Mac/Linux. Clone boards with the CH340 chip need a CH340 driver (search “CH340 driver”) and appear as “USB-SERIAL CH340”.
5. **Open an example:** **File ▶ Examples ▶ 01.Basics ▶ Blink**.
6. **Click Verify (✓)** to compile without uploading: it catches typos. Then click **Upload (→)**. The RX/TX LEDs flicker; the IDE reports “Done uploading”.
7. **Look at the board:** the “L” LED next to pin 13 should now blink once per second. Your first program is running.

Uploading a sketch is like **copying a recipe into the chef's permanent notebook**. The IDE (your word processor) writes the recipe, the compiler translates it into the language the chef understands (machine code), and the bootloader (a helper who is always waiting after reset) writes it into the notebook (flash memory). From then on, the chef follows it every time the power turns on, without needing the computer.

What happens when you press Upload

1. The IDE **preprocesses** your sketch (adds missing declarations and `#include "Arduino.h"`) and turns it into C++.
2. The **compiler (avr-gcc)** translates C++ to AVR machine code, **linking** in the Arduino core functions and libraries.
3. The result is a **.hex** file: the raw bytes of the program.
4. **avrdude** resets the board (the USB chip pulses the RESET line); the **bootloader** running in the first moments after reset accepts the new program from USB and writes it to flash.
5. The bootloader starts your program. Reset or power cycle: it still runs (flash is non-volatile).

14.4 The anatomy of a sketch

An Arduino program is called a **sketch**. Every sketch has two required functions:

- `void setup()` runs **once** when the board powers up or is reset. Use it for configuration: setting pin modes, starting Serial, initialising sensors.
- `void loop()` runs **over and over forever** after `setup()` finishes. This is where your main logic lives: read inputs, decide, set outputs.

Blink with an external LED on pin 8

```
/* Blink: turns an LED on for one second, then off for one second, repeatedly.
   A multi-line comment starts with slash-star and ends with star-slash. */

const int LED_PIN = 8;           // a named constant for the LED pin (single-line comment)

void setup() {                   // runs once at start
  pinMode(LED_PIN, OUTPUT);     // make pin 8 an output
}

void loop() {                   // runs forever
  digitalWrite(LED_PIN, HIGH);  // pin 8 = 5 V: LED on
  delay(1000);                  // wait 1000 milliseconds (1 second)
  digitalWrite(LED_PIN, LOW);   // pin 8 = 0 V: LED off
  delay(1000);                  // wait another second
}
```

Line by line:

- `const int LED_PIN = 8;` defines a **constant** named LED_PIN with the integer value 8. Named constants make code readable and easy to change: you write the pin number once. (Chapter 15 explains all data types.)
- `pinMode(LED_PIN, OUTPUT);` configures the pin as an **output**: the chip will drive it HIGH (5 V) or LOW (0 V). Pins default to **INPUT**. Forgetting `pinMode` is the #1 reason an LED does not light.
- `digitalWrite(LED_PIN, HIGH);` sets the pin to 5 V; `LOW` sets it to 0 V.
- `delay(1000);` pauses the program for 1000 milliseconds. The CPU does nothing else in the meantime (so avoid long delays in serious projects; Chapter 16 shows how to do better).

- **Semicolons** end statements; **braces** { } group them; **//** starts a comment to the end of the line; **/* ... */** makes a multi-line comment.
- C++ is **case sensitive**: `digitalWrite` works, `DigitalWrite` does not. And **HIGH**, **LOW**, **INPUT**, **OUTPUT** are written in capitals.

Build the external LED circuit

The Uno's built-in LED is connected to pin 13. To blink an LED you can see and control, wire it on a breadboard. It is the circuit from Chapter 8, with Arduino pin 8 as the source:

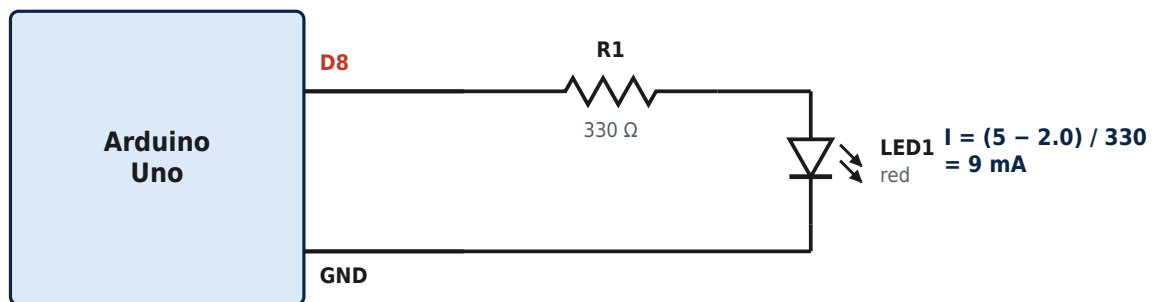


Figure 14.2 Blink circuit: pin 8 → 330 Ω → LED anode; LED cathode → GND.

On the breadboard: connect a jumper from Arduino **GND** to the blue rail; push the LED's **long leg** into a column, the resistor between that column and another; a jumper from the resistor's far column to **pin 8**, and the LED's **short leg** to the GND rail. **Unplug USB while wiring**, then plug in again. Upload the sketch above: the LED blinks.

COMMON MISTAKE

LED backwards: the LED does not light (but is not harmed). Turn it around. **No resistor:** may work for a while but stresses the LED and the pin. **Wrong pin number:** the LED stays off: check that the code's pin matches your wire. **pinMode missing:** the LED glows very dimly (internal pull-up) instead of turning on fully.

HANDS-ON EXPERIMENT | EXPERIMENT 10: PLAY WITH BLINK

1. Upload the stock Blink example. Watch the L LED. Then change `delay(1000)` to `delay(200)` and upload again: the blink is faster. Try 50: the LED seems constantly on but slightly dim (persistence of vision).
2. Make the LED **on** for 100 ms and **off** for 900 ms (a "heartbeat"). Then an S.O.S. pattern (three short, three long, three short).
3. Connect a second LED with its own resistor to pin 9 and make the two LEDs alternate (one on while the other is off).
4. Replace `delay()` values with variables at the top of the sketch: `int onTime = 100;`. Modify and upload several times to see how fast the round trip is.
5. Open **Tools ► Serial Monitor** and use the sketch below to print a message every second.

```
Serial Monitor hello world
```

```
void setup() {  
  Serial.begin(9600);           // start serial communication at 9600 baud  
  Serial.println("Hello, Arduino!");  
}  
  
void loop() {  
  Serial.print("Time since power-on (ms): ");  
  Serial.println(millis());     // number of milliseconds since the program started  
  delay(1000);  
}
```

Set the Serial Monitor's baud rate to **9600** (bottom right): text appears. `Serial.print()` shows values on your computer: your main debugging tool.

14.5 Electrical limits: keeping your board alive

The datasheet lists **absolute maximum ratings**: beyond these the chip can be destroyed. Stay well inside them.

Parameter	Safe / recommended	Absolute maximum	What to do
Current per I/O pin	≤ 20 mA	40 mA	Drive bigger loads through a transistor or MOSFET (Chapter 9)
Total current, all I/O pins	≤ 150 mA	200 mA	Budget your current (Chapter 13)
Voltage on any I/O pin	0 – 5 V	–0.5 V to Vcc + 0.5 V (5.5 V)	Never apply 12 V or a 3.3 V-limited signal above its limit; use dividers or level shifters
Current from 5V pin (on USB)	≤ 400 mA	500 mA (polyfuse trips)	Use an external 5 V supply for motors
Current from 3.3V pin	≤ 50 mA	50 mA	Small sensors only
VIN / barrel jack	7 – 12 V	6 – 20 V	Higher voltage overheats the regulator

SAFETY WARNING

Do not connect **anything that produces more than 5 V or less than 0 V** directly to an Arduino pin: motors (spikes), relay coils, mains-derived signals, sensors with 12 V outputs. Use divider resistors, optocouplers or transistors as appropriate. And never connect **5V to GND** with a wire: it is a short circuit that trips the USB fuse and can damage your computer's port.

14.6 When things go wrong: upload and IDE problems

Symptom / message	Likely cause	Fix
No port listed in Tools ► Port	Charge-only cable; missing driver; faulty USB port	Try another cable and port; install the CH340/FTDI driver; check the ON LED
<code>avrdude: stk500_getsync() ... not in sync: resp=0x00</code>	Wrong board or port selected; pins 0/1 have something connected; bootloader mismatch	Choose the correct board and port; disconnect anything on pins 0 and 1; on Nano try the "Old Bootloader"
<code>avrdude: ser_open(): can't open device ... Access denied (or busy)</code>	Another program (Serial Monitor, another IDE) is using the port	Close other serial programs; unplug and replug
Upload OK but nothing happens	Sketch logic bug; wrong pin; missing <code>pinMode</code> ; wiring	Check with Serial prints; test with built-in LED (pin 13) first

Symptom / message	Likely cause	Fix
error: expected ; before ...	Missing semicolon on the line above the reported line	Check the previous line
error: xyz was not declared in this scope	Misspelled name; wrong capitalisation; missing library or declaration	Check spelling and case; install/include the library
Sketch too big	More than 32 KB of flash	Remove libraries, use F() for strings, or use a Mega
Board resets randomly	Power problems, short circuit, motor noise	See Chapters 13 and 25

TIP

When in doubt, load the Blink example. If Blink uploads and the L LED blinks, your board, cable, driver and IDE are fine, and the problem is in your sketch or wiring. Also use a **simulator: Tinkercad Circuits** (tinkercad.com) and **Wokwi** (wokwi.com) let you build and run Arduino circuits in the browser, free and risk-free: excellent for testing ideas before touching hardware.

CHAPTER SUMMARY

- A microcontroller is a computer on a chip; the Arduino Uno is a board around the ATmega328P (16 MHz, 32 KB flash, 2 KB SRAM, 14 digital and 6 analog pins, 5 V logic).
- Pins: D0-D13 digital (~ = PWM), A0-A5 analog in, 5V, 3.3V, GND, VIN. Avoid pins 0/1 during upload. Each I/O pin gives 20 mA at most.
- Install the IDE, choose **Board** and **Port**, open an example, **Verify**, **Upload**.
- A sketch has `setup()` (runs once) and `loop()` (runs forever). `pinMode`, `digitalWrite`, `delay`, `Serial.println` are your first four tools.
- External LED: pin → 330 Ω → LED anode; cathode → GND. Respect the absolute maximum ratings; never apply > 5 V to a pin.

ASSIGNMENTS | CHAPTER 14

14.1 [Practice] List three differences between a microcontroller and the microprocessor in a laptop.

14.2 [Practice] Which Uno pins can output PWM? Which are used for I2C? Which two should be avoided while uploading?

14.3 [Calculate] The Uno's 16 MHz clock: how long does one clock cycle take? How many cycles does 1 ms correspond to?

14.4 [Calculate] You connect six LEDs, each drawing 15 mA, to six pins. Is each pin within limits? Is the total?

14.5 [Calculate] You want an LED ($V_F = 2.2$ V, 12 mA) on a 5 V pin. Choose a resistor from E12 and calculate the actual current.

14.6 [Practice] Modify Blink so the LED is on for 250 ms and off for 750 ms. Write the `loop()` function.

14.7 [Practice] Explain the difference between Verify and Upload in the IDE.

14.8 [Think] Your Blink sketch uploads without errors, but the external LED never lights. List four things you would check, in order.

14.9 [Think] Why is it a bad idea to apply 12 V directly to a digital input pin?

14.10 [Hands-on] Complete Experiment 10 items 1–3 and record what the changes do. Take a photo of your working breadboard for your notebook.

Programming basics: C++ for Arduino

Arduino sketches are written in C++ (a slightly simplified form of it). You do not need to become a professional programmer, but you do need to be fluent in about a dozen ideas. This chapter teaches them in the order you will need them, with tiny programs you can upload and test at once. If you have never programmed before, take it slowly: type each example yourself rather than copy-pasting, change a number and predict the effect before you run it. That is how programming is learned.

WHAT YOU WILL LEARN IN THIS CHAPTER

- **Variables and data types:** int, long, float, byte, bool, char and their limits
- **Operators:** arithmetic, comparison, logical and bitwise; integer-division pitfalls
- **Decisions and loops:** if/else, switch, for, while, do-while
- **Functions** with parameters and return values; **variable scope**
- **Arrays** and text (char arrays and the String class)
- Useful built-ins (`map`, `constrain`, `random`) and **debugging with the Serial Monitor**

15.1 How a program works

A program is a list of instructions the microcontroller carries out **one after the other**, very fast: the Uno executes roughly 16 million simple instructions a second. In `loop()`, when the last instruction is done, execution jumps back to the first. The instructions can store values in named boxes (**variables**), make decisions (**if**), repeat things (**loops**) and call reusable blocks (**functions**). Everything else in this chapter is a detail on those four ideas.

ANALOGY | THE RECIPE

A program is a **recipe**. Variables are the labelled bowls holding ingredients; `if` is “if the sauce is too thick, add water”; a loop is “stir until smooth”; a function is a mini-recipe such as “make the dough” that the main recipe can call by name whenever it needs it.

15.2 Variables and data types

A **variable** is a named location in memory that holds a value. Before using one you **declare** it, stating its **type** (what kind of value) and **name**, optionally giving it an initial value. C++ needs the type because memory is limited: an `int` uses 2 bytes, a `long` uses 4, and the compiler must know how much room to reserve.

Declaring variables

```
int    ledPin    = 8;           // whole number, -32768 to 32767
float  voltage   = 3.14;       // number with a decimal point
bool   isPressed = false;      // true or false
char   letter    = 'A';        // a single character
byte   level     = 200;        // whole number 0 to 255
```

Type	Size	Range	Use for
bool (or boolean)	1 byte	false (0) or true (1)	On/off states, flags
byte (uint8_t)	1 byte	0 to 255	Small counts, PWM values, port bits
char	1 byte	−128 to 127 (ASCII characters)	Single letters: 'A'
int	2 bytes	−32,768 to 32,767	Everyday integers, ADC readings
unsigned int (uint16_t)	2 bytes	0 to 65,535	Larger positive counts
long	4 bytes	−2,147,483,648 to 2,147,483,647	Large integers
unsigned long (uint32_t)	4 bytes	0 to 4,294,967,295	Time from millis() (Chapter 16)
float (double is the same on the Uno)	4 bytes	$\pm 3.4 \times 10^{38}$, ~6-7 significant digits	Measurements with fractions
String	varies	text	Text (with caution: it uses dynamic memory)

SAFETY WARNING | NUMBERS DO NOT STRETCH

Overflow. Numbers wrap around when they exceed the range: an `int` at 32,767 plus 1 becomes −32,768. Two classic bugs: computing `int x = 300 * 200;` (60,000 does not fit in an int) and storing `millis()` in an `int` (it overflows after 32 s). Use `long` or `unsigned long`, and remember `300L * 200` forces a long calculation.

Constants

Use `const` for values that must never change (pin numbers, thresholds). It is typed, safe and takes no RAM on the AVR when the compiler can substitute the value. The older `#define` does text replacement without types; it works but `const` is preferred.

const versus #define

```
const int BUTTON_PIN = 2;      // can never be changed later
const float VREF = 5.0;
#define LED_COUNT 4            // old style; no semicolon, no equals sign
```

Naming. Use meaningful names (`sensorValue`, not `x`), start variables with a lowercase letter (`sensorValue`), write constants in CAPITALS (`BUTTON_PIN`), no spaces, do not start with a digit. Good names make comments almost unnecessary.

Scope: where a variable lives

- **Global** variables are declared outside any function (at the top of the sketch). All functions can see them and they keep their value forever.
- **Local** variables are declared inside a function or block `{ }`. They exist only there and are recreated each time the function runs. Use locals by default (they avoid unintended interference), and globals for values that must persist between calls to `loop()`.
- **static** locals are declared inside a function but keep their value between calls.

Scope

```
int counter = 0; // global: remembers between loop() runs

void loop() {
  int temp = analogRead(A0); // local: created fresh every time loop() runs
  static int calls = 0; // local but remembers its value
  calls++; // increments each time
  counter = counter + 1;
}
```

15.3 Operators

Arithmetic

Operator	Meaning	Example	Result
+ - * /	add, subtract, multiply, divide	7 / 2 (both int)	3 (integer division truncates!)
%	remainder (modulo)	7 % 2	1
=	assignment (NOT equality!)	x = 5	store 5 in x
+= -= *= /=	shorthand	x += 3 means x = x + 3	
++ --	increment / decrement by 1	i++	i becomes i + 1

COMMON MISTAKE

Integer division. If both operands are integers, the result is an integer: 5 / 2 is 2, not 2.5, and 1 / 3 is 0. Make at least one operand a float: 5 / 2.0 or (float)5 / 2 gives 2.5. And float v = 512 / 1023 * 5; gives 0 (because 512/1023 = 0). Write 512 * 5.0 / 1023, multiplying before dividing, to preserve precision.

Comparison and logical operators

Operator	Meaning
== !=	equal to, not equal to (two equals signs for comparison!)
< > <= >=	less than, greater than, and or-equal versions
&&	logical AND: true if both sides are true
	logical OR: true if at least one side is true
!	logical NOT: reverses true/false

COMMON MISTAKE

= **versus** ==. `if (x = 5)` **assigns** 5 to x (and is always true), a very common bug. You mean `if (x == 5)`.

Bitwise operators (used with registers and shift registers)

Operator	Meaning	Example	Result
&	AND bit by bit	0b1100 & 0b1010	0b1000
	OR bit by bit	0b1100 0b1010	0b1110
^	XOR (toggles bits)	0b1100 ^ 0b1010	0b0110
~	invert all bits	~0b00001111 (byte)	0b11110000

Operator	Meaning	Example	Result
<< >>	shift left / right	1 << 3	8 (0b1000)

Handy idioms: **set bit** `n x |= (1 << n);` **clear bit** `n x &= ~(1 << n);` **toggle** `x ^= (1 << n);` **test if** `(x & (1 << n)).`

15.4 Making decisions

if / else if / else

if / else if / else

```
int temperature = analogRead(A0);

if (temperature > 700) {
  Serial.println("Too hot!");
} else if (temperature > 400) {
  Serial.println("Comfortable");
} else {
  Serial.println("Cold");
}
```

The condition in parentheses is evaluated as true (non-zero) or false (zero). The statements in braces run only if it is true. Only the **first** true branch runs. If the body is a single statement the braces are optional, but always using them prevents bugs when you add lines later.

switch / case

When one variable is compared with many specific values, **switch** is tidier than a chain of **ifs**:

switch / case

```
switch (mode) {
  case 0:
    Serial.println("Off");
    break;           // without break, execution "falls through" to the next case
  case 1:
    Serial.println("Slow blink");
    break;
  case 2:
  case 3:           // cases 2 and 3 share the same action
    Serial.println("Fast blink");
    break;
  default:
    Serial.println("Unknown mode");
}
```

15.5 Repeating things: loops

for loop: a known number of repetitions

for loop

```
for (int i = 0; i < 5; i++) {           // start; keep going while; after each pass
  digitalWrite(LED_PIN, HIGH);
  delay(100);
  digitalWrite(LED_PIN, LOW);
  delay(100);
}                                       // blinks 5 times, i takes values 0,1,2,3,4
```

The three parts inside `for ( ; ; )` are: **initialisation** (run once), **condition** (checked before each pass), **update** (run after each pass). Note that C++ counts from **0**: 5 passes have `i = 0...4`.

while and do-while: repeat until a condition changes

while / do-while

```
while (digitalRead(BUTTON_PIN) == HIGH) { // stay here as long as the button is NOT pressed
    // wait
}

do { // runs at least once, then tests
    reading = analogRead(A0);
} while (reading < 100);
```

SAFETY WARNING | INFINITE LOOPS

A loop whose condition never becomes false freezes your program: `while (true);`. Sometimes that is intended, but it usually signals a bug: make sure something inside the loop can change the condition. Note that `loop()` itself is an infinite loop by design.

Loops can be **nested**, and you can leave a loop early with `break`; or skip to the next pass with `continue`;

WORKED EXAMPLE | KNIGHT-RIDER LED CHASE

Four LEDs on pins 8, 9, 10, 11, lighting one at a time back and forth. Using a `for` loop and `pinMode` over a range: see the listing below.

Chaser: loops replace repetition

```
const int FIRST_LED = 8;
const int LAST_LED = 11;

void setup() {
    for (int pin = FIRST_LED; pin <= LAST_LED; pin++) {
        pinMode(pin, OUTPUT); // set up all four pins in 3 lines
    }
}

void loop() {
    for (int pin = FIRST_LED; pin <= LAST_LED; pin++) { // go right
        digitalWrite(pin, HIGH);
        delay(80);
        digitalWrite(pin, LOW);
    }
    for (int pin = LAST_LED - 1; pin > FIRST_LED; pin--) { // come back left
        digitalWrite(pin, HIGH);
        delay(80);
        digitalWrite(pin, LOW);
    }
}
```

15.6 Functions

A **function** is a named block of code you can run (“call”) from anywhere, optionally handing it values (**parameters**) and getting a value back (**return value**). Functions remove duplication, make programs readable and let you test parts separately. `setup()` and `loop()` are themselves functions.

Defining and calling functions

```
// A function with no return value (void) and one parameter
void blink(int pin, int times) {
  for (int i = 0; i < times; i++) {
    digitalWrite(pin, HIGH);
    delay(150);
    digitalWrite(pin, LOW);
    delay(150);
  }
}

// A function that returns a value: converts ADC reading to volts
float toVolts(int adcValue) {
  return adcValue * 5.0 / 1023.0;
}

void loop() {
  blink(8, 3); // blink pin 8 three times
  float v = toVolts(analogRead(A0)); // call and store the result
  Serial.println(v);
}
```

The parts: the **return type** (`void` for none, `int`, `float`...), the **name**, the **parameter list** with types, and the **body**. The Arduino IDE automatically adds function declarations (prototypes), so you can call a function before defining it, unlike in plain C++.

TIP

Write a function whenever you copy-paste code more than once, or when a block of `loop()` has a nameable purpose (`readTemperature()`, `updateDisplay()`, `checkButton()`). A good `loop()` reads like an outline of the whole program.

15.7 Arrays

An **array** holds a numbered list of values of the same type; the numbering (index) starts at **0**. Arrays and loops go together like LEDs and resistors.

Arrays

```
const int ledPins[] = {8, 9, 10, 11}; // array of 4 ints; size worked out automatically
const int NUM_LEDS = sizeof(ledPins) / sizeof(ledPins[0]); // = 4

int readings[10]; // room for 10 ints (indices 0..9)

void setup() {
  for (int i = 0; i < NUM_LEDS; i++) {
    pinMode(ledPins[i], OUTPUT); // ledPins[0] is 8, ledPins[3] is 11
  }
}
```

SAFETY WARNING | OFF-BY-ONE ERRORS

Array index out of range. `readings[10]` in an array of size 10 does not exist (indices 0 to 9). C++ does not stop you; you overwrite other variables silently and the program behaves bizarrely. Always keep indices inside the array size.

15.8 Text: char arrays and String

Text is a sequence of characters. Two ways to handle it:

- **C-string (char array):** `char name[] = "Arduino";`. Compact, predictable, ends with an invisible `\0` character. Use `strcpy`, `strlen`, `strcmp` from the standard library.
- **String object:** `String name = "Arduino";`. Easy: `name + " Uno"`, `name.length()`, `name.toUpperCase()`. But it allocates memory dynamically and, in long-running programs on 2 KB of RAM, can **fragment memory** and cause random crashes. Use it in short sketches and prototypes; prefer `char` arrays for permanent devices.

Text output

```
Serial.println("Text literal");           // strings in quotes use RAM!
Serial.println(F("Saved in flash"));      // F() keeps the text in flash memory, saving RAM

String s = "Temp: ";
s += 23.5;                               // "Temp: 23.50"
Serial.println(s);
```

15.9 Handy built-in functions

Function	What it does	Example
<code>map(x, inMin, inMax, outMin, outMax)</code>	Rescales x linearly (integer arithmetic)	<code>map(analogRead(A0), 0, 1023, 0, 255)</code>
<code>constrain(x, lo, hi)</code>	Limits x to the range lo..hi	<code>constrain(val, 0, 255)</code>
<code>min(a,b), max(a,b)</code>	Smaller / larger of two	<code>max(0, x)</code>
<code>abs(x)</code>	Absolute value	<code>abs(-5) → 5</code>
<code>pow(base, exp), sqrt(x)</code>	Power, square root (float)	<code>sqrt(2.0)</code>
<code>random(max) / random(min, max)</code>	Pseudo-random integer in [min, max)	<code>random(1, 7) → dice 1-6</code>
<code>randomSeed(analogRead(A5))</code>	Makes the sequence differ per run (read an unconnected pin)	call once in <code>setup()</code>
<code>millis(), micros()</code>	Time since start in ms / μ s (unsigned long)	<code>unsigned long t = millis();</code>
<code>delay(ms), delayMicroseconds(us)</code>	Pause the program	<code>delay(500);</code>
<code>sin, cos, tan, log</code>	Trigonometry, logarithm (radians)	<code>sin(PI / 2)</code>

`map()` **example.** A potentiometer gives 0–1023; a servo needs 0–180°: `angle = map(pot, 0, 1023, 0, 180);` At `pot = 512` you get 90. Remember `map` does **not** limit the output: if the input goes out of range, so does the output, so combine it with `constrain()`. It also uses integer maths, so fractions are lost.

15.10 Talking to your computer: the Serial Monitor

The Serial Monitor is your window into the running program. It costs nothing but a few lines:

Serial output

```
void setup() {
  Serial.begin(9600);           // must match the baud setting in the Serial Monitor
}

void loop() {
  int value = analogRead(A0);
  Serial.print("A0 = ");        // print: no newline
  Serial.print(value);
  Serial.print(" voltage = ");
  Serial.print(value * 5.0 / 1023.0, 2); // 2 decimal places
  Serial.println(" V");         // println: adds a newline
  delay(200);
}
```

And for input, typed characters arrive in a buffer:

Controlling an LED from the keyboard

```
void setup() {
  Serial.begin(9600);
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  if (Serial.available() > 0) { // is there at least one character waiting?
    char c = Serial.read();     // read one character
    if (c == '1') digitalWrite(LED_BUILTIN, HIGH);
    if (c == '0') digitalWrite(LED_BUILTIN, LOW);
  }
}
```

Type **1** in the Serial Monitor's input box and press Send: the LED turns on. This is how you test outputs without any buttons connected. **Debugging tip:** when something misbehaves, print the variables you think you know. `Serial.print` turns guesses into facts.

15.11 Style, comments and structure

- **Indent consistently** (2 spaces). In the IDE press **Ctrl+T** (§T) to auto-format.
- **Comment the why, not the what.** `// wait 10 ms for the switch to settle helps; // delay 10` does not.
- **Top-of-file header comment:** name, purpose, pin assignments, date.
- **Name your pins** at the top with `const int`. Then a change of wiring is a change to one line.
- **Avoid “magic numbers”:** `if (temp > 30)` is better as `const int MAX_TEMP = 30;` and `if (temp > MAX_TEMP)`.
- **One idea per function; small functions;** and keep `loop()` short.
- **Test small.** Compile and run after every few lines.

A tidy small sketch

```
/*
 Project:   Night light
 Purpose:   LED comes on when the room is dark
 Hardware:  LDR divider on A0, LED + 330 ohm on D9
 Author/Date: ...
*/
const int LDR_PIN = A0;
const int LED_PIN = 9;
const int DARK_THRESHOLD = 400;      // tune after measuring (Chapter 17)

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  int light = analogRead(LDR_PIN);
  digitalWrite(LED_PIN, light < DARK_THRESHOLD ? HIGH : LOW);  // ternary: condition ? a : b
}
```

15.12 Libraries

A **library** is a collection of ready-made code for a specific task: driving a servo, an LCD, a sensor. Include it with `#include <Servo.h>`, create objects, and call its functions. The IDE ships with many (Servo, Wire, SPI, LiquidCrystal, EEPROM, SoftwareSerial). Others are installed from **Sketch ► Include Library ► Manage Libraries** (search “DHT sensor library”, “LiquidCrystal I2C”...). You will use libraries constantly from Chapter 18 onward. Look under **File ► Examples** for each library’s example sketches: the best documentation there is.

CHAPTER SUMMARY

- Variables have **types** (int 2 bytes, long 4, float 4, byte 1, bool) and can overflow. Use `const` for fixed values and `unsigned long` for `millis()`.
- Beware `=` vs `==`, and **integer division** ($5/2 = 2$).
- **if / else, switch, for, while** control the flow; C++ counts from 0.
- **Functions** (`void` or returning a value) organise code; **arrays** + loops handle sets of pins.
- `map`, `constrain`, `random`, `millis`, `Serial.print` are your everyday helpers; `String` can fragment RAM; use `F()` for constant text.
- Style: meaningful names, named pin constants, comment the reason, small functions. Test in small steps.

ASSIGNMENTS | CHAPTER 15

15.1 [Calculate] What are the values of: (a) `7 / 2`, (b) `7 / 2.0`, (c) `7 % 3`, (d) `10 / 4 * 4`, (e) `(float)10 / 4 * 4`?

15.2 [Calculate] A sketch declares `int x = 32000; x = x + 1000;`. What is `x` afterwards? Why?

15.3 [Calculate] Use `map()` to convert a potentiometer reading of 300 (0-1023) into a servo angle 0-180. Compute by hand (integer maths).

15.4 [Calculate] Write the expression that converts an ADC reading `raw` into millivolts (5 V reference) without using floating point, avoiding overflow.

15.5 [Calculate] What is `0b10110000 | 0b00001111`? And `0b11111111 & ~(1 << 3)`?

15.6 [Practice] Write a for loop that turns on LEDs on pins 2-7 one after another with 100 ms between them (pins already set as outputs).

ASSIGNMENTS | CHAPTER 15

15.7 [Practice] Write a function `int average(int a, int b)` that returns the mean of two integers, and show how to call it with two analog readings.

15.8 [Practice] Write a sketch fragment using an array of 3 pins {9, 10, 11} and a loop that sets them as outputs and then turns all of them off.

15.9 [Practice] Find the bug: `if (buttonState = HIGH) { digitalWrite(led, HIGH); }`

15.10 [Practice] Find the bug: `int readings[5]; for (int i = 0; i <= 5; i++) readings[i] = analogRead(A0);`

15.11 [Think] Why does `float volts = analogRead(A0) / 1023 * 5;` always give 0 (except at exactly 1023)? Give a corrected line.

15.12 [Think] When would you prefer a char array to a String object?

15.13 [Hands-on] Upload the Serial keyboard-control sketch and control the LED. Then extend it so that typing `b` makes the LED blink three times and `s` prints the current time (millis) to the Serial Monitor.

Digital inputs and outputs

Digital pins are the Arduino's bread and butter: they read switches, drive LEDs, control transistors and talk to chips. This chapter turns the pull-up/pull-down theory of Chapter 10 into working code and then goes further, tackling the problem every beginner meets around week three: how do I blink an LED **and** watch a button at the same time? The answer, `millis()`, is the most important programming technique in this book.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How `pinMode`, `digitalWrite` and `digitalRead` work, and what a pin does electrically in each mode
- Reading a **button** correctly with `INPUT_PULLUP`; detecting presses and releases (edge detection)
- **Debouncing** in software
- Why `delay()` is a trap and how to do several things at once with `millis()`
- **State machines** and toggling: a clean way to structure behaviour
- Driving many outputs: LED patterns and using arrays

16.1 The three pin modes

Each digital pin can be configured by `pinMode(pin, mode)` in one of three ways:

Mode	Electrical behaviour	Use
OUTPUT	Pin actively driven to 5 V (HIGH) or 0 V (LOW) by transistors in the chip. Can source or sink up to 20 mA.	LEDs (through resistor), transistor bases/gates, buzzers, logic signals to chips
INPUT	High-impedance ($\approx 100\text{ M}\Omega$). Reads voltage. Floats if nothing is connected.	Signals that are actively driven HIGH and LOW (sensors with digital outputs) or with external pull resistors
INPUT_PULLUP	Input with a 20 – 50 k Ω resistor to 5 V enabled inside the chip.	Buttons and switches wired to GND (pressed = LOW), open-collector sensors

ANALOGY | A SWITCH AND A PEEPHOLE

Think of a pin as a **door with a light switch**. As an OUTPUT the Arduino is the person flipping the switch. As an INPUT it is a person who only looks at whether the light is on. INPUT_PULLUP is like a weak spring that holds the switch in the “on” position until something pushes it to “off”.

Sourcing and sinking current

An output pin can push current **out** into a load referenced to ground (**sourcing**: pin HIGH $\rightarrow$ current flows out of the pin), or pull current **in** from a load connected to +5 V (**sinking**: pin LOW $\rightarrow$ current flows into the pin). Both are limited to about 20 mA. Wiring an LED from the pin to ground, as we did, is sourcing; wiring it from +5 V to the pin through a resistor is sinking (the LED lights when the pin is LOW). Some modules (active-LOW relay boards, common-anode RGB LEDs) work in this inverted manner.

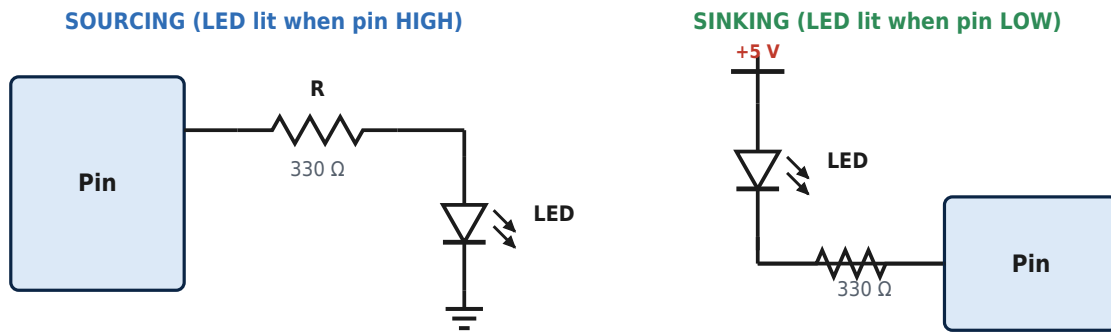


Figure 16.1 Sourcing (left) and sinking (right). In the sinking arrangement the logic is inverted: LOW turns the LED on.

16.2 Reading a button

The recommended way: INPUT_PULLUP

Connect one side of the button to the pin and the other to **GND**. No external resistor is needed. With the button open, the internal pull-up holds the pin **HIGH**; pressing it connects the pin to ground, reading **LOW**. So **pressed = LOW**: the logic is inverted, but that is only a matter of getting used to it.

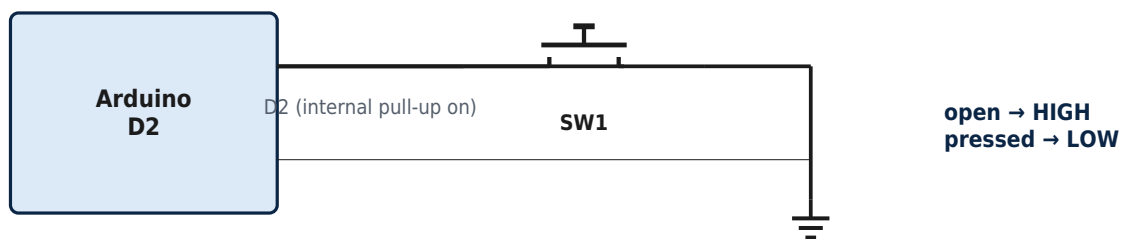


Figure 16.2 Button wired between pin and GND, using INPUT_PULLUP: the simplest, most robust input.

LED follows the button

```
const int BUTTON_PIN = 2;
const int LED_PIN = 8;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP); // internal pull-up: no external resistor
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) { // LOW means pressed
    digitalWrite(LED_PIN, HIGH); // LED on while the button is held
  } else {
    digitalWrite(LED_PIN, LOW);
  }
}
```

TIP

A neat one-liner: `digitalWrite(LED_PIN, !digitalRead(BUTTON_PIN));` copies the inverted button state to the LED. With `INPUT_PULLUP`, a wire touched between the pin and GND also “presses” the button, so you can test quickly with a jumper.

The external pull-down alternative

With an external 10 kΩ pull-down to GND and the button to +5 V, pressed reads **HIGH**. Use `pinMode(pin, INPUT)` and compare with **HIGH**. Pull-ups are generally preferred because the chip already has them and idle switch wiring at HIGH is more noise-resistant with long wires.

16.3 Edge detection: reacting to the moment of change

Often you do not care whether the button **is** pressed, but that it **just became** pressed (once per press). Otherwise, holding the button would repeat the action a thousand times a second. The technique: remember the previous state and act only when it **changes**.

Toggle an LED with each press (without debouncing)

```
const int BUTTON_PIN = 2;
const int LED_PIN = 8;

bool ledState = false;           // current state of the LED
bool lastButton = HIGH;         // previous reading (HIGH = released with pull-up)

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  bool button = digitalRead(BUTTON_PIN);

  if (lastButton == HIGH && button == LOW) { // falling edge: just pressed
    ledState = !ledState;                   // toggle
    digitalWrite(LED_PIN, ledState);
  }
  lastButton = button;                     // remember for the next pass
}
```

Try it: mostly it works, but sometimes the LED toggles twice or seems to ignore a press. That is **contact bounce** (Chapter 10) and the Arduino is fast enough to see it.

16.4 Debouncing

After the first contact change, the signal may chatter for 1 – 20 ms. Solutions:

- **Hardware:** a 100 nF capacitor across the switch, with the pull-up resistor forming an RC filter ($\tau = 10 \text{ k}\Omega \times 100 \text{ nF} = 1 \text{ ms}$). Works, but adds parts.
- **Software, delay-based:** after detecting a change, wait ~20 ms and test again. Simple but blocks the program.
- **Software, time-based (best):** ignore any change that happens less than 20–50 ms after the last accepted one, using `millis()`.

Robust debounced toggle

```
const int BUTTON_PIN = 2;
const int LED_PIN    = 8;
const unsigned long DEBOUNCE_MS = 30;

bool ledState = false;
bool stableState = HIGH;           // debounced state
bool lastReading = HIGH;           // raw reading last time
unsigned long lastChange = 0;       // when the raw reading last changed

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  bool reading = digitalRead(BUTTON_PIN);

  if (reading != lastReading) {      // raw signal changed: restart the timer
    lastChange = millis();
    lastReading = reading;
  }

  if (millis() - lastChange > DEBOUNCE_MS && reading != stableState) {
    stableState = reading;           // signal has been steady long enough: accept it
    if (stableState == LOW) {        // accepted a press
      ledState = !ledState;
      digitalWrite(LED_PIN, ledState);
    }
  }
}
```

Read the logic in words: “when the raw pin changes, note the time. If it has stayed unchanged for more than 30 ms and it differs from the state we last accepted, accept it.” The program never waits; `loop()` keeps spinning thousands of times per second. The Arduino library **Bounce2** wraps the same idea in three lines if you prefer.

16.5 The `delay()` problem, and `millis()`

`delay(1000)` stops the processor for a second: it cannot read a button, update a display or watch a sensor. Suppose you want to blink an LED once a second **and** react to a button immediately. With `delay()` the button is only checked once per second, and pressing between checks is missed. The cure is to stop waiting and start **checking the clock**.

ANALOGY | STARING AT THE POT VS. THE KITCHEN TIMER

With `delay()` you are like a cook who **stares at the pot** for ten minutes until it boils, ignoring everything else in the kitchen. With `millis()` you **set a kitchen timer** and get on with chopping vegetables, glancing at the clock now and then: “has ten minutes passed yet? No... not yet... now yes: take the pot off.”

`millis()` returns the number of milliseconds since the board started, as an `unsigned long`. It overflows after about 49.7 days, back to 0. The standard non-blocking pattern:

The millis() timing pattern

```
unsigned long previous = 0;           // the last time we toggled
const unsigned long INTERVAL = 500;

void loop() {
  unsigned long now = millis();
  if (now - previous >= INTERVAL) {  // has the interval elapsed?
    previous = now;                  // restart the interval
    // do the periodic thing here
  }
  // ... the rest of loop() runs continually, so other tasks stay responsive
}
```

GOOD TO KNOW

Why `now - previous` and not `now >= previous + INTERVAL`? With unsigned arithmetic, the subtraction gives the correct elapsed time even across the 49-day rollover, because both numbers wrap the same way. This is the “rollover-safe” form; always use it.

WORKED EXAMPLE | BLINK AND RESPOND AT THE SAME TIME

Below, the LED blinks at 1 Hz with `millis()` while a button (INPUT_PULLUP on D2) instantly lights a second LED. Neither blocks the other.

Two tasks at once

```
const int BLINK_LED = 8, BUTTON_LED = 9, BUTTON_PIN = 2;
unsigned long previous = 0;
bool blinkState = false;

void setup() {
  pinMode(BLINK_LED, OUTPUT);
  pinMode(BUTTON_LED, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  // Task 1: blink every 500 ms without blocking
  if (millis() - previous >= 500) {
    previous = millis();
    blinkState = !blinkState;
    digitalWrite(BLINK_LED, blinkState);
  }
  // Task 2: button LED responds within microseconds
  digitalWrite(BUTTON_LED, digitalRead(BUTTON_PIN) == LOW);
}
```

You can run many independent timers this way, each with its own `previous` variable: sensor readings every 200 ms, display refresh every second, a heartbeat every 500 ms. This “cooperative multitasking” is how most well-behaved Arduino projects are written.

COMMON MISTAKE

Using `delay()` inside a `loop()` that also has to react to inputs, and storing time in an `int` (overflow after 32 s). Use `unsigned long` for every variable that holds a `millis()` value.

16.6 State machines

As programs grow, tangles of `ifs` and flags become confusing. A **state machine** is a tidy way to organise behaviour: the system is always in exactly **one state** (Idle, Counting, Alarm...), and **events** (button pressed, time elapsed, sensor above threshold) cause **transitions** between them. You draw it as a diagram first, then code it as a `switch`.

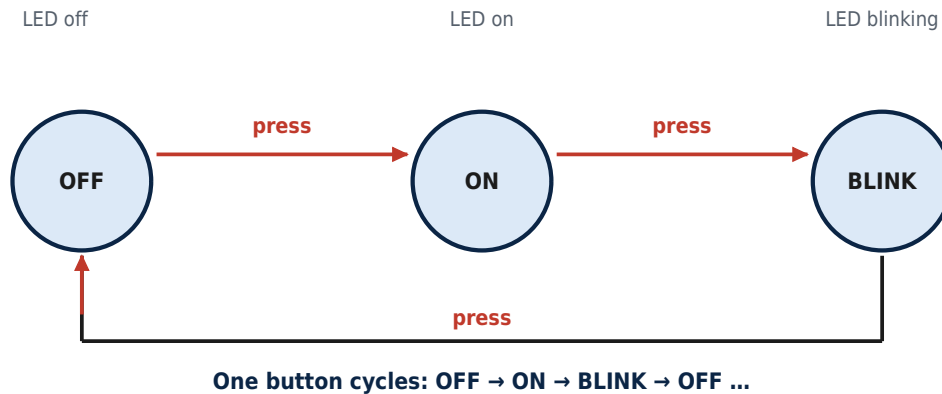


Figure 16.3 A three-state machine controlled by a single button.

State machine: one button, three LED modes

```
enum State { OFF, ON, BLINK };           // named states instead of magic numbers
State state = OFF;

const int BUTTON_PIN = 2, LED_PIN = 8;
bool lastRaw = HIGH, stable = HIGH;
unsigned long lastChange = 0, lastBlink = 0;
bool blinkLevel = false;

void setup() {
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(LED_PIN, OUTPUT);
}

bool buttonPressed() {                   // returns true once per debounced press
    bool raw = digitalRead(BUTTON_PIN);
    if (raw != lastRaw) { lastChange = millis(); lastRaw = raw; }
    if (millis() - lastChange > 30 && raw != stable) {
        stable = raw;
        if (stable == LOW) return true;
    }
    return false;
}

void loop() {
    if (buttonPressed()) {               // event -> transition
        if (state == OFF) state = ON;
        else if (state == ON) state = BLINK;
        else state = OFF;
    }

    switch (state) {                     // behaviour of each state
        case OFF: digitalWrite(LED_PIN, LOW); break;
        case ON: digitalWrite(LED_PIN, HIGH); break;
        case BLINK:
            if (millis() - lastBlink >= 250) {
                lastBlink = millis();
                blinkLevel = !blinkLevel;
                digitalWrite(LED_PIN, blinkLevel);
            }
            break;
    }
}
```

REAL-LIFE EXAMPLE | STATE MACHINES EVERYWHERE

Traffic lights (Red → Red+Amber → Green → Amber), washing machines (fill → wash → rinse → spin), vending machines, ATMs and elevators are state machines. Once you learn to think this way, complicated behaviour becomes a list of states and rules.

16.7 Many outputs: patterns with arrays

Combining arrays (Chapter 15) with loops lets you drive rows of LEDs with tiny code. Store patterns as data rather than as instructions:

Pattern table driven by millis()

```
const int ledPins[] = {8, 9, 10, 11};
const int N = 4;
// Each row is one step in the pattern: 1 = LED on
const byte pattern[][4] = {
  {1,0,0,0}, {0,1,0,0}, {0,0,1,0}, {0,0,0,1}, {0,0,1,0}, {0,1,0,0}
};
const int STEPS = sizeof(pattern) / sizeof(pattern[0]);
unsigned long last = 0;
int step = 0;

void setup() {
  for (int i = 0; i < N; i++) pinMode(ledPins[i], OUTPUT);
}

void loop() {
  if (millis() - last >= 120) {
    last = millis();
    for (int i = 0; i < N; i++) digitalWrite(ledPins[i], pattern[step][i]);
    step = (step + 1) % STEPS; // wrap around after the last step
  }
}
```

Modify the table to make new animations without touching the logic: this “data-driven” style is much easier to maintain. If you need more than ~10 outputs, use a shift register (Chapter 20) or an I/O expander (MCP23017), rather than more pins.

Port manipulation (advanced, optional)

`digitalWrite()` takes about 4 μ s. To change many pins at once or run faster, write directly to the port registers: pins 8–13 are **PORTB** bits 0–5, pins 0–7 are **PORTD**, A0–A5 are **PORTC**. For example `PORTB = 0b00001111`; sets pins 8–11 HIGH and 12–13 LOW in one instruction (about 62 ns), and `DDRB = 0b00111111`; makes pins 8–13 outputs. It is fast, but less readable, so use it only when needed.

Arduino function	Direct register (pin 13 = PB5)	Time
<code>pinMode(13, OUTPUT)</code>	<code>DDRB = (1 << 5);</code>	—
<code>digitalWrite(13, HIGH)</code>	<code>PORTB = (1 << 5);</code>	$\approx 4 \mu\text{s}$ vs $\approx 0.1 \mu\text{s}$
<code>digitalWrite(13, LOW)</code>	<code>PORTB &= ~(1 << 5);</code>	same
<code>digitalRead(2)</code>	<code>(PIND >> 2) & 1</code>	$\approx 4 \mu\text{s}$ vs $\approx 0.1 \mu\text{s}$

HANDS-ON EXPERIMENT | EXPERIMENT 11: A REACTION-TIME GAME

You need: Arduino, breadboard, 1 LED + 330 Ω , 1 push button, jumper wires.

Goal: After a random wait of 2 – 5 s the LED lights; you press the button as fast as you can; the Serial Monitor prints your reaction time in ms.

Wire: LED on D8 (with resistor to GND), button between D2 and GND. Use `INPUT_PULLUP`.

Hints: (1) In `setup()` call `randomSeed(analogRead(A5))`; (2) Wait with `delay(random(2000, 5000))`; (delay is fine here because nothing else is happening). (3) Turn the LED on and record `unsigned long t0 = millis()`; (4) Wait with `while (digitalRead(2) == HIGH) {}`. (5) Compute `millis() - t0` and print it. (6) Detect cheating: if the button is already pressed before the LED lights, print “Too early!”.

A typical human reaction time is 200 – 300 ms. Record your best of ten tries.

CHAPTER SUMMARY

- Modes: `OUTPUT` drives, `INPUT` reads, `INPUT_PULLUP` reads with an internal pull-up (button to GND: pressed = LOW).
- Detect **edges** by remembering the last state; **debounce** with a 20–50 ms time filter based on `millis()`.
- `delay()` blocks everything; use `if (millis() - previous >= interval)` to run periodic tasks without blocking. Store times in `unsigned long`.
- **State machines** (enum + switch) make multi-mode behaviour clear and reliable.
- Arrays hold pin lists and patterns; the modulus operator `%` wraps counters; port registers give speed when needed.

ASSIGNMENTS | CHAPTER 16

16.1 [Practice] A button is wired between pin 4 and GND with `INPUT_PULLUP`. Write the condition that is true while the button is pressed.

16.2 [Practice] Modify the LED-follows-button sketch to use an external pull-down and a button to +5 V. What changes in the code and in the wiring?

16.3 [Practice] Write a sketch that counts button presses (debounced) and prints the count to the Serial Monitor each time it changes.

16.4 [Calculate] How long does it take for `millis()` to overflow? (unsigned long, 4,294,967,295 ms)

16.5 [Calculate] The RC debounce filter uses $R = 10\text{ k}\Omega$ (the pull-up) and $C = 100\text{ nF}$. What is its time constant, and how long to settle to 95 %?

16.6 [Calculate] A sketch reads a button 5,000 times per second using `loop()`. If the bounce lasts 8 ms, roughly how many times could the raw reading flip during one press?

16.7 [Design] Write a program with two LEDs: LED A blinks at 2 Hz, LED B blinks at 0.5 Hz, both without using `delay()`.

16.8 [Design] Design (as a state diagram) a traffic light with states RED (5 s), GREEN (4 s), YELLOW (1 s). Then write the code using `millis()`.

16.9 [Think] Why is it better to declare `unsigned long previous` as a global variable rather than inside `loop()`?

16.10 [Think] Explain in words why `if (now - previous >= interval)` still works when `millis()` rolls over from 4,294,967,295 to 0.

16.11 [Hands-on] Build and run Experiment 11. Add a second attempt mode: after each game, print the average reaction time of all your tries so far.

Analog inputs and sensors

Analog inputs let the Arduino sense the continuous world: how bright, how hot, how far a knob is turned. You already know the theory: a 10-bit ADC compares the input voltage with a reference and returns a number 0 – 1023 (Chapter 12), and a voltage divider turns a changing resistance into a changing voltage (Chapter 5). This chapter puts them together to build real sensing circuits, and teaches the practical tricks that make readings stable and accurate.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How `analogRead()` works, its range, timing and reference voltage options
- Reading a **potentiometer** and converting to volts and to useful ranges
- Reading resistive sensors with a **voltage divider**: LDR, thermistor, flex/force sensors
- Using voltage-output sensors such as the **LM35** and **TMP36**, and the thermistor equation
- **Smoothing and filtering** noisy readings (average, moving average, exponential filter)
- **Calibration**, thresholds with hysteresis, and getting the best accuracy from the ADC

17.1 The `analogRead()` function

`analogRead(pin)` reads one of A0 – A5 and returns an integer **0 – 1023** proportional to the voltage between 0 V and the ADC reference (5 V by default). It takes about **100 µs** (the ADC is a 13-cycle successive-approximation converter at 125 kHz) and needs no `pinMode`.

Read and print an analog voltage

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  int raw = analogRead(A0);           // 0..1023
  float volts = raw * 5.0 / 1023.0;   // convert to volts
  Serial.print(raw);
  Serial.print(" ");
  Serial.print(volts, 3);              // 3 decimal places
  Serial.println(" V");
  delay(250);
}
```

Reference voltage

The ADC measures the input **relative to the reference**, so the reference sets the full-scale range and the resolution. Options with `analogReference(type)` (call it in `setup()` before `analogRead`):

Reference	Call	Full scale	Resolution (10 bit)	Remarks
DEFAULT	(nothing)	5 V (the supply, so its accuracy depends on your USB/regulator)	4.88 mV	Ratiometric sensors (pots, divider sensors powered from 5 V) are unaffected by supply variations

Reference	Call	Full scale	Resolution (10 bit)	Remarks
INTERNAL	<code>analogReference(INTERNAL);</code>	1.1 V (built-in bandgap reference)	1.07 mV	Very stable; ideal for small signals such as LM35 (10 mV/°C) and current shunts. Never apply more than 1.1 V!
EXTERNAL	<code>analogReference(EXTERNAL);</code>	voltage on the AREF pin	$V_{ref}/1024$	For a precision reference IC. Never apply voltage to AREF while using DEFAULT/INTERNAL settings without the matching call: it shorts the internal reference

SAFETY WARNING

With `analogReference(INTERNAL)` an input above 1.1 V just reads 1023 (it saturates), and voltages above 5 V still damage the pin. If you connect an external voltage to AREF you must select EXTERNAL **before** any `analogRead`, otherwise the internal and external references fight each other.

Ratiometric measurement

When a sensor is powered from the same 5 V as the ADC reference (a potentiometer or a divider), a change in the supply voltage changes both the signal and the reference, so **the ratio (the reading) stays the same**. That is a happy accident which makes such circuits robust against a sagging USB supply. In contrast, when you measure an absolute voltage (a battery, a LM35 output), an inaccurate reference gives an inaccurate result. Calibrate with a multimeter, or use the internal 1.1 V reference.

17.2 Potentiometers: converting readings

A pot wired as a divider (outer legs to 5 V and GND, wiper to A0) gives a reading proportional to the knob position. Typical conversions:

Typical conversions

```
int raw = analogRead(A0); // 0..1023

int brightness = map(raw, 0, 1023, 0, 255); // for analogWrite (Chapter 18)
int angle      = map(raw, 0, 1023, 0, 180);  // for a servo (Chapter 18)
float percent  = raw * 100.0 / 1023.0;      // 0..100 %
float volts    = raw * 5.0 / 1023.0;        // 0..5 V
int level      = raw / 128;                 // 8 discrete levels 0..7
```

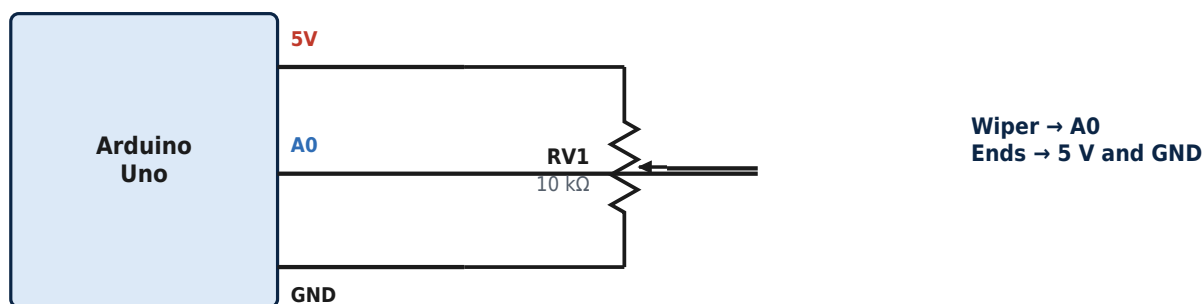


Figure 17.1 Potentiometer as a voltage divider feeding an analog input.

TIP

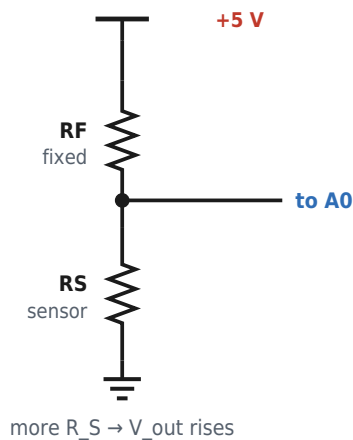
If the reading goes the “wrong way” when you turn the knob, swap the 5 V and GND connections to the outer pins. And if it jitters by ± 1 –2 counts, that is normal ADC noise: see the filtering section below.

17.3 Resistive sensors and the voltage divider

Many sensors change **resistance**. The Arduino measures voltage, not resistance, so we build a divider (Chapter 5) with a fixed resistor R_F and the sensor R_S , and read the junction. Which arrangement (sensor at the top or bottom) decides whether the reading rises or falls with the stimulus:

$$V_{\text{out}} = 5 \text{ V} \times R_{\text{bottom}} / (R_{\text{top}} + R_{\text{bottom}})$$

Sensor at the BOTTOM



Sensor at the TOP

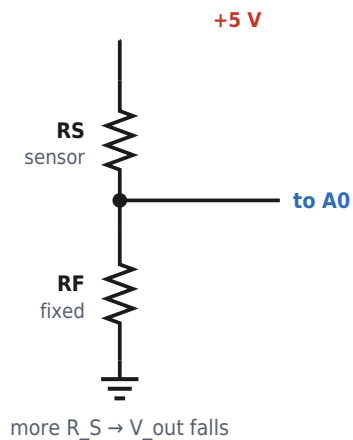


Figure 17.2 Two ways to read a resistive sensor. Choose the arrangement so that the reading rises with the stimulus you care about.

TIP

Choosing the fixed resistor: pick R_F close to the sensor’s resistance in the middle of its range. That maximises the swing of V_{out} (a divider changes most when both resistors are similar) and gives the best use of the ADC’s range. For an LDR that goes from 1 k Ω (bright) to 100 k Ω (dark), a 10 k Ω fixed resistor is a good compromise.

The LDR (light-dependent resistor)

An LDR is a piece of cadmium-sulphide film whose resistance falls as light increases: about 1 k Ω in bright light and 100 k Ω – 1 M Ω in darkness (the values vary greatly between parts). It is slow (tens of milliseconds) and not linear, but perfect for “is it light or dark?” tasks. With the LDR on top and 10 k Ω at the bottom, the reading **rises with more light**.

Automatic night light with hysteresis

```
const int LDR_PIN = A0;
const int LED_PIN = 9;
const int DARK_ON = 300;    // turn ON when reading falls below this
const int DARK_OFF = 350;   // turn OFF when it rises above this (hysteresis)

bool lightOn = false;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int light = analogRead(LDR_PIN);    // higher = brighter (LDR on the top)
  if (!lightOn && light < DARK_ON) lightOn = true;
  if (lightOn && light > DARK_OFF) lightOn = false;
  digitalWrite(LED_PIN, lightOn);
  Serial.println(light);
  delay(100);
}
```

Hysteresis (two thresholds) prevents the light from flickering on and off when the reading hovers around a single threshold: it switches on below 300 and off only above 350, leaving a “dead band”. Note that turning on the LED may itself change the light seen by the LDR (feedback): with hysteresis wide enough, this is harmless; otherwise shield the LDR from the LED.

The thermistor

A **thermistor** is a resistor whose resistance changes strongly with temperature. An **NTC** (negative temperature coefficient) thermistor’s resistance falls as it warms; the common **10 kΩ NTC** has 10 kΩ at 25 °C. The relationship is exponential, described well by the **beta equation**:

$$1/T = 1/T_0 + (1/\beta) \times \ln(R / R_0)$$

T in kelvin; $T_0 = 298.15$ K (25 °C); $R_0 = 10$ kΩ; $\beta \approx 3950$ K (from the datasheet)

Wiring: 10 kΩ NTC on top of a 10 kΩ fixed resistor (to GND), reading at the junction. Then R_{NTC} is found from the reading:

$$R_{NTC} = R_F \times (1023 / ADC - 1)$$

For the NTC on the TOP and the fixed resistor on the bottom

Temperature from an NTC thermistor

```
const int THERM_PIN = A0;
const float R_FIXED = 10000.0; // 10k fixed resistor to GND
const float R0 = 10000.0; // thermistor resistance at 25 °C
const float T0 = 298.15; // 25 °C in kelvin
const float BETA = 3950.0; // from the datasheet

void setup() { Serial.begin(9600); }

void loop() {
  int adc = analogRead(THERM_PIN);
  float r = R_FIXED * (1023.0 / adc - 1.0); // thermistor resistance
  float tK = 1.0 / (1.0 / T0 + log(r / R0) / BETA); // beta equation (log = natural log)
  float tC = tK - 273.15;
  Serial.print("Temperature: ");
  Serial.print(tC, 1);
  Serial.println(" °C");
  delay(500);
}
```

WORKED EXAMPLE | CHECKING BY HAND

At 25 °C the thermistor has 10 kΩ, so the divider gives exactly half: ADC = 511.5. Compute: $r = 10\,000 \times (1023 / 511.5 - 1) = 10\,000\,\Omega \rightarrow \ln(1) = 0 \rightarrow 1/T = 1/298.15 \rightarrow T = 25\,^{\circ}\text{C}$ ✓.

At 50 °C $R \approx 3.6\,\text{k}\Omega$ ($\beta = 3950$): $\text{ADC} = 1023 \times 10 / (3.6 + 10) = 752$; the formula returns 3.6 kΩ and $T \approx 50\,^{\circ}\text{C}$ ✓. Near 25 °C the divider output changes by about 11 counts per °C, i.e. $\sim 0.09\,^{\circ}\text{C}$ resolution, though absolute accuracy is limited to about $\pm 1\,^{\circ}\text{C}$ by tolerance.

Self-heating warning: a thermistor dissipates $I^2 R$; at 5 V and 10 kΩ + 10 kΩ, $I = 0.25\,\text{mA}$ and the sensor dissipates 0.6 mW, negligible.

Other resistive sensors

Sensor	Resistance range	Fixed resistor	Notes
LDR (GL5528)	1 kΩ (bright) – 1 MΩ (dark)	10 kΩ	Slow; non-linear; not calibrated
10 kΩ NTC thermistor	30 kΩ (5 °C) – 1 kΩ (100 °C)	10 kΩ	Use beta equation or Steinhart-Hart
Force-sensitive resistor (FSR 402)	> 1 MΩ (no force) – 200 Ω (hard press)	10 kΩ – 100 kΩ	Sensitivity, not accuracy: detects pressure levels
Flex sensor (2.2")	25 kΩ (flat) – 100 kΩ (bent)	47 kΩ – 100 kΩ	Gloves, bend detection
Soil moisture (resistive)	varies with wetness	built into the module	Corrodes quickly: prefer capacitive types
Potentiometer / slide pot	0 – 10 kΩ	divider itself	Position sensing

17.4 Voltage-output sensors

Some sensors integrate the electronics and give a voltage proportional to the quantity. No divider needed:

Sensor	Output	Range	Conversion
LM35	10 mV per °C, 0 V at 0 °C	2 – 150 °C (basic)	$^{\circ}\text{C} = (\text{analogRead} \times 5.0 / 1023) \times 100$
TMP36	10 mV per °C with 500 mV offset	–40 – 125 °C	$^{\circ}\text{C} = (V - 0.5) \times 100$
MCP9700	10 mV per °C, 500 mV at 0 °C	–40 – 125 °C	like TMP36

Sensor	Output	Range	Conversion
ACS712 current sensor	2.5 V $\pm$ 66–185 mV per A	$\pm$ 5/20/30 A	$A = (V - 2.5) / \text{sensitivity}$
Analog accelerometer (ADXL335)	$\approx 1.65 \text{ V} \pm 0.33 \text{ V}$ per g (3.3 V)	$\pm 3 \text{ g}$	$g = (V - 1.65) / 0.33$
Sharp GP2Y0A21 IR distance	non-linear voltage (0.4 – 3 V)	10 – 80 cm	lookup / power-law fit

LM35 temperature sensor, high resolution

```
// LM35: Vout = 10 mV/degC. Use the 1.1 V internal reference for better resolution:
// 1.1 V / 1024 = 1.07 mV per step -> 0.107 degC per step (vs 0.49 degC with a 5 V reference)
void setup() {
  analogReference(INTERNAL);           // range 0..1.1 V = 0..110 degC
  Serial.begin(9600);
}

void loop() {
  int raw = analogRead(A0);
  float tempC = raw * 1.1 * 100.0 / 1023.0;
  Serial.print(tempC, 1);
  Serial.println(" C");
  delay(1000);
}
```

TIP

Power the sensor from the same 5 V rail, put a 100 nF capacitor across its supply pins, and keep the wire short. When you switch the reference, the first one or two readings after `analogReference()` can be wrong: throw them away.

17.5 Noise and filtering

Analog readings jitter: thermal noise, supply ripple, radio pickup and the ADC's quantisation. A pot at rest may flicker between 511, 512 and 513; a sensor near a motor may swing by 20. Software filtering trades responsiveness for stability.

1. Averaging N samples

Average of several readings

```
int readAveraged(int pin, int samples) {
  long sum = 0;           // long: 16 x 1023 would overflow an int
  for (int i = 0; i < samples; i++) {
    sum += analogRead(pin);
    delayMicroseconds(200); // let the ADC input capacitor settle
  }
  return sum / samples;
}
```

Averaging N independent readings reduces random noise by a factor of about $\sqrt{N}$: 16 samples give ~ 4 times less noise. It costs time ($16 \times 100 \mu\text{s} = 1.6 \text{ ms}$), which is negligible for slowly changing signals.

2. Exponential smoothing (a cheap low-pass filter)

$$y_{\text{new}} = \alpha \times x + (1 - \alpha) \times y_{\text{old}}$$

x = new reading; y = filtered value; α between 0 (very smooth) and 1 (no filtering)

Exponential moving average

```
float filtered = 0;

void loop() {
  int raw = analogRead(A0);
  filtered = 0.1 * raw + 0.9 * filtered; // alpha = 0.1: strong smoothing
  Serial.println(filtered);
  delay(10);
}
```

This is the digital version of the RC filter in Chapter 6: with a sample time T_s the equivalent time constant is $\tau \approx T_s (1 - \alpha) / \alpha$. For $\alpha = 0.1$ and 10 ms sampling, $\tau \approx 90$ ms: the output reaches 63 % of a step change in 90 ms. Very little memory is used: one variable.

3. Median filter (against spikes)

A median of 3–5 readings ignores occasional wild spikes (from motor noise, for instance) that would distort an average. Sort the readings and take the middle one.

4. Hardware filtering

- A **100 nF ceramic capacitor** from the analog pin to GND (with a source impedance of 1–10 k Ω) forms an RC low-pass that removes high-frequency noise.
- **Keep sensor wires short** and away from motors, relays and mains wiring; use twisted pairs or shielded cable for long runs.
- **Do not read the ADC while switching heavy loads.** Sample before or after, not during.
- **Source impedance:** the ADC works best with a source resistance below ~ 10 k Ω . A large divider (100 k Ω) can give sloppy readings; add a 100 nF capacitor at the pin, or a buffer (op-amp follower, Chapter 11).
- **When switching between analog channels,** the first reading after the switch can be contaminated by the previous channel: read twice and discard the first.

17.6 Calibration and thresholds

No two sensors are identical. Calibration links readings to reality. The simplest method is a **two-point calibration**: measure the raw reading at two known conditions (for a thermistor: ice water at 0 °C and a reference thermometer at 40 °C), then use `map()` or the line equation to convert.

WORKED EXAMPLE | TWO-POINT CALIBRATION

A sensor reads 210 in ice water (0 °C) and 625 at 40 °C. Then $T = (\text{raw} - 210) \times 40 / (625 - 210) = (\text{raw} - 210) \times 0.0964$. A reading of 400 means $(400 - 210) \times 0.0964 = \mathbf{18.3\text{ }^{\circ}\text{C}}$.

In code: `float t = (raw - 210) * 40.0 / (625 - 210);` (use float arithmetic to avoid integer truncation).

For threshold-based devices (dark $\rightarrow$ LED on), **print the values** over a day's range with Serial and choose thresholds from actual data, not guesses. Always include hysteresis to prevent chatter.

HANDS-ON EXPERIMENT | EXPERIMENT 12: A DARKNESS METER AND A THERMOMETER

You need: Arduino, LDR, 10 k Ω , 10 k Ω NTC thermistor (or LM35/TMP36), 5 LEDs with 330 Ω , breadboard.

1. **LDR bar graph:** wire the LDR (top) and 10 k Ω (bottom) to A0. Read with the Serial Monitor in bright and dark conditions (cover with your hand) and record the range. Use `map(raw, darkValue, brightValue, 0, 5)` to decide how many of 5 LEDs (pins 8–12) to light: a light-level bar graph.
2. **Smoothing:** add the exponential filter ($\alpha = 0.1$). Compare the Serial output of raw and filtered values (print both separated by a comma and use Tools ► Serial Plotter to view them as a graph).
3. **Thermometer:** with the thermistor on A0 use the sketch above. Compare its reading with a known thermometer at room temperature, then hold the thermistor between your fingers and watch the value climb towards 32 °C. Measure the time constant (how long to reach 63 % of the total rise).
4. **Two-point calibration:** apply the calibration procedure and compare accuracy before and after.

CHAPTER SUMMARY

- `analogRead()` returns 0 – 1023 for 0 – 5 V (default reference); volts = raw $\times$ 5.0 / 1023. It takes $\sim$ 100 μ s.
- Use `analogReference(INTERNAL)` (1.1 V) for small signals like the LM35. Ratiometric sensors ignore supply variations.
- Resistive sensors are read via a **voltage divider**; choose the fixed resistor near the sensor's mid-range value; decide which one is on top to set the direction of change.
- LDR: light/dark switching with **hysteresis**. NTC thermistor: $R = R_F(1023/\text{ADC} - 1)$ then the **beta equation**. LM35/TMP36: 10 mV per °C.
- Reduce noise by averaging, exponential smoothing, median filtering, a 100 nF capacitor and short wires. Calibrate with two known points.

ASSIGNMENTS | CHAPTER 17

17.1 [Calculate] The Arduino reads 341 on A0 (5 V reference). What is the voltage? What would it read for 2.0 V?

17.2 [Calculate] What is the resolution (volts per count) with the 1.1 V internal reference? With an LM35, what temperature step is that?

17.3 [Calculate] An LDR of 20 k Ω (dim room) is on top of a 10 k Ω resistor to ground with a 5 V supply. What voltage and ADC value result? Repeat for bright light, LDR = 2 k Ω .

17.4 [Calculate] An NTC (10 k Ω at 25 °C) is on top of a 10 k Ω resistor. The ADC reads 620. Find the thermistor resistance, and say whether the temperature is above or below 25 °C.

17.5 [Calculate] A TMP36 outputs 0.75 V. What is the temperature? What ADC value does the Arduino read (5 V reference)?

17.6 [Calculate] Using $\alpha = 0.2$, the filtered value is 500 and a new reading of 600 arrives. What is the new filtered value? After a second reading of 600?

17.7 [Calculate] You average 25 samples. By what factor does random noise fall (approximately)?

17.8 [Practice] Write code that reads a potentiometer and outputs the message "LOW", "MEDIUM" or "HIGH" on the Serial Monitor when the reading is in the lower, middle or upper third.

17.9 [Practice] Modify the night-light sketch so that it turns the LED on after the room has been dark for 5 seconds (not immediately). Hint: use `millis()`.

17.10 [Think] Why can't you measure a 12 V battery directly with an analog pin? How can you fix it?

ASSIGNMENTS | CHAPTER 17

17.11 [Think] A divider reading jumps around by ± 30 counts when a nearby motor runs. List three measures.

17.12 [Hands-on] Complete Experiment 12 and record raw values at three temperatures (ice water, room, body). Are they in the order you expect for an NTC on top of the divider?

PWM: analog-style outputs, servos and sound

The Uno has no true analog output: its pins are either 0 V or 5 V. Yet we can dim LEDs, set motor speeds and make musical notes. The trick is to switch a pin on and off so fast that the load only feels the average. This technique, pulse-width modulation, is one of the most important in all of electronics, used in phone chargers, LED lighting, motor drives and audio amplifiers.

WHAT YOU WILL LEARN IN THIS CHAPTER

- What **pulse-width modulation (PWM)** is, duty cycle, and why the average voltage is proportional to it
- How to use `analogWrite()` to dim LEDs, mix colours on an RGB LED and control motor speed
- The Uno's PWM pins, frequencies and timers; PWM limits
- How to turn PWM into a real analog voltage with an RC filter
- Controlling **servo motors** with the Servo library
- Making sound: `tone()`, `noTone()`, notes and melodies with a buzzer or speaker

18.1 What is PWM?

A PWM signal is a square wave with a fixed **frequency** (how often the on/off cycle repeats) in which the fraction of each cycle spent HIGH, the **duty cycle**, is varied. The average voltage is proportional to the duty cycle:

$$V_{\text{average}} = V_{\text{supply}} \times \text{Duty cycle}$$

$$\text{Duty cycle} = t_{\text{ON}} \div (t_{\text{ON}} + t_{\text{OFF}}) = t_{\text{ON}} \div T$$

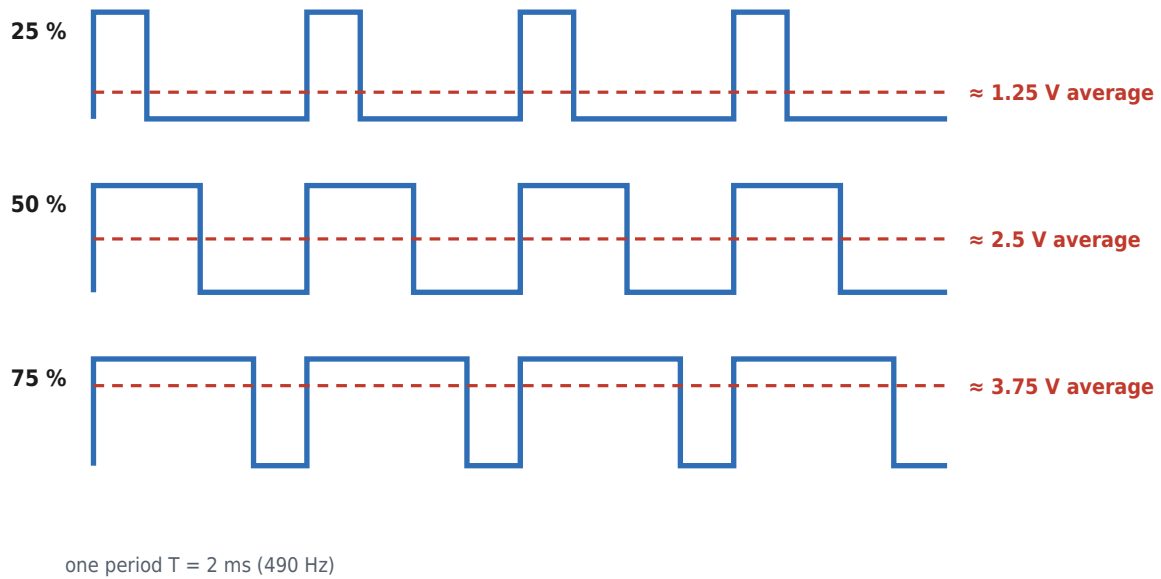


Figure 18.1 PWM signals with 25 %, 50 % and 75 % duty cycle. The dashed red line is the average voltage that a slow load (LED, motor, RC filter) responds to.

ANALOGY | FLICKING A LIGHT SWITCH FAST

Imagine switching a room light on and off very quickly. If it is on for a quarter of the time, the room looks dim; on for three quarters, bright; and if you flick fast enough, your eye can't see the flicker and only perceives the **average** brightness. The same goes for a motor: its inertia averages the pulses. It is like controlling the speed of a car by pressing the accelerator for short bursts: the car's inertia smooths them into a steady speed.

Why not just lower the voltage with a resistor? Because PWM is **efficient**: the transistor is either fully on (no voltage across it, no loss) or fully off (no current, no loss), so it dissipates almost no heat. A resistor or a linear regulator would burn the difference as heat (Chapter 13).

18.2 analogWrite() on the Uno

`analogWrite(pin, value)` outputs a PWM signal on pins **3, 5, 6, 9, 10, 11** (marked ~). The value is 0 – 255 (8-bit): 0 = always LOW, 255 = always HIGH, 128 $\approx$ 50 %.

$$\text{Duty cycle} = \text{value} \div 255$$

value = the number given to `analogWrite` (0 – 255)

$$V_{\text{average}} \approx 5 \text{ V} \times \text{value} \div 255$$

Pins	Timer	Default frequency
5, 6	Timer 0 (also used by <code>millis()</code> and <code>delay()</code>)	$\approx$ 980 Hz
9, 10	Timer 1 (16-bit; also used by the Servo library)	$\approx$ 490 Hz
3, 11	Timer 2 (used by <code>tone()</code>)	$\approx$ 490 Hz

GOOD TO KNOW

The `analogWrite()` name is misleading: there is no digital-to-analog converter. Also, **`analogWrite` on a pin makes it an output automatically** (still do `pinMode` for clarity). Using the **Servo library disables PWM on pins 9 and 10**, and `tone()` disables PWM on pins 3 and 11.

Fading an LED (a "breathing" light)

```
const int LED_PIN = 9;           // a PWM pin (~)

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  for (int b = 0; b <= 255; b++) { // fade up
    analogWrite(LED_PIN, b);
    delay(5);
  }
  for (int b = 255; b >= 0; b--) { // fade down
    analogWrite(LED_PIN, b);
    delay(5);
  }
}
```

Perceived brightness is not linear

The eye responds roughly logarithmically to light, so a duty of 50 % looks much brighter than “half as bright”. For smooth fades, apply a gamma correction: `int out = pow(b / 255.0, 2.2) * 255;`. With a linear ramp you will notice that most of the visual change happens at the low end.

Dimming with a potentiometer

Knob controls brightness

```
const int POT = A0, LED = 9;

void setup() { pinMode(LED, OUTPUT); }

void loop() {
  int raw = analogRead(POT);           // 0..1023
  int duty = map(raw, 0, 1023, 0, 255); // rescale to 0..255
  analogWrite(LED, duty);
}
```

RGB LED colour mixing

An RGB LED contains three LEDs; feeding each with its own PWM level mixes any colour (additive mixing). With a **common-cathode** part, connect the longest lead to GND and each colour pin through its own resistor (Chapter 8) to a PWM pin. For a **common-anode** part, connect the common lead to +5 V and *invert* the values (`255 - value`).

RGB colours

```
const int R_PIN = 9, G_PIN = 10, B_PIN = 11;

void setColor(int r, int g, int b) {          // values 0..255
  analogWrite(R_PIN, r);
  analogWrite(G_PIN, g);
  analogWrite(B_PIN, b);
}

void setup() {
  pinMode(R_PIN, OUTPUT); pinMode(G_PIN, OUTPUT); pinMode(B_PIN, OUTPUT);
}

void loop() {
  setColor(255, 0, 0); delay(700); // red
  setColor(0, 255, 0); delay(700); // green
  setColor(0, 0, 255); delay(700); // blue
  setColor(255, 255, 0); delay(700); // yellow = red + green
  setColor(0, 255, 255); delay(700); // cyan
  setColor(255, 0, 255); delay(700); // magenta
  setColor(255, 255, 255); delay(700); // white (all three)
  setColor(255, 80, 0); delay(700); // orange
}
```

18.3 PWM for power: motors and LED strips

The same `analogWrite()` values can control the speed of a DC motor or the brightness of a strip of LEDs, **if the PWM pin drives a transistor/MOSFET** and not the load directly (Chapter 9, Figure 10.4). Each pulse turns the MOSFET fully on or off, so it stays cool. Motor speed is roughly proportional to the average voltage, but motors need a minimum duty cycle to overcome friction (typically 20 – 40 %). Below that they just hum.

Motor speed with a pot (through a MOSFET)

```
const int MOTOR_PIN = 5;          // PWM pin -> 220 ohm -> logic-level MOSFET gate
const int POT = A0;

void setup() { pinMode(MOTOR_PIN, OUTPUT); }

void loop() {
  int speed = map(analogRead(POT), 0, 1023, 0, 255);
  if (speed < 40) speed = 0;        // dead zone: motor would only hum
  analogWrite(MOTOR_PIN, speed);
}
```

TIP

PWM frequency matters. Motors are happiest at 1 – 20 kHz; below ~500 Hz you may hear a whine and feel the motor "chatter"; at very high frequency, the MOSFET switching losses rise. LEDs are best at ≥ 490 Hz to avoid visible flicker in cameras. You can change the frequency by changing the timer prescaler (e.g., `TCCR1B = (TCCR1B & 0b11111000) | 0x01`; sets pins 9 & 10 to 31 kHz), but this affects timing functions on Timer 0 (pins 5 & 6), so only change Timer 1 or 2 if you can.

18.4 A real analog voltage from PWM: the RC filter

If you need an actual DC voltage (for example to set a reference or control an analog input of another circuit), smooth the PWM with a low-pass RC filter (Chapter 6). The filter's corner frequency must be far below the PWM frequency, and the more attenuation you need for the ripple, the slower the response.

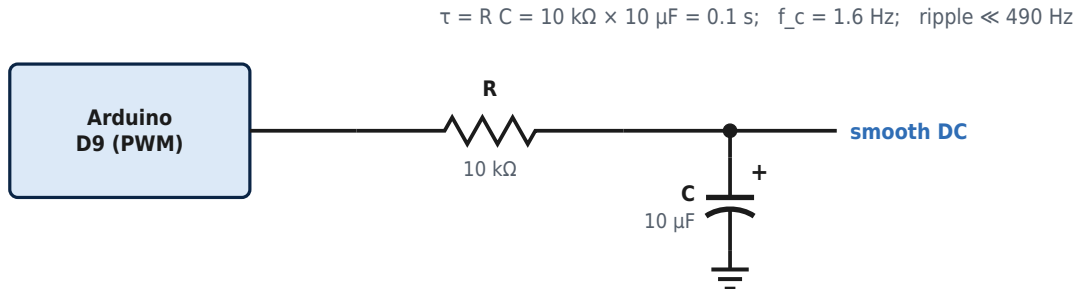


Figure 18.2 A first-order RC filter turns PWM into a steady analog voltage ($V_{\text{out}} \approx 5\text{ V} \times \text{duty}$). Average voltage appears after roughly $5\tau = 0.5\text{ s}$.

WORKED EXAMPLE | RIPPLE ESTIMATE

For a first-order RC filter with a corner frequency $f_c = 1/(2\pi RC)$ and PWM at f_{PWM} , the ripple attenuation is about f_c/f_{PWM} . Here $f_c = 1.6\text{ Hz}$ and $f_{\text{PWM}} = 490\text{ Hz}$ gives a factor of 300; the 5 V peak-to-peak PWM shows about $5\text{ V} / 300 \approx 17\text{ mV}$ of ripple at 50 % duty (worst case is a little larger). That is about 3 ADC counts: acceptable for many uses, and a two-stage filter makes it much better.

18.5 Servo motors

A hobby servo (Chapter 10) positions its shaft according to the width of a pulse repeated every 20 ms. The **Servo library** generates the correct pulses on any pin with a simple interface:

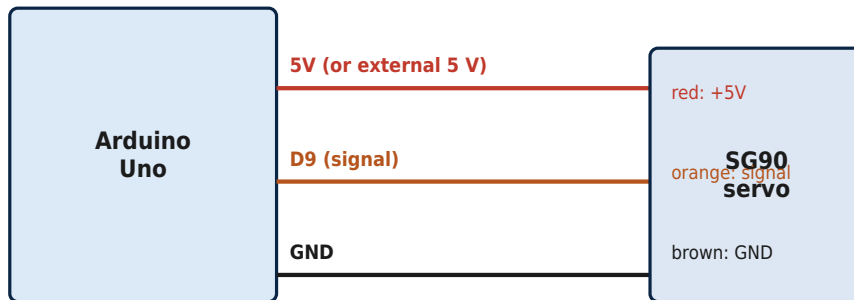
Servo basics

```
#include <Servo.h>

Servo myServo;                // create a servo object
const int SERVO_PIN = 9;

void setup() {
  myServo.attach(SERVO_PIN);    // signal wire to pin 9 (attach can also take min/max pulse widths)
}

void loop() {
  myServo.write(0);             // go to 0 degrees
  delay(1000);
  myServo.write(90);            // centre
  delay(1000);
  myServo.write(180);           // 180 degrees
  delay(1000);
}
```



add 470–1000 μ F
across the servo
supply for larger
servos

Figure 18.3 Servo wiring: red to 5 V, brown/black to GND, orange/yellow to a signal pin. For more than one small servo, use a separate 5 V supply and join the grounds.

Sweeping smoothly and using a potentiometer

Servo follows a potentiometer

```
#include <Servo.h>
Servo s1;

void setup() {
  s1.attach(9);
}

void loop() {
  int raw = analogRead(A0);
  int angle = map(raw, 0, 1023, 0, 180);
  s1.write(angle);           // servo follows the knob
  delay(15);                 // give the servo time to move
}
```

Slow sweep with millis()

```
// Slow sweep without blocking (millis-based)
#include <Servo.h>
Servo s1;
int angle = 0, dir = 1;
unsigned long last = 0;

void setup() { s1.attach(9); }

void loop() {
  if (millis() - last >= 20) {           // 20 ms per degree: 3.6 s for a full sweep
    last = millis();
    angle += dir;
    if (angle >= 180 || angle <= 0) dir = -dir; // reverse at the ends
    s1.write(angle);
  }
}
```

TIP

`write()` takes degrees (0–180), `writeMicroseconds(1500)` takes the pulse width directly (more precise; 1000–2000 μ s is nominal, real servos range 500 – 2500 μ s). Servos jitter or hum when the supply is inadequate or the command is beyond the mechanical limit: keep the range slightly inside 0–180 if it buzzes at the ends. Use `detach()` to stop the pulses and save power when the servo doesn't need to hold position.

SAFETY WARNING

An SG90 servo can draw 500 – 700 mA when stalled or when moving a heavy load. Powering two or more from the Arduino 5V pin often causes brownouts and resets. Provide 5 V from a separate source (4×AA pack, a 5 V 2 A adapter or a buck converter), **connect its ground to the Arduino GND**, and add a 470 µF capacitor.

18.6 Making sound

Sound is vibration of the air. A speaker or buzzer vibrates when driven by an alternating signal; the **frequency** of the alternation is the **pitch** (Hz): the note A4 is 440 Hz. The Arduino can generate a square wave of any frequency with `tone(pin, frequency)` (or `tone(pin, frequency, duration_ms)`), and stop with `noTone(pin)`.

Note	C4	D4	E4	F4	G4	A4	B4	C5
Frequency (Hz)	262	294	330	349	392	440	494	523

Each semitone multiplies the frequency by $2^{1/12} \approx 1.0595$; an octave doubles it. Humans hear roughly 20 Hz – 20 kHz, with small piezo buzzers loudest around 2 – 4 kHz.

A piezo buzzer needs no resistor; a speaker needs a series resistor (or a transistor + amplifier for volume).

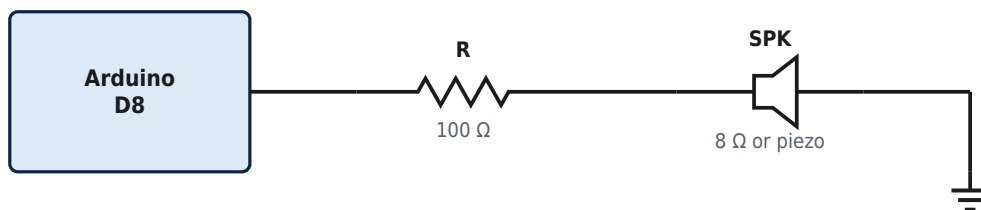


Figure 18.4 A small speaker or passive buzzer on a digital pin. For more volume use a transistor driver (Chapter 9) or an audio amplifier module (LM386, PAM8403).

tone() basics

```
const int BUZZER = 8;

void setup() {
  // nothing: tone() sets the pin mode itself
}

void loop() {
  tone(BUZZER, 440);           // play 440 Hz (A4) until noTone() is called
  delay(500);
  noTone(BUZZER);             // silence
  delay(500);

  tone(BUZZER, 880, 200);      // play 880 Hz for exactly 200 ms (returns immediately)
  delay(1000);
}
```

A melody from arrays

```
// Play a scale and a short melody
const int BUZZER = 8;
const int NOTE_C4 = 262, NOTE_D4 = 294, NOTE_E4 = 330, NOTE_F4 = 349,
      NOTE_G4 = 392, NOTE_A4 = 440, NOTE_B4 = 494, NOTE_C5 = 523;

// "Ode to Joy" (start): E E F G G F E D C C D E E D D
const int melody[] = {NOTE_E4, NOTE_E4, NOTE_F4, NOTE_G4, NOTE_G4, NOTE_F4, NOTE_E4, NOTE_D4,
      NOTE_C4, NOTE_C4, NOTE_D4, NOTE_E4, NOTE_E4, NOTE_D4, NOTE_D4};
const int beats[] = {1,1,1,1,1,1,1, 1,1,1,1, 2,1,3}; // note lengths in beats
const int N = sizeof(melody) / sizeof(melody[0]);
const int BEAT_MS = 300;

void setup() {
  for (int i = 0; i < N; i++) {
    int duration = beats[i] * BEAT_MS;
    tone(BUZZER, melody[i], duration * 0.9); // play for 90% of the beat
    delay(duration); // pause for the full beat (silence between notes)
  }
}

void loop() {} // play once
```

COMMON MISTAKE

`tone()` can play only **one tone at a time** and it uses Timer 2, disabling PWM on pins 3 and 11. To play two notes together you need external hardware or a different library. Also, `tone()` on a pin driving a large speaker directly can draw too much current: always use a resistor or a transistor.

HANDS-ON EXPERIMENT | EXPERIMENT 13: LIGHT, COLOUR AND SOUND

You need: Arduino, breadboard, LED + 330 Ω on D9, RGB LED + three 220 Ω , potentiometer, servo, passive buzzer, jumper wires.

1. Fade the LED up and down; then control the brightness with the potentiometer. Watch the LED with a phone camera: at 490 Hz you may see banding lines.
2. Connect the RGB LED. Use three potentiometers (A0, A1, A2) to mix colours. Find the values for pure white: usually the mix is not perfectly white because each colour has a different brightness; adjust the resistors or the values.
3. Connect the servo (external supply if you have one). Make it sweep 0 – 180° and then follow the pot. Measure the real range of your servo.
4. Build a **theremin-style instrument**: read an LDR on A0 (Chapter 17), map the value to 200 – 2000 Hz with `map()`, and call `tone(8, freq)`. Wave your hand over the LDR to play.
5. **Challenge**: combine all three: pot → servo angle, servo angle → tone pitch, and RGB colour according to the angle.

CHAPTER SUMMARY

- PWM switches a pin on/off rapidly; average voltage = supply × duty cycle. `analogWrite(pin, 0.255)` on pins 3, 5, 6, 9, 10, 11 (~).
- PWM is efficient (a switch has no loss). Use MOSFETs for motors/LED strips, compensate for non-linear brightness, and mind the PWM frequency.
- An RC low-pass filter turns PWM into a real DC voltage.
- Servo: `#include <Servo.h>`, `attach(pin)`, `write(angle)`; needs 5 V with up to 700 mA: use a separate supply and a common ground.
- `tone(pin, Hz[, ms])` and `noTone(pin)` make one square-wave tone at a time; note A4 = 440 Hz; +1 semitone = $\times 1.0595$.

ASSIGNMENTS | CHAPTER 18

18.1 [Calculate] What is the average voltage for `analogWrite(9, 64)` on a 5 V Uno? For `analogWrite(9, 200)`?

18.2 [Calculate] A 490 Hz PWM signal has a period of what duration? If the duty cycle is 30 %, how long is the HIGH time?

18.3 [Calculate] You want to create 2.0 V with a filtered PWM output from a 5 V Arduino. What `analogWrite` value do you use?

18.4 [Calculate] Design an RC filter for PWM at 490 Hz with a corner frequency of 5 Hz. Choose $R = 10\text{ k}\Omega$ and find C.

18.5 [Calculate] A servo needs a pulse of 1.0 ms for 0° and 2.0 ms for 180° . What pulse width corresponds to 60° ? What duty cycle is this at 50 Hz?

18.6 [Calculate] What is the frequency of the note one octave above A4? Of the note one semitone above A4 (A#4)?

18.7 [Practice] Write code to blink an RGB LED through red, green, blue in sequence, with each colour fading in and out over about 1 second.

18.8 [Practice] Write a program that plays the tone corresponding to the potentiometer position, quantised to the 8 notes of the C major scale.

18.9 [Think] Why does a servo sometimes twitch when the Arduino also drives a motor from the same supply?

18.10 [Think] Why is PWM dimming of an LED more energy-efficient than putting a larger series resistor?

18.11 [Hands-on] Build the LDR theremin (Experiment 13 step 4). Determine the frequency range you obtain and adjust the `map()` so that it spans exactly two octaves.

A tour of common sensors

Chapter 17 taught you the principles of analog sensing. This chapter is a practical tour of the sensors you will use most often, each with its wiring, code and quirks. Every sensor here follows the same pattern: power it correctly, connect its output, read it, convert the reading to physical units, and filter it. If you understand that recipe you can use almost any sensor on the market.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How to use the **HC-SR04 ultrasonic** distance sensor, and the physics behind it
- How to detect motion with a **PIR** sensor and read **DHT11/DHT22** temperature and humidity
- How to use digital-output modules, sound and gas sensors, and read a **rotary encoder**
- How to install and use **libraries** for sensors, and read the library's example code
- How to choose and combine sensors sensibly

19.1 Ultrasonic distance: HC-SR04

The **HC-SR04** measures distance by **echo-location**, exactly as a bat does. It emits a burst of 40 kHz ultrasound (beyond human hearing) and times how long the echo takes to return. Sound travels at about **343 m/s** in air at 20 °C (0.0343 cm/μs), and the pulse goes out and back, so:

$$\text{distance} = (t_{\text{echo}} \times 0.0343) \div 2 \text{ [cm, with } t \text{ in } \mu\text{s]}$$

$$\text{or distance} \approx t_{\text{echo}} \div 58 \text{ [cm]}$$

HC-SR04 pin	Connect to	Function
VCC	5 V	Power (15 mA)
TRIG	Any digital pin (e.g. D9)	A ≥ 10 μs HIGH pulse starts a measurement
ECHO	Any digital pin (e.g. D10)	Output pulse whose width equals the echo time (5 V output: fine for Uno; use a divider for 3.3 V boards)
GND	GND	Ground

HC-SR04 distance meter

```
const int TRIG = 9;
const int ECHO = 10;

void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  Serial.begin(9600);
}

float readDistanceCm() {
  digitalWrite(TRIG, LOW);           // make sure the trigger starts low
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);          // 10 us pulse starts the measurement
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  unsigned long t = pulseIn(ECHO, HIGH, 30000UL); // echo pulse in us; timeout 30 ms (~5 m)
  if (t == 0) return -1;               // no echo: out of range
  return t * 0.0343 / 2.0;             // convert to cm
}

void loop() {
  float d = readDistanceCm();
  if (d < 0) Serial.println("Out of range");
  else {
    Serial.print(d, 1);
    Serial.println(" cm");
  }
  delay(100);                         // at least 60 ms between measurements
}
```

- **Range** 2 – 400 cm, resolution about 0.3 cm, accuracy ± 1 cm at best. **Beam** about 15° wide.
- **Temperature** affects the speed of sound ($v \approx 331.3 + 0.606 T$ m/s, T in °C): at 30 °C it is 349.5 m/s, 2 % faster than at 20 °C. Compensate if you need accuracy.
- **Trouble:** soft surfaces (cloth, foam) absorb sound; angled surfaces (over ~30° tilt) deflect it away; very close objects (< 2 cm) and thin objects (chair legs) are not detected reliably.
- **Multiple sensors:** read them one at a time, at least 30 ms apart, to avoid cross-talk.
- **Alternative:** the `NewPing` library handles timeouts and averaging; the VL53L0X laser sensor (I2C) is precise at short range.

WORKED EXAMPLE | PARKING-SENSOR BEEPER

Combine the sensor with a buzzer: the closer the object, the faster the beeps. Compute the interval as `interval = map(d, 5, 100, 50, 800)` after `constrain(d, 5, 100)`; beep with `tone()` for 30 ms every `interval` ms using a `millis()` timer (Chapter 16).

19.2 Motion detection: the PIR sensor

A **PIR (passive infrared) sensor** such as the **HC-SR501** detects the change in infrared radiation when a warm body (person or animal) moves across its field of view. It contains a pyroelectric element that produces a voltage when the infrared pattern changes, split into zones by a Fresnel lens, plus a small chip that outputs a **digital HIGH** when motion is detected.

Pin / adjustment	Function
VCC (5 – 20 V), GND	Power (65 μ A idle, ~2 mA while active)
OUT	3.3 V digital output: HIGH during and for a few seconds after motion (fine for the Uno's inputs)

Pin / adjustment	Function
Trimmer 1 (Sx)	Sensitivity / range (3 – 7 m)
Trimmer 2 (Tx)	Hold time of the output (roughly 0.3 s – 5 min)
Jumper	H = repeat trigger (output stays HIGH while motion continues); L = single trigger

PIR motion light

```

const int PIR_PIN = 2;
const int LED_PIN = 13;

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
  delay(30000);           // PIR needs 30-60 s to stabilise after power-up
}

void loop() {
  bool motion = digitalRead(PIR_PIN);
  digitalWrite(LED_PIN, motion);
  if (motion) Serial.println("Motion detected!");
  delay(100);
}

```

TIP

Give the PIR up to a minute to “warm up” after power-on: it may trigger falsely in this period. Keep it away from heaters, sunlit windows and air-conditioner vents, which create infrared changes. A PIR detects **movement of warm objects**, not presence: a person sitting perfectly still is eventually ignored.

19.3 Temperature and humidity: DHT11 and DHT22

The DHT family combines a humidity sensor and a thermistor with a small chip that sends a 40-bit digital reading over one wire. The **DHT11** is cheap (temperature ± 2 °C, humidity ± 5 %, integer resolution, 0 – 50 °C); the **DHT22 (AM2302)** is better (± 0.5 °C, ± 2 –5 % RH, 0.1 resolution, –40 – 80 °C). Both can be read only once every 1 s (DHT11) or 2 s (DHT22).

Wiring: VCC → 5 V, GND → GND, DATA → D2 (with a **10 kΩ pull-up** from DATA to VCC; most 3-pin modules include it). Install “**DHT sensor library**” by Adafruit (and “Adafruit Unified Sensor”) via Library Manager.

DHT22 temperature and humidity

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT22           // change to DHT11 if that is what you have
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  delay(2000);                  // the sensor is slow
  float h = dht.readHumidity();
  float t = dht.readTemperature(); // Celsius (use true for Fahrenheit)

  if (isnan(h) || isnan(t)) {   // reading failed: wiring or timing problem
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  Serial.print("Humidity: "); Serial.print(h); Serial.print(" % ");
  Serial.print("Temperature: "); Serial.print(t); Serial.println(" C");
}
```

Relative humidity is the amount of water vapour in the air as a percentage of the maximum it can hold at that temperature. Comfortable indoor humidity is 40 – 60 %. Warm air holds more moisture, so the same water content gives a **lower** RH at a higher temperature.

19.4 Sound: the microphone module

A common **KY-038 / LM393 sound module** contains an electret microphone, an amplifier, a comparator with a trimmer, and two outputs: **AO** (analog, following the sound level around a bias) and **DO** (digital, HIGH/LOW according to the threshold set by the trimmer). Use DO for “clap detection” and AO for sound-level measuring: sample rapidly and compute the difference between the maximum and minimum values in a 50 ms window (the “peak-to-peak”), which is proportional to loudness. Better microphone boards use the **MAX4466** (adjustable-gain amplifier, output 2.5 V bias) for accurate audio levels.

Sound level meter

```
const int MIC = A0;
const int LED = 8;
const unsigned long WINDOW_MS = 50;

void setup() { pinMode(LED, OUTPUT); Serial.begin(9600); }

void loop() {
  unsigned long start = millis();
  int lo = 1023, hi = 0;
  while (millis() - start < WINDOW_MS) { // sample for 50 ms
    int v = analogRead(MIC);
    if (v < lo) lo = v;
    if (v > hi) hi = v;
  }
  int peakToPeak = hi - lo; // loudness measure
  Serial.println(peakToPeak);
  digitalWrite(LED, peakToPeak > 200); // LED on for loud noises (tune the threshold)
}
```

19.5 Gas and air quality

The **MQ-series** sensors (MQ-2 smoke/LPG, MQ-3 alcohol, MQ-7 CO, MQ-135 air quality) contain a heated tin-dioxide element whose resistance falls in the presence of the target gas. The module gives an analog output (AO) and a threshold output (DO). They need **5 V and about 150 mA** for the heater and a **24 - 48 hour burn-in**, and readings need calibration in clean air. They indicate relative levels; they are **not** certified safety devices. For measuring CO₂, choose an NDIR sensor (MH-Z19, SCD30) or the I2C **BME680 / CCS811** for VOCs.

SAFETY WARNING

Never rely on a hobby gas sensor as your only smoke, LPG or carbon-monoxide protection. Use certified detectors for safety.

19.6 Rotary encoder

A **rotary encoder** (KY-040) outputs two square waves, **A (CLK)** and **B (DT)**, 90° out of phase, as the knob turns. The order of the edges tells the **direction**, and each “click” is one step. It has no end stops: unlike a potentiometer it can rotate endlessly and gives a relative position. A push button is usually built in (SW).

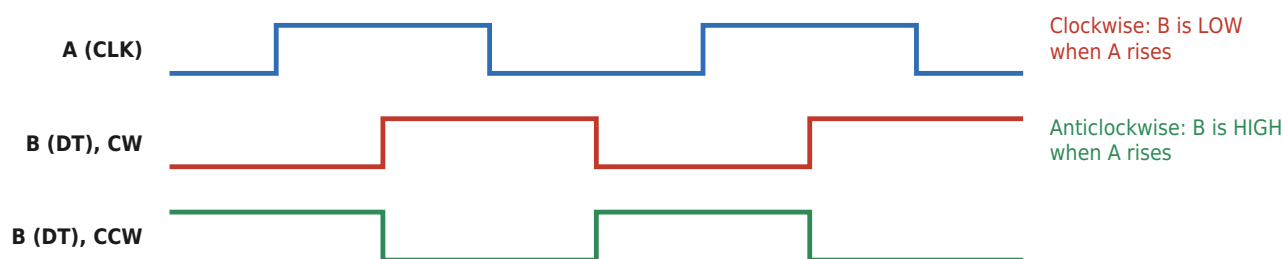


Figure 19.1 Quadrature outputs of a rotary encoder. The relative timing of A and B gives the direction.

Rotary encoder with an interrupt

```
const int CLK = 2, DT = 3;
volatile int position = 0;           // shared with the interrupt routine: volatile
int lastPos = 0;

void onClkRise() {                  // called on every rising edge of CLK
  if (digitalRead(DT) == LOW) position++; // clockwise
  else position--;                  // anticlockwise
}

void setup() {
  pinMode(CLK, INPUT_PULLUP);
  pinMode(DT, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(CLK), onClkRise, RISING); // Chapter 23
  Serial.begin(9600);
}

void loop() {
  if (position != lastPos) {
    lastPos = position;
    Serial.println(position);
  }
}
```

Encoders bounce too. For a robust reader use an interrupt (Chapter 23) with a short debounce, add 10 nF capacitors from CLK and DT to GND, or use the Encoder library.

19.7 Other common sensors

Sensor	What it does	Interface	Notes
Reed switch / Hall (A3144)	Detects a magnet (door open/closed, speed pick-up)	Digital	Use INPUT_PULLUP; A3144 is open-collector and needs the pull-up
Tilt switch (SW-520D)	Detects orientation (ball inside)	Digital	Debounce in software
Flame sensor module	Detects flame IR (760 – 1100 nm)	Analog + digital	Also triggered by sunlight: indoors only
Soil moisture (capacitive)	Water content of soil	Analog	Calibrate: dry air vs. glass of water
Water level / rain sensor	Wet/dry traces	Analog + digital	Traces corrode with DC: power only while reading
Touch (TTP223)	Capacitive touch button	Digital	Works through thin plastic
IR obstacle sensor	Reflective IR: obstacle within 2–30 cm	Digital	Line-following robots (TCRT5000)
IR remote receiver (VS1838B)	Decodes 38 kHz remote commands	Digital (IRremote library)	Use to control projects with a TV remote
MPU6050	3-axis accelerometer + 3-axis gyro	I2C (0x68)	3.3 V logic OK on most breakouts; Chapter 22
BMP280 / BME280	Pressure, temperature, humidity (BME)	I2C / SPI	Weather stations; altitude
DS18B20	Digital waterproof temperature probe, ± 0.5 °C	1-Wire (OneWire library)	Needs 4.7 k Ω pull-up; many on one pin
DS3231 RTC	Accurate real-time clock with battery	I2C (0x68)	Time stamps, alarms
HX711 + load cell	Weight (kitchen-scale style)	2-wire serial	Calibrate with a known mass
GPS (NEO-6M)	Position and time	UART (9600 baud)	Use TinyGPS++

19.8 Working with libraries and datasheets

Most sensors are used through a **library**. The workflow, which works for every sensor in the table above:

1. Find the sensor's **datasheet** and the **library** most people use (Library Manager: search the part number).
2. Check the **supply voltage** and **logic level**, and the **pin names** on your particular breakout board.
3. Open **File ▶ Examples ▶ [Library name]** and upload the simplest example first. Do not write your own code until this works.
4. Read the example: the library's main calls (`begin()`, `read...()`) are shown.
5. Copy the essentials into your own sketch and **print values to the Serial Monitor** while you test.
6. Handle failures (e.g. `isnan()`, timeouts), and use `millis()` timing for slow sensors.

HANDS-ON EXPERIMENT | EXPERIMENT 14: PARKING SENSOR WITH AN ULTRASONIC SENSOR AND BUZZER

You need: Arduino, HC-SR04, passive buzzer (or LED + 330 Ω), 3 LEDs (green/yellow/red, each 330 Ω), breadboard.

1. Wire the sensor (TRIG D9, ECHO D10, VCC 5 V, GND). Upload the distance sketch and check that the Serial Monitor readings agree with a ruler at 10, 30 and 100 cm.
2. Add three LEDs on D2, D3, D4: green when distance > 60 cm, yellow between 30 and 60 cm, red below 30 cm.
3. Add the buzzer on D8: beep at 2 beeps/s below 60 cm rising to continuous below 10 cm, using a `millis()`-based timer.
4. **Improve:** smooth the distance with an exponential filter (Chapter 17) and ignore readings of `-1` (timeouts).

CHAPTER SUMMARY

- **HC-SR04:** 10 μ s trigger pulse $\rightarrow$ echo pulse; distance (cm) = $t(\mu\text{s}) \times 0.0343 / 2$; use `pulseIn` with a timeout.
- **PIR** outputs HIGH on motion; allow ~ 30 s warm-up; trimmers set sensitivity and hold time.
- **DHT11/22** read with the DHT library, once every 1–2 s; check `isnan()`. **DS18B20** uses OneWire; **MQ** sensors need burn-in.
- Sound modules give an analog level (use peak-to-peak) and a digital threshold; **encoders** give quadrature pulses handled with interrupts.
- Use libraries and their examples; verify the supply voltage and logic level; debug via the Serial Monitor.

ASSIGNMENTS | CHAPTER 19

- 19.1 [Calculate]** An HC-SR04 echo pulse is 1,750 μ s. What is the distance? (speed of sound 343 m/s)
- 19.2 [Calculate]** What is the maximum echo pulse width for an object at 4 m?
- 19.3 [Calculate]** At 35 $^{\circ}\text{C}$ the speed of sound is $331.3 + 0.606 \times 35 = 352.5$ m/s. If your code assumes 343 m/s, what distance will it report for an object truly at 100 cm?
- 19.4 [Calculate]** A DHT22 can be read every 2 s. If your loop reads it on every pass with no delay, what happens? How would you fix it?
- 19.5 [Calculate]** A rotary encoder gives 20 steps per revolution. The position counter shows 55. How many full turns is that, and in which direction (positive = clockwise)?
- 19.6 [Practice]** Write a program that lights an LED for 10 seconds after each PIR trigger (without `delay()`).
- 19.7 [Practice]** Modify the ultrasonic sketch to print “Object too close!” when the distance is below 15 cm, and to ignore readings above 400 cm.
- 19.8 [Think]** Why might an HC-SR04 fail to detect a soft sofa or a person wearing a thick coat?
- 19.9 [Think]** Why should a PIR sensor be given time to stabilise after power-up?
- 19.10 [Hands-on]** Build Experiment 14 and measure at ten known distances (10 – 200 cm) with a ruler. Compute the error at each and plot measured versus true distance.

Displays: 7-segment, shift registers and LCD

Outputs that you can read are what make a project feel finished: a thermometer that shows the temperature, a timer that counts down, a game with a score. This chapter covers the classic displays. Along the way you will meet two important ideas: the **shift register**, which multiplies your pins, and **multiplexing**, which lets a few pins drive many LEDs.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How a **7-segment display** works (common cathode/anode) and how to drive it
- How a **74HC595 shift register** turns 3 pins into 8 outputs, and how to chain them
- **Multiplexing** several digits and the persistence of vision
- How to use a **16×2 character LCD** with an I2C backpack
- Displaying numbers, text and custom characters; choosing a display

20.1 The seven-segment display

A 7-segment display has seven LED bars (segments **a - g**) plus a decimal point (**dp**) arranged in a figure-of-eight. Lighting the right combination shows the digits 0 – 9 and some letters. The eight LEDs share one common pin:

- **Common cathode (CC):** all LED cathodes joined; the common goes to **GND**; a segment lights when its pin is **HIGH**.
- **Common anode (CA):** all anodes joined; the common goes to **+5 V**; a segment lights when its pin is **LOW**.

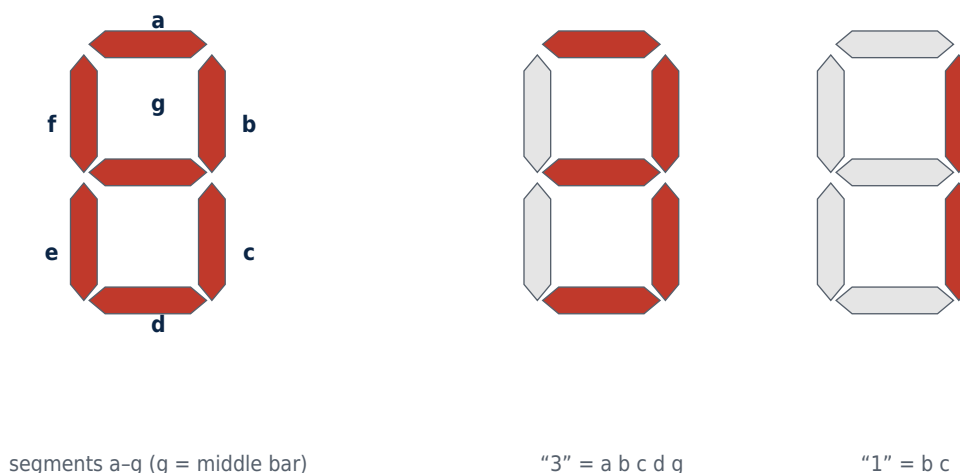


Figure 20.1 The seven segments a – g, and the patterns for 3 and 1.

Each segment is an LED, so **each needs its own current-limiting resistor** (220 – 330 Ω). A single digit requires 8 pins from the Arduino (7 + dp) or, with the 74HC595, just 3. To display a digit you need a **lookup**

table mapping each number to the segments that light. Encoding a pattern as a byte, with bit 0 = a, bit 1 = b ... bit 6 = g, bit 7 = dp:

Digit	Segments on	Binary (dp g f e d c b a)	Hex
0	a b c d e f	0011 1111	0x3F
1	b c	0000 0110	0x06
2	a b d e g	0101 1011	0x5B
3	a b c d g	0100 1111	0x4F
4	b c f g	0110 0110	0x66
5	a c d f g	0110 1101	0x6D
6	a c d e f g	0111 1101	0x7D
7	a b c	0000 0111	0x07
8	a b c d e f g	0111 1111	0x7F
9	a b c d f g	0110 1111	0x6F

GOOD TO KNOW

For a **common-anode** display, invert the bits: `~pattern`. Datasheets differ in how pins are numbered: use the multimeter's diode test or a 5 V source with a resistor to identify which pin lights which segment and where the common pin is.

Counting on a 7-segment digit

```
// Common-cathode single digit, segments a..g on pins 2..8 (dp unused)
const int segPins[] = {2, 3, 4, 5, 6, 7, 8}; // a b c d e f g
const byte digits[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

void showDigit(int n) {
    byte pattern = digits[n];
    for (int i = 0; i < 7; i++) {
        digitalWrite(segPins[i], (pattern >> i) & 1); // pick bit i
    }
}

void setup() {
    for (int i = 0; i < 7; i++) pinMode(segPins[i], OUTPUT);
}

void loop() {
    for (int n = 0; n <= 9; n++) { // count 0..9 once per second
        showDigit(n);
        delay(1000);
    }
}
```

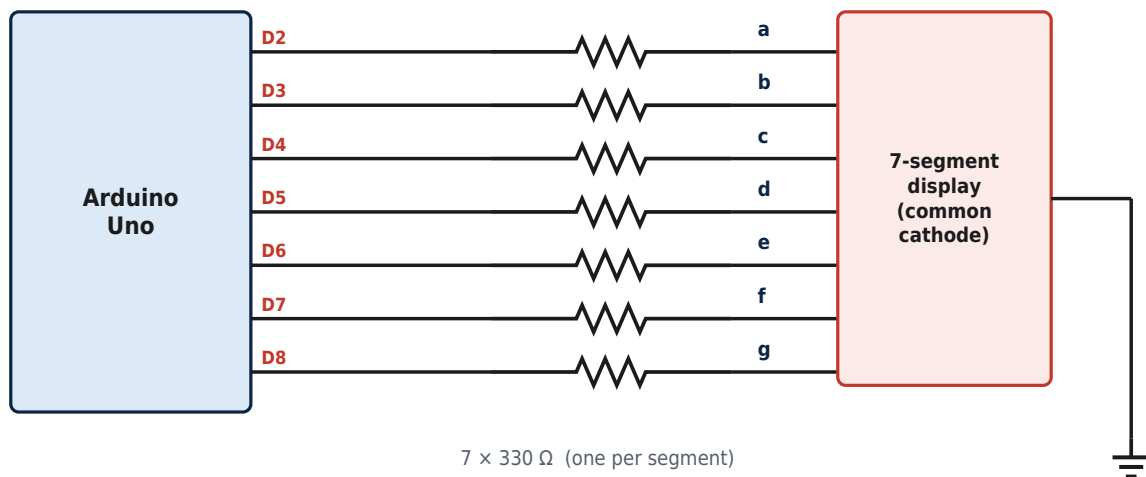


Figure 20.2 Direct drive of one digit: 7 pins, 7 resistors. Common cathode to GND.

20.2 The 74HC595 shift register

Driving eight LEDs directly uses eight pins. A **shift register** solves that: the **74HC595** is a serial-in, parallel-out 8-bit register. You send bits one at a time on **one data line** with a clock pulse for each; then a **latch** pulse copies all eight bits at once to the eight output pins Q0 – Q7. Three Arduino pins (DATA, CLOCK, LATCH) drive 8 outputs; and chaining registers gives 16, 24... outputs, still with three pins.

ANALOGY | CONVEYOR BELT AND PHOTOGRAPH

A shift register is like a **conveyor belt with eight slots**. Each clock “tick” shifts everything one slot along and puts the new item on the belt’s start. After eight ticks the belt holds eight items. The latch pulse then “photographs” the belt and holds up all eight items at the same time (the outputs), so the audience never sees the belt moving.

74HC595 pin	Name	Connect to	Function
14	DS (SER)	Arduino D11 (data)	Serial data input
11	SH_CP (SRCLK)	Arduino D12 (clock)	Shift clock: rising edge shifts a bit in
12	ST_CP (RCLK)	Arduino D8 (latch)	Latch: rising edge copies bits to the outputs
13	OE (active LOW)	GND	Output enable: tie LOW to enable outputs
10	MR (SRCLR, active LOW)	+5 V	Master reset: tie HIGH (LOW clears the register)
15, 1–7	Q0 – Q7	LEDs (each with 330 Ω)	Parallel outputs (max ~6 mA each when all on; 70 mA total)
9	Q7' (QH')	Next chip's DS	Serial output for daisy-chaining
16 / 8	VCC / GND	+5 V / GND	Power (100 nF across them)

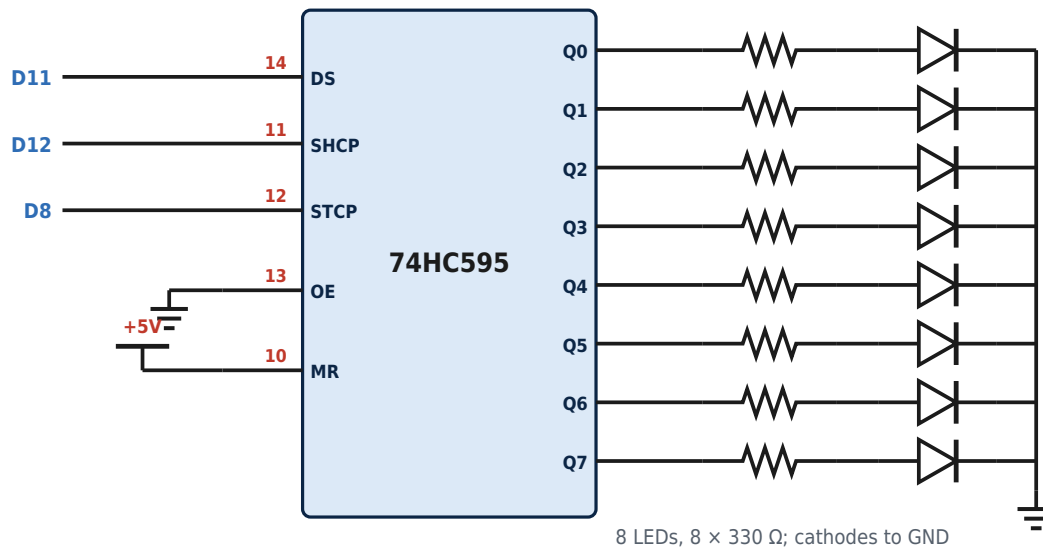


Figure 20.3 74HC595 driving eight LEDs from three Arduino pins (the LED symbols are simplified).

Shifting data: shiftOut()

74HC595: 8 outputs from 3 pins

```
const int DATA_PIN = 11;
const int CLOCK_PIN = 12;
const int LATCH_PIN = 8;

void setup() {
  pinMode(DATA_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
}

void writeRegister(byte value) {
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, value);
  digitalWrite(LATCH_PIN, HIGH);
}

void loop() {
  for (int i = 0; i < 8; i++) {
    writeRegister(1 << i);
    delay(100);
  }
  writeRegister(0b10101010); delay(500);
  writeRegister(0b01010101); delay(500);
  for (int n = 0; n < 256; n++) {
    writeRegister(n);
    delay(50);
  }
}
```

// outputs hold while we shift
// send 8 bits, most significant first
// rising edge: update the outputs

// running light
// 00000001, 00000010, ...

// alternate LEDs

// binary counter: watch it count

Chaining: connect the first chip's Q7' (pin 9) to the second chip's DS, and tie the clock and latch pins of both together. Then call `shiftOut` twice (16 bits: first the byte for the **last** chip in the chain) before one latch pulse. **Driving a 7-segment display:** connect Q0-Q6 to segments a-g via 330 Ω resistors and send `digits[n]`; two chips drive a two-digit display.

TIP

The 74HC595 can sink or source about 6 – 20 mA per output but only ~70 mA in total, so it is not for big loads. For heavy loads, add a **ULN2803** (8-channel Darlington array with flyback diodes) after the register. For inputs (reading many buttons) use the **74HC165** (parallel-in, serial-out).

20.3 Multiplexing multi-digit displays

A four-digit display would need $4 \times 8 = 32$ pins directly. **Multiplexing** solves this by lighting **one digit at a time**, extremely fast, so that the eye sees all of them at once (**persistence of vision**). All digits share the segment lines a – g; each digit has its own common pin, switched on by a transistor (or a spare Arduino pin for small displays). The program cycles: put digit 1's pattern on the segment lines and enable digit 1 for ~2–5 ms; then digit 2, and so on, at least 60 refreshes per second per digit to avoid flicker.

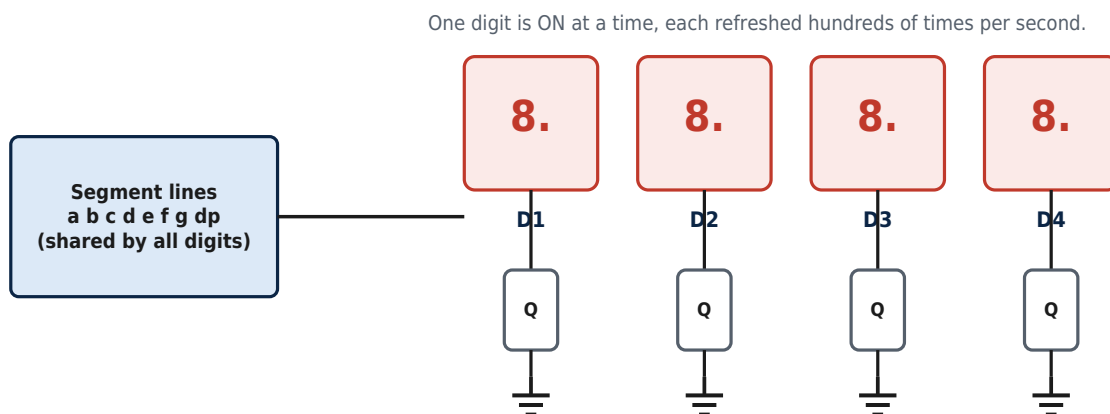


Figure 20.4 Multiplexing: shared segment lines, and one transistor per digit that connects that digit's common cathode to ground.

The savings are enormous: a 4-digit display needs 8 segment lines + 4 digit lines = 12 pins instead of 32 (or 3 pins with two shift registers). The cost: the CPU must keep refreshing (use a timer interrupt, Chapter 23, or the **SevSeg** library) and each segment is on only 1/4 of the time, so drive them with a higher current (up to 30 – 40 mA pulses within the ratings of the chip and the segment). Ready-made modules such as the **TM1637** (4-digit, two wires CLK/DIO) and **MAX7219** (8 digits or an 8×8 matrix, SPI) contain all the multiplexing electronics so you never have to.

20.4 The 16×2 character LCD with I2C

The **HD44780** character LCD (16 columns × 2 rows, 5×8-pixel characters) is the classic text display. Connecting it directly needs 6 – 10 pins; a small **I2C backpack (PCF8574)** soldered on the back reduces it to **4 wires: VCC, GND, SDA, SCL**. Wire SDA → **A4**, SCL → **A5** (Uno). The backpack's I2C address is usually **0x27** (or **0x3F**): run an I2C scanner (Chapter 22) if the screen stays blank. The blue trimmer on the backpack sets **contrast**: turn it until the characters appear (a common cause of "blank display").

Hello LCD

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>           // install "LiquidCrystal I2C" from Library Manager

LiquidCrystal_I2C lcd(0x27, 16, 2);     // address, columns, rows

void setup() {
    lcd.init();                         // start the LCD (older library versions: lcd.begin())
    lcd.backlight();
    lcd.setCursor(0, 0);                // column 0, row 0 (top-left)
    lcd.print("Hello, world!");
    lcd.setCursor(0, 1);                // second row
    lcd.print("Arduino LCD");
}

void loop() {
    lcd.setCursor(12, 1);
    lcd.print(millis() / 1000);         // seconds since start
    lcd.print(" ");                     // spaces erase leftover digits
}
```

Method	Effect
<code>lcd.clear()</code>	Clear the screen (slow: ~2 ms) and home the cursor
<code>lcd.setCursor(col, row)</code>	Position the cursor (0-based)
<code>lcd.print(x)</code>	Print text or numbers at the cursor (<code>lcd.print(3.14159, 2)</code> for 2 decimals)
<code>lcd.backlight()</code> / <code>noBacklight()</code>	Backlight on/off (saves about 20 mA)
<code>lcd.scrollDisplayLeft()</code>	Scroll the text (useful for messages > 16 characters)
<code>lcd.createChar(n, bitmap)</code>	Define one of 8 custom characters (5×8 pixels) and show with <code>lcd.write(byte(n))</code>

LCD thermometer with a custom character

```
// Thermometer display: a custom degree symbol and a stable layout
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

byte degree[8] = {0b00110, 0b01001, 0b01001, 0b00110, 0, 0, 0, 0}; // a small circle

void setup() {
    lcd.init(); lcd.backlight();
    lcd.createChar(0, degree);
    lcd.setCursor(0, 0); lcd.print("Temperature:");
}

void loop() {
    int raw = analogRead(A0);
    float t = raw * 5.0 * 100.0 / 1023.0; // LM35 on A0
    lcd.setCursor(0, 1);
    lcd.print(t, 1);
    lcd.write(byte(0)); // custom degree symbol
    lcd.print("C "); // trailing spaces erase old characters
    delay(500);
}
```

COMMON MISTAKE

Calling `lcd.clear()` every loop makes the display flicker. Instead, overwrite with `setCursor` and pad with spaces to erase old characters. Also, when the LCD is blank: check the contrast trimmer, the I2C address (0x27 vs 0x3F), 5 V power, and that SDA/SCL are on A4/A5 (Uno).

20.5 Other displays: how to choose

Display	Interface	Pins	Best for	Notes
LEDs	Direct / 74HC595	1 each	Status indicators	Simple, bright
7-segment (1 digit)	Direct / 595	7-8 / 3	A single number	Cheap
TM1637 4-digit	2-wire	2	Clocks, counters	Brightness control
MAX7219 (8 digits / 8×8)	SPI	3	Larger numbers; scrolling text matrix	LedControl library
16×2 / 20×4 LCD (I2C)	I2C	2 (A4, A5)	Text and values	Backlit; contrast trimmer
0.96" OLED (SSD1306)	I2C (0x3C) / SPI	2	Text and small graphics	Adafruit_SSD1306 + GFX; high contrast; low power
TFT LCD (ILI9341, 2.4")	SPI	5-6	Colour graphics, touch	Needs more RAM; Mega or 3.3 V boards recommended
E-paper	SPI	6-8	Low-power static info	Slow refresh; retains image with no power

HANDS-ON EXPERIMENT | EXPERIMENT 15: DIGITAL COUNTER AND THERMOMETER

You need: Arduino, one 7-segment display (common cathode), 7 × 330 Ω, or a 74HC595 + 8 resistors; a push button; an I2C LCD (optional).

1. Wire the 7-segment display directly to D2-D8 as in Figure 20.2 and run the 0-9 counter. Find out which segment is which by trying values 1, 2, 4, 8... in `showDigit`.
2. Add a button on D9 (INPUT_PULLUP): each debounced press increments the digit (0 → 9 → 0).
3. Now replace the seven direct wires by the **74HC595** (Figure 20.3): three Arduino pins. Send `digits[n]` through `shiftOut`. Verify the display works identically.
4. **LCD thermometer:** connect an LM35 or the thermistor and display the temperature with a custom degree symbol as in the listing. Show the minimum and maximum temperature recorded on the second line.

CHAPTER SUMMARY

- A 7-segment display has 7 LED segments + dp; common-cathode lights on HIGH, common-anode on LOW; each segment needs a resistor; a lookup table maps digits to bit patterns.
- The **74HC595** turns 3 pins (DATA, CLOCK, LATCH) into 8 outputs; `shiftOut(data, clock, MSBFIRST, value)` then a latch pulse. Chain chips for more outputs.
- **Multiplexing** lights one digit at a time fast enough to look continuous: fewer pins but constant refreshing. Use TM1637/MAX7219 modules.
- 16×2 LCD with I2C backpack: 4 wires; `LiquidCrystal_I2C lcd(0x27,16,2); init, backlight, setCursor, print`; adjust contrast; avoid `clear()` in loops.
- Choose displays by interface pins, information type, power and readability (OLED for graphics, LCD for text, 7-segment for numbers).

ASSIGNMENTS | CHAPTER 20

20.1 [Practice] Which 7-segment pattern (bits d p g f e d c b a) displays the letter “A”? And “H”?

20.2 [Practice] You use a common-anode display. What changes in the code and in the wiring compared to common-cathode?

20.3 [Calculate] Each segment of a 7-segment display ($V_F = 2.0\text{ V}$) should carry 10 mA from 5 V. What resistor? What is the worst-case total current when the digit shows “8”?

20.4 [Calculate] How many bits must be shifted to update a chain of three 74HC595 chips? How many latch pulses are needed?

20.5 [Calculate] A 4-digit multiplexed display refreshes each digit for 2 ms in turn. What is the refresh frequency per digit? What is the duty cycle of each digit?

20.6 [Calculate] To get an average of 8 mA per segment in a 4-digit multiplexed display with 25 % duty cycle, what pulsed current is needed per segment?

20.7 [Practice] Write a function that sends the byte for a decimal digit to a 74HC595 driving a common-cathode 7-segment display.

20.8 [Practice] Write code to display the number of seconds since start on an LCD, as mm:ss on the second row.

20.9 [Think] Your I2C LCD lights up its backlight but shows only dark rectangles in the top row. What is wrong and how do you fix it?

20.10 [Think] Why can a multiplexed display flicker if your loop() contains a long delay()?

20.11 [Hands-on] Build Experiment 15 steps 1–3 and photograph the working display. Measure the current through one segment with your multimeter (in series) and compare with the calculated value.

Driving motors

Motion is the most exciting output. This chapter gathers what you learned about transistors (Chapter 9), inductive kick (Chapter 7), PWM (Chapter 18) and power (Chapter 13) and applies it to real motors. By the end you can make a wheeled robot go forwards, backwards, turn, and stop when it sees a wall.

WHAT YOU WILL LEARN IN THIS CHAPTER

- Choosing between DC motors, servos and steppers for a job
- The **H-bridge**: how it reverses a motor, and how to use the **L293D** / **L298N** / **TB6612** drivers
- Speed and direction control with PWM; braking and coasting
- How to drive a **stepper motor** (28BYJ-48 + ULN2003) with the Stepper library
- Power supply, current, noise and thermal considerations for robots
- Building a small two-wheel robot base with obstacle avoidance

21.1 Which motor for which job?

Motor	Controls	Strengths	Weaknesses	Typical use
DC motor (brushed)	Speed (voltage/PWM), direction (polarity)	Cheap, simple, high speed, strong	No position feedback; needs a driver; noisy	Wheels, fans, pumps
Geared DC motor	As above, but slower with much more torque	Direct drive for robots	Backlash	Robot chassis (TT motors)
Hobby servo	Angle 0 – 180° (pulse width)	Built-in feedback; one signal wire	Limited rotation (or continuous-rotation version with speed control)	Arms, steering, flaps
Stepper motor	Exact steps; speed by step rate	Precise open-loop position; holds torque	Lower speed; power hungry; needs a driver	3D printers, CNC, camera sliders
Brushless (BLDC)	Speed (ESC)	Efficient, powerful, long-lived	Needs an ESC; more complex	Drones, fans, gimbals

21.2 Reversing a DC motor: the H-bridge

A single transistor (Chapter 9) can switch a motor on and off in one direction. To **reverse** the direction you must reverse the polarity of the voltage across it. The circuit that does this electronically is the **H-bridge**: four switches arranged in an “H” with the motor as the crossbar. Closing the diagonal pair **S1 + S4** sends current left-to-right; closing **S2 + S3** sends it right-to-left. Closing both switches on one side would short the supply: **shoot-through**, the one forbidden state.

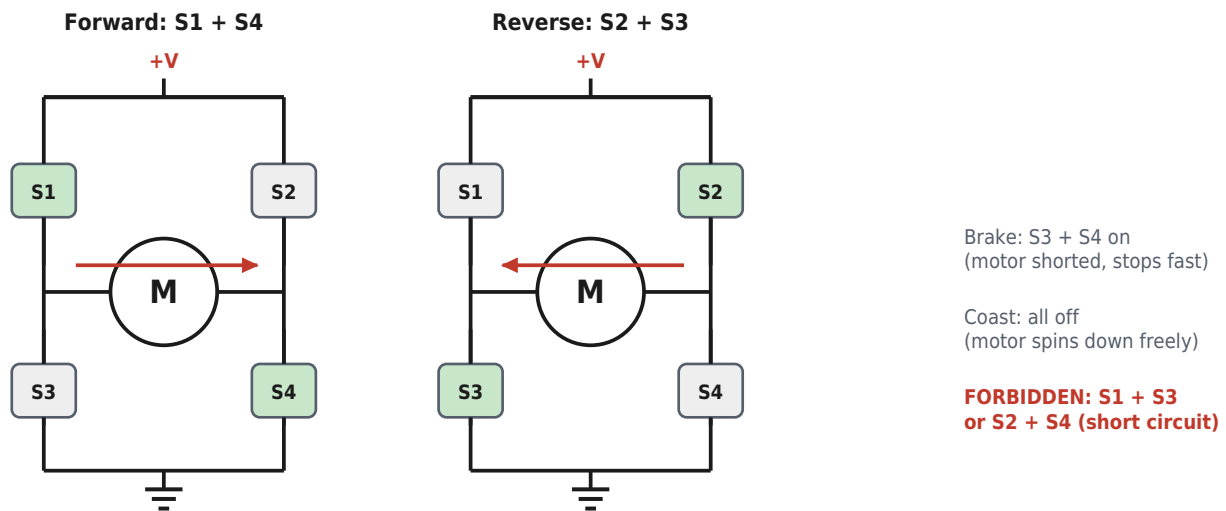


Figure 21.1 H-bridge operation. Green switches are closed. Current flows through the motor in a direction set by which diagonal pair is on.

In practice the four switches are transistors or MOSFETs with flyback diodes (built into MOSFETs), and control logic prevents shoot-through. Rather than build one, use an integrated **motor driver** that contains one or two complete H-bridges.

Driver	Channels	Motor voltage	Current per channel	Notes
L293D	2 (dual H-bridge)	4.5 – 36 V	600 mA (1.2 A peak)	DIP-16; built-in flyback diodes; 1.4 V+ voltage drop (bipolar); cool for small motors; 74HC-compatible inputs
L298N (module)	2	5 – 35 V	2 A	Big heat sink; 2 V+ drop; on-board 5 V regulator (jumper); cheap popular robot module
TB6612FNG (module)	2	4.5 – 13.5 V	1.2 A (3.2 A peak)	MOSFET-based: low drop, efficient; preferred for battery robots
DRV8833	2	2.7 – 10.8 V	1.5 A	Very small; good for 3.3 V systems
BTS7960	1 (high power)	6 – 27 V	43 A	Large motors, power wheelchairs

Using an L298N module

The L298N module has inputs **IN1**, **IN2** (direction, motor A), **ENA** (enable/PWM speed for motor A) and **IN3**, **IN4**, **ENB** for motor B. The truth table for one channel:

ENA	IN1	IN2	Motor A
PWM (0–255)	HIGH	LOW	Forward at the PWM speed
PWM	LOW	HIGH	Reverse at the PWM speed
HIGH	LOW	LOW	Brake (or coast, depending on chip)
HIGH	HIGH	HIGH	Brake
LOW	x	x	Disabled: motor coasts

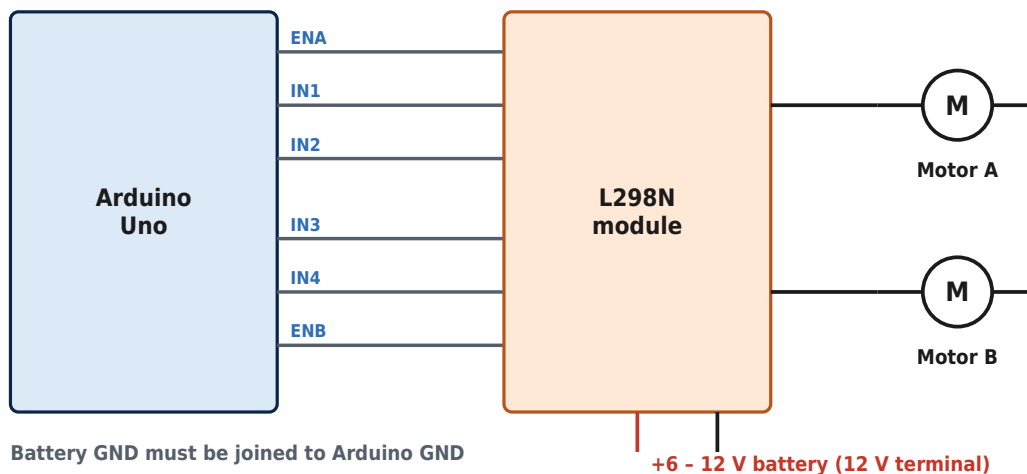


Figure 21.2 Two motors on an L298N module. Remove the ENA/ENB jumpers to use PWM speed control. Battery ground and Arduino ground must be connected.

Simple robot driving functions

```
// Two-motor drive with an L298N
const int ENA = 5, IN1 = 7, IN2 = 8;    // motor A (ENA is a PWM pin)
const int ENB = 6, IN3 = 9, IN4 = 10;  // motor B

void setup() {
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
}

// speed: -255 (full reverse) .. +255 (full forward)
void driveA(int speed) {
  digitalWrite(IN1, speed > 0);
  digitalWrite(IN2, speed < 0);
  analogWrite(ENA, abs(speed));
}
void driveB(int speed) {
  digitalWrite(IN3, speed > 0);
  digitalWrite(IN4, speed < 0);
  analogWrite(ENB, abs(speed));
}

void forward(int s) { driveA(s); driveB(s); }
void backward(int s) { driveA(-s); driveB(-s); }
void turnLeft(int s) { driveA(-s); driveB(s); } // spin on the spot
void turnRight(int s) { driveA(s); driveB(-s); }
void stopAll() { driveA(0); driveB(0); }

void loop() {
  forward(180); delay(2000);
  stopAll(); delay(500);
  turnRight(150); delay(600);
  backward(180); delay(1500);
  stopAll(); delay(2000);
}
```

SAFETY WARNING

Power: the L298N has an on-board 5 V regulator that is only valid when the motor supply is ≤ 12 V (jumper in place); the motor supply goes to the “12 V” terminal, GND to GND, and **the module’s GND must be connected to Arduino GND**. Motors need many times more current than an Arduino can supply: power the module from a battery pack (for example 4×AA or 2×18650), not from the USB port. **Motor stall current** can be 5 × running current: choose a battery and driver that can handle it.

Do not remove the drivers’ heat sink, and beware: the L298N wastes ~ 2 V per channel: 12 V in gives ≈ 10 V at the motor and about 2 W of heat per amp.

Speed, torque and the dead zone

Motor torque is proportional to current, and speed to voltage. With PWM, a duty below about 30 – 40 % often does not overcome static friction (the motor whines but does not turn), so many programs start the motor with a short kick at full power, or map the speed range 0 – 255 to 60 – 255. Gearboxes trade speed for torque: a 1:48 gear reduces speed 48× and raises torque nearly 48×, which is why small robots use geared TT motors (about 200 rpm at 6 V).

Electrical noise and protection

- **Add a 100 nF ceramic capacitor across each motor’s terminals** (soldered close to the motor) and, for noisy motors, one from each terminal to the case. This reduces spikes that reset the Arduino.
- **Decouple** the driver’s supply with a 100 – 470 μ F electrolytic close to the module.
- **Flyback diodes** are inside L293D/L298N (L298N needs external ones: the module has them); MOSFET H-bridges have body diodes.
- **Separate battery** for the motors, or a robust regulator: if the Arduino resets when the motors start, that is the classic sign of a supply that sags (Chapter 13).

21.3 Stepper motors

A **stepper motor** has several coils; energising them in sequence makes the rotor advance one small angle (a **step**) per change. It moves exactly as commanded (as long as it does not lose steps by being overloaded). Two families:

- **Unipolar** (5-, 6-wire; e.g. 28BYJ-48): each coil has a centre tap; driven by transistor arrays like the **ULN2003**. Simple, low torque.
- **Bipolar** (4-wire; NEMA 17): needs an H-bridge per coil: **A4988** or **DRV8825** drivers take STEP and DIR pins and handle microstepping and current limiting: the standard for 3D printers.

The **28BYJ-48** with its ULN2003 board has 4 control inputs IN1–IN4 and needs 5 V. Thanks to its internal 1:64 gearbox it makes about **2048 steps per revolution in full-step mode** (4096 in half-step mode). The half-step sequence energises the coils in the order below; the Stepper library uses the full-step sequence.

Half-step	IN1	IN2	IN3	IN4
1	1	0	0	0
2	1	1	0	0
3	0	1	0	0
4	0	1	1	0
5	0	0	1	0
6	0	0	1	1

Half-step	IN1	IN2	IN3	IN4
7	0	0	0	1
8	1	0	0	1

Stepper library with a 28BYJ-48 + ULN2003

```
#include <Stepper.h>

const int STEPS_PER_REV = 2048;           // 28BYJ-48 in full-step mode
// NOTE: for the 28BYJ-48 the coil order is IN1, IN3, IN2, IN4
Stepper motor(STEPS_PER_REV, 8, 10, 9, 11);

void setup() {
    motor.setSpeed(10);                   // 10 revolutions per minute (max ~15 for this motor)
}

void loop() {
    motor.step(STEPS_PER_REV);             // one full turn clockwise
    delay(1000);
    motor.step(-STEPS_PER_REV / 2);        // half a turn counter-clockwise
    delay(1000);
}
```

With a driver like the **A4988**: `digitalWrite(DIR, HIGH); for (int i = 0; i < 200; i++) { digitalWrite(STEP, HIGH); delayMicroseconds(800); digitalWrite(STEP, LOW); delayMicroseconds(800); }` makes one revolution (200 steps of 1.8°). For smooth non-blocking motion use the **AccelStepper** library (acceleration, several motors). Set the A4988's **current limit** with its trimmer ($V_{ref} = I_{max} \times 8 \times R_{sense}$) and never disconnect the motor while it is powered: that can destroy the driver.

WORKED EXAMPLE | ANGLE TO STEPS

A stepper with 200 steps/rev (1.8°) and 1/16 microstepping has $200 \times 16 = 3200$ steps per revolution. To move a lead-screw carriage 5 mm with an 8 mm-per-turn screw: turns = $5/8 = 0.625 \rightarrow$ steps = $0.625 \times 3200 =$ **2000 steps**.

21.4 A small robot: obstacle avoidance

Combine everything: an ultrasonic sensor on a servo (or fixed in front), two motors with an H-bridge, and simple decision logic (a state machine, Chapter 16): **drive forward**; if an obstacle is closer than 20 cm, **stop**, **back up briefly**, **turn** (random direction), then resume.

Robot state machine

```
// Obstacle-avoiding robot: L298N + HC-SR04 (uses the driving functions above)
const int TRIG = 12, ECHO = 13;
enum State { FORWARD, BACKING, TURNING };
State state = FORWARD;
unsigned long stateStart = 0;
int turnDir = 1;

float distanceCm() {
    digitalWrite(TRIG, LOW); delayMicroseconds(2);
    digitalWrite(TRIG, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG, LOW);
    unsigned long t = pulseIn(ECHO, HIGH, 25000UL);
    return (t == 0) ? 400 : t * 0.0343 / 2.0;
}

void setup() {
    pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);
    // ... motor pin setup as before ...
    randomSeed(analogRead(A0));
}
```

```

void loop() {
  unsigned long now = millis();
  switch (state) {
    case FORWARD:
      forward(170);
      if (distanceCm() < 20) {           // obstacle ahead
        stopAll();
        state = BACKING; stateStart = now;
      }
      break;
    case BACKING:
      backward(150);
      if (now - stateStart > 400) {
        turnDir = random(0, 2) ? 1 : -1; // pick left or right at random
        state = TURNING; stateStart = now;
      }
      break;
    case TURNING:
      if (turnDir > 0) turnRight(150); else turnLeft(150);
      if (now - stateStart > random(350, 700)) { stopAll(); state = FORWARD; }
      break;
  }
  delay(30); // gives the sensor time between pings
}

```

TIP

Build the robot in stages: (1) motors turn correctly with the test sketch (swap a motor's two wires if it runs backwards); (2) sensor prints sensible distances; (3) combine. Put the battery low and centred, secure the wires, and add a power switch. Test on blocks with the wheels off the table first.

HANDS-ON EXPERIMENT | EXPERIMENT 16: MOTOR DRIVER AND SPEED CONTROL

You need: Arduino, L298N or L293D, a small DC motor (3–6 V) or two, battery pack (4×AA), 10 kΩ potentiometer, breadboard, 100 nF capacitor.

1. Wire the driver and motor A as in Figure 21.2 with the separate battery pack (grounds joined). Solder or clip 100 nF across the motor's terminals.
2. Run the `driveA()` test: the motor should spin forward, stop, then reverse. If the direction is opposite to what you expect, swap the motor's wires.
3. Add the potentiometer on A0: map 0–1023 to –255..+255 so the middle position stops the motor, one end runs it forward and the other reverse. Add a dead zone around the centre.
4. Measure the motor current with your multimeter (mA/A range) in series with the battery, at three speeds, and while holding the shaft (briefly!) to measure stall current. Compare with the driver's rating.

CHAPTER SUMMARY

- DC motors (speed via PWM, reversible), servos (angle), steppers (steps) each suit different tasks. Motors need drivers, separate supplies, and a common ground.
- An **H-bridge** reverses polarity; never turn on both switches of one side. Use L293D (small), L298N (2 A, wasteful), TB6612 (efficient).
- `digitalWrite(IN1/IN2)` sets direction; `analogWrite(EN)` sets speed; motors need a minimum duty to start. Add 100 nF across motor terminals and bulk capacitors.
- Steppers move in steps: 28BYJ-48 + ULN2003 with the Stepper library (2048 steps/rev); A4988/DRV8825 with STEP/DIR pins; set the current limit.
- Structure robot behaviour as a **state machine**; build and test in stages.

ASSIGNMENTS | CHAPTER 21

21.1 [Calculate] A motor stalls at 1.2 A from a 6 V supply. What is its winding resistance? What power does the driver's H-bridge dissipate if its voltage drop is 2 V at that current (L298N)?

21.2 [Calculate] The motor's free-running current is 200 mA and stall current is 1.2 A. A TB6612 with $R_{DS(on)} = 0.5\ \Omega$ per switch pair is used. Compare its loss at stall with the L298N.

21.3 [Calculate] A geared motor is 1:48 and the motor shaft turns at 9600 rpm. What is the wheel speed? If the wheel diameter is 65 mm, how fast does the robot move (m/s)?

21.4 [Calculate] A 28BYJ-48 is set to 10 rpm with 2048 steps/rev. What is the step rate? What is the delay between steps?

21.5 [Calculate] You need to rotate a NEMA 17 (200 steps/rev, 1/8 microstep) by 45° . How many microsteps?

21.6 [Practice] Write a function `setMotor(int speed)` for one motor with IN1, IN2, EN pins that handles positive, negative and zero speed (zero = brake by setting both inputs LOW and EN HIGH).

21.7 [Practice] Modify the robot code so that it reverses if the obstacle is closer than 10 cm and turns if between 10 and 25 cm.

21.8 [Think] Why does the Arduino often reset the instant a motor starts, and what are the fixes?

21.9 [Think] Explain shoot-through and how motor-driver ICs avoid it.

21.10 [Hands-on] Complete Experiment 16. Draw a table of PWM value, motor speed (estimated rpm or by counting turns for 10 s), and current. At which duty cycle does your motor start to turn?

Communication: Serial, I2C and SPI

Sensors, displays, memory cards, radios and other microcontrollers contain their own chips. They exchange data with the Arduino through **communication buses**: agreed rules for sending bits over a few wires. You have already met one, without noticing, every time you used the Serial Monitor. This chapter explains the three standards found on almost every module you will ever buy: UART, I2C and SPI.

WHAT YOU WILL LEARN IN THIS CHAPTER

- Why chips need **communication buses**, and the difference between serial and parallel, synchronous and asynchronous
- **UART / Serial**: baud rate, start/stop bits, the Serial Monitor, `SoftwareSerial`, and RS-232/USB
- **I2C**: two wires, addresses, pull-ups, scanning the bus, reading an RTC and an MPU6050
- **SPI**: four wires, chip select, modes; an SD card or MAX7219
- How to choose between them, and **logic-level** problems between 5 V and 3.3 V devices

22.1 Serial and parallel; synchronous and asynchronous

To send a byte (8 bits) from one chip to another, you can use **8 wires at once** (**parallel**: fast but many pins) or send the bits **one after another over a single wire** (**serial**: few pins). Almost all modern buses are serial. Serial links differ in how the receiver knows *when* each bit arrives:

- **Asynchronous**: no shared clock; both sides agree beforehand on the speed (the **baud rate**). Each byte is framed by start and stop bits so the receiver can synchronise. (UART)
- **Synchronous**: a separate **clock** wire tells the receiver exactly when to read each bit. Faster and more robust, one extra wire. (SPI, I2C)

Bus	Wires	Speed (typical)	Devices	Distance	Used for
UART	2 (TX, RX) + GND	9600 – 115200 baud (up to 1 Mbaud)	2 (point-to-point)	Up to ~15 m (RS-232), 1 km (RS-485)	PC, GPS, Bluetooth HC-05, ESP8266, fingerprint sensors
I2C (TWI)	2 (SDA, SCL) + GND	100 kHz, 400 kHz (up to 3.4 MHz)	Many (up to 112 by address)	Board-level: < 1 m	Sensors, RTC, EEPROM, OLED/LCD, port expanders
SPI	4 (MOSI, MISO, SCK, SS) per device	1 – 20+ MHz	Many (one SS pin each)	Board-level	SD cards, TFT displays, radios (NRF24), ADC/DAC, MAX7219
1-Wire	1 (+ GND)	~16 kbit/s	Many	~100 m	DS18B20 temperature probes

22.2 UART: the Serial port

A **UART** (universal asynchronous receiver/transmitter) sends bytes on the **TX** line and receives on the **RX** line; two devices connect **TX → RX and RX → TX (crossed)**, plus a common GND. The idle line is HIGH. To send a byte the transmitter pulls the line LOW for one bit time (**start bit**), sends the 8 data bits (least significant first), and finishes with a HIGH **stop bit**. The standard “9600 8N1” means 9600 bits per second, 8 data bits, no parity, 1 stop bit.

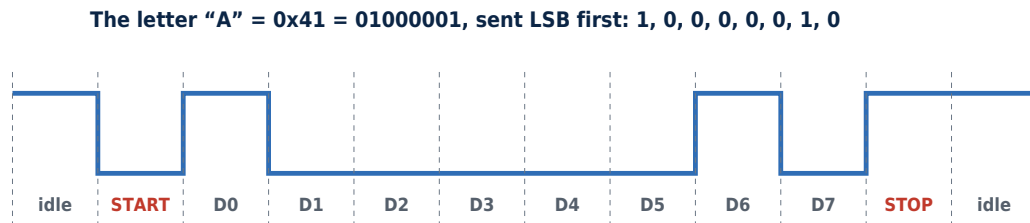


Figure 22.1 One UART frame (8N1) carrying the byte 0x41. At 9600 baud each bit lasts 104 μ s; a byte takes 10 bits = 1.04 ms.

Bytes per second $\approx$ baud $\div$ 10

9600 baud $\approx$ 960 bytes/s (8 data bits + start + stop = 10 bits per byte)

On the Uno, the hardware UART is on **pins 0 (RX) and 1 (TX)**, shared with the USB-serial chip. That is why the Serial Monitor works, and why connecting other devices to pins 0/1 interferes with uploading. To talk to a second serial device (GPS, Bluetooth), use `SoftwareSerial` on any other two pins, or a board with several hardware UARTs (Mega has four).

A serial bridge (PC $\leftrightarrow$ Bluetooth module)

```
#include <SoftwareSerial.h>

SoftwareSerial bt(10, 11);           // RX = pin 10, TX = pin 11 (Arduino side)

void setup() {
  Serial.begin(9600);                // to the PC
  bt.begin(9600);                    // to the Bluetooth module (HC-05 default 9600)
}

void loop() {
  if (bt.available()) Serial.write(bt.read()); // module -> PC
  if (Serial.available()) bt.write(Serial.read()); // PC -> module
}
```

SAFETY WARNING

Logic levels on UART. The Uno’s TX outputs 5 V; many modules (HC-05 RX, ESP8266, NEO-6M GPS on some boards) accept only 3.3 V. Use a divider (1 k Ω from TX to the module’s RX, 2 k Ω from there to GND) on the line that goes from the 5 V Arduino to the 3.3 V module. The other direction (3.3 V TX → Arduino RX) works without conversion.

Serial functions you will use

Function	What it does
<code>Serial.begin(9600)</code>	Start the port at the given baud rate (monitor must match)
<code>Serial.print(x) / println(x)</code>	Send text/numbers, <code>println</code> adds a newline; <code>print(x, HEX) / (x, BIN) / (f, 2)</code> format numbers
<code>Serial.write(b)</code>	Send a raw byte
<code>Serial.available()</code>	Number of bytes waiting to be read
<code>Serial.read()</code>	Read one byte (–1 if none)
<code>Serial.parseInt() / parseFloat()</code>	Read a number typed by the user (blocks up to <code>Serial.setTimeout</code> , default 1 s)
<code>Serial.readStringUntil("\n")</code>	Read a line of text

Command from the Serial Monitor

```
// Ask the user for a brightness value 0..255 and apply it
const int LED = 9;

void setup() {
  Serial.begin(9600);
  pinMode(LED, OUTPUT);
  Serial.println("Type a number 0-255 and press Enter (set line ending to 'Newline')");
}

void loop() {
  if (Serial.available() > 0) {
    int value = Serial.parseInt();           // read the number
    Serial.readStringUntil('\n');           // discard the newline
    value = constrain(value, 0, 255);
    analogWrite(LED, value);
    Serial.print("Brightness set to ");
    Serial.println(value);
  }
}
```

TIP

The **Serial Plotter** (Tools ► Serial Plotter) draws numbers you print as live graphs: print several values separated by commas or tabs on one line: `Serial.print(a); Serial.print(","); Serial.println(b);`. Great for watching sensors and filters (Chapter 17).

22.3 I2C: two wires, many devices

I2C (Inter-Integrated Circuit, “I-squared-C”, invented by Philips in 1982) connects up to about 100 devices on **two wires**: **SDA** (data) and **SCL** (clock), plus ground. One device is the **controller** (formerly “master”: your Arduino); others are **peripherals** (“slaves”). Every peripheral has a fixed **7-bit address** (for example 0x3C for an OLED, 0x68 for the DS3231 clock and MPU6050, 0x27 for an LCD backpack, 0x76 for a BME280). The controller starts a transaction by sending the address and a read/write bit; only the addressed device replies.

ANALOGY | A CLASSROOM

I2C is like a **classroom**: one teacher (the controller) and many students (peripherals) sharing one conversation. The teacher calls a student by name (address); only that student answers; the rest stay silent. Everyone hears the same clock ticks from the teacher's metronome (SCL).

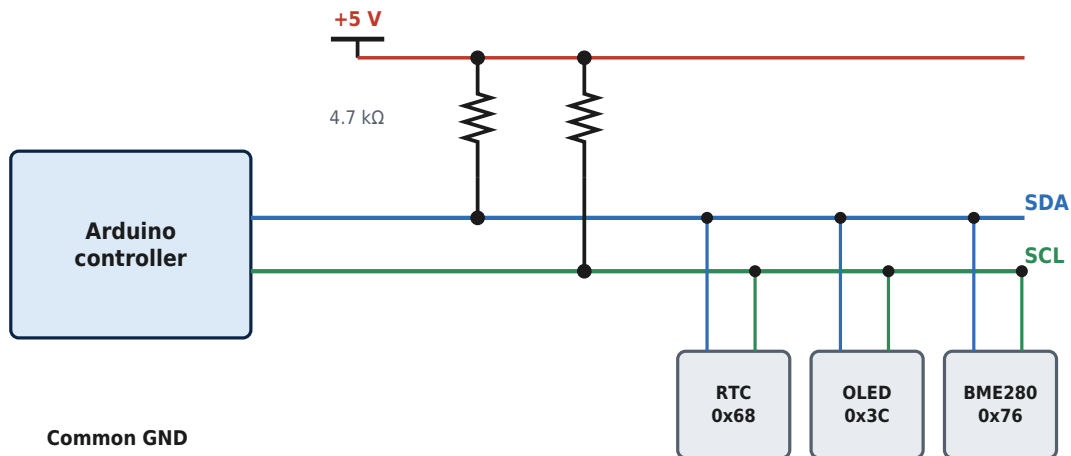


Figure 22.2 An I2C bus: two shared wires with pull-up resistors to +5 V; each device has its own address. (Ground wires not drawn.)

Pull-up resistors are essential. I2C devices only pull the lines LOW (**open-drain**); the lines return HIGH through pull-up resistors (typically 4.7 kΩ, 2.2 kΩ – 10 kΩ). Most breakout modules include them, and the Uno's internal pull-ups are enabled by the Wire library (`weak`). If several modules with their own pull-ups are connected in parallel, the effective resistance falls; 3 modules with 4.7 kΩ give 1.6 kΩ, which is still acceptable, but 10 modules might be too heavy a load.

Uno pin	Function
A4	SDA
A5	SCL
GND, 5V or 3.3V	Power the module according to its rating (check!)

Step 1: find the devices with an I2C scanner

I2C scanner: find your device's address

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("I2C scanner");
  int found = 0;
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {           // 0 = device acknowledged
      Serial.print("Device found at 0x");
      Serial.println(addr, HEX);
      found++;
    }
  }
  if (found == 0) Serial.println("No I2C devices found: check wiring and power");
}

void loop() {}
```

Step 2: talk to a device

Most of the time a library handles the protocol. But you should understand the raw operation: write a **register address**, then read the bytes that follow. Here is the raw method for the **DS3231 real-time clock** (address 0x68), whose registers 0 – 6 hold seconds, minutes, hours, weekday, date, month and year in **binary-coded decimal (BCD)**:

Reading the time from a DS3231 over I2C

```
#include <Wire.h>

byte bcdToDec(byte b) { return (b >> 4) * 10 + (b & 0x0F); } // BCD: each nibble is a digit

void setup() {
  Wire.begin();
  Serial.begin(9600);
}

void loop() {
  Wire.beginTransmission(0x68);
  Wire.write(0); // point to register 0 (seconds)
  Wire.endTransmission();

  Wire.requestFrom(0x68, 3); // read 3 bytes: seconds, minutes, hours
  byte sec = bcdToDec(Wire.read() & 0x7F);
  byte min = bcdToDec(Wire.read());
  byte hr = bcdToDec(Wire.read() & 0x3F);

  char buf[9];
  sprintf(buf, "%02d:%02d:%02d", hr, min, sec); // formatted as hh:mm:ss
  Serial.println(buf);
  delay(1000);
}
```

In real projects you would use the **RTClib** library (`rtc.now().hour()`). Reading raw registers, however, is how you use any I2C chip whose library does not exist: read the datasheet's register table.

WORKED EXAMPLE | THE MPU6050 ACCELEROMETER/GYRO

Address 0x68 (0x69 if AD0 is high). Wake it: write 0 to register 0x6B (power management). Read 6 bytes starting at 0x3B: ACCEL_X high/low, Y, Z. Each axis is a 16-bit signed number: `int16_t ax = (Wire.read() << 8) | Wire.read();` At ± 2 g range the sensitivity is 16384 LSB/g, so a reading of 16384 on Z with the sensor flat means 1 g (gravity). The tilt angle in X: `angle = atan2(ax, az) * 180/PI`.

Watch for the 3.3 V vs 5 V on the module: most GY-521 boards have a regulator and level shifting for SDA/SCL and accept 5 V on VCC.

Two devices with the same address

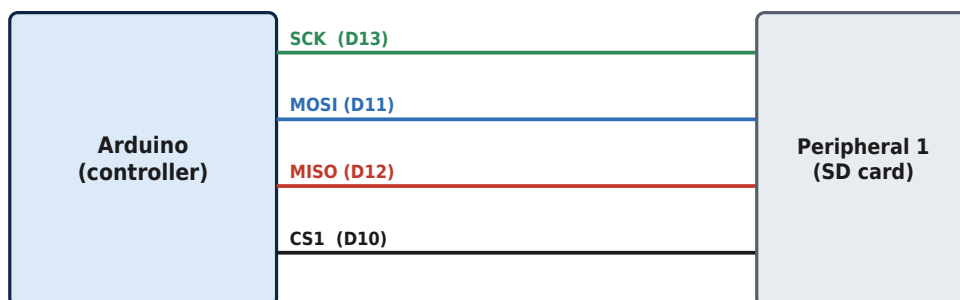
If two identical modules share an address (say two OLED displays at 0x3C), you cannot use both on the same bus. Solutions: choose the alternate address (solder jumper on the board: 0x3C/0x3D), use an **I2C multiplexer (TCA9548A)** to give each device its own branch, or use separate buses on another board (Mega has one; ESP32 has two).

COMMON MISTAKE

Forgetting the pull-ups on bare chips, using wires longer than ~1 m (capacitance rounds off the edges), mixing 3.3 V devices on a 5 V bus without a level shifter, and wrong address (0x27 vs 0x3F is a classic).

22.4 SPI: fast and simple

SPI (Serial Peripheral Interface, from Motorola) is faster than I2C because it uses **separate lines for each direction** and no addresses. The controller drives a shared **clock (SCK)** and **data out (MOSI, controller-out peripheral-in)**, reads **data in (MISO)**, and selects one peripheral by pulling its dedicated **chip-select (CS / SS) line LOW**. Data is shifted simultaneously in both directions: as the controller shifts a byte out on MOSI, the peripheral shifts a byte back on MISO.



A second peripheral shares SCK, MOSI and MISO; it needs its own CS pin (D9, for example).

Each device: SCK, MOSI, MISO shared; CS one per device

Figure 22.3 SPI connections. Only the selected peripheral (CS LOW) responds and drives MISO.

Uno pin	SPI signal	Alternative names
D13	SCK	CLK, SCLK
D12	MISO	SDO, DO (peripheral out)
D11	MOSI	SDI, DI (peripheral in)
D10 (or any pin)	CS	SS, CSN, NSS

SPI has four **modes** (0 – 3) defined by the clock's idle level (CPOL) and which edge samples data (CPHA); the datasheet states which one a device needs (most use mode 0). The Arduino **SPI** library sets it with **SPISettings(clock, bitOrder, mode)**.

SPI transaction skeleton

```
#include <SPI.h>

const int CS = 10;

void setup() {
    pinMode(CS, OUTPUT);
    digitalWrite(CS, HIGH);           // deselect the peripheral
    SPI.begin();
    Serial.begin(9600);
}

byte readRegister(byte reg) {
    SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0)); // 1 MHz, mode 0
    digitalWrite(CS, LOW);           // select the device
    SPI.transfer(reg | 0x80);         // send the register address (bit 7 = read on many chips)
    byte value = SPI.transfer(0x00); // send a dummy byte while receiving the reply
    digitalWrite(CS, HIGH);          // deselect
    SPI.endTransaction();
    return value;
}

void loop() {
    Serial.println(readRegister(0x0F), HEX); // e.g. the "WHO_AM_I" register of a sensor
    delay(1000);
}
```

For the most common SPI devices you use libraries: **SD** (microSD card modules; data logging), **MFRC522** (RFID card reader), **RF24** (NRF24L01 radios), **TFT_eSPI** / **Adafruit_ILI9341** (displays), **LedControl** / **MD_MAX72XX** (MAX7219). Note that SD card modules are usually 3.3 V devices: the common cheap modules have a level shifter and a regulator on board, so power them from 5 V.

	UART	I2C	SPI
Clock	None (agreed baud)	Shared, from the controller	Shared, from the controller
Addresses	None (point-to-point)	7-bit address per device	Chip-select line per device
Wires for n devices	2 per link	2 in total	3 + n
Speed	≈ 115 kbit/s common	100 – 400 kbit/s (I2C fast: 1 – 3.4 Mbit/s)	Up to tens of Mbit/s
Complexity	Low	Medium (pull-ups, addresses)	Low, but more wires
Best when	Two devices, long-ish distance	Many slow sensors, few pins	Fast data: SD, displays, radio

22.5 Two Arduinos talking

Two Arduinos can exchange data via UART (TX↔RX crossed, common GND), or I2C (one controller, one peripheral using `Wire.begin(address)` and `Wire.onReceive()`), or SPI. UART is the simplest: connect Arduino A pin 10 (SoftwareSerial TX) to Arduino B pin 11 (RX) and vice versa, join grounds, and send bytes or lines. Keep messages short and structured (for example one line per reading, such as T=23.5) and parse them with `Serial.readStringUntil` using a newline terminator.

Line-based messaging

```
// Sender: sends the temperature once per second as a line of text
void setup() { Serial.begin(9600); }
void loop() {
  float t = analogRead(A0) * 5.0 * 100.0 / 1023.0;
  Serial.print("T=");
  Serial.println(t, 1);
  delay(1000);
}

// Receiver (use SoftwareSerial if the hardware port is busy):
// String line = link.readStringUntil('\n');
// if (line.startsWith("T=")) { float t = line.substring(2).toFloat(); ... }
```

HANDS-ON EXPERIMENT | EXPERIMENT 17: SCAN THE BUS AND READ A CLOCK

You need: Arduino, a DS3231 RTC module (with CR2032 backup battery), or any I2C sensor/LCD backpack, jumper wires.

1. Connect VCC → 5 V, GND → GND, SDA → A4, SCL → A5. Upload the I2C scanner: note the address(es) found. (A DS3231 board often shows 0x68 plus 0x57 for its EEPROM.)
2. Upload the raw register-reading sketch and confirm it prints a time (it may show 00:00:00 or 165:85:85 until set: the RTC must be initialised).
3. Install **RTCLib** (Adafruit). Run the example **ds3231**: it sets the time from the compile time; read it back and unplug USB for a minute, then plug back: the time keeps counting thanks to the backup battery.
4. Combine with the LCD from Chapter 20: both are I2C devices sharing A4/A5 (addresses 0x68 and 0x27). Display the date and time. This is your first two-device bus.
5. **Challenge:** log the temperature every 10 s to the Serial Plotter, with a time stamp.

CHAPTER SUMMARY

- **UART** (asynchronous): TX/RX crossed + GND; baud rate must match; 8N1 frame = 10 bits per byte; Uno hardware pins 0/1, **SoftwareSerial** elsewhere.
- **I2C**: SDA (A4) + SCL (A5); 7-bit addresses; needs pull-up resistors; **Wire** library; scan the bus to find addresses; devices can share it.
- **SPI**: SCK, MOSI, MISO + CS per device (D13, D11, D12 on the Uno); fast; **SPI** library and **SPISettings**.
- Choose UART for point-to-point links, I2C for many slow sensors on two pins, SPI for speed. Always check voltage levels (5 V vs 3.3 V).
- Use libraries, but learn the register-read pattern: write register address, then read bytes.

ASSIGNMENTS | CHAPTER 22

22.1 [Calculate] How long does it take to send the 12-character string “Hello, world” (with newline = 13 bytes) at 9600 baud (8N1)? At 115200 baud?

22.2 [Calculate] You send one 200-byte packet per second over serial. What is the lowest standard baud rate that would keep up with continuous transmission? (assume 8N1)

22.3 [Calculate] A 16-bit ADC reading is sent over I2C at 100 kHz. Roughly how long does the transaction take (address byte + register byte + 2 data bytes, 9 clock cycles per byte)?

22.4 [Calculate] An I2C bus has 4 modules, each with 10 k Ω pull-ups. What is the effective pull-up resistance? Is it OK for 400 kHz operation?

22.5 [Calculate] An SPI display is updated at 8 MHz. How long does it take to send a 320 × 240 frame of 16-bit pixels?

22.6 [Practice] Wiring: an Arduino Uno with a BME280 (I2C, 3.3 V module with regulator) and an SSD1306 OLED (I2C). Which pins are used, what addresses might they have, and do you need extra pull-ups?

22.7 [Practice] Write code that reads an integer from the Serial Monitor and turns an LED on for that many seconds without using `delay()` for the timing.

22.8 [Think] Why do you cross TX and RX in a UART connection but not SDA and SCL in I2C?

22.9 [Think] What happens if two I2C devices with the same address are connected to the same bus?

22.10 [Hands-on] Do Experiment 17 and produce a table of all I2C addresses found with a description of each device.

Interrupts and timers

So far your program has asked “has anything happened?” over and over inside `loop()`: this is **polling**. It is simple, but it fails if you miss a short event while busy doing something else (a 50 μ s pulse while the program runs a 100 ms task), or it wastes time checking. **Interrupts** reverse the arrangement: the hardware taps the processor on the shoulder the moment an event occurs, the processor pauses whatever it was doing, handles the event, and resumes. Combined with hardware **timers**, interrupts give you precise timing and reliable event capture. This is where an Arduino starts to feel like a real-time system.

WHAT YOU WILL LEARN IN THIS CHAPTER

- The difference between **polling** and **interrupts**, and when each is appropriate
- How to write an **interrupt service routine (ISR)** with `attachInterrupt()`: the rules that keep it safe
- **volatile**, **atomic access**, and sharing data between an ISR and `loop()`
- The ATmega328P’s **timers**: prescalers, compare-match and using the **TimerOne** library
- The **watchdog**, **sleep modes**, and how to reduce power
- Practical uses: counting pulses, measuring frequency, RPM, debouncing, fixed-rate sampling

23.1 Polling vs interrupts

ANALOGY | CHECKING THE DOOR VS. THE DOORBELL

Polling is checking the door every minute to see whether the delivery has arrived: you waste time and might miss the moment. An **interrupt** is a **doorbell**: you do other things, and when someone rings, you stop what you are doing, answer the door, and then go back to your task exactly where you left it.

	Polling (in <code>loop()</code>)	Interrupt
How	Program checks the input repeatedly	Hardware triggers a function at the moment of the event
Response time	Up to one <code>loop()</code> pass (can be ms or more)	Within a few microseconds
Risk	Missing short events	Complex bugs if the ISR is long or shares data badly
Use for	Buttons, slow sensors	Fast pulses, encoders, flow sensors, urgent stops, timing

23.2 External interrupts on the Uno

The Uno can trigger a function on a pin event on **two dedicated pins: D2 (INT0) and D3 (INT1)**. Additionally, **pin-change interrupts** exist on every pin (libraries such as `PinChangeInterrupt` or `EnableInterrupt` make them easy). You choose what triggers it:

Mode	Triggers when...
RISING	the pin goes from LOW to HIGH

Mode	Triggers when...
FALLING	the pin goes from HIGH to LOW (a press with INPUT_PULLUP)
CHANGE	the pin changes state (either way)
LOW	the pin is LOW (repeatedly while it remains LOW)

An interrupt-driven button

```

const int BUTTON_PIN = 2;           // INT0
const int LED_PIN    = 13;
volatile bool ledState = false;     // changed in the ISR, so: volatile

void toggleLed() {                  // the ISR: must be short and fast
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
}

void setup() {
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(LED_PIN, OUTPUT);
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), toggleLed, FALLING);
}

void loop() {
    // The processor can do anything here (even delay(5000)); the button is never missed.
}

```

Line by line: `digitalPinToInterrupt(pin)` converts a pin number to the interrupt number (important for portability between boards). The second argument is the name of the **ISR** (a function with no parameters and no return value). The third is the trigger mode. The ISR runs whenever the event occurs, **even in the middle of `delay()` or any other code**.

SAFETY WARNING | INTERRUPTS DO NOT REMOVE BOUNCE

The button still **bounces**: each bounce is an interrupt, so the LED may toggle several times per press. Ignore interrupts arriving within 50 ms of the previous one (see below), or add a hardware debounce (10 kΩ + 100 nF).

23.3 The rules of ISRs

RULES FOR INTERRUPT SERVICE ROUTINES

- **Keep it short:** a few lines. Set a flag or update a counter and leave; do the real work in `loop()`.
- **No `delay()`** (it relies on interrupts which are disabled inside an ISR) and **no `Serial.print()`** (it may block or misbehave). `millis()` works but does not advance during the ISR; `micros()` works for short periods.
- **Variables shared with `loop()` must be `volatile`** so the compiler does not keep stale copies in registers.
- **Multi-byte variables need protection:** reading an `int` or `long` takes several instructions; if an interrupt changes the value midway, you read garbage. Disable interrupts briefly when copying (atomic access).
- **Avoid heavy computation** (floating-point maths, dynamic memory) in an ISR.

volatile and atomic access

Safe copy of a multi-byte shared variable

```
volatile unsigned long pulseCount = 0;    // modified by the ISR

void onPulse() { pulseCount++; }

unsigned long readCount() {
  noInterrupts();                        // pause interrupts
  unsigned long copy = pulseCount;       // a 4-byte read cannot be interrupted now
  interrupts();                          // allow them again
  return copy;
}
```

Why is this needed? On an 8-bit processor, **unsigned long** is 4 bytes read one at a time. If the ISR increments the count from 0x00FF to 0x0100 between your reading the low byte and the high byte, you might get 0x01FF: a value that never existed. **noInterrupts()/interrupts()** around the copy prevents that. Single-byte variables (**byte**, **bool**) are read in one instruction and are safe already.

Debouncing inside an interrupt

Time-based debounce in an ISR

```
volatile unsigned long lastInterrupt = 0;
volatile int presses = 0;

void onPress() {
  unsigned long now = millis();
  if (now - lastInterrupt > 50) {        // ignore anything within 50 ms of the last event
    presses++;
    lastInterrupt = now;
  }
}
```

23.4 Measuring a frequency or RPM

A classic use: counting pulses from a flow sensor, a tachometer (one pulse per rotation of a magnet past a Hall sensor) or a wheel encoder. Count pulses in the ISR; once per second, read and reset the count in **loop()**.

Tachometer: counting pulses over one second

```
const int SENSOR_PIN = 2;           // INT0
volatile unsigned long pulses = 0;
unsigned long lastCalc = 0;
const int PULSES_PER_REV = 1;       // one magnet on the wheel

void countPulse() { pulses++; }

void setup() {
  Serial.begin(9600);
  pinMode(SENSOR_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(SENSOR_PIN), countPulse, FALLING);
}

void loop() {
  if (millis() - lastCalc >= 1000) {
    noInterrupts();
    unsigned long p = pulses;        // copy and reset atomically
    pulses = 0;
    interrupts();
    lastCalc = millis();

    float rpm = p * 60.0 / PULSES_PER_REV; // pulses per second * 60 = pulses per minute
    Serial.print("RPM: ");
    Serial.println(rpm);
  }
}
```

WORKED EXAMPLE | RESOLUTION

Counting for 1 s with one pulse per revolution gives an RPM resolution of 60 rpm (one pulse = 60 rpm). At low speeds the error is big. **Measure the time between pulses instead:** $\text{rpm} = 60,000,000 / \text{period_us}$ using `micros()` in the ISR. At 120 rpm, the period is 500,000 μs and the resolution is $\pm(1 \mu\text{s} \div 500,000 \mu\text{s}) \approx 0.0002 \%$, far better.

A water-flow sensor YF-S201 gives about 7.5 pulses per second per L/min: $\text{flow} = \text{pulses per second} / 7.5$.

23.5 Hardware timers

The ATmega328P has three hardware timers that count clock pulses independently of your program. They are the engine behind `millis()`, PWM, `tone()` and `Servo`. A timer is a counter register that increments with a clock derived from the 16 MHz system clock divided by a **prescaler** (1, 8, 64, 256, 1024). It can trigger an interrupt when it **overflows** or when it equals a **compare value**.

Timer	Size	Used by	PWM pins
Timer0	8-bit	<code>millis()</code> , <code>micros()</code> , <code>delay()</code>	5, 6
Timer1	16-bit	Servo library, TimerOne library	9, 10
Timer2	8-bit	<code>tone()</code>	3, 11

$$f_{\text{interrupt}} = f_{\text{clock}} \div (\text{prescaler} \times (\text{compare} + 1))$$

For a timer in CTC (clear-timer-on-compare) mode

WORKED EXAMPLE | A 1 KHZ INTERRUPT FROM TIMER1

$f_{\text{clock}} = 16 \text{ MHz}$. Choose prescaler 8: timer clock = 2 MHz. For 1 kHz we need 2,000 counts: compare = 2000 – 1 = 1999. So $f = 16,000,000 \div (8 \times 2000) = \mathbf{1000 \text{ Hz}}$ exactly. A 16-bit timer can count up to 65,535, so this fits.

Another: 100 Hz with prescaler 64: timer clock 250 kHz; compare = 2500 – 1 = 2499.

Timer1 compare interrupt (register level)

```
// Toggle an LED from a timer interrupt at exactly 2 Hz (Timer1, CTC mode). Register-level.
const int LED = 13;

void setup() {
  pinMode(LED, OUTPUT);
  noInterrupts();
  TCCR1A = 0; // normal port operation
  TCCR1B = 0;
  TCNT1 = 0; // reset the counter
  OCR1A = 15624; // compare value: 16e6 / (1024 * 1 Hz) - 1 = 15624 -> 1 Hz
  TCCR1B |= (1 << WGM12); // CTC mode: clear timer on compare match
  TCCR1B |= (1 << CS12) | (1 << CS10); // prescaler = 1024
  TIMSK1 |= (1 << OCIE1A); // enable the compare-match interrupt
  interrupts();
}

ISR(TIMER1_COMPA_vect) { // interrupt vector for Timer1 compare match A
  digitalWrite(LED, !digitalRead(LED)); // toggles every second -> blinks at 0.5 Hz (1 s on, 1 s off)
}

void loop() {
  // free for other work
}
```

Check the arithmetic: timer clock = 16 MHz / 1024 = 15,625 Hz. Counting to 15,625 (OCR1A = 15,624) takes exactly 1 s. So the ISR fires once a second; toggling gives a 2-second period (0.5 Hz blinking). To toggle at 2 Hz you would fire it four times a second (OCR1A = 3905).

The easy way: the TimerOne library

Fixed-rate sampling at 100 Hz

```
#include <TimerOne.h> // install "TimerOne" from the Library Manager

volatile bool sampleNow = false;

void timerISR() { sampleNow = true; } // just set a flag

void setup() {
  Serial.begin(9600);
  TimerOne.initialize(10000); // period in microseconds: 10 ms -> 100 Hz
  TimerOne.attachInterrupt(timerISR);
}

void loop() {
  if (sampleNow) { // executed exactly 100 times per second
    sampleNow = false;
    int v = analogRead(A0);
    Serial.println(v);
  }
}
```

TIP

A timer interrupt guarantees **regular timing without jitter**, unlike `millis()`-based timing that can be delayed by other code in `loop()`. Use it for sampling, refreshing multiplexed displays (Chapter 20), generating waveforms, and control loops. Remember: the ISR should only set a flag or do minimal work.

23.6 The watchdog timer

A **watchdog** is a timer that resets the microcontroller unless the program “pets” it regularly. If the program hangs (an infinite loop, a stuck sensor), the watchdog times out and restarts the chip: an essential feature for unattended devices. On the Uno use `<avr/wdt.h>`:

Watchdog: automatic recovery from hangs

```
#include <avr/wdt.h>

void setup() {
  Serial.begin(9600);
  wdt_enable(WDTO_2S);           // reset if not petted within 2 seconds
}

void loop() {
  wdt_reset();                  // "pet the dog": call regularly when all is well
  doWorkThatMightHang();
}
```

SAFETY WARNING

On the original Uno bootloader the watchdog can get stuck in a reset loop if it resets the board before the bootloader can clear it: test carefully, and consider the newer Optiboot bootloader (standard on recent Unos).

23.7 Sleep and low power

A project running on a battery should **sleep** between tasks. The ATmega328P has sleep modes; in **power-down** mode it draws about 0.1 μA (the chip alone), waking on an external interrupt or a watchdog timeout. The Uno board itself draws $\sim 45\text{ mA}$ because of its regulator, USB chip and LED, so for real battery operation use a bare ATmega328P or a Pro Mini with the regulator and LED removed. Use the **LowPower** library:

Sleeping between measurements

```
#include <LowPower.h>           // "Low-Power" by Rocket Scream

void loop() {
  takeMeasurement();           // ~10 ms of work
  LowPower.powerDown(SLEEP_8S, ADC_OFF, BOD_OFF); // sleep 8 s, using ~6 uA total (chip alone)
}
```

WORKED EXAMPLE | BATTERY LIFE ESTIMATE

A sensor node wakes for 50 ms every 8 s drawing 15 mA, and sleeps at 10 μA . Average current = $(0.05 \times 15\text{ mA} + 7.95 \times 0.01\text{ mA})/8\text{ s} = (0.75 + 0.0795)/8 = \mathbf{0.104\text{ mA}}$. On a 2000 mAh battery: $2000/0.104 \approx 19,200\text{ h} \approx \mathbf{2.2\text{ years}}$ (ignoring self-discharge). Without sleep (15 mA continuous): 133 hours $\approx$ 5.5 days. This is why sleeping matters.

HANDS-ON EXPERIMENT | EXPERIMENT 18: PULSES, COUNTS AND TIMING

You need: Arduino, push button, LED, 330 Ω , 10 k Ω , 100 nF, jumper wires. (For the tachometer: a Hall sensor A3144 + magnet, or simply a button tapped at a steady rhythm.)

1. Upload the interrupt button toggle. Press the button 20 times and count how many times the LED actually toggles: you will see errors due to bounce. Add a 100 nF capacitor across the button and compare. Then add the 50 ms software debounce inside the ISR.
2. Add `delay(3000)` in `loop()`, and press the button during the delay: the LED still responds immediately. Compare with the polling version of Chapter 16, where the button is ignored during a delay.
3. Write the tachometer: tap the button at about 2 taps per second; the Serial Monitor should show ≈ 120 "RPM".
4. Use `TimerOne` to print a number every 100 ms exactly. Add a `delay(50)` in `loop()` and see that the timer keeps perfect time while `loop()` does not.

CHAPTER SUMMARY

- **Interrupts** run an ISR at the moment of an event; on the Uno, `attachInterrupt(digitalPinToInterrupt(2 or 3), isr, mode)`; modes RISING/FALLING/CHANGE/LOW.
- **ISR rules:** short; no `delay()` or `Serial`; shared variables are `volatile`; copy multi-byte data with interrupts disabled.
- Mechanical switches bounce, even in interrupts: use a time-based or hardware debounce.
- Measure frequency/RPM by counting pulses over a fixed time, or better by measuring the period with `micros()`.
- **Timers** count the 16 MHz clock divided by a prescaler; $f = 16 \text{ MHz} / (\text{prescaler} \times (\text{OCR}+1))$; `TimerOne` makes periodic interrupts easy.
- The **watchdog** resets a hung program; **sleep modes** cut power to microamps.

ASSIGNMENTS | CHAPTER 23

23.1 [Calculate] A `Timer1` in CTC mode uses prescaler 256 and `OCR1A = 6249`. What is the interrupt frequency?

23.2 [Calculate] Choose a prescaler and `OCR1A` value to generate a 50 Hz interrupt. (`Timer1`, 16-bit)

23.3 [Calculate] A tachometer has 2 pulses per revolution. In 1 s you count 130 pulses. What is the RPM?

23.4 [Calculate] Using period measurement: the time between pulses from a 1-pulse-per-revolution sensor is 25 ms. What is the RPM?

23.5 [Calculate] A flow sensor produces 7.5 pulses per second for each L/min. You count 255 pulses in 10 s. What is the flow in L/min and the volume in 10 s?

23.6 [Calculate] A node draws 20 mA for 100 ms every 10 s and sleeps at 5 μA . Find the average current and the life on a 1000 mAh battery.

23.7 [Practice] Why is `Serial.println()` inside an ISR a bad idea?

23.8 [Practice] Find the bug: `int count; void isr() { count++; } loop() { Serial.println(count); }` What is wrong and how do you fix it?

23.9 [Think] Why does `millis()` stop advancing inside a very long ISR and what are the consequences?

23.10 [Think] When should you prefer polling to an interrupt?

23.11 [Hands-on] Do Experiment 18 step 4: print with TimerOne at 100 Hz and in loop() add delay(50). Count how many samples arrive in 10 s.

Designing your own circuits

You have spent twenty-three chapters collecting skills. Design is the act of combining them. The difference between a beginner and an experienced maker is not cleverness: it is **process**. Experienced people never start by wiring; they start by writing down what the system must do, then work systematically from requirements to parts to numbers to a tested result. This chapter gives you that process, a library of interface recipes you can reuse for almost any part, and a full worked example. Follow it on your next three projects and you will design circuits without needing a tutorial.

WHAT YOU WILL LEARN IN THIS CHAPTER

- A repeatable **design process**: requirements → architecture → parts → schematic → firmware → test
- How to **choose components** and verify them with the calculations you have learned
- How to do a **pin assignment**, a **current budget** and a **protection review**
- Standard **interface recipes** for connecting any input or output to an Arduino safely
- How to read a **datasheet** and how to draw a clean **schematic**
- A complete worked design from a blank page: a greenhouse monitor

24.1 The design process

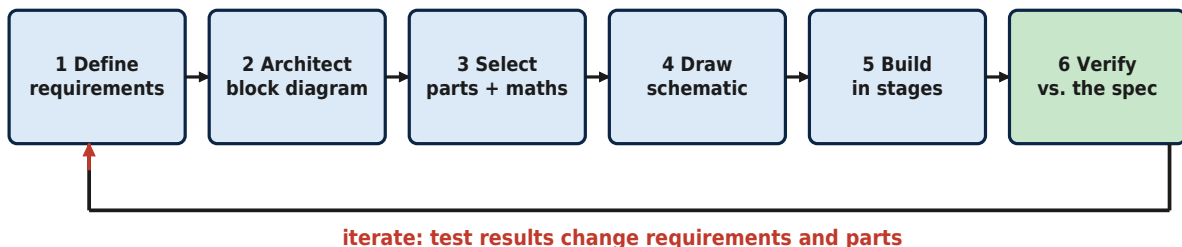


Figure 24.1 The design loop. Skipping steps 1–4 and jumping to “build” is the most common cause of wasted evenings.

Step 1: Define the requirements

Write, in plain sentences, **what the device must do** and under what conditions. Put numbers on everything. A requirement is good if you can **test** it with a yes/no answer.

Vague	Testable requirement
“Measure temperature”	Measure air temperature from 0 to 50 °C with ± 1 °C accuracy, once every 10 s
“Turn on a fan”	Switch a 12 V, 300 mA fan on above 28 °C and off below 26 °C (hysteresis 2 °C)
“Battery powered”	Run for ≥ 7 days on 4 × AA (2000 mAh) → average current ≤ 11 mA
“Show the value”	16×2 LCD shows temperature (one decimal) and humidity every 2 s
“Safe”	No exposed mains; 12 V fan has a 1 A fuse; all inputs protected to 12 V

Step 2: Architecture: the block diagram

Draw boxes for each function and arrows for what flows between them (power, analog signals, digital signals, data buses). Always begin with the same generic pattern: **inputs** → **controller** → **outputs**, plus **power**. A block diagram lets you see the interfaces before choosing any part.

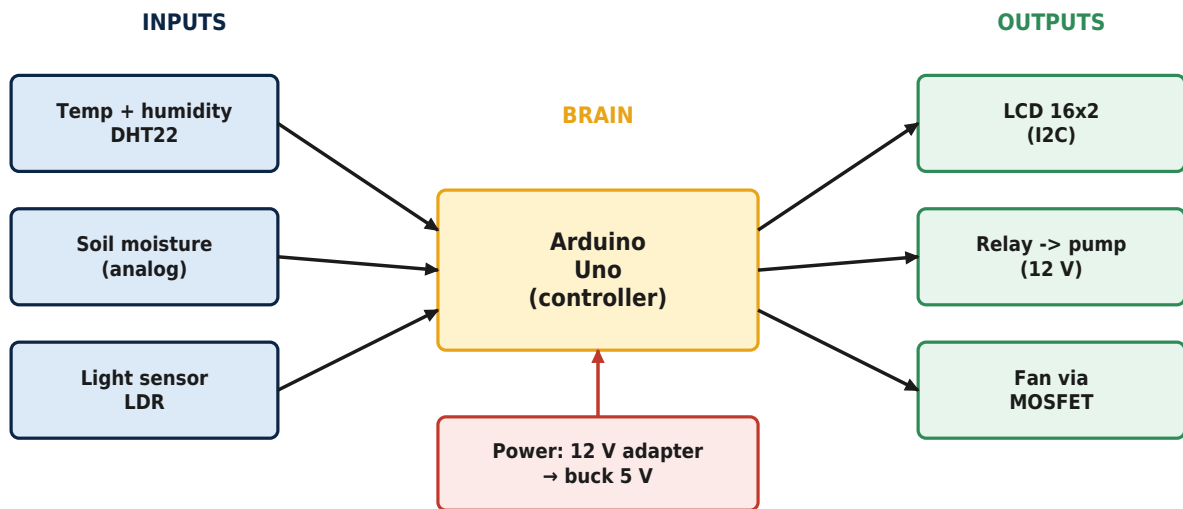


Figure 24.2 A block diagram for a greenhouse controller: inputs on the left, outputs on the right, power underneath.

Step 3: Select the parts and do the numbers

For every block, choose a part **and prove it fits** with arithmetic. The checklist below catches nearly every beginner error before a single wire is connected:

PER-PART DESIGN CHECKLIST

- **Voltage:** supply voltage within the part's range? Logic level compatible (5 V vs 3.3 V)? Any voltage on any pin $\leq$ its absolute maximum?
- **Current:** what does it draw? What can its driver (pin, regulator, transistor) supply? Is the total within the power budget?
- **Interface:** which pins, which protocol (digital, analog, I2C, SPI, UART)? Are libraries available? Are addresses unique?
- **Support components:** resistors (LED, pull-ups), capacitors (decoupling), diodes (flyback, reverse-polarity), fuse, connectors.
- **Power dissipation:** resistors, transistors and regulators within their ratings ($P = V \times I$) with 50 % margin?
- **Timing and speed:** read-rate limits (DHT22: 2 s), settle times, PWM frequency?
- **Availability and cost:** can you actually buy it? Is there an alternative?

Step 4: Draw the schematic and assign pins

A schematic drawn in a program (KiCad is free; EasyEDA and Fritzing are friendly) or even on paper with a ruler is a **contract** between you and the circuit. Rules for a readable schematic:

- **Positive supply at the top, ground at the bottom, signals left to right.**
- **Every part has a reference designator** (R1, C2, Q1) and a **value** (10 k Ω , 100 nF, 2N2222).
- **Label nets** (D7_LED, SDA) instead of drawing long wires; every ground symbol is the same net.
- **Show connector pin numbers** and supply pins of ICs; include decoupling capacitors.
- **One function per area** (power section, inputs, outputs) and a title block with name, date, revision.
- **Keep a pin table** next to it: each Arduino pin, its function and its part.

Arduino pin	Net name	Connects to	Type	Note
D2	DHT_DATA	DHT22 data (+10 kΩ pull-up)	digital in/out	Reads every 2 s
D3	FAN_PWM	MOSFET gate via 220 Ω	PWM out	Timer2 pin
D4	PUMP_RELAY	Relay module IN	digital out	Active LOW module
D5, D6	—	reserved (future)	—	
A0	SOIL	Soil moisture sensor AO	analog in	Power from 5 V
A1	LDR	LDR divider (10 kΩ)	analog in	
A4 / A5	SDA / SCL	LCD 0x27	I2C	Pull-ups on backpack
D0/D1	—	leave free for USB	—	Do not use

Pin-assignment rules: avoid D0/D1 (USB serial); use PWM pins (3, 5, 6, 9, 10, 11) for PWM only when needed; reserve D2/D3 for interrupts; keep analog inputs on A0–A5 away from noisy outputs; leave spare pins for later; avoid pin 13 for loads that would drag the built-in LED (fine for small loads).

Step 5: Build stage by stage

Build **one block at a time** and test it alone with a tiny sketch before adding the next: (1) power, measure 5 V on the rail; (2) first sensor prints a sensible number; (3) next sensor; (4) outputs with test sketches; (5) the integrated logic. Keep the previous step working; if a new block breaks things, the culprit is the last change.

Step 6: Verify against the requirements

Go back to the testable requirements and **test them** one at a time (“does it switch on at 28 °C? how long does it run on the batteries?”). Record the results. If a requirement is not met, decide: change the design or the requirement.

24.2 Interface recipes

Nearly every connection to an Arduino falls into one of the following patterns. Memorise them; the design of a new project is mostly choosing the right recipe for each part.

You want to connect...	Recipe	Chapter
LED (indicator)	Pin → resistor $R = (5 - V_F)/I_F$ ($\approx 330\ \Omega$) → LED → GND	8, 14
High-power LED / strip / motor / lamp ($\geq 20\ \text{mA}$)	Pin → 220 Ω → logic-level N-MOSFET gate (+10 kΩ pull-down); load between +V and drain; source to GND; flyback diode for inductive loads	9, 18, 21
Relay / solenoid	Relay module (transistor + diode on board) or pin → 1 kΩ → NPN base with 10 kΩ pull-down; flyback diode across the coil	7, 9
Buzzer	Active: pin → buzzer (+ transistor if $> 20\ \text{mA}$). Passive: <code>tone()</code> through 100 Ω	18
Button / switch	Pin to GND, <code>INPUT_PULLUP</code> ; debounce in software	10, 16
Potentiometer	Outer legs 5 V/GND, wiper → analog pin	17
Resistive sensor (LDR, NTC, FSR)	Voltage divider with a fixed resistor $\approx$ sensor mid-range; middle → analog pin; 100 nF to GND for noise	5, 17
Voltage up to 25 V (battery monitor)	Divider $R1/R2$ with $V_{\text{out}} \leq 5\ \text{V}$: e.g. 30 kΩ + 10 kΩ → ÷4; 100 nF at the pin	5, 17
3.3 V sensor output into the Arduino	Direct (reads HIGH above 3 V)	12

You want to connect...	Recipe	Chapter
5 V Arduino output into 3.3 V device	Divider 1 k Ω /2 k Ω or level-shifter module	12, 22
Digital sensor with open-collector output (Hall, some IR)	INPUT_PULLUP or 10 k Ω pull-up to 5 V	10, 19
Servo	Signal $\rightarrow$ pin; red $\rightarrow$ external 5 V; brown $\rightarrow$ common GND; 470 μ F across supply	18
DC motor	H-bridge module; separate supply; 100 nF across motor; common GND	21
I2C device	SDA $\rightarrow$ A4, SCL $\rightarrow$ A5, power per board; pull-ups 4.7 k Ω if not present; unique address	22
Mains load (AC)	Certified, enclosed relay module or SSR, fused, by or with an expert only ; never bare circuits	7, 13

24.3 Reading a datasheet

A **datasheet** is the manufacturer's specification of a part. Do not fear them: you only need a few sections, and you already know the vocabulary.

Section	What to look for
Features / description (page 1)	What is it? Voltage range, current, package. Decide if it is the right part.
Absolute maximum ratings	The limits you must never exceed (supply voltage, pin voltage, current, temperature). Stay well below.
Recommended operating conditions	The range where performance is guaranteed.
Electrical characteristics	The numbers for design: V_F , $V_{CE(sat)}$, $R_{DS(on)}$, quiescent current, V_{IH}/V_{IL} , dropout, typical vs maximum.
Pin configuration	Which pin is which; top view vs. bottom view!
Typical application circuit	A reference design with recommended component values: copy it.
Timing diagrams / register map	For digital and communication devices: how to talk to it.
Package dimensions, thermal data	For PCB design and heat (θ_{JA} in $^{\circ}\text{C}/\text{W}$).

TIP

Use the **typical** values for estimation and the **maximum / minimum** values for worst-case design. Beware of graphs with log axes and notes about test conditions ("at $T_A = 25^{\circ}\text{C}$, $V_{CC} = 5\text{ V}$ "): your circuit may differ.

24.4 Worked design: a greenhouse monitor

Let us apply the process to a real example. A gardener wants to monitor a small greenhouse and automate its ventilation.

1. Requirements

1. Measure air temperature (0 – 50 °C, ± 1 °C) and humidity every 5 s.
2. Measure soil moisture (dry / OK / wet) and light level (day/night).
3. Switch a 12 V, 300 mA ventilation fan **on above 28 °C, off below 26 °C**.
4. Switch a 12 V water pump through a relay when the soil is dry and it is daytime, for at most 10 s once per hour.
5. Show temperature, humidity and soil status on a 16×2 LCD.
6. Powered from a 12 V, 2 A adapter. Non-mains electronics only. Safe: fuse on the 12 V input, reverse-polarity protection.

2. Architecture

Figure 24.2 shows the blocks. Power: a 12 V adapter feeds the fan and pump directly; a buck converter module (12 → 5 V) feeds the Arduino and sensors (a linear regulator would waste $(12 - 5) \times 0.15 = 1$ W).

3. Parts and calculations

Block	Part	Checks
Temperature + humidity	DHT22 on D2	5 V, 1.5 mA; read every 2 s or more ✓ (5 s); needs 10 k Ω pull-up (module has it)
Soil moisture	Capacitive sensor on A0	5 V, 5 mA; output 1.2 – 3 V (calibrate in dry air/water) ✓
Light	LDR 10 k Ω / 10 k Ω divider on A1	Output 0.5 – 4.5 V; 0.25 mA ✓
Fan	IRLZ44N MOSFET on D3 (PWM), 220 Ω gate, 10 k Ω pull-down, 1N4007 across fan	Fan 300 mA: $I^2R_{DS(on)} = 0.09 \times 0.03 \approx 3$ mW ✓; gate 5 V $\geq$ logic level ✓; flyback diode since a fan is inductive
Pump	5 V relay module on D4, pump on the relay NO contact from the 12 V rail; 1N4007 across the pump	Relay coil 70 mA (module LED + transistor) ✓; contact rating 10 A ✓; pump current 0.5 A assumed
Display	I2C LCD 0x27 on A4/A5	5 V, 25 mA with backlight ✓
Power	Fuse 2 A; 1N5819 series diode; LM2596 module set to 5.0 V; 470 μ F on the 12 V rail; 100 μ F on 5 V	Buck delivers up to 2 A: load $41 + 25 + 70 + 1.5 + 5 + 0.3 \approx 150$ mA ✓; Schottky drop 0.3 V: input 11.7 V fine

Current budget (5 V rail): Arduino 45 mA + LCD 25 mA + DHT22 1.5 mA + soil 5 mA + LDR 0.3 mA + relay coil 70 mA = **147 mA**, well within what the buck converter can deliver, and within 500 mA if ever powered from USB (for programming). **12 V rail:** fan 300 mA + pump 500 mA + buck input ≈ 147 mA $\times 5/12/0.85 = 72$ mA → worst case (everything on) $\approx$ **0.9 A** → a 2 A adapter and a 2 A fuse are fine.

4. The code outline (a state machine with hysteresis)

Greenhouse controller (structure)

```
#include <DHT.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const int DHT_PIN = 2, FAN_PIN = 3, PUMP_PIN = 4, SOIL_PIN = A0, LDR_PIN = A1;
const float FAN_ON = 28.0, FAN_OFF = 26.0;           // hysteresis
const int SOIL_DRY = 600;                             // calibrate: higher = drier for this sensor
const int DAY_LEVEL = 400;                             // LDR above this = daytime
const unsigned long PUMP_MS = 10000UL;                // pump runs at most 10 s
const unsigned long PUMP_LOCKOUT = 3600000UL;         // at most once per hour

DHT dht(DHT_PIN, DHT22);
```

```

LiquidCrystal_I2C lcd(0x27, 16, 2);

bool fanOn = false;
bool pumpOn = false;
unsigned long pumpStart = 0, lastPump = 0, lastRead = 0;
float temp = 0, hum = 0;

void setup() {
  pinMode(FAN_PIN, OUTPUT);
  pinMode(PUMP_PIN, OUTPUT);
  digitalWrite(PUMP_PIN, HIGH);           // relay module is active-LOW: HIGH = off
  dht.begin();
  lcd.init(); lcd.backlight();
  lastPump = -PUMP_LOCKOUT;               // allow watering right after start (unsigned wraps)
}

void loop() {
  unsigned long now = millis();

  if (now - lastRead >= 5000) {           // task 1: measure every 5 s
    lastRead = now;
    float t = dht.readTemperature(), h = dht.readHumidity();
    if (!isnan(t)) temp = t;              // keep the previous value on a bad read
    if (!isnan(h)) hum = h;
    int soil = analogRead(SOIL_PIN);
    int light = analogRead(LDR_PIN);

    if (!fanOn && temp > FAN_ON) fanOn = true; // task 2: fan with hysteresis
    if (fanOn && temp < FAN_OFF) fanOn = false;
    digitalWrite(FAN_PIN, fanOn);

    bool dry = soil > SOIL_DRY, day = light > DAY_LEVEL; // task 3: start pump if allowed
    if (!pumpOn && dry && day && now - lastPump >= PUMP_LOCKOUT) {
      pumpOn = true; pumpStart = now;
      digitalWrite(PUMP_PIN, LOW);
    }

    lcd.setCursor(0, 0);                  // task 4: display
    lcd.print(temp, 1); lcd.print("C ");
    lcd.print(hum, 0); lcd.print("% ");
    lcd.print(fanOn ? "FAN " : "    ");
    lcd.setCursor(0, 1);
    lcd.print(dry ? "Soil:DRY " : "Soil:OK ");
    lcd.print(pumpOn ? "PUMP" : "    ");
  }

  if (pumpOn && now - pumpStart >= PUMP_MS) { // safety: pump stops after 10 s, independent of sensors
    pumpOn = false; lastPump = now;
    digitalWrite(PUMP_PIN, HIGH);
  }
}

```

Notice the design ideas: **hysteresis** prevents the fan from chattering; a **maximum run time** and **lock-out** make the pump safe even if a sensor fails (**fail-safe**); values are named constants; all timing uses `millis()` so nothing blocks; bad sensor reads are ignored; relay initial state is set in `setup()` so the pump does not pulse at power-up.

5 and 6. Build and verify

Build and test: (1) 5 V rail measured with the multimeter before connecting the Arduino; (2) DHT22 alone; (3) soil and light sensors with values printed; (4) fan with the MOSFET circuit using a bench test sketch; (5) relay with a lamp/LED, not yet with the pump; (6) integrated program; (7) test each requirement: heat the DHT22 with your hand/warm air and see that the fan switches at 28 °C and releases at 26 °C; put the soil sensor in water and dry air; time the pump.

24.5 Design patterns worth knowing

- **Fail-safe defaults:** outputs that can cause harm (heaters, pumps, motors) default to OFF at power-up and when sensors fail; add hardware limits (fuse, thermal cutout) independent of software.
- **Hysteresis** everywhere you compare a noisy signal with a threshold.
- **Separate the “what” from the “how”:** functions `readSensors()`, `decide()`, `actuate()`, `display()`.
- **Watchdog** (Chapter 23) for anything unattended.
- **Configuration at the top:** all pins, thresholds and timings as constants, so tuning never means hunting through code.
- **Serial debug output** controlled by a flag (`#define DEBUG 1`) so you can turn it off.
- **Calibrate** sensors and record the calibration values in the code comments.
- **Reuse tested blocks:** keep a folder of small, working sketches for each sensor/actuator (your “cookbook”).

HANDS-ON EXPERIMENT | EXPERIMENT 19: DESIGN IT YOURSELF (MINI DESIGN EXERCISE)

Pick one of these: (A) **night light that fades with an LDR and a buzzer alarm if the door (reed switch) opens**; (B) **plant-watering reminder** (soil sensor + LED + buzzer); (C) **temperature-triggered fan** for your desk.

1. Write 5 testable requirements with numbers.
2. Draw the block diagram and choose the parts from your kit.
3. Make the pin table and compute: each resistor (LED, pull-ups, dividers), the current budget and the power ratings.
4. Draw the schematic (paper is fine) using the conventions in this chapter. Ask a friend to read it: can they build it without asking you anything?
5. Build it in stages; write the code as functions; test each requirement; note what you had to change and why.

CHAPTER SUMMARY

- Follow the loop: **define** → **architect** → **select** → **draw** → **build in stages** → **verify**. Write testable requirements with numbers.
- For each part check voltage, current, interface, support components, dissipation, timing; do the arithmetic before connecting anything.
- Use **interface recipes**: LED + resistor; MOSFET for loads; button to GND with INPUT_PULLUP; dividers for sensors and voltage monitors; level shifting; flyback diodes; separate motor supply with common ground.
- Draw clear schematics (+ at top, GND at bottom, labels, values); keep a pin table; read datasheets for ratings, pinout and typical circuit.
- Design for safety: fail-safe defaults, hysteresis, limits, watchdog, fuses; never build bare mains circuits.

ASSIGNMENTS | CHAPTER 24

24.1 [Design] Write five testable requirements for “a doorbell with a button, a melody and a light” (include numbers: loudness not required, but timing, voltage and power should be).

24.2 [Calculate] You need to read a 24 V battery with the Arduino’s 5 V ADC. Choose R1 (top) and R2 (bottom) from E12 so that 30 V maps to ≤ 5 V, and compute the divider current at 24 V.

24.3 [Calculate] A pump draws 1.2 A from 12 V. You switch it with an IRLZ44N ($R_{DS(on)} = 0.035 \Omega$). Compute MOSFET dissipation, the required gate resistor if the gate capacitance charge must not exceed 20 mA peak from a 5 V pin, and whether a heatsink is needed.

ASSIGNMENTS | CHAPTER 24

24.4 [Calculate] Compute the 5 V current budget of: Arduino (45 mA), OLED (20 mA), two DS18B20 (1.5 mA each), one LED at 10 mA, and a 5 V relay module (70 mA). Can the USB port supply it? Can a 7805 from 9 V run it without a heatsink?

24.5 [Design] Draw a block diagram and pin table for a **motion-activated light with a manual override button**: PIR sensor, LDR to allow operation only when dark, a 12 V LED strip (1 A), a push button to force the light on, 2-minute timeout.

24.6 [Design] List which parts of your greenhouse design (Section 24.4) would make it unsafe if the DHT22 failed, and how the design protects against this.

24.7 [Practice] Read the datasheet of a part in your kit (2N2222, LM35, or IRLZ44N) and record: absolute maximum V and I, the pinout, and one typical application circuit. Write them in your notebook.

24.8 [Think] Why should you test each block separately before integrating the whole system?

24.9 [Think] Why is a hardware safety (fuse, thermal cut-out) important even if your code already limits the heater power?

24.10 [Hands-on] Carry out Experiment 19. Submit (to yourself) the requirements, block diagram, pin table, schematic, code and a test table showing each requirement and whether it passed.

Debugging and troubleshooting

Everything you build will, at some point, not work. That is not a sign that you are bad at electronics; it is the normal state of the craft. Professional engineers spend more time debugging than designing. What separates them from beginners is not luck but **method**: they ask the right questions in the right order, measure instead of guess, and change one thing at a time. This chapter gives you that method and a catalogue of the faults you will meet again and again.

WHAT YOU WILL LEARN IN THIS CHAPTER

- A systematic **debugging method** that works for hardware and software
- The **top 12 beginner faults** and how to find each in minutes
- How to use the **multimeter**, **Serial** prints and simple tricks to isolate a fault
- Recognising **power problems**, **noise**, **floating inputs** and **memory problems**
- How to test components and modules, and how to ask for help effectively

25.1 The debugging mindset

SEVEN RULES OF DEBUGGING

1. **Stay calm and stop changing things randomly.** Every random change adds a new possible fault.
2. **Observe before you act.** What exactly happens? What did you expect? Write down the difference.
3. **Divide and conquer.** Split the system in two halves, decide which half contains the fault, and repeat: 16 suspects become 8, 4, 2, 1 in four steps.
4. **Measure, do not assume.** Put the multimeter on it: is it really 5 V there? Print the variable: is it really what you think?
5. **Change one thing at a time**, then retest. Undo the change if it did not help.
6. **Simplify.** Remove parts until something works, then add them back one by one. Test sensors and outputs with the smallest possible sketch.
7. **Assume the simple explanation first:** power, ground, polarity, a wire in the wrong hole, a typo. Exotic explanations come last.

ANALOGY | A DOCTOR OR A MECHANIC

A doctor diagnoses by asking about symptoms, taking measurements (temperature, blood pressure) and testing the simplest causes before rare ones. A mechanic checks fuel, spark and air before dismantling the engine. Debugging is **diagnosis**: symptoms → measurements → hypothesis → test → conclusion.

25.2 A general procedure

1. **Power:** Is the power LED on? Measure the 5 V and GND pins with the multimeter. Measure the supply at the far end of the circuit (at the sensor) to check for voltage drops.
2. **Does the basic board work?** Upload **Blink**. If it works, board, cable and IDE are fine. If not, see Chapter 14's table.
3. **Isolate the circuit from the code:** for an output, use a trivial sketch (set the pin HIGH) and measure the pin with the meter; for an input, print the raw value.
4. **Check the wiring against the schematic,** one connection at a time, with a pencil tick on the printed schematic. Use the continuity buzzer to verify that the breadboard connections are what you think.
5. **Check the code:** pins, pin modes, logic (pull-up inverted?), timing, variable types, libraries and I2C addresses. Add `Serial.println` at each decision point.
6. **Swap parts:** try another jumper wire, another LED, another breadboard position, another sensor of the same type. Breadboard contacts and cheap jumpers fail often.
7. **Search:** the exact error message or symptom plus the part number. Somebody has met it before.
8. **Ask for help** with the information listed in section 25.8.

25.3 The top 12 faults

#	Symptom	Probable cause	How to check / fix
1	Nothing works; power LED off or dim	Bad USB cable/port; short circuit; overloaded 5V	Remove everything from the board; measure 5 V; check for shorts with continuity (5V-GND)
2	LED does not light	LED reversed; missing <code>pinMode</code> ; wrong pin number; no resistor path to GND; dead LED	Reverse LED; print code pin; test the LED with a battery + resistor
3	Button reads random values	Floating input (no pull resistor); missing ground	<code>INPUT_PULLUP</code> and button to GND; check the ground wire
4	Button registers many presses per touch	Contact bounce	Debounce in software (30 ms) or add 100 nF
5	Analog readings jump around or are always 0/1023	Floating pin; missing divider resistor; noise; sensor wired to the wrong pin; 3.3 V vs 5 V sensor	Measure the pin voltage; add 100 nF and averaging; check that the pin is A0-A5
6	Motor/servo runs, Arduino resets	Supply sags; motor noise; shared ground impedance	Separate supply, common GND, 470 μ F across supply, 100 nF across motor, flyback diode
7	Motor does not spin, or spins the wrong way	Missing common ground; driver enable not high; motor supply off; polarity	Measure the voltage at the motor terminals; swap motor wires; check EN pin
8	I2C device not found	Wrong address; SDA/SCL swapped or on the wrong pins; no power; no pull-ups	Run the scanner; check A4/A5; 3.3 V vs 5 V; try another cable
9	LCD lit but no text	Contrast trimmer; wrong address; <code>init()</code> missing	Turn the trimmer; scanner; correct address
10	Program behaves strangely after many minutes; random resets	RAM exhausted (strings, arrays); <code>String</code> fragmentation; <code>int</code> overflow of <code>millis()</code> ; brown-out	Use <code>F()</code> and char arrays; <code>unsigned long</code> for time; check free RAM; check supply
11	Upload fails	Wrong board/port; pins 0/1 in use; cable; another program using the port	Chapter 14 table; disconnect pins 0/1; close Serial Monitor

#	Symptom	Probable cause	How to check / fix
1 2	Works on the breadboard but not when moved / vibrated	Loose connections; oxidised breadboard contacts; long jumper wires	Reseat wires; shorter wires; solder the circuit (Chapter 27)

25.4 Hardware debugging tools and techniques

The multimeter: your eyes

- **Continuity test** finds broken wires and verifies connections **before** power: test every node of your schematic. Also finds **shorts**: measure between 5 V and GND with the circuit **unpowered**; a beep means a short. Measure again after disconnecting sections to find it.
- **Voltage checks** with power on: VCC at each IC (must be within $\pm 5\%$ of its specification), logic levels at signal pins, the voltage across each resistor (Ohm's law tells you the current). A voltage that is half of expected often means a missing connection plus a pull-up/pull-down, or a floating node.
- **Current checks** (in series): is the circuit drawing what you calculated? A very high current = short/overload; zero current = open circuit.
- **Compare with a known-good unit or simulation.** If you have two identical modules, swap them.

Software “probes”

Printing your way to the answer

```
// 1. Print the raw value and what you derive from it
Serial.print("raw="); Serial.print(raw);
Serial.print(" volts="); Serial.print(volts, 3);
Serial.print(" state="); Serial.println(state);

// 2. Show that code paths are reached
Serial.println(F("entered FORWARD state")); // F() saves RAM

// 3. Use the built-in LED as a signal, even with no Serial (e.g. inside an ISR or at boot)
digitalWrite(LED_BUILTIN, HIGH); // "got here"

// 4. Report free RAM: if this number shrinks over time you have a memory leak/fragmentation
extern int __heap_start, *__brkval;
int freeRam() {
    int v;
    return (int)&v - (__brkval == 0 ? (int)&__heap_start : (int)__brkval);
}
```

Logic analyser and oscilloscope

When you need to see **signals over time** (is that PWM really 490 Hz? what does the I2C waveform look like? is the supply dipping when the motor starts?), a scope or a logic analyser is invaluable. An inexpensive **8-channel USB logic analyser** (24 MHz, ~\$10, used with the free **PulseView/sigrok** software) decodes UART, I2C and SPI automatically and is an excellent beginner's tool; a cheap USB or handheld **oscilloscope** shows voltage waveforms, noise and ripple. You do not need them for this book, but you will appreciate them later.

25.5 Power and noise problems

More than half of the “mysterious” faults in microcontroller projects are power problems. Clues: random resets, erratic readings, a display that goes dim or garbled when something switches on, programs that work on USB but not on a battery.

Suspect	Test	Cure
Voltage too low (weak battery, long thin wires)	Measure VCC at the chip while the load is active	Fresh batteries, thicker/shorter wires, regulator with headroom
Voltage dips when motors/servos/relays switch	Watch VCC on a scope/meter min-hold; or see resets at the exact moment of switching	Separate supply for power loads; 470–2200 μ F capacitor; 100 nF across the motor; diode across relay coil
Regulator overheating (hot, shuts down)	Touch test (carefully); compute $(V_{in} - V_{out}) \times I$	Lower input voltage, heat sink, or buck converter
No common ground between supplies	Continuity between the ground of both supplies	Join the grounds at one point
USB port current limit (500 mA)	Power LED flickers; board disconnects	External 5 V supply with adequate current
Noise on analog readings	Print many readings; large swings	100 nF at the pin, averaging, shorter and separated wires, star ground, analog supply filtering

Grounding: the hidden half of every circuit

Current flows in a loop, so every signal shares a **return path** through ground. A thin ground wire carrying a motor’s current has a voltage drop $V = I R$; if a sensor shares that wire, its “0 V” rises and falls with the motor. Use a **star ground**: bring each section’s ground wire separately to one point (the supply). Keep **high-current return paths separate from sensitive analog returns**, and keep all loops small.

25.6 Software bugs and how to catch them

Bug	Example	Clue	Fix
= instead of ==	<code>if (x = 3)</code>	Condition always true	Use ==
Missing semicolon / brace	Compile error on the next line	Error message “expected ‘;’ before ...”	Look at the previous line
Integer division	<code>float v = 5 / 2;</code> → 2.0	Results are whole numbers	Use <code>5.0 / 2</code>
Variable type overflow	<code>int ms = millis();</code>	Breaks after 32 s	<code>unsigned long</code>
Off-by-one	<code>for (i=0; i<=N; i++)</code> with arrays	Corrupted data, crashes	<code>i < N</code>
Blocking delay	<code>delay(1000)</code> while waiting for a button	Presses missed	<code>millis()</code> timing
Uninitialised variable	<code>int count;</code> used before set (globals start at 0; locals do not)	Random values	Initialise always
Wrong pin mode	Reading a pin not set as INPUT; LED never set as OUTPUT	Dim LED, random reads	<code>pinMode</code> in <code>setup()</code>
Inverted logic	Active-LOW relay/LED, pull-up button	Everything seems reversed	Read the module’s truth table
Timer conflicts	<code>tone()</code> with PWM on pins 3/11; Servo with PWM on 9/10	PWM not working	Move to other pins

Bug	Example	Clue	Fix
Out of RAM	Large arrays, Strings	Strange behaviour; resets	<code>F ()</code> , <code>PROGMEM</code> , smaller buffers; Mega

TIP

Rubber-duck debugging: explain your code line by line, out loud, to a rubber duck (or a patient friend). Half the time you find the bug as you speak. When you truly cannot see it, take a break, eat, sleep: fresh eyes find bugs in minutes.

25.7 Testing components

Component	Quick test
Resistor	Multimeter Ω (out of circuit): within tolerance of the marked value
Capacitor (electrolytic)	Multimeter capacitance mode, or watch the resistance reading climb when measuring Ω (charging); leaky ones show low resistance
Diode / LED	Diode-test mode: 0.5–0.7 V forward (silicon), 1.6–3 V (LED lights faintly); OL in reverse. Both directions low = shorted; both OL = open
NPN transistor	Diode-test base-emitter and base-collector: ≈ 0.6 V with red on base; OL reversed; C-E OL both ways. Try the hFE socket
MOSFET	G-D-S all OL; briefly touching the gate to a voltage can turn it on (drain-source then reads low): static-sensitive
Relay	Coil resistance (50–400 Ω); click when powered; continuity on COM-NO/NC
Potentiometer	Resistance between outer legs = rated; wiper varies smoothly 0 $\rightarrow$ max
Jumper wire / breadboard row	Continuity beep. Many “dead circuits” are broken jumper wires
Arduino pin	Blink test: set the pin HIGH, measure 5 V; test each pin with an LED + resistor
Sensor module	Run the library’s simplest example; check power LED; measure the output with the meter

25.8 Asking for help the right way

Forums, communities and classmates are a huge resource, but people can only help if you give them information. Include:

1. **What you want to achieve** (the goal), and what you **expected** to happen.
2. **What actually happens** (exact symptoms, error messages copied in full).
3. **Hardware:** board model, every module (with a link or part number), power source, and a **clear photo** or a **schematic** (not just a description).
4. **Code:** the **complete** sketch (or a minimal version that shows the problem), formatted as code.
5. **What you have tried** and the measurements you have taken (voltages, Serial output).

HANDS-ON EXPERIMENT | EXPERIMENT 20: FIND THE FAULTS (DEBUGGING DRILL)

Ask a friend (or do it yourself using the list below after a break!) to build the LED-and-button circuit with one deliberate fault each time; find the fault using the procedure above:

1. LED reversed. 2. One jumper wire broken inside the insulation. 3. Resistor in the wrong row so it is not connected to anything. 4. Button wired to a different pin than in the code. 5. `pinMode` missing. 6. A 10 k Ω resistor used instead of 330 Ω (LED dim). 7. The ground wire removed from the Arduino. 8. Wrong pull-up logic (**HIGH** instead of **LOW**).

For each, write the symptom, the measurement that revealed it, and the fix. Time yourself: the goal is to find each fault in under five minutes using the method instead of guessing.

CHAPTER SUMMARY

- Debug with a method: observe, isolate, measure, change one thing at a time, simplify, check the simple things first (power, ground, polarity, pins).
- The top faults: power/ground, LED direction, floating inputs, bounce, wrong pin/mode, common-ground errors, noisy supplies, I2C address, overflow and RAM.
- Use the multimeter (continuity, voltage, current), Serial prints, the Blink test, and a logic analyser/scope when available.
- Most random resets and strange readings trace back to power, noise, or grounding: use separate supplies with a star ground and decoupling capacitors.
- Ask for help with the goal, symptoms, photos/schematic, complete code and what you have tested.

ASSIGNMENTS | CHAPTER 25

25.1 [Think] An LED connected to pin 9 never lights. List six possible causes in order of how quickly you can check them.

25.2 [Think] Your robot works on the bench connected to USB, but resets when you run it from a 9 V battery and the motors start. Give a hypothesis and a test.

25.3 [Calculate] A 5 V Arduino circuit draws 350 mA through 1 m of thin jumper wire (two wires, total 0.5 Ω). What is the voltage drop and what voltage arrives at the load?

25.4 [Calculate] Your ADC reading on A0 is 512 when it should be 768. The sensor is a divider with $R_{\text{top}} = 10$ k Ω and a sensor as the bottom resistor. What might be wrong? (Suppose the expected bottom resistor is 30 k Ω .)

25.5 [Practice] Write a tiny sketch to test whether pin 6 can drive an LED, printing a message to Serial each time it toggles. Why is this a good first step?

25.6 [Practice] Explain how to use the multimeter to find a short circuit between 5 V and GND on an unpowered breadboard.

25.7 [Practice] What information would you include when asking for help on a forum about an LCD that stays blank?

25.8 [Think] Why should you fix one thing at a time and retest?

25.9 [Hands-on] Do the debugging drill (Experiment 20). Record the time to locate each fault and the clue that led you to it.

Capstone projects

This chapter is a workshop. Each project follows the design process of Chapter 24: requirements, block diagram, parts and calculations, wiring, code, tests, and extensions. Build them in order; each one adds new skills. Resist the urge to copy the code without thinking: read every decision, change a parameter, break something on purpose and fix it. When you finish, you will have three working devices and, more importantly, the ability to invent your own.

WHAT YOU WILL LEARN IN THIS CHAPTER

- Apply the whole design process to three complete projects
- **Project 1:** an adjustable temperature-controlled fan with LCD (analog sensing, MOSFET driver, PWM, hysteresis)
- **Project 2:** a **Simon Says** memory game (arrays, buttons, LEDs, tones, game logic)
- **Project 3:** a **security alarm** with entry delay and event log (state machine, PIR, EEPROM, `millis()` timing)
- A list of further project ideas, and a checklist to confirm that you can now design on your own

26.1 Project 1: Temperature-controlled fan

Requirements

1. Measure room temperature from 10 – 50 °C, resolution better than 0.5 °C.
2. The user sets the **target temperature** between 18 °C and 35 °C with a knob.
3. A 12 V, 250 mA DC fan runs when the temperature exceeds the target by more than 1 °C, and stops when it falls within 0.3 °C of the target (hysteresis). Above that, speed rises with the error: minimum 35 % duty, full speed at 5 °C over.
4. A 16×2 LCD shows target, current temperature and fan speed, updated twice a second.
5. No component may exceed 50 % of its ratings; all inductive loads are protected.

Parts

Qty	Part	Purpose
1	Arduino Uno	Controller
1	LM35 temperature sensor (TO-92)	10 mV/°C analog output
1	10 kΩ linear potentiometer	Target temperature knob
1	16×2 LCD with I2C backpack	Display
1	IRLZ44N logic-level N-MOSFET (or 2N7000 for a small fan)	Fan switch
1	12 V DC fan (≤ 300 mA) and 12 V, 1 A supply	Load and power
1	1N4007 diode	Flyback protection
2	220 Ω and 10 kΩ resistors	Gate resistor and gate pull-down
2	100 nF ceramic and 470 μF electrolytic (25 V)	Sensor filtering; fan supply decoupling

Qty	Part	Purpose
1	Buck converter (12 → 5 V) or use the Arduino's VIN (12 V) with the on-board regulator	Logic supply

Design decisions and calculations

- **Sensor:** the LM35 gives 10 mV/°C. At 5 V reference one ADC step is 4.88 mV = 0.49 °C: too coarse. **Averaging 32 readings** brings the resolution to ≈ 0.1 °C (thanks to ADC noise acting as dither) and reduces noise by $\sqrt{32} \approx 5.7\times$. Alternative: the 1.1 V internal reference (0.107 °C per step), but then the pot (0 - 5 V) would saturate because the reference is global: use a divider on the pot or read it first. We keep the 5 V reference and average.
- **Fan driver:** $I = 250$ mA; the MOSFET $R_{DS(on)} \approx 0.03 \Omega \rightarrow P = 0.25^2 \times 0.03 = 1.9$ mW. Gate resistor 220 Ω ; 10 k Ω pull-down; flyback diode across the fan (motor = inductor). PWM at 490 Hz on D9 (Timer1) works; a whine may be audible: if so set 31 kHz as explained in Chapter 18.
- **Power budget:** 12 V rail: 250 mA (fan) + Arduino via VIN ≈ 50 mA $\rightarrow 0.3$ A; adapter 1 A is plenty. If the Arduino regulator dissipates $(12 - 5) \times 0.07$ A = 0.5 W that is acceptable; a buck converter is better.
- **Noise:** fan motors are noisy: put 470 μ F across the fan supply and 100 nF across the fan terminals, and keep the LM35 wires short, with a 100 nF capacitor at the ADC pin.

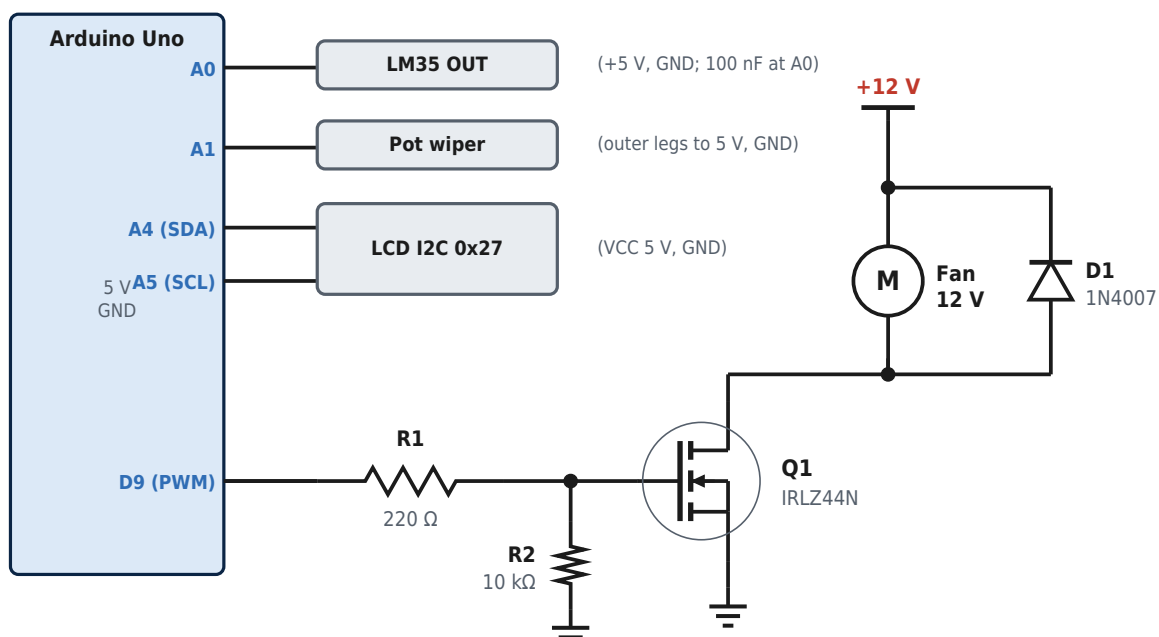


Figure 26.1 Project 1 wiring. The LM35 and pot feed A0 and A1, the LCD uses I2C, and D9 drives the fan MOSFET with its flyback diode. All grounds are common.

Code

Project 1: thermostat fan

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const int SENSOR_PIN = A0;           // LM35
const int SETPOINT_PIN = A1;         // potentiometer
const int FAN_PIN = 9;               // PWM to the MOSFET gate
const float HYST_ON = 1.0;           // fan starts when T > target + 1.0
const float HYST_OFF = 0.3;          // fan stops when T < target + 0.3
const float FULL_SPEED_ERR = 5.0;    // full speed at 5 degrees above target
const int MIN_DUTY = 90;             // about 35 % of 255
```

```

LiquidCrystal_I2C lcd(0x27, 16, 2);
bool fanRunning = false;
unsigned long lastUpdate = 0;

float readTemperature() { // average 32 samples
    long sum = 0;
    for (int i = 0; i < 32; i++) {
        sum += analogRead(SENSOR_PIN);
        delayMicroseconds(300);
    }
    float raw = sum / 32.0;
    return raw * 5.0 * 100.0 / 1023.0; // LM35: 10 mV per degree C
}

float readSetpoint() { // 18 .. 35 degrees C
    return 18.0 + analogRead(SETPOINT_PIN) * (35.0 - 18.0) / 1023.0;
}

int computeDuty(float error) {
    if (!fanRunning && error > HYST_ON) fanRunning = true; // hysteresis logic
    if ( fanRunning && error < HYST_OFF) fanRunning = false;
    if (!fanRunning) return 0;
    float frac = constrain(error / FULL_SPEED_ERR, 0.0, 1.0);
    return MIN_DUTY + (int)(frac * (255 - MIN_DUTY));
}

void setup() {
    pinMode(FAN_PIN, OUTPUT);
    analogWrite(FAN_PIN, 0); // fail-safe: fan off at start
    lcd.init(); lcd.backlight();
}

void loop() {
    float temp = readTemperature();
    float target = readSetpoint();
    int duty = computeDuty(temp - target);
    analogWrite(FAN_PIN, duty);

    if (millis() - lastUpdate >= 500) { // refresh the display twice a second
        lastUpdate = millis();
        lcd.setCursor(0, 0);
        lcd.print("Set:"); lcd.print(target, 1); lcd.print("C ");
        lcd.setCursor(0, 1);
        lcd.print("Now:"); lcd.print(temp, 1); lcd.print("C ");
        lcd.print(duty * 100 / 255); lcd.print("% ");
    }
}

```

Test plan

Requirement	Test	Expected
Temperature accuracy $\pm 0.5\text{ }^{\circ}\text{C}$	Compare with a reference thermometer at room temperature and in warm water ($\sim 35\text{ }^{\circ}\text{C}$)	Within $\pm 0.5\text{ }^{\circ}\text{C}$ (after calibration offset if needed)
Setpoint range 18 – 35 $^{\circ}\text{C}$	Turn the knob fully both ways, read the LCD	18.0 ... 35.0
Hysteresis	Set the target 2 $^{\circ}\text{C}$ below room temperature: fan must start; raise the target: fan must stop when $T < \text{target} + 0.3$	No chattering
Speed $\propto$ error	Hold the LM35 (warm fingers) to raise temperature; watch the duty	Between 35 % and 100 %
Protection	Stop the fan by hand while running; touch the MOSFET	Cool ($< 40\text{ }^{\circ}\text{C}$); no resets

Extensions

- Add a **buzzer alarm** above 45 °C. Add a **minimum/maximum** display. Log temperature to the Serial Plotter.
- Replace the averaging by the **exponential filter** of Chapter 17 and compare the response time.
- Convert it into a **heater controller** using a relay and a low-voltage heater pad: the same hysteresis logic, inverted.

26.2 Project 2: “Simon Says” memory game

Requirements

1. Four coloured LEDs with four buttons; each LED has its own musical tone.
2. The game plays a sequence; the player repeats it. Each round adds one more step. Maximum 32.
3. A wrong press plays a low “game over” sound and restarts the game.
4. The player has 5 s per press. The sequence is random each time.
5. Runs from USB or a 9 V battery.

Design

Four LED + resistor pairs on pins 8 – 11, four buttons on pins 2 – 5 (**INPUT_PULLUP**, other side to GND), a passive buzzer on pin 6. LED current: $(5 - 2)/330 \approx 9$ mA each (green/blue LEDs use 220 Ω). Only one LED is lit at a time, so the pin budget is safe. The design is a **state machine** (show sequence → wait for player → check → next level or game over), written here as clearly ordered steps inside `loop()`; the sequence is stored in an array.

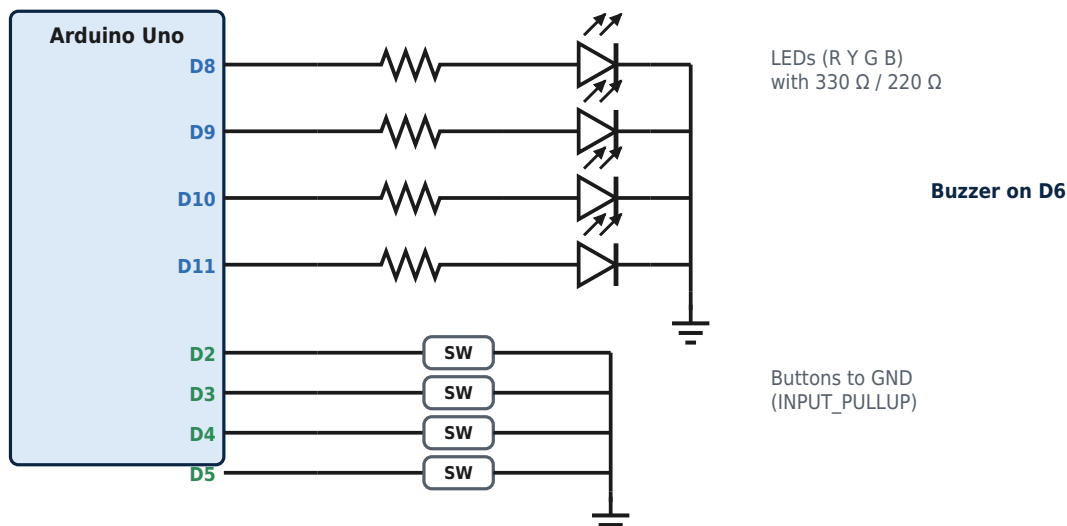


Figure 26.2 Project 2 wiring summary: four LEDs with resistors, four buttons to ground and a passive buzzer on D6 (not drawn).

Project 2: Simon Says

```
const int ledPins[4] = {8, 9, 10, 11};
const int btnPins[4] = {2, 3, 4, 5};
const int BUZZ = 6;
const int tones[4] = {262, 330, 392, 523}; // C4 E4 G4 C5
const int MAX_LEVEL = 32;

byte sequence[MAX_LEVEL];
int level = 0;

void playStep(int i, int ms) { // light LED i and play its tone
    digitalWrite(ledPins[i], HIGH);
```

```

    tone(BUZZ, tones[i], ms);
    delay(ms);
    digitalWrite(ledPins[i], LOW);
    delay(60);
}

int waitForPress() {                                // returns 0..3, or -1 after 5 s with no press
    unsigned long start = millis();
    while (millis() - start < 5000) {
        for (int i = 0; i < 4; i++) {
            if (digitalRead(btnPins[i]) == LOW) {
                digitalWrite(ledPins[i], HIGH);
                tone(BUZZ, tones[i]);
                while (digitalRead(btnPins[i]) == LOW) {} // wait for release
                noTone(BUZZ);
                digitalWrite(ledPins[i], LOW);
                delay(30); // debounce the release
                return i;
            }
        }
    }
    return -1;
}

void gameOver() {
    for (int i = 0; i < 4; i++) digitalWrite(ledPins[i], HIGH);
    tone(BUZZ, 110, 700); // low growl
    delay(900);
    for (int i = 0; i < 4; i++) digitalWrite(ledPins[i], LOW);
    level = 0;
    delay(800);
}

void setup() {
    for (int i = 0; i < 4; i++) {
        pinMode(ledPins[i], OUTPUT);
        pinMode(btnPins[i], INPUT_PULLUP);
    }
    randomSeed(analogRead(A0)); // unconnected pin: random seed
    for (int i = 0; i < 4; i++) playStep(i, 150); // start-up jingle
}

void loop() {
    if (level == MAX_LEVEL) { // the player won!
        for (int r = 0; r < 3; r++) for (int i = 0; i < 4; i++) playStep(i, 80);
        level = 0;
        return;
    }
    sequence[level++] = random(4); // add one random step
    delay(700);
    for (int i = 0; i < level; i++) playStep(sequence[i], 350); // play the sequence

    for (int i = 0; i < level; i++) { // player repeats it
        int p = waitForPress();
        if (p != sequence[i]) { // wrong (or timeout)
            gameOver(); // loop() restarts with level = 0
            return;
        }
    }
    delay(600); // success: next round
}

```

- **Why `randomSeed(analogRead(A0))`?** Without it `random()` gives the same sequence after every reset. A floating analog pin reads noise, which differs each time.
- **Why wait for release?** Otherwise one long press would be counted as several.
- **Speed-up:** reduce the step duration as the level increases: `playStep(sequence[i], max(120, 350 - level * 10))`.
- **Extension:** display the level on a 7-segment or LCD, add a high-score stored in EEPROM, or make the game two-player.

26.3 Project 3: Security alarm with entry delay and event log

Requirements

1. A button arms and disarms the system. When arming there is a 10 s **exit delay** (red LED blinks) so the user can leave.
2. When armed, a PIR sensor detects motion. On motion, a 10 s **entry delay** starts (fast beeps) during which the user can disarm by pressing the button.
3. If not disarmed in time, the **alarm** sounds (siren) for up to 30 s or until the button is pressed.
4. The number of alarms ever triggered is stored permanently (**EEPROM**) and reported on the Serial Monitor.
5. The green LED is on when disarmed; the red LED on when armed.
6. Non-blocking code: the button must always respond.

State machine

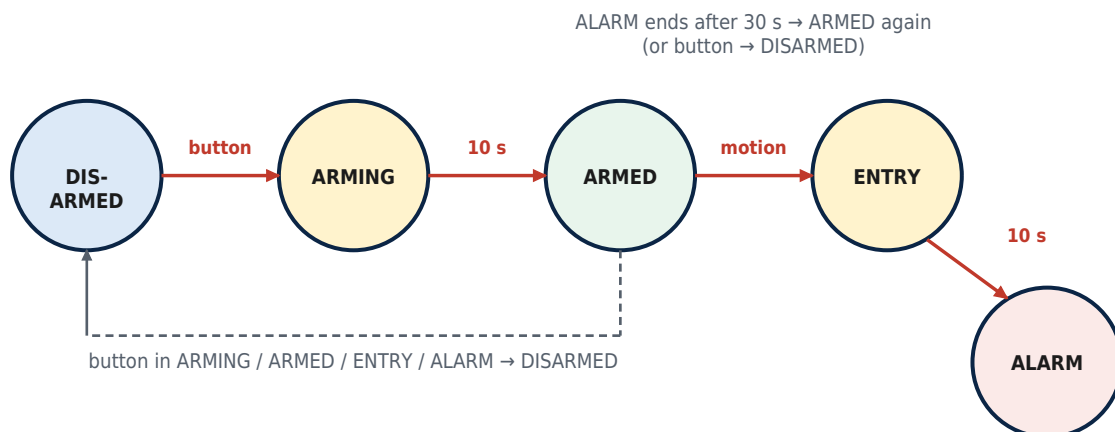


Figure 26.3 Project 3 state diagram. The “button” event from any active state returns the system to DISARMED.

Project 3: alarm system

```
#include <EEPROM.h>

const int PIR_PIN = 2, BTN_PIN = 3, GREEN_LED = 4, RED_LED = 5, BUZZ = 6;
const unsigned long EXIT_DELAY = 10000UL, ENTRY_DELAY = 10000UL, ALARM_TIME = 30000UL;

enum State { DISARMED, ARMING, ARMED, ENTRY, ALARM };
const char* names[] = {"DISARMED", "ARMING", "ARMED", "ENTRY", "ALARM"};
State state = DISARMED;
unsigned long stateStart = 0, lastBeep = 0;
unsigned int alarmCount = 0;

// --- debounced button: true once per press ---
bool lastRaw = HIGH, stable = HIGH;
unsigned long lastChange = 0;
bool buttonPressed() {
    bool raw = digitalRead(BTN_PIN);
    if (raw != lastRaw) { lastChange = millis(); lastRaw = raw; }
    if (millis() - lastChange > 30 && raw != stable) {
        stable = raw;
        if (stable == LOW) return true;
    }
    return false;
}

void setState(State s) {
    state = s;
```

```

    stateStart = millis();
    Serial.print("State: "); Serial.println(names[s]);
}

void setup() {
    pinMode(PIR_PIN, INPUT);
    pinMode(BTN_PIN, INPUT_PULLUP);
    pinMode(GREEN_LED, OUTPUT); pinMode(RED_LED, OUTPUT);
    Serial.begin(9600);
    EEPROM.get(0, alarmCount); // read the stored count
    if (alarmCount == 0xFFFF) alarmCount = 0; // blank EEPROM reads 0xFFFF
    Serial.print("Alarms recorded so far: "); Serial.println(alarmCount);
    setState(DISARMED);
}

void loop() {
    unsigned long now = millis();
    unsigned long elapsed = now - stateStart;
    bool pressed = buttonPressed();
    bool motion = digitalRead(PIR_PIN) == HIGH;

    switch (state) {
        case DISARMED:
            digitalWrite(GREEN_LED, HIGH); digitalWrite(RED_LED, LOW);
            if (pressed) setState(ARMING);
            break;

        case ARMING: // exit delay: red LED blinks
            digitalWrite(GREEN_LED, LOW);
            digitalWrite(RED_LED, (now / 250) % 2);
            if (pressed) setState(DISARMED);
            else if (elapsed >= EXIT_DELAY) setState(ARMED);
            break;

        case ARMED:
            digitalWrite(RED_LED, HIGH);
            if (pressed) setState(DISARMED);
            else if (motion) setState(ENTRY);
            break;

        case ENTRY: // fast beeps: disarm now!
            digitalWrite(RED_LED, (now / 100) % 2);
            if (now - lastBeep >= 500) { lastBeep = now; tone(BUZZ, 1800, 100); }
            if (pressed) setState(DISARMED);
            else if (elapsed >= ENTRY_DELAY) {
                alarmCount++;
                EEPROM.put(0, alarmCount); // store permanently
                Serial.print("ALARM #"); Serial.println(alarmCount);
                setState(ALARM);
            }
            break;

        case ALARM: // rising/falling siren
            digitalWrite(RED_LED, (now / 150) % 2);
            tone(BUZZ, 700 + (now / 4) % 700);
            if (pressed) { noTone(BUZZ); setState(DISARMED); }
            else if (elapsed >= ALARM_TIME) { noTone(BUZZ); setState(ARMED); }
            break;
    }

    if (state == DISARMED) noTone(BUZZ);
}

```

Requirement	Test
Exit delay 10 s, red LED blinks	Press the button; time 10 s; check the state log on Serial
Motion detection when armed	After ARMED, wave a hand in front of the PIR; state → ENTRY within ~1 s
Entry delay and disarm	Press the button within 10 s: state → DISARMED, no alarm
Alarm and log	Let the entry delay expire: siren, count increases; unplug and re-plug USB: count persists

Requirement	Test
Responsiveness	Press the button at random moments in all states; response is immediate (no delay())

- **PIR note:** after boot, wait 30–60 s before arming the first time; the PIR stabilises.
- **EEPROM wear:** each cell survives ~100,000 writes; we write once per alarm, so it lasts forever. Never write EEPROM in a fast loop.
- **Extensions:** add a keypad for a PIN code; a reed switch for the door; an SMS or Bluetooth notification; an RTC to timestamp each alarm; a battery with sleep mode.

26.4 Your turn: project ideas

The best way to consolidate what you have learned is to design something of your own. Pick a project that excites you, write requirements, and follow the process. Here are ideas ordered by difficulty, with the main skills each one exercises:

Level	Project	Key skills
★	Digital dice with a seven-segment display or LEDs	Buttons, random(), arrays, display
★	Light-following servo (two LDRs)	Analog inputs, servo, comparison logic
★	Clap switch	Sound sensor, timing, relay/LED
★★	Digital thermometer with min/max memory and alarm	Sensors, LCD, EEPROM, state machine
★★	Automatic plant watering with a pump and moisture sensor	Sensors, relay/MOSFET, fail-safe timing
★★	Line-following robot (IR sensors + L298N)	Sensors, PWM, motors, control logic
★★	Reaction-time tester with LCD and high-score	millis(), interrupts, EEPROM
★★	Weather station (BME280 + RTC + SD card)	I2C, SPI, time stamps, data logging
★★★	Self-balancing robot (MPU6050 + stepper/DC + PID)	I2C, control theory, timing
★★★	Wireless sensor node (NRF24 or ESP-NOW) with sleep	SPI, radio, low-power design
★★★	MIDI instrument or drum pad (piezo sensors)	Analog peaks, timing, serial protocol
★★★	Bluetooth-controlled car/lamp with a phone app	UART, protocol design, drivers

26.5 The “Can I design by myself?” checklist

Tick each item honestly. If you cannot, go back to the chapter indicated and repeat its experiment.

I can...	Chapter
☐ Explain voltage, current and resistance and use Ohm’s law and $P = VI$ to calculate any simple circuit	1, 2
☐ Use a breadboard and a multimeter to build and measure a circuit from a schematic	3
☐ Calculate series/parallel networks and design a voltage divider (and know its limits)	4, 5
☐ Choose resistor values (LED, pull-up, dividers) and check their power ratings	4, 8, 10
☐ Use capacitors for decoupling, smoothing and time constants	6
☐ Protect circuits against inductive kick (flyback diodes) and reverse polarity	7, 8
☐ Design a transistor/MOSFET switch to drive a relay, motor or LED strip from a pin	9
☐ Give the Arduino a proper power source and budget the current	13, 14
☐ Write C++ with variables, conditions, loops, functions and arrays	15
☐ Read buttons (debounced, pull-up) and structure non-blocking code with millis() and state machines	16

I can...	Chapter
☐ Read and calibrate analog sensors and filter noise	17
☐ Produce PWM, drive servos, steppers and motors through drivers	18, 21
☐ Use common sensors and displays via libraries (I2C, SPI, UART)	19, 20, 22
☐ Use interrupts and timers where needed	23
☐ Take an idea through requirements → block diagram → parts + calculations → schematic → staged build → test	24
☐ Debug systematically with the multimeter, Serial prints and isolation	25

CHAPTER SUMMARY

- You designed, calculated, built and programmed three complete devices: a thermostat fan, a memory game and an alarm.
- Each project used the same pattern: testable requirements → block diagram → part selection with calculations → wiring → tidy non-blocking code → test plan → extensions.
- Common building blocks recur: hysteresis, averaging, debouncing, flyback diodes, MOSFET drivers, millis() timers, state machines, EEPROM for permanent data.
- To keep improving: choose a new project every few weeks, increase the difficulty gradually and always finish by testing against your requirements.

ASSIGNMENTS | CHAPTER 26

26.1 [Design] Modify Project 1 so that the fan speed uses a proportional band of 3 °C instead of 5 °C and the minimum duty is 50 %. Which constants change? What is the duty at an error of 1.5 °C?

26.2 [Calculate] In Project 1, how much power does the MOSFET dissipate at full PWM duty if $R_{DS(on)} = 0.035 \Omega$ and $I = 0.25 \text{ A}$? How much would the 220Ω gate resistor dissipate on average?

26.3 [Practice] Explain in your own words what hysteresis does in Project 1 and what would happen without it.

26.4 [Calculate] Project 2: if the game continues to level 10 and each step lasts 350 ms + 60 ms pause, how long does playing the sequence at level 10 take?

26.5 [Design] Change Project 2 so the LED durations shorten as the level increases: from 350 ms at level 1 down to a minimum of 120 ms. Write the expression.

26.6 [Practice] Project 3: what happens if the user presses the button during ALARM? And if nobody does for 30 s? Trace the state changes and the LEDs/buzzer.

26.7 [Design] Add an “arming confirmation beep” to Project 3 when entering ARMED, and a double beep when DISARMED. Where in the code do you put it?

26.8 [Design] Choose one project idea from the table in 26.4. Write five testable requirements, a block diagram, a pin table and a parts list with calculations for every interface.

26.9 [Hands-on] Build Project 1, 2 or 3 (your choice) and complete the test plan. Record any deviations between the expected and measured behaviour and what you changed.

26.10 [Hands-on] Build your own design from the previous assignment. Document it with a schematic, photos, code and test results.

Soldering, PCBs and where to go next

A breadboard is perfect for experiments, but it is not a place for a finished project: connections come loose, wires tangle and the circuit cannot be moved. Making a permanent version is what turns an experiment into a device you can use, give away or sell. In this closing chapter you learn the practical craft of making circuits permanent, then look ahead at where your new skills can take you.

WHAT YOU WILL LEARN IN THIS CHAPTER

- How to **solder** well: tools, technique, and how to recognise a good or bad joint
- How to move from breadboard to **perfboard** and a **printed circuit board (PCB)**
- Basic PCB design rules and how to order boards
- Enclosures, connectors, cables, and making a project robust and safe
- A roadmap for continued learning: topics, tools, communities and parts

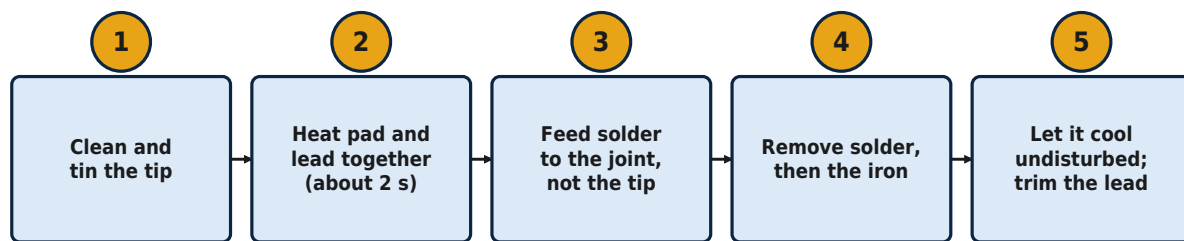
27.1 Soldering

Soldering joins metal parts by melting a low-melting-point alloy (**solder**) that flows between them and solidifies into a strong electrical and mechanical bond. Solder is a mix of tin and lead (60/40, melts at ~190 °C) or lead-free (tin-silver-copper; melts at ~220 °C); electronics solder contains a core of **flux**, a cleaning agent that removes oxide so the solder wets the metal.

Tools and materials

Item	Recommendation
Soldering iron	Temperature-controlled, 40–60 W, with a fine conical or chisel tip (≈ 1.5 mm). Set to about 330–350 °C for leaded and 350–370 °C for lead-free solder.
Solder wire	0.6–0.8 mm, rosin-core, 60/40 or SAC305 (lead-free). Use leaded only with good ventilation and washing hands afterwards.
Stand + brass-wool tip cleaner	Never rest a hot iron on the bench. Brass wool cleans without cooling the tip as a wet sponge does.
Flux pen/paste, desoldering wick, solder sucker	Flux helps difficult joints; wick/sucker removes solder to fix mistakes.
Helping hands / PCB holder, side cutters, tweezers	Hold the work still; trim leads after soldering.
Fume extractor or ventilation, safety glasses	Flux fumes irritate lungs; hot solder can spit.
Multimeter	Continuity check after soldering, to catch bridges and cold joints.

The technique: a good joint in five steps



Total time per joint: about 2 - 4 seconds

Figure 27.1 The five steps of soldering a through-hole joint.

ANALOGY | HEAT THE WORK, NOT THE SOLDER

Soldering is like gluing with a **flowing metal glue**. Heat the parts, not the solder: the heated parts draw the molten solder into the gap by capillary action (as a paper towel draws water). If you melt solder on the tip and “paint” it on cold metal it will sit in a blob without bonding.

Joint	Appearance	Cause	Fix
Good	Shiny, smooth, concave fillet like a small volcano; solder flows onto both pad and lead	—	—
Cold joint	Dull, grainy, ball-shaped; may be loose electrically	Not enough heat; parts moved while cooling	Reheat with a touch of fresh solder/flux
Too much solder	Large blob, covers the lead	Too much wire	Remove with wick
Bridge	Solder connecting two adjacent pads	Too much solder; dirty tip	Wick it away; clean the tip
Insufficient / dry	Pad not fully covered	Not enough solder or heat	Add solder with a hot iron

Order of assembly and safety

- **Lowest-profile parts first** (resistors, diodes, then sockets and ICs, capacitors, transistors, connectors last). Flip the board over on a flat surface so parts stay in place, solder, then trim leads.
- **Use IC sockets** for chips you might replace; never solder an ATmega directly unless sure.
- **Mind polarity** (diodes, electrolytic capacitors, LEDs, ICs) before soldering: reversing a soldered part is tedious.
- **Heat-sensitive parts** (LEDs, sockets, plastic connectors, transistors) should be soldered quickly; clip a crocodile clip to the lead as a heat sink if needed.
- **Always test with the multimeter** (continuity, no short between 5 V and GND) before first power-up.
- **Safety:** the tip is hot (350 °C); keep it in its stand; ventilate; tie back hair; keep solder away from children and pets; wash hands after handling solder; avoid breathing the smoke.

27.2 From breadboard to permanent circuit

Method	How	Pros	Cons	Good for
Breadboard	Push-in	Instant; reusable	Loose contacts; bulky	Experiments
Stripboard / veroboard	Copper strips; cut with a drill bit	Cheap; compact	Planning needed; strips limit layouts	Simple circuits
Perfboard	Isolated pads; wire links	Flexible; cheap; one-offs	Wiring is laborious	Prototypes on Arduino shields
Arduino prototyping shield	Perfboard that plugs onto an Uno	Mechanically solid	Limited space	Finished Arduino projects
Custom PCB	Designed in EDA software; manufactured	Neat; reliable; repeatable	Learning curve; 1-2 weeks	Final devices; many copies

A good path: (1) breadboard prototype and test; (2) draw the final schematic; (3) transfer to perfboard or a shield and test again; (4) when you need several copies or a neat result, design a PCB.

27.3 Designing a printed circuit board

A **PCB** is a sheet of fibreglass (FR-4) with copper tracks etched on it that replace wires. Tools like **KiCad** (free, professional), **EasyEDA** (web-based, linked to manufacturers), and **Fritzing** (beginner-friendly) take you from schematic to manufacturing files.

The workflow

1. **Draw the schematic** in the EDA tool. Assign each symbol a **footprint** (the physical pad pattern: through-hole 0.1", 0805 SMD resistor, SOIC-8, etc.).
2. **Run the Electrical Rules Check (ERC)**: catches unconnected pins and conflicting outputs.
3. **Define the board outline** (size, mounting holes) and **place components**: group by function, put connectors at the edges, decoupling capacitors next to the IC pins, keep sensitive analog parts away from switching parts.
4. **Route the tracks**: short and direct. Use wide tracks for power (0.5–1 mm or more for amps), narrower (0.25 mm) for signals. Use a **ground plane** (a copper pour connected to GND) on one layer.
5. **Run the Design Rules Check (DRC)**: minimum track widths, clearances, drill sizes for your manufacturer.
6. **Add silkscreen labels** (reference designators, polarity marks, pin names, version).
7. **Export Gerber files** and a drill file; upload to a manufacturer (JLCPCB, PCBWay, OSH Park: a few dollars for five boards, shipped in 1–2 weeks).
8. **Solder, test in stages** (power rail first), and note problems for revision B.

Basic layout rules

Rule	Why
Track width : 0.25 mm for signals; for current use roughly 0.3 mm per ampere of 1 oz copper as an absolute minimum (wider is better: 1 mm carries $\approx 2\text{--}3\text{ A}$ with modest temperature rise)	Narrow tracks have more resistance and heat up ($P = I^2R$)
Clearance : $\geq 0.2\text{ mm}$ (8 mil) between tracks; larger for higher voltage ($\geq 1\text{ mm}$ for mains: and then you must understand creepage/clearance rules)	Prevents shorts and arcing
Decoupling capacitor 100 nF directly at each IC's power pin	Chapter 6: supplies fast current locally

Rule	Why
Avoid sharp 90° bends; use 45° corners	Better manufacturing and signal behaviour
Solid ground plane; make return paths short; keep loops small	Lower noise and EMI
Separate motor/power ground return from sensitive analog grounds; join at one point (star)	Chapter 25 grounding
Keep crystal and fast signals short; no tracks under a crystal	Prevents oscillation problems
Add test points and mounting holes; label connectors	Makes testing and mounting easy

You can even put a bare **ATmega328P** on your own PCB with a 16 MHz crystal, two 22 pF capacitors, a 10 kΩ reset pull-up, 100 nF decoupling and a 6-pin ISP header: a “minimal Arduino” costing a few dollars. It is programmed through an FTDI/USB-serial adapter after burning the Arduino bootloader with another Arduino (as ISP).

27.4 Connectors, wiring and enclosures

- **Connectors:** use polarised connectors (JST-XH, screw terminals, DC barrel jack) so you cannot plug them in backwards. Use **screw terminals** for motors and power; **Dupont headers** for sensors; **JST-PH** for batteries.
- **Wire:** stranded wire for anything that moves; solid for fixed breadboard links. Choose gauge by current: 22 AWG (0.33 mm²) ≈ 3 A max; 18 AWG (0.82 mm²) ≈ 10 A. Use colours consistently (red +, black GND). Add **strain relief** (cable ties, hot glue) near connectors. Use **heat-shrink** tubing on soldered splices.
- **Fusing:** put a fuse or resettable polyfuse on every battery or power input.
- **Enclosures:** project boxes (ABS), 3D-printed cases or laser-cut acrylic. Leave ventilation for regulators; mount the board on standoffs; provide access to the USB port; label inputs and outputs; use cable glands.
- **Documentation:** keep the schematic, code, photos and a bill of materials in a folder (or a Git repository). Revisit it a year from now and you will be glad you did.

27.5 Where to go next

You now have the foundations of electronics and the ability to design Arduino-based circuits. The field is huge; here are directions to explore, each building on what you know.

Direction	What you will learn	Starter step
More analog electronics	Op-amp circuits, filters, amplifiers, power supply design, oscilloscope use	Build an active filter and a small audio amplifier; buy a basic oscilloscope
Digital design	Logic ICs, FPGAs, state machines in hardware	Build a counter with 74HC chips; try an FPGA board (iCEBreaker, Tang Nano)
Better microcontrollers	ESP32 (WiFi/Bluetooth), Raspberry Pi Pico (RP2040), STM32; direct register programming; RTOS	Port a project to an ESP32 and send data to the internet
IoT and wireless	MQTT, HTTP, LoRa, BLE, ESP-NOW; cloud dashboards	Weather station posting to a web dashboard
Embedded C/C++	Pointers, structs, classes, interrupts, memory, debuggers, unit testing	Write your own library for a sensor
Control systems	PID control, sensor fusion, Kalman filters	Self-balancing robot; temperature controller with PID
Robotics	Kinematics, mapping, ROS, computer vision	Line follower → obstacle avoider → mapping robot

Direction	What you will learn	Starter step
Power electronics	Buck/boost design, motor control (BLDC, FOC), battery management	Design a buck converter with an off-the-shelf controller IC
PCB and product design	Multilayer boards, SMD soldering, EMC, certification	Design and order your first PCB with KiCad
Signal processing	ADC sampling, FFT, digital filters, audio	Spectrum analyser with an FFT library
Computer vision / AI at the edge	TinyML, ESP32-CAM, Raspberry Pi	Gesture or keyword detection with TensorFlow Lite Micro

Resources

- **Arduino Reference and tutorials:** arduino.cc (Language Reference, Built-in Examples, Libraries).
- **Datasheets:** the manufacturer's site (Microchip, TI, ST); search "[part number] datasheet".
- **Simulators:** Tinkercad Circuits, Wokwi, Falstad circuit simulator, LTspice for analog.
- **Learning:** Adafruit Learning System, SparkFun tutorials, EEVblog and GreatScott! on video, "The Art of Electronics" (Horowitz & Hill) as the long-term reference, "Make: Electronics" (Platt) as a companion.
- **Communities:** Arduino Forum, Electronics Stack Exchange, r/arduino and r/electronics, local maker spaces and hackathons.
- **Parts:** reputable suppliers (Digi-Key, Mouser, Farnell/element14, RS, and local distributors); AliExpress for cheap modules (test them!).

GOOD TO KNOW

Learning electronics is a long game. Keep a notebook, build something every month, read other people's schematics and code, and teach someone else what you have learned. Teaching is the fastest way to find out what you really understand.

CHAPTER SUMMARY

- Good solder joints: clean, heat both parts, add solder to the joint, remove solder then iron, let cool. Check with continuity.
- Permanent builds: perfboard/shield for one-offs; custom PCB for neat, repeatable results (schematic → footprints → layout → DRC → Gerbers).
- PCB basics: short tracks, adequate width for current, decoupling at every IC, ground plane, star grounding, clearances.
- Use polarised connectors, proper wire gauge, fuses, strain relief and an enclosure; document your project.
- Keep learning: analog, digital design, other MCUs (ESP32, RP2040), IoT, control, robotics, PCB design.

ASSIGNMENTS | CHAPTER 27

27.1 [Practice] List the five steps of making a good through-hole solder joint, in order.

27.2 [Practice] You see a dull, lumpy solder joint on a resistor lead. What is it called, what probably caused it, and how do you fix it?

27.3 [Calculate] A PCB track must carry 2 A. Using the rule of thumb of 0.3 mm per ampere as an absolute minimum and 1 mm as a safe choice for 2–3 A, what track width would you choose? Estimate the voltage drop over a 100 mm track of 1 mm width (1 oz copper ≈ 0.5 mΩ per square; resistance = squares × 0.5 mΩ).

27.4 [Calculate] You solder 40 joints at 3 s each, and also spend 10 minutes preparing. How long is that in total?

ASSIGNMENTS | CHAPTER 27

27.5 [Practice] Which is better for 5 identical copies of a finished project: perfboard or a custom PCB? Give two reasons.

27.6 [Think] Why should you place a decoupling capacitor as close as possible to each IC's power pins on a PCB?

27.7 [Think] Why do professionals use IC sockets for expensive or experimental chips but not for finished mass-produced boards?

27.8 [Design] Create a plan to turn your best Arduino project into a permanent device: list the steps (including the wiring changes, soldering, enclosure and testing) and the parts you need.

27.9 [Hands-on] Practise soldering: on a piece of perfboard solder 20 resistors, an LED and a header and check each joint with continuity. Then desolder two joints with wick and re-solder them.

APPENDICES

Reference

Answers to every assignment, a formula sheet, standard value tables, an Arduino quick reference and a glossary. Keep these pages handy at your workbench.

IN THIS PART

- A Answers and hints to all assignments
- B Formula cheat sheet
- C Standard values, codes and tables
- D Arduino quick reference
- E Component pinouts
- F Glossary

Answers and hints to all assignments

Try every assignment before you look. Numerical answers are rounded; small differences from your result usually come from rounding, standard resistor values or the exact constants you used. “Design” and “Hands-on” answers describe what a good result looks like rather than a single right answer.

Chapter 1

1.1 A wire carries a constant current of 0.5 A for 2 minutes. How much charge passes a point in...

$$Q = I \times t = 0.5 \text{ A} \times 120 \text{ s} = \mathbf{60 \text{ C}}.$$

1.2 A 1.5 V cell moves 200 C of charge through a circuit. How much energy does it deliver? (Use...

$$E = 1.5 \text{ V} \times 200 \text{ C} = \mathbf{300 \text{ J}}.$$

1.3 How many electrons pass a point each second when the current is 1 mA? (One coulomb $\approx 6.24 \times \dots$

$$1 \text{ mA} = 0.001 \text{ C/s, so } 0.001 \times 6.24 \times 10^{18} = \mathbf{6.24 \times 10^{15} \text{ electrons per second}}.$$

1.4 Find the resistance of 2 m of copper wire with cross-sectional area 1 mm² ($1 \times 10^{-6} \text{ m}^2$)....

$$R = \rho L/A = (1.68 \times 10^{-8} \times 2) / (1 \times 10^{-6}) = \mathbf{0.0336 \Omega}.$$

1.5 If the wire in the previous question were replaced with aluminium ($\rho = 2.82 \times 10^{-8} \Omega \cdot \text{m}$)...

$$R = (2.82 \times 10^{-8} \times 2) / (1 \times 10^{-6}) = \mathbf{0.0564 \Omega}, \text{ about } \mathbf{1.68 \text{ times}} \text{ bigger than copper}.$$

1.6 Mains in India has $f = 50 \text{ Hz}$. What is the period of one cycle? How long do 10 cycles take?

$$T = 1/50 = 0.02 \text{ s} = \mathbf{20 \text{ ms}}. \text{ Ten cycles take } \mathbf{0.2 \text{ s}}.$$

1.7 A friend says: “The battery uses up the electrons, so eventually it runs out of electrons.”...

Electrons are not consumed: they flow around the loop and return to the battery. The battery’s chemical energy is what is used up. When the chemicals are exhausted the battery can no longer supply a push (voltage), even though it still contains just as many electrons as before.

1.8 Why does a lamp light almost instantly when you flip a switch even though electrons drift...

The wire is already full of free electrons. The electric push (the field) travels through the wire at close to the speed of light, and every electron along the whole circuit starts moving together, like water pushed into a pipe that is already full.

1.9 In the water analogy, what corresponds to (a) a battery, (b) a switch, (c) an open circuit,...

(a) a pump; (b) a tap/valve; (c) a blocked (closed) pipe: no flow; (d) a big pipe bypassing everything: a huge, uncontrolled flow.

1.10 Use a multimeter’s resistance mode to measure five household objects (e.g. a coin, a pencil...

Sample results: coin $\approx 0 \Omega$ (conductor); pencil graphite a few Ω to hundreds of Ω depending on length (poor conductor); plastic/paper: out of range (insulator); skin: 100 k Ω – 1 M Ω dry, falls sharply when wet.

Chapter 2

2.1 Convert: (a) 4.7 k Ω to Ω , (b) 330 mA to A, (c) 2200 μF to mF, (d) 47 nF to μF , (e) 1.2 M Ω ...

(a) 4700 Ω ; (b) 0.33 A; (c) 2.2 mF; (d) 0.047 μF ; (e) 1200 k Ω ; (f) 0.015 V.

2.2 A 470 Ω resistor is connected across a 12 V supply. What current flows?

$$I = 12 / 470 = 0.02553 \text{ A} \approx \mathbf{25.5 \text{ mA}}.$$

2.3 A current of 3 mA flows through a 2.2 k Ω resistor. What is the voltage across it?

$$V = 3 \text{ mA} \times 2.2 \text{ k}\Omega = \mathbf{6.6 \text{ V}}.$$

2.4 A resistor has 9 V across it and carries 30 mA. What is its resistance?

$$R = 9 / 0.030 = \mathbf{300 \Omega}.$$

2.5 A 150 Ω resistor carries 20 mA. What power does it dissipate? Is a $\frac{1}{4} \text{ W}$ resistor adequate?

$$P = I^2 R = 0.02^2 \times 150 = 0.06 \text{ W} = \mathbf{60 \text{ mW}}. \text{ Yes, well below } 250 \text{ mW}.$$

2.6 A 100 Ω resistor is placed across a 9 V battery. What power is dissipated, and what is the...

$P = 81/100 = 0.81$ W. Choose at least **1 W**, preferably 2 W for a safety margin.

2.7 A 1500 W heater runs from 230 V. Find (a) its current, (b) its resistance.

(a) $I = P/V = 1500/230 = \mathbf{6.52}$ A. (b) $R = V^2/P = 230^2/1500 = \mathbf{35.3}$ Ω .

2.8 A 3.7 V, 2500 mAh lithium cell powers a circuit that draws 125 mA. Estimate the runtime,...

Runtime $\approx 2500 / 125 = \mathbf{20}$ h (ideal). Energy = $3.7 \times 2.5 = \mathbf{9.25}$ Wh.

2.9 A 60 W bulb is on for 5 hours daily for 30 days. How many kWh does it use? How much would...

$60 \text{ W} \times 5 \text{ h} \times 30 = 9000 \text{ Wh} = \mathbf{9}$ kWh. LED: $9 \times 5 \times 30 = 1350 \text{ Wh} = 1.35$ kWh. Saving = **7.65 kWh**.

2.10 You measure an unknown component: (1 V, 4.7 mA), (2 V, 9.5 mA), (4 V, 18.9 mA), (8 V, 38...

$V/I = 213, 211, 212, 211$ Ω . The ratio is constant, so it is **ohmic**, $R \approx 211$ Ω ; probably a **220 Ω** resistor with a tolerance of 5 % or better.

2.11 If you double the voltage across a fixed resistor, what happens to (a) the current, (b) the...

(a) The current doubles ($I = V/R$). (b) The power **quadruples**, because $P = V^2/R$ and V is squared.

2.12 Why does a phone charger cable get warm when charging at 3 A, but stays cool at 0.3 A? Use...

The cable has a small fixed resistance R. Heat = I^2R . Ten times the current gives a hundred times the heat.

2.13 You have a 5 V supply and want exactly 10 mA to flow through a resistor. Calculate the...

$R = 5/0.010 = 500$ Ω . Nearest E12 value is **470 Ω** (gives 10.6 mA) or 560 Ω (8.9 mA). Power = $0.01^2 \times 500 = 50$ mW, so a $\frac{1}{4}$ W part is fine.

Chapter 3

3.1 On a breadboard, which of these holes are connected: (a) A5 and E5, (b) A5 and F5, (c) A5...

(a) Yes, same column-half. (b) No, the centre gap separates them. (c) No, adjacent columns are different strips. (d) Yes. (e) Yes (unless the rail is split in the middle of the board).

3.2 You want to measure the current through an LED circuit powered by a 5 V supply. Describe...

Break the circuit at one point (for example between the supply + and the resistor) and insert the meter (in mA mode, red lead in the mA jack) so all current flows through it: series connection. Across the supply, the meter acts as a short circuit and blows its fuse.

3.3 Your meter displays "OL" when measuring the resistance of a component. Give two possible...

The component is open (broken, or not touching the probes), or its resistance is higher than the selected range.

3.4 Explain the difference between measuring resistance in a powered and an unpowered circuit,...

The meter's Ω mode supplies its own tiny test current. If the circuit is powered, the circuit's own voltage and current corrupt the reading and can even damage the meter. The circuit must be unpowered (and preferably the part isolated).

3.5 Draw the schematic of a 9 V battery in series with a 1 k Ω resistor and a switch, with a...

Your diagram should contain: B1 (9 V), SW1, R1 (1 k Ω), an ammeter symbol in series in the loop, and a voltmeter symbol drawn in parallel across R1. Compare with Figure 3.3.

3.6 In Experiment 1 you measured 8.9 V across a resistor and 8.6 mA through it. What is the...

$R = 8.9 / 0.0086 = \mathbf{1035}$ Ω . Error = $(1035 - 1000)/1000 = \mathbf{+3.5}$ % (within the common 5 % tolerance).

3.7 Carry out Experiment 1 for four resistor values and fill in a table with columns R...

You should find V/I equal to the measured R within a few percent for all four resistors, confirming Ohm's law. Small differences come from meter accuracy and battery internal resistance.

3.8 Use the continuity mode to trace the connections on your own breadboard: confirm that...

You should find that A-E of a column beep together, F-J beep together, but A-F do not. Rails beep along their length unless the board has a break at the middle.

3.9 Why is it a bad idea to build a big circuit all at once and then power it up? Give at least...

A single wiring mistake can damage parts or the supply; if it fails you have no idea where the fault is; a stage-by-stage build isolates faults to the last stage added.

Chapter 4

4.1 Decode these 4-band resistors: (a) red-red-brown-gold, (b) orange-orange-yellow-gold, (c)...

(a) 220 $\Omega \pm 5$ %; (b) 330 k $\Omega \pm 5$ %; (c) 10 k $\Omega \pm 5$ %; (d) 5.6 k $\Omega \pm 5$ %; (e) 680 $\Omega \pm 5$ %.

4.2 Decode the 5-band resistor yellow-violet-black-red-brown.

Digits 4, 7, 0; multiplier red = $\times 100$; tolerance brown = ± 1 %. Value = $470 \times 100 = \mathbf{47}$ k $\Omega \pm 1$ %.

4.3 Give the 4-band colours (with a gold tolerance band) for: 4.7 k Ω , 100 k Ω , 1 M Ω , 330 Ω , 2.2 k Ω .

4.7 k Ω : yellow-violet-red-gold. 100 k Ω : brown-black-yellow-gold. 1 M Ω : brown-black-green-gold. 330 Ω : orange-orange-brown-gold. 2.2 k Ω : red-red-red-gold.

4.4 What do these SMD codes mean: 103, 471, 4R7, 1002?

103 = 10 k Ω ; 471 = 470 Ω ; 4R7 = 4.7 Ω ; 1002 = 10.0 k Ω (10 000 Ω).

4.5 A 1 k Ω resistor has a tolerance of 5 %. What is the range of values it might actually have?

950 Ω to 1050 Ω .

4.6 Find the total resistance of 47 Ω , 100 Ω and 220 Ω in series, and the current when...

$R = 367 \Omega$. $I = 5/367 = \mathbf{13.6 \text{ mA}}$.

4.7 Find the voltage across each resistor of the previous question.

$I = 0.01362 \text{ A}$. $V(47) = 0.640 \text{ V}$; $V(100) = 1.362 \text{ V}$; $V(220) = 2.997 \text{ V}$. Sum = 5.0 V.

4.8 Find the equivalent of 330 Ω in parallel with 330 Ω , and of 1 k Ω in parallel with 4.7 k Ω .

$330 \parallel 330 = \mathbf{165 \Omega}$. $1 \text{ k} \parallel 4.7 \text{ k} = (1000 \times 4700)/5700 = \mathbf{824.6 \Omega}$.

4.9 Three 1 k Ω resistors are wired in parallel across a 5 V supply. Find the total resistance,...

$R = 333.3 \Omega$. $I_{\text{total}} = 15 \text{ mA}$; each branch 5 mA.

4.10 A 12 V battery supplies $R1 = 220 \Omega$ in series with ($R2 = 470 \Omega \parallel R3 = 470 \Omega$). Find total...

$R_{23} = 235 \Omega$. Total = 455 Ω . $I = 12/455 = \mathbf{26.4 \text{ mA}}$. Voltage across the pair = $26.4 \text{ mA} \times 235 \Omega = \mathbf{6.20 \text{ V}}$ (and 5.80 V across $R1$).

4.11 You need 150 Ω but only have 100 Ω , 47 Ω , 330 Ω and 300 Ω resistors. Give two different...

(a) $300 \Omega \parallel 300 \Omega = 150 \Omega$ exactly (two 300 Ω). (b) $100 \Omega + 47 \Omega = 147 \Omega$ (2 % low) in series.

4.12 You need a resistor of about 1.5 k Ω and only have 1 k Ω and 470 Ω parts in $\frac{1}{4} \text{ W}$. What...

$1 \text{ k} + 470 \Omega = 1470 \Omega \approx 1.5 \text{ k} \Omega$ (2 % off).

4.13 A string of ten identical bulbs in series stops working when one bulb burns out, but the...

In series the same current must pass through every bulb; a broken bulb opens the only path. In a house the bulbs are in parallel, so each has its own path across the mains and the others are unaffected.

4.14 Carry out Experiment 2 and produce a table of nominal, measured, predicted and...

Measured values should agree with predictions within the component tolerances (typically 1–5 %). Larger differences probably indicate a misread colour code or your fingers touching the leads during resistance measurement.

Chapter 5

5.1 At a node, 8 mA flows in through one wire and 3 mA flows in through another. Two more wires...

$I_{\text{in}} = 11 \text{ mA}$. Out = 5 mA + I . So $I = \mathbf{6 \text{ mA}}$.

5.2 A 9 V battery is connected across three resistors in series: 1 k Ω , 2 k Ω and 6 k Ω . Use KVL...

$I = 9/9000 = \mathbf{1 \text{ mA}}$. Voltages: 1 V, 2 V, 6 V (sum 9 V).

5.3 A divider uses $R1 = 22 \text{ k}\Omega$ (top) and $R2 = 10 \text{ k}\Omega$ (bottom) from 9 V. Find V_{out} .

$9 \times 10/32 = \mathbf{2.81 \text{ V}}$.

5.4 You want to make 3.3 V from a 5 V source with a divider. Choose $R1$ and $R2$ from the E12...

Ratio 0.66. E.g. $R1 = 5.6 \text{ k}\Omega$ (or $4.7 \text{ k} + 1 \text{ k}$), $R2 = 10 \text{ k}\Omega$: $5 \times 10/15.6 = 3.21 \text{ V}$; or $R1 = 1 \text{ k}\Omega$, $R2 = 2 \text{ k}\Omega$: $5 \times 2/3 = \mathbf{3.33 \text{ V}}$ (using standard 1 k and 2.2 k gives 3.44 V). Any close value is acceptable.

5.5 A 10 k Ω potentiometer is wired between 5 V and GND. The wiper is 30 % of the way from the...

No load: $0.3 \times 5 = \mathbf{1.5 \text{ V}}$. With load: bottom part $3 \text{ k}\Omega \parallel 10 \text{ k}\Omega = 2.31 \text{ k}\Omega$; top part 7 k Ω . $V_{\text{out}} = 5 \times 2.31/(7 + 2.31) = \mathbf{1.24 \text{ V}}$.

5.6 A divider of 100 k Ω (top) and 100 k Ω (bottom) from 5 V is connected to a load of 100 k Ω ...

100 k: bottom = $100 \parallel 100 = 50 \text{ k}\Omega$; $V = 5 \times 50/150 = \mathbf{1.67 \text{ V}}$. 1 k Ω pair: bottom = $1 \text{ k} \parallel 100 \text{ k} = 990 \Omega$; $V = 5 \times 0.990/1.990 = \mathbf{2.49 \text{ V}}$. Lower resistances are less affected by the load.

5.7 A battery has an open-circuit voltage of 6.0 V. When it drives a 20 Ω load, the terminal...

$I = 5.4/20 = 0.27 \text{ A}$. Lost voltage = 0.6 V. $R_{\text{int}} = 0.6/0.27 = \mathbf{2.22 \Omega}$.

5.8 60 mA arrives at a parallel pair of 47 Ω and 120 Ω . Find the current in each.

$I(47) = 60 \times 120/167 = \mathbf{43.1 \text{ mA}}$; $I(120) = 60 \times 47/167 = \mathbf{16.9 \text{ mA}}$.

5.9 Find the Thevenin equivalent (V_{th} and R_{th}) of a 12 V source feeding a divider of $R1 = 2 \text{ k}\Omega$...

$V_{\text{th}} = 12 \times 4/6 = \mathbf{8 \text{ V}}$. $R_{\text{th}} = 2 \text{ k} \parallel 4 \text{ k} = \mathbf{1.33 \text{ k}\Omega}$.

5.10 Refer to Figure 5.3. Suppose $V2$ is removed and replaced with a wire (short circuit). Find...

$R1$ in series with ($R2 \parallel R3$): $R2 \parallel R3 = 132 \Omega$, total 232 Ω , $I = 10/232 = 43.1 \text{ mA}$; voltage across $R2 = 43.1 \text{ mA} \times 132 \Omega =$

5.69 V; $I(R_2) = 5.69/220 = \mathbf{25.9\text{ mA}}$.

5.11 Why can't you power a 100 Ω motor from a voltage divider made of two 1 k Ω resistors? What...

The motor (100 Ω) in parallel with the lower 1 k Ω would form roughly 91 Ω , so the output would collapse to a fraction of a volt, and the resistors would burn power. Use a proper supply, a regulator or a transistor-driven battery connection.

5.12 A student measures an almost-flat 9 V battery with a multimeter and gets 8.7 V, but the...

The meter draws negligible current, so it shows the open-circuit voltage. The battery's internal resistance has become large; under the bulb's current the terminal voltage falls to a low value.

5.13 Build the divider $R_1 = R_2 = 10\text{ k}\Omega$ from 5 V (Arduino 5 V pin or a battery pack, 9 V is also...

Expected (5 V): unloaded 2.5 V; 100 k Ω : 2.38 V; 10 k Ω : 1.67 V; 1 k Ω : 0.24 V.

5.14 Wire a 10 k Ω potentiometer as a divider on 5 V. With a multimeter on the wiper, rotate from...

The voltage should sweep smoothly from 0 to 5 V. Linear-taper pots (marked B) give an almost linear relationship; audio pots (marked A) are logarithmic.

Chapter 6

6.1 Convert: (a) 0.1 μF to nF, (b) 2200 nF to μF , (c) 470 pF to nF, (d) 4.7 μF to pF.

(a) 100 nF; (b) 2.2 μF ; (c) 0.47 nF; (d) 4,700,000 pF.

6.2 How much charge does a 470 μF capacitor hold at 12 V? How much energy?

$Q = 470\text{e-}6 \times 12 = 5.64\text{ mC}$. $E = \frac{1}{2} \times 470\text{e-}6 \times 144 = \mathbf{33.8\text{ mJ}}$.

6.3 Find the time constant of $R = 4.7\text{ k}\Omega$ and $C = 220\text{ }\mu\text{F}$. How long until the capacitor is...

$\tau = 4700 \times 220\text{e-}6 = 1.034\text{ s}$. $5\tau \approx \mathbf{5.2\text{ s}}$.

6.4 A 100 μF capacitor charges through 22 k Ω from 5 V. What is its voltage after 2.2 s? ($\tau = 2.2\text{ s}$)

After exactly 1 τ : 63.2 % of 5 V = **3.16 V**.

6.5 Using the same circuit, how long does it take to reach 4 V? (Use $t = -\tau \ln(1 - V/V_s)$.)

$t = -2.2 \times \ln(1 - 0.8) = -2.2 \times \ln 0.2 = 2.2 \times 1.609 = \mathbf{3.54\text{ s}}$.

6.6 Find the equivalent capacitance of 22 μF and 47 μF in (a) parallel and (b) series.

(a) 69 μF ; (b) $22 \times 47/69 = \mathbf{14.99\text{ }\mu\text{F}}$.

6.7 Design an RC low-pass filter with a cut-off of about 100 Hz. Choose $R = 10\text{ k}\Omega$ and find C ...

$C = 1/(2\pi \times 100 \times 10\,000) = 159\text{ nF}$. Nearest standard: **150 nF** ($f_c = 106\text{ Hz}$) or 180 nF (88 Hz).

6.8 What is the reactance of a 100 nF capacitor at (a) 50 Hz, (b) 5 kHz, (c) 1 MHz?

(a) 31.8 k Ω ; (b) 318 Ω ; (c) 1.59 Ω .

6.9 Decode: (a) 104, (b) 473, (c) 221, (d) a small disc capacitor marked "22".

(a) 100 nF; (b) 47 nF; (c) 220 pF; (d) 22 pF.

6.10 A capacitor is marked "470 μF 10 V" and you plan to use it on a 9 V battery line. Is that a...

Marginal: 9 V battery voltage can be up to 9.6 V, leaving almost no safety margin. Choose a 16 V or 25 V rating.

6.11 Why do we place a 100 nF capacitor right next to the supply pins of a chip rather than at...

The wire between connector and chip has inductance and resistance, so it cannot supply the very fast current spikes.

The capacitor next to the pins supplies them locally over a very short path.

6.12 A capacitor is described as "blocking DC and passing AC". Use the reactance formula to explain.

$X_C = 1/(2\pi fC)$. For DC, $f = 0$, so X_C is infinite and no steady current flows. For high f , X_C is small, so AC passes easily.

6.13 Do Experiment 3. Plot the measured voltage against time on graph paper and compare with the...

Your curve should pass through about 63 % at 10 s, 86 % at 20 s, and 95 % at 30 s (with $\tau = 10\text{ s}$). Deviations come from capacitor tolerance ($-20\%/+80\%$) and meter loading.

Chapter 7

7.1 A coil of 22 mH carries 300 mA. How much energy does it store?

$E = \frac{1}{2} \times 0.022 \times 0.09 = \mathbf{0.99\text{ mJ}}$.

7.2 An RL circuit has $L = 50\text{ mH}$ and $R = 25\text{ }\Omega$. Find τ . How long until the current is essentially...

$\tau = 0.05/25 = \mathbf{2\text{ ms}}$. $5\tau = \mathbf{10\text{ ms}}$.

7.3 What is the reactance of a 10 mH inductor at (a) 100 Hz, (b) 10 kHz?

(a) $2\pi \times 100 \times 0.01 = 6.28\text{ }\Omega$; (b) 628 Ω .

7.4 A transformer has 500 primary turns and 25 secondary turns and a 230 V input. Find the...

$V_s = 230 \times 25/500 = \mathbf{11.5\text{ V}}$. Power = 23 W; primary current = $23/230 = \mathbf{0.1\text{ A}}$ (ideal).

7.5 A 12 V relay has a coil resistance of 240 Ω . What current does the coil draw and what power...

$I = 12/240 = 50 \text{ mA}$; $P = 0.6 \text{ W}$. No: 50 mA exceeds the pin limit of 20 mA (and the Arduino's 5 V is too low). Use a transistor and a 12 V supply.

7.6 Two 10 mH inductors are connected in series, then in parallel. What are the equivalent...

Series 20 mH; parallel 5 mH.

7.7 Draw the schematic of a 12 V relay coil, driven by a battery via a switch, with a flyback...

The diode is connected in parallel with the coil with its cathode (bar) toward the + side of the supply; see Figure 7.3.

7.8 A relay module has terminals labelled COM, NO and NC. You want a lamp to be ON only while...

Supply + to COM; lamp between NO and supply -. When the coil energises, COM connects to NO and the lamp lights.

7.9 Why is a diode across a coil connected "backwards" with respect to the supply?

In normal operation the diode must not conduct (otherwise it would short the supply through the diode). It only conducts for the reversed-polarity spike at turn-off, giving the coil current a safe path.

7.10 Why does a transformer not work on DC? What happens if you connect a mains transformer to a...

A transformer works through a changing magnetic field. Steady DC produces a constant field and no induced voltage. With DC the primary only has its (low) winding resistance limiting current, so it can overheat and burn out.

7.11 Explain why relay contacts rated 10 A / 250 V AC may be rated for only 10 A / 30 V DC.

AC current passes through zero 100 or 120 times a second, so an arc across opening contacts extinguishes naturally. DC has no zero crossing, so an arc can sustain itself and damage the contacts, hence the lower DC voltage rating.

7.12 With a relay module powered from 5 V, use a jumper wire to connect IN to GND (or 5 V,...

You should hear a click and see the module LED light; COM-NC beeps at rest and COM-NO beeps when energised. If IN to GND turns it on, the module is active-LOW (most common).

Chapter 8

8.1 A red LED ($V_F = 2.0 \text{ V}$) must run at 12 mA from a 5 V supply. Compute R and choose the...

$R = 3/0.012 = 250 \Omega \rightarrow$ use **270 Ω** . $I = 3/270 = 11.1 \text{ mA}$.

8.2 A green LED ($V_F = 2.2 \text{ V}$) is powered from a 9 V battery at 10 mA. Find R, the standard...

$R = 6.8/0.01 = 680 \Omega$ (an E12 value). Power = $6.8 \times 0.01 = 68 \text{ mW}$.

8.3 A white LED ($V_F = 3.2 \text{ V}$) is driven from the 3.3 V pin of an Arduino at about 2 mA. Find R....

$R = 0.1/0.002 = 50 \Omega$ — but a tiny change of V_F ($\pm 0.1 \text{ V}$) swings the current between 0 and 4 mA (or more), so it is unreliable. Better to power a white LED from 5 V with $(5 - 3.2)/0.01 = 180 \Omega \rightarrow 180 \Omega/220 \Omega$.

8.4 Four red LEDs ($V_F = 2 \text{ V}$ each) are placed in series and powered from 12 V at 20 mA. Find R...

Total $V_F = 8 \text{ V}$. $R = (12 - 8)/0.02 = 200 \Omega$. $P = 4 \times 0.02 = 80 \text{ mW}$.

8.5 Ten LEDs each needing 10 mA are wired in parallel with separate resistors on 5 V. What...

Total = 100 mA. No: a pin supplies only ~20 mA. Use a transistor, or divide across several pins (respect the 200 mA chip limit).

8.6 A 12 V, 1 A power adapter is fed through a bridge rectifier whose diodes each drop 0.7 V,...

Peak = $9 \times 1.414 = 12.7 \text{ V}$; minus $2 \times 0.7 \text{ V} = 11.3 \text{ V}$.

8.7 Design a Zener regulator: input 12 V, Zener 5.1 V, load up to 15 mA, minimum Zener current...

$R_S = (12 - 5.1)/(0.015 + 0.005) = 345 \Omega \rightarrow 330 \Omega$. Worst-case current (no load) = $6.9/330 = 20.9 \text{ mA}$; $P_Z = 107 \text{ mW}$; $P_R = 0.0209^2 \times 330 = 0.144 \text{ W}$ ($\frac{1}{4} \text{ W}$ part is enough).

8.8 You lose the label on a diode; it has a black stripe near one end. Which lead is the...

The stripe end is the cathode. Diode-test mode: red on anode, black on cathode gives ~0.6 V; reversed gives OL.

8.9 Draw the schematic of one LED with a resistor on an Arduino pin (pin 8), showing polarity....

Pin 8 $\rightarrow$ resistor $\rightarrow$ LED anode; LED cathode $\rightarrow$ GND. Conventional current flows from the pin, through the resistor and the LED, to ground.

8.10 Why can an LED be destroyed almost instantly if connected directly to a 5 V supply, while a...

The LED's current rises exponentially with voltage above V_F . At 5 V the current is limited only by its tiny internal resistance and supply, so it is far beyond the rated 20 mA and overheats the junction. A resistor obeys Ohm's law, so current is limited by its resistance.

8.11 Two LEDs in parallel share one resistor and, after a few hours, one of them dies. Explain...

The LED with the lower V_F carried more current and failed (or dimmed). The remaining LED now carries the entire resistor current, which is more than its rating, so it also fails.

8.12 Do Experiment 4 for a red LED and a blue LED and fill in a table of V_F , current, and...

Red $V_F \approx 1.8\text{--}2.0\text{ V}$; blue $\approx 2.9\text{--}3.2\text{ V}$. Current follows $(5 - V_F)/R$. Brightness rises with current but flattens at the high end; at $10\text{ k}\Omega$ ($\approx 0.3\text{ mA}$) the LEDs are dim but visible.

Chapter 9

9.1 A BJT has $\beta = 200$. If the base current is 0.05 mA , what is the collector current (active...

$I_C = 200 \times 0.05 = \mathbf{10\text{ mA}}$. $I_E = I_C + I_B = 10.05\text{ mA}$.

9.2 You want to switch a 12 V buzzer that draws 40 mA using an Arduino pin (5 V). Choose I_B ...

$I_B = 4\text{ mA}$. $R_B = (5 - 0.7)/0.004 = 1075\ \Omega \rightarrow$ use $\mathbf{1\text{ k}\Omega}$ ($I_B = 4.3\text{ mA}$).

9.3 A 5 V relay has a coil resistance of $125\ \Omega$. Design the driver with a 2N2222: give I_C , I_B ,...

$I_C = 5/125 = 40\text{ mA}$. $I_B = 4\text{ mA}$. $R_B = 4.3/0.004 = 1075\ \Omega \rightarrow \mathbf{1\text{ k}\Omega}$. Power = $V_{CE(sat)} \times I_C \approx 0.2 \times 0.04 = \mathbf{8\text{ mW}}$.

9.4 A 12 V , 250 mA lamp is switched by a 2N2222 from a 5 V Arduino pin using forced $\beta = 10$. Is...

$I_B = 25\text{ mA}$, which exceeds the 20 mA pin limit (unacceptable). Use a MOSFET, a Darlington, or a two-transistor arrangement, or accept a forced β of 20–25 with $I_B = 10\text{--}12\text{ mA}$ (check the datasheet saturation curves).

9.5 A MOSFET with $R_{DS(on)} = 0.04\ \Omega$ carries 3 A . What is the power dissipated? Would it need a...

$P = 3^2 \times 0.04 = 0.36\text{ W}$. No heatsink needed.

9.6 The same MOSFET is fed 12 V . If it were replaced by an IRF540 (needs 10 V gate) driven from...

$P = 9 \times 0.5 = \mathbf{4.5\text{ W}}$: it would overheat quickly. This is why the logic-level type matters.

9.7 Draw an Arduino pin controlling a 12 V solenoid (300 mA) via an N-channel logic-level...

Pin $\rightarrow 220\ \Omega \rightarrow$ gate; $10\text{ k}\Omega$ gate to GND; source to GND; drain to solenoid $-$; solenoid $+$ to $+12\text{ V}$; 1N4007 across the solenoid, cathode to $+12\text{ V}$; connect the Arduino GND to the 12 V supply GND.

9.8 Explain why the emitter of an NPN low-side switch is connected to ground and why the ground...

The base-emitter voltage (or gate-source) must be measured relative to the emitter, so the pin signal and the emitter share a reference. Without a common ground, no voltage difference exists between the pin and the emitter and the transistor cannot turn on properly.

9.9 A student tests an NPN with the multimeter diode test. Red on pin 1, black on pin 2: 0.65 ...

Pin 1 is the base (both junctions show a diode drop with red probe on it): the transistor is **NPN**.

9.10 Why must the transistor be driven hard into saturation when used as a switch, rather than...

In the active region V_{CE} is large while I_C is large, so the transistor dissipates a lot of power ($P = V_{CE} \times I_C$) and gets hot. In saturation V_{CE} is $\approx 0.2\text{ V}$, so power loss is minimal.

9.11 What is the purpose of a $10\text{ k}\Omega$ resistor from the base (or gate) to ground?

It defines the input level (0 V) when the driving pin is floating or in a high-impedance state (e.g., during Arduino reset or boot-loader), keeping the load off, and drains leakage charge.

9.12 Do Experiment 5 parts A–C. Record base current, collector current and β for at least three...

Expected: with a large base resistor ($100\text{ k}\Omega$) $\beta \approx 100\text{--}300$ (LED dim); as the base resistor decreases the LED brightness saturates at the limit set by its series resistor, and computed β falls (transistor saturated).

Chapter 10

10.1 You connect a button between an Arduino input pin and GND with no other components, and...

The pin floats when the button is open. Fixes: (1) add a $10\text{ k}\Omega$ pull-up resistor from the pin to 5 V , or (2) enable the internal pull-up with `pinMode(pin, INPUT_PULLUP)`. Remember that pressed then reads LOW.

10.2 A $10\text{ k}\Omega$ pull-up is connected to 5 V and a button pulls it to ground. What current flows...

$I = 5/10\ 000 = 0.5\text{ mA}$. $P = 5 \times 0.0005 = 2.5\text{ mW}$.

10.3 Suppose the pull-up in the previous question were $100\ \Omega$ instead. What current and power...

$I = 50\text{ mA}$; $P = 250\text{ mW}$: wasteful; overheats the $\frac{1}{4}\text{ W}$ resistor and draws unnecessary current from the supply.

10.4 A 28BYJ-48 stepper has 2048 steps per revolution. How many steps rotate the shaft by 90° ?

$90^\circ = 2048/4 = \mathbf{512\text{ steps}}$. $10^\circ = 2048 \times 10/360 = \mathbf{56.9 \approx 57\text{ steps}}$.

10.5 A servo expects a pulse from 1 ms (0°) to 2 ms (180°). What pulse width would you send to...

Width = $1\text{ ms} + \text{angle}/180\text{ ms}$. $45^\circ \rightarrow 1.25\text{ ms}$; $135^\circ \rightarrow 1.75\text{ ms}$.

10.6 A DC motor draws 150 mA free-running and 900 mA stalled at 6 V . You use a MOSFET with...

$P = I^2 R = 0.81 \times 0.05 = \mathbf{0.04\text{ W}}$.

10.7 Identify a suitable sensor and output type for each: (a) is it dark outside? (b) is someone...

(a) LDR in a voltage divider (analog) or a light module; (b) PIR sensor (digital); (c) HC-SR04 ultrasonic (timed pulse); (d)

LM35/TMP36 (analog voltage) or DS18B20/DHT22 (digital).

10.8 Why should motors be powered from a supply separate from the Arduino's 5 V pin, and why...

Motors draw large, noisy currents that can brown out or reset the Arduino. A separate supply isolates that load. The grounds must be joined so that the control signal (referenced to Arduino GND) is measured against the same 0 V that the driver uses.

10.9 Explain why the stall current is so much higher than the running current.

A spinning motor generates back-EMF that opposes the supply and reduces the net voltage across the winding resistance. At stall there is no rotation, so no back-EMF: the full supply voltage drives current through the low winding resistance ($I = V/R$).

10.10 Do Experiment 6 and record voltages for the pull-up and pull-down circuits with the button...

Pull-down: ~0 V open, ~5 V pressed. Pull-up: ~5 V open, ~0 V pressed. A floating input touched by a finger can read anything from about 0 to 2 V or more on a high-impedance meter.

Chapter 11

11.1 A non-inverting amplifier has $R_1 = 2.2 \text{ k}\Omega$ and $R_2 = 22 \text{ k}\Omega$. What is its gain? What is V_{out} ...

Gain = $1 + 22/2.2 = 11$. $V_{\text{out}} = 2.75 \text{ V}$.

11.2 Design a non-inverting amplifier with a gain of 6 using $R_1 = 10 \text{ k}\Omega$. Which standard E24...

$R_2 = 5 \times 10 \text{ k}\Omega = 50 \text{ k}\Omega$. E24 value: **51 k Ω** (gain 6.1).

11.3 An inverting amplifier uses $R_1 = 4.7 \text{ k}\Omega$ and $R_2 = 47 \text{ k}\Omega$. Find the gain. If $V_{\text{in}} = -0.3 \text{ V}$,...

Gain = -10 . $V_{\text{out}} = +3 \text{ V}$.

11.4 A thermistor divider gives 0.8 V at 25 °C and 1.1 V at 40 °C. You want the output to span...

The signal changes by 0.3 V and you want a 5 V span: gain $\approx 5/0.3 = 16.7$, and the 0.8 V base offset must be subtracted (differential amplifier or a reference at the $-$ input). Alternatively read the sensor directly with the Arduino ADC and scale in software.

11.5 An LM358 runs from 5 V. What is the highest output voltage you can expect? Would a gain-5...

About 3.5 V ($V_{\text{cc}} - 1.5 \text{ V}$). A 1 V input would want 5 V, so no: the output would clip at $\approx 3.5 \text{ V}$. Use a rail-to-rail op-amp or a gain of 3.

11.6 A 555 in monostable mode has $R = 220 \text{ k}\Omega$ and $C = 4.7 \mu\text{F}$. How long is the output pulse?

$t = 1.1 \times 220\,000 \times 4.7 \times 10^{-6} = 1.14 \text{ s}$.

11.7 You want a 10-second monostable delay using a 100 μF capacitor. Find R and choose a...

$R = 10/(1.1 \times 100 \times 10^{-6}) = 90.9 \text{ k}\Omega \rightarrow \mathbf{91 \text{ k}\Omega}$ (E24), $t = 10.05 \text{ s}$.

11.8 A 555 astable uses $R_1 = 4.7 \text{ k}\Omega$, $R_2 = 47 \text{ k}\Omega$, $C = 0.1 \mu\text{F}$. Find f , t_{HIGH} , t_{LOW} and duty cycle.

$R_1 + 2R_2 = 98.7 \text{ k}\Omega$. $f = 1.44/(98.7 \text{ k} \times 0.1 \mu) = \mathbf{146 \text{ Hz}}$. $t_{\text{HIGH}} = 0.693 \times 51.7 \text{ k} \times 0.1 \mu = 3.58 \text{ ms}$; $t_{\text{LOW}} = 0.693 \times 47 \text{ k} \times 0.1 \mu = 3.26 \text{ ms}$. Duty = $51.7/98.7 = \mathbf{52.4 \%}$.

11.9 Design a 555 astable that flashes an LED once per second (1 Hz). Use $C = 100 \mu\text{F}$ and $R_1 = 1...$

$R_1 + 2R_2 = 1.44/(1 \times 100 \mu) = 14.4 \text{ k}\Omega \rightarrow R_2 = 6.7 \text{ k}\Omega$: choose **6.8 k Ω** ($f = 1.44/(14.6 \text{ k} \times 100 \mu) = 0.986 \text{ Hz}$).

11.10 Why does an op-amp with no feedback act as a comparator rather than an amplifier?

Because of its enormous open-loop gain ($\geq 100\,000$): any tiny difference between the inputs saturates the output at one of the rails. Negative feedback is needed to bring the gain down to a controlled value.

11.11 In the inverting amplifier, why is the $-$ input called a "virtual ground"?

Feedback holds the $-$ input at the same voltage as the $+$ input (0 V) without any direct connection to ground. Current flows toward that node but the node stays at 0 V, which is why R_1 alone sets the input current.

11.12 Build Experiment 7 and record the blink frequency for three different C values. Compare...

Errors of 10–30 % are typical, mainly because electrolytic capacitors have a wide tolerance ($-20 \%/+80 \%$) and resistors $\pm 5 \%$. A ceramic/film capacitor would give better agreement.

Chapter 12

12.1 Convert to decimal: (a) 1010, (b) 11111111, (c) 10000000, (d) 01100110.

(a) 10; (b) 255; (c) 128; (d) $64 + 32 + 4 + 2 = 102$.

12.2 Convert to binary (8 bits): (a) 5, (b) 37, (c) 100, (d) 255.

(a) 00000101; (b) 00100101; (c) 01100100; (d) 11111111.

12.3 Convert: (a) 0x2F to decimal, (b) 0xA0 to binary, (c) 173 to hex, (d) 0b11010110 to hex.

(a) 47; (b) 10100000; (c) 0xAD; (d) 0xD6.

12.4 How many different values can a 12-bit number take? What is the largest?

$2^{12} = 4096$ values; largest 4095.

12.5 An Arduino analog input reads 768. What voltage is applied (5 V reference)? What would the...

$768 \times 5/1023 = \mathbf{3.75\text{ V}}$. $1.0\text{ V} \rightarrow 1.0/5 \times 1023 = \mathbf{205}$.

12.6 What is the voltage resolution (step size) of a 12-bit ADC with a 3.3 V reference? How much...

$3.3/4096 = \mathbf{0.806\text{ mV}}$; $4.88/0.806 \approx 6$ times finer.

12.7 You want to read a signal with frequency components up to 8 kHz. What is the minimum...

Nyquist: $> 16\text{ kS/s}$. An Uno at 9.6 kS/s cannot; you need a faster ADC or a lower-bandwidth signal.

12.8 Complete the truth table for $Y = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$ for all 8 input combinations.

A B C $\rightarrow$ Y: 000 $\rightarrow$ 1, 001 $\rightarrow$ 0, 010 $\rightarrow$ 1, 011 $\rightarrow$ 0, 100 $\rightarrow$ 1, 101 $\rightarrow$ 0, 110 $\rightarrow$ 1, 111 $\rightarrow$ 1.

12.9 Write the Arduino condition (C++) that is true when a button is pressed (pin 2 reads LOW...

if (digitalRead(2) == LOW && v > 500) { ... }

12.10 Using De Morgan's law, rewrite NOT(A AND B) using only OR and NOT.

(NOT A) OR (NOT B).

12.11 Design a logic circuit for a car dome-light: it turns on if any door is open (D1, D2, D3)...

Light = (D1 + D2 + D3 + S) · NOT I. Arduino: `if ((d1 || d2 || d3 || s) && !ignition) ...`

12.12 Explain why a 5 V Arduino output connected to a 3.3 V sensor input may damage the sensor,...

The 5 V output exceeds the 3.3 V chip's maximum input rating ($\approx V_{cc} + 0.3\text{ V}$), so its protection diodes conduct heavily.

A 3.3 V output is above the Arduino's V_{IH} (3.0 V), so it reads as HIGH, though with little margin.

12.13 Why is it important to tie unused CMOS inputs to +V or ground?

A CMOS input is very high-impedance. If floating, it picks up noise, can sit at a mid-level voltage where both transistors conduct (wasting current and heating the chip), and produces erratic outputs.

12.14 Do Experiment 8 and fill in the truth table for the series and the parallel switch...

Series (AND): on only for 1,1. Parallel (OR): on for 01, 10 and 11.

Chapter 13

13.1 A project draws 250 mA average. You power it from a 2600 mAh Li-ion cell (3.7 V) through a...

Battery energy = $3.7 \times 2.6 = 9.62\text{ Wh}$. Load power = $5 \times 0.25 = 1.25\text{ W}$. Time = $9.62 \times 0.85/1.25 = \mathbf{6.5\text{ h}}$ (theoretical; derate a little).

13.2 A 7805 is fed with 9 V and supplies 200 mA. How much power does it dissipate? What...

$P = (9 - 5) \times 0.2 = 0.8\text{ W}$. $\Delta T = 0.8 \times 65 = \mathbf{52\text{ }^{\circ}\text{C}}$ (hot but survivable; add a heatsink or use lower voltage).

13.3 The same 7805 is fed 12 V and supplies 500 mA. Power? Is a heatsink necessary?

$P = 7 \times 0.5 = 3.5\text{ W}$; $\Delta T = 227\text{ }^{\circ}\text{C}$ without a heatsink: impossible. A heatsink of $\leq 10\text{ }^{\circ}\text{C/W}$ or a buck converter is required.

13.4 A 12 V, 1 A adapter feeds a buck converter (90 % efficient) that supplies 5 V at 1 A. What...

$P_{\text{out}} = 5\text{ W}$; $P_{\text{in}} = 5/0.9 = 5.56\text{ W}$; $I_{\text{in}} = 5.56/12 = \mathbf{0.46\text{ A}}$. A linear regulator would need 1 A and waste 7 W.

13.5 Design the smoothing capacitor for a 300 mA supply from 50 Hz mains (full-wave rectifier)...

$C = I/(2 f V_r) = 0.3/(2 \times 50 \times 0.5) = 0.006\text{ F} = \mathbf{6000\text{ }\mu\text{F}}$; use 6800 μF (or 4700 μF + 2200 μF).

13.6 Total the current budget for: Arduino (45 mA), 3 LEDs (10 mA each), one relay module (70...

$45 + 30 + 70 + 65 + 30 = 240\text{ mA}$ typical; well below 500 mA. Yes, with margin.

13.7 Explain why the Arduino Uno gets hot when powered with 12 V and driving several LEDs and a...

The on-board linear regulator drops 7 V and passes all the 5 V-rail current: $P = 7\text{ V} \times I$. At 300 mA that is 2.1 W, which overheats it.

13.8 You measure a "12 V" unregulated adapter with no load and read 17 V. Is it safe for the...

The board's maximum recommended input is 12 V (20 V absolute limit); 17 V is inside the absolute limit but makes the regulator run very hot. Use a regulated 9 V adapter instead.

13.9 Why must a battery pack of two cells connected in parallel use identical cells at equal charge?

A cell with a higher voltage will push large current into the lower one through their small internal resistances, wasting energy and heating both; in Li-ion this can be dangerous.

13.10 Your Arduino resets every time a small DC motor connected to the same 5 V supply starts....

(1) Power the motor from a separate supply (common ground); (2) add a 470 – 1000 μF capacitor across the Arduino's supply and a 100 nF capacitor across the motor terminals; (3) add a flyback diode and use a MOSFET/driver, and thicker wiring to lower voltage drop.

13.11 Carry out Experiment 9. Report the output voltage of the 7805 at three load currents and...

Expected: output stays at 4.95 – 5.05 V for all loads; loss at 50, 106 mA: 0.2 and 0.43 W; warmth becomes noticeable above about 0.5 W.

Chapter 14

14.1 List three differences between a microcontroller and the microprocessor in a laptop.

A microcontroller integrates memory, I/O and peripherals on the chip, runs at MHz with milliwatts, and runs one dedicated program; a laptop processor needs external RAM, storage and I/O chips, runs at GHz with watts, and runs a general-purpose operating system.

14.2 Which Uno pins can output PWM? Which are used for I2C? Which two should be avoided while...

PWM: 3, 5, 6, 9, 10, 11. I2C: A4 (SDA) and A5 (SCL). Avoid 0 and 1 (RX/TX).

14.3 The Uno's 16 MHz clock: how long does one clock cycle take? How many cycles does 1 ms...

$T = 1/16 \text{ MHz} = 62.5 \text{ ns}$. $1 \text{ ms} = 16,000 \text{ cycles}$.

14.4 You connect six LEDs, each drawing 15 mA, to six pins. Is each pin within limits? Is the total?

Each pin $15 \text{ mA} < 20 \text{ mA}$ ✓. Total $90 \text{ mA} < 150 \text{ mA}$ (recommended) ✓.

14.5 You want an LED ($V_F = 2.2 \text{ V}$, 12 mA) on a 5 V pin. Choose a resistor from E12 and calculate...

$R = 2.8/0.012 = 233 \Omega \rightarrow 270 \Omega$ (E12) gives 10.4 mA ; 220Ω gives 12.7 mA .

14.6 Modify Blink so the LED is on for 250 ms and off for 750 ms. Write the loop() function.

```
void loop() { digitalWrite(LED_PIN, HIGH); delay(250); digitalWrite(LED_PIN, LOW); delay(750); }
```

14.7 Explain the difference between Verify and Upload in the IDE.

Verify compiles the sketch and reports errors but does not touch the board. Upload compiles and then transfers the program to the board's flash memory.

14.8 Your Blink sketch uploads without errors, but the external LED never lights. List four...

1) Does the built-in L LED blink? (proves the code runs) 2) LED polarity (long leg to resistor side) 3) Wiring: correct pin number, GND connected, breadboard column connections 4) pinMode(..., OUTPUT) present, resistor value, LED works (test with a battery + resistor).

14.9 Why is it a bad idea to apply 12 V directly to a digital input pin?

The pin's protection diodes and transistors are rated for at most $V_{cc} + 0.5 \text{ V}$ (about 5.5 V). 12 V would force large current through the protection diode into the 5 V rail, damaging the pin or the entire chip.

14.10 Complete Experiment 10 items 1–3 and record what the changes do. Take a photo of your...

The LED should follow your timing changes; the alternating LED pattern requires that one is HIGH exactly when the other is LOW.

Chapter 15

15.1 What are the values of: (a) $7 / 2$, (b) $7 / 2.0$, (c) $7 \% 3$, (d) $10 / 4 * 4$, (e) $(\text{float})10 / \dots$

(a) 3; (b) 3.5; (c) 1; (d) 8 ($10/4=2$, $\times 4$); (e) 10.0.

15.2 A sketch declares `int x = 32000; x = x + 1000;`. What is x afterwards? Why?

$32000 + 1000 = 33000$ exceeds 32,767; it wraps to $33000 - 65536 = -32536$.

15.3 Use map() to convert a potentiometer reading of 300 (0–1023) into a servo angle 0–180....

$300 \times 180 / 1023 = 52.78 \rightarrow 52$.

15.4 Write the expression that converts an ADC reading raw into millivolts (5 V reference)...

```
long mv = (long)raw * 5000 / 1023;
```

15.5 What is `0b10110000 | 0b00001111`? And `0b11111111 & ~(1 << 3)`?

`0b10111111`; `0b11110111`.

15.6 Write a for loop that turns on LEDs on pins 2–7 one after another with 100 ms between them...

```
for (int pin = 2; pin <= 7; pin++) { digitalWrite(pin, HIGH); delay(100); }
```

15.7 Write a function `int average(int a, int b)` that returns the mean of two integers, and show...

```
int average(int a, int b) { return (a + b) / 2; } ... int m = average(analogRead(A0), analogRead(A1));
```

15.8 Write a sketch fragment using an array of 3 pins {9, 10, 11} and a loop that sets them as...

```
const int pins[] = {9, 10, 11}; for (int i = 0; i < 3; i++) { pinMode(pins[i], OUTPUT); digitalWrite(pins[i], LOW); }
```

15.9 Find the bug: `if (buttonState = HIGH) { digitalWrite(led, HIGH); }`

Single = assigns HIGH to buttonState, so the condition is always true. Should be `==`.

15.10 Find the bug: `int readings[5]; for (int i = 0; i <= 5; i++) readings[i] = analogRead(A0);`

The loop runs $i = 0..5$ (six passes), writing `readings[5]` outside the array. Use `i < 5`.

15.11 Why does `float volts = analogRead(A0) / 1023 * 5;` always give 0 (except at exactly 1023)?...

`analogRead` returns an `int`, and `int / 1023` is integer division, giving 0 for values below 1023. Correct: `float volts = analogRead(A0) * 5.0 / 1023;`

15.12 When would you prefer a `char` array to a `String` object?

For long-running devices with little RAM, because `String` uses dynamic allocation that can fragment memory and eventually crash the program; `char` arrays have fixed, predictable memory use.

15.13 Upload the *Serial keyboard-control sketch* and control the LED. Then extend it so that...

Add: `if (c == 'b') { for (int i=0;i<3;i++){ digitalWrite(LED_BUILTIN,HIGH); delay(150); digitalWrite(LED_BUILTIN,LOW); delay(150);} }` and `if (c == 's') Serial.println(millis());`

Chapter 16

16.1 A button is wired between pin 4 and GND with `INPUT_PULLUP`. Write the condition that is true...

`digitalRead(4) == LOW` (or `!digitalRead(4)`).

16.2 Modify the *LED-follows-button sketch* to use an external pull-down and a button to +5 V....

Wiring: 10 k Ω from pin to GND, button from pin to +5 V. Code: `pinMode(pin, INPUT);` and test `== HIGH` for pressed.

16.3 Write a sketch that counts button presses (debounced) and prints the count to the Serial...

Use the debounce sketch, add `int count = 0;` and inside the accepted-press branch `count++;`

`Serial.println(count);`

16.4 How long does it take for `millis()` to overflow? (unsigned long, 4,294,967,295 ms)

$4,294,967,296 \text{ ms} \div 86,400,000 \text{ ms/day} \approx \mathbf{49.7 \text{ days}}$.

16.5 The RC debounce filter uses $R = 10 \text{ k}\Omega$ (the pull-up) and $C = 100 \text{ nF}$. What is its time...

$\tau = 10^4 \times 10^{-7} = 1 \text{ ms}$; $3\tau = 3 \text{ ms}$ for 95 %.

16.6 A sketch reads a button 5,000 times per second using `loop()`. If the bounce lasts 8 ms,...

$8 \text{ ms} \times 5 \text{ readings/ms} = 40 \text{ readings}$, of which several can be different; typical bounce gives 2 – 10 false edges. This is why debouncing matters.

16.7 Write a program with two LEDs: LED A blinks at 2 Hz, LED B blinks at 0.5 Hz, both without...

Use two `millis()` timers: intervals 250 ms (A toggles every 250 ms $\rightarrow$ 2 Hz) and 1000 ms (B toggles every 1000 ms $\rightarrow$ 0.5 Hz), each with its own previous variable and state.

16.8 Design (as a state diagram) a traffic light with states RED (5 s), GREEN (4 s), YELLOW (1...

States: RED $\rightarrow$ GREEN $\rightarrow$ YELLOW $\rightarrow$ RED. Keep `stateStart = millis();` in each state compare `millis() - stateStart` with that state's duration and switch to the next state when it elapses, setting the LED pins accordingly.

16.9 Why is it better to declare unsigned long previous as a global variable rather than inside...

A local variable would be re-created and reset every pass of `loop()`, so it would not remember when the last event happened. Globals (or `static` locals) keep their values between calls.

16.10 Explain in words why `if (now - previous >= interval)` still works when `millis()` rolls over...

Unsigned subtraction wraps modulo 2^{32} , so the difference always equals the true elapsed time (e.g. $5 - 4,294,967,290 = 11$ in unsigned arithmetic).

16.11 Build and run Experiment 11. Add a second attempt mode: after each game, print the average...

Keep `unsigned long total;` `int games;` — after each game add the reaction time, increment games and print `total / games`.

Chapter 17

17.1 The Arduino reads 341 on A0 (5 V reference). What is the voltage? What would it read for 2.0 V?

$341 \times 5/1023 = \mathbf{1.667 \text{ V}}$. $2.0 \text{ V} \rightarrow 2.0/5 \times 1023 = \mathbf{409}$.

17.2 What is the resolution (volts per count) with the 1.1 V internal reference? With an LM35,...

$1.1/1024 = 1.07 \text{ mV}$; LM35 = 10 mV/ $^{\circ}\text{C}$, so **0.107 $^{\circ}\text{C}$ per count**.

17.3 An LDR of 20 k Ω (dim room) is on top of a 10 k Ω resistor to ground with a 5 V supply. What...

Dim: $5 \times 10/30 = 1.667 \text{ V} \rightarrow 341$. Bright: $5 \times 10/12 = 4.167 \text{ V} \rightarrow 853$.

17.4 An NTC (10 k Ω at 25 $^{\circ}\text{C}$) is on top of a 10 k Ω resistor. The ADC reads 620. Find the...

$R = 10\,000 \times (1023/620 - 1) = 6\,500 \Omega < 10 \text{ k}\Omega$, so the thermistor is **warmer** than 25 $^{\circ}\text{C}$ (about 35 $^{\circ}\text{C}$ with $\beta = 3950$).

17.5 A TMP36 outputs 0.75 V. What is the temperature? What ADC value does the Arduino read (5 V...

$T = (0.75 - 0.5) \times 100 = \mathbf{25\text{ }^{\circ}\text{C}}$. $\text{ADC} = 0.75/5 \times 1023 = \mathbf{153}$.

17.6 Using $\alpha = 0.2$, the filtered value is 500 and a new reading of 600 arrives. What is the new...

$0.2 \times 600 + 0.8 \times 500 = \mathbf{520}$; then $0.2 \times 600 + 0.8 \times 520 = \mathbf{536}$.

17.7 You average 25 samples. By what factor does random noise fall (approximately)?

$\sqrt{25} = \mathbf{5\text{ times}}$.

17.8 Write code that reads a potentiometer and outputs the message "LOW", "MEDIUM" or "HIGH" on...

if (raw < 341) ...("LOW"); else if (raw < 682) ...("MEDIUM"); else ...("HIGH");

17.9 Modify the night-light sketch so that it turns the LED on after the room has been dark for...

Record the time when the reading first drops below the threshold; if it stays below for 5000 ms, turn on; if it rises above, reset the start time.

17.10 Why can't you measure a 12 V battery directly with an analog pin? How can you fix it?

Above 5.5 V would damage the pin. Use a voltage divider (e.g. 20 k Ω + 10 k Ω , ratio 1/3, maps 15 V to 5 V) and multiply the result by 3 in software.

17.11 A divider reading jumps around by ± 30 counts when a nearby motor runs. List three measures.

100 nF capacitor on the analog pin; move/shield the wires and separate the motor supply; averaging or median filtering; sample when the motor is not switching; add a flyback diode and noise capacitor at the motor.

17.12 Complete Experiment 12 and record raw values at three temperatures (ice water, room, body)...

With the NTC on top and the fixed resistor at the bottom, warming lowers the NTC resistance and raises the divider output: ice water gives the lowest ADC value, room temperature more, and your fingers the most. (If you swap the resistors the trend reverses.)

Chapter 18

18.1 What is the average voltage for analogWrite(9, 64) on a 5 V Uno? For analogWrite(9, 200)?

$64/255 \times 5 = \mathbf{1.25\text{ V}}$; $200/255 \times 5 = \mathbf{3.92\text{ V}}$.

18.2 A 490 Hz PWM signal has a period of what duration? If the duty cycle is 30 %, how long is...

$T = 1/490 = 2.04\text{ ms}$; $\text{HIGH} = 0.3 \times 2.04 = \mathbf{0.61\text{ ms}}$.

18.3 You want to create 2.0 V with a filtered PWM output from a 5 V Arduino. What analogWrite...

$2.0/5 \times 255 = \mathbf{102}$.

18.4 Design an RC filter for PWM at 490 Hz with a corner frequency of 5 Hz. Choose $R = 10\text{ k}\Omega$ and...

$C = 1/(2\pi \times 5 \times 10\,000) = 3.18\text{ }\mu\text{F} \rightarrow \mathbf{3.3\text{ }\mu\text{F}}$.

18.5 A servo needs a pulse of 1.0 ms for 0° and 2.0 ms for 180°. What pulse width corresponds to...

$\text{width} = 1 + 60/180 = 1.333\text{ ms}$. $\text{Duty} = 1.333/20 = \mathbf{6.7\text{ \%}}$.

18.6 What is the frequency of the note one octave above A4? Of the note one semitone above A4 (A#4)?

880 Hz; $440 \times 1.0595 = \mathbf{466\text{ Hz}}$.

18.7 Write code to blink an RGB LED through red, green, blue in sequence, with each colour...

For each colour pin, loop v from 0 to 255 then back to 0 with delay(2), using analogWrite(pin, v).

18.8 Write a program that plays the tone corresponding to the potentiometer position, quantised...

int idx = map(analogRead(A0), 0, 1023, 0, 7); tone(8, scale[idx]); with const int scale[] = {262,294,330,349,392,440,494,523};

18.9 Why does a servo sometimes twitch when the Arduino also drives a motor from the same supply?

The motor's current spikes and back-EMF disturb the supply and ground, corrupting the servo's control pulses or resetting the microcontroller. Use separate supplies with a common ground, add capacitors and flyback diodes.

18.10 Why is PWM dimming of an LED more energy-efficient than putting a larger series resistor?

With a resistor the extra voltage is dissipated as heat in the resistor all the time. With PWM the LED is fully on (in its efficient operating point) or off, so no power is wasted in a series resistor beyond the small one needed for current limiting.

18.11 Build the LDR theremin (Experiment 13 step 4). Determine the frequency range you obtain and...

Two octaves is a factor of 4; e.g. map from 220 Hz to 880 Hz. Read the LDR's min and max values with Serial first, then use map(reading, min, max, 220, 880).

Chapter 19

19.1 An HC-SR04 echo pulse is 1,750 μ s. What is the distance? (speed of sound 343 m/s)

$d = 1750 \times 0.0343/2 = \mathbf{30.0\text{ cm}}$.

19.2 What is the maximum echo pulse width for an object at 4 m?

$t = 2 \times 400\text{ cm} / 0.0343 = \mathbf{23,300\text{ }\mu\text{s}}$ (23.3 ms).

19.3 At 35 °C the speed of sound is $331.3 + 0.606 \times 35 = 352.5\text{ m/s}$. If your code assumes 343...

The code computes with 343 while the true speed is 352.5, so it reads $343/352.5 \times 100 = \mathbf{97.3\text{ cm}}$ (error -2.7%).

19.4 A DHT22 can be read every 2 s. If your loop reads it on every pass with no delay, what...

Requests faster than 2 s return old values or fail (NaN). Use a millis() timer to read only every 2000 ms.

19.5 A rotary encoder gives 20 steps per revolution. The position counter shows 55. How many...

$55/20 = 2.75$ turns clockwise.

19.6 Write a program that lights an LED for 10 seconds after each PIR trigger (without delay()).

On PIR HIGH set `lastMotion = millis(); ledOn = true;` in loop, if ledOn and `millis() - lastMotion > 10000` turn the LED off.

19.7 Modify the ultrasonic sketch to print "Object too close!" when the distance is below 15 cm,...

if ($d > 400$ || $d < 0$) return; if ($d < 15$) Serial.println("Object too close!");

19.8 Why might an HC-SR04 fail to detect a soft sofa or a person wearing a thick coat?

Soft materials absorb ultrasound rather than reflecting it, so the returning echo is too weak to be detected.

19.9 Why should a PIR sensor be given time to stabilise after power-up?

Its circuit needs time to learn the room's baseline infrared pattern; initial transients may produce false triggers.

19.10 Build Experiment 14 and measure at ten known distances (10 – 200 cm) with a ruler. Compute...

The plot should be nearly a straight line with slope ≈ 1 ; errors of $\pm 1\text{ cm}$; larger deviations at extreme range or oblique surfaces.

Chapter 20

20.1 Which 7-segment pattern (bits d p g f e d c b a) displays the letter "A"? And "H"?

A = segments a b c e f g = 0111 0111 = 0x77. H = b c e f g = 0111 0110 = 0x76.

20.2 You use a common-anode display. What changes in the code and in the wiring compared to...

Wiring: the common pin goes to +5 V. Code: invert the pattern (`~digits[n]`) because a segment lights when its pin is LOW.

20.3 Each segment of a 7-segment display ($V_F = 2.0\text{ V}$) should carry 10 mA from 5 V. What...

$R = 3/0.01 = 300\text{ }\Omega \rightarrow 330\text{ }\Omega$ (9.1 mA). "8" lights 7 segments: 64 mA total (plus dp): but each pin carries only 9 mA, well within limits.

20.4 How many bits must be shifted to update a chain of three 74HC595 chips? How many latch...

24 bits (three bytes), and **one** latch pulse.

20.5 A 4-digit multiplexed display refreshes each digit for 2 ms in turn. What is the refresh...

Cycle = $4 \times 2\text{ ms} = 8\text{ ms} \rightarrow 125\text{ Hz}$ per digit; duty cycle = 25 %. The average current is one quarter of the pulsed current.

20.6 To get an average of 8 mA per segment in a 4-digit multiplexed display with 25 % duty...

$8 / 0.25 = \mathbf{32\text{ mA}}$ (within the segment's pulsed rating, but above the 20 mA pin limit: use transistors/driver ICs).

20.7 Write a function that sends the byte for a decimal digit to a 74HC595 driving a...

```
void showDigit(int n) { digitalWrite(LATCH,LOW); shiftOut(DATA,CLOCK,MSBFIRST,digits[n]); digitalWrite(LATCH,HIGH); }
```

20.8 Write code to display the number of seconds since start on an LCD, as mm:ss on the second row.

```
unsigned long s = millis()/1000; lcd.setCursor(0,1); if (s/60 < 10) lcd.print('0'); lcd.print(s/60); lcd.print(':'); if (s%60 < 10) lcd.print('0'); lcd.print(s%60);
```

20.9 Your I2C LCD lights up its backlight but shows only dark rectangles in the top row. What is...

The controller has not been initialised or the contrast is too high. Check `lcd.init()` runs, and turn the contrast trimmer on the backpack until characters appear; verify the I2C address.

20.10 Why can a multiplexed display flicker if your loop() contains a long delay()?

The display is only lit one digit at a time as long as loop() keeps refreshing it. A long delay pauses refreshing, so a single digit stays lit (very bright) while the others are dark.

20.11 Build Experiment 15 steps 1–3 and photograph the working display. Measure the current...

Expect about 9 mA per segment with $330\text{ }\Omega$ on 5 V ($V_F \approx 2\text{ V}$); differences follow the LED's actual forward voltage.

Chapter 21

21.1 A motor stalls at 1.2 A from a 6 V supply. What is its winding resistance? What power does...

$R = 6/1.2 = 5\ \Omega$. Driver loss = $2\text{ V} \times 1.2\text{ A} = \mathbf{2.4\text{ W}}$ (needs the heat sink).

21.2 The motor's free-running current is 200 mA and stall current is 1.2 A. A TB6612 with...

TB6612 loss = $I^2 R = 1.44 \times 0.5 = 0.72\text{ W}$ (vs 2.4 W for the L298N): about a third.

21.3 A geared motor is 1:48 and the motor shaft turns at 9600 rpm. What is the wheel speed? If...

$9600/48 = 200\text{ rpm}$. Circumference $\pi \times 0.065 = 0.204\text{ m}$; speed = $200 \times 0.204/60 = \mathbf{0.68\text{ m/s}}$.

21.4 A 28BYJ-48 is set to 10 rpm with 2048 steps/rev. What is the step rate? What is the delay...

Rate = $10 \times 2048/60 = 341\text{ steps/s}$; delay = 2.93 ms per step.

21.5 You need to rotate a NEMA 17 (200 steps/rev, 1/8 microstep) by 45°. How many microsteps?

$(45/360) \times 200 \times 8 = \mathbf{200\text{ microsteps}}$.

21.6 Write a function `setMotor(int speed)` for one motor with IN1, IN2, EN pins that handles...

```
if (speed == 0) { digitalWrite(IN1,LOW); digitalWrite(IN2,LOW); analogWrite(EN,255);} else { digitalWrite(IN1,speed>0);  
digitalWrite(IN2,speed<0); analogWrite(EN,abs(speed)); }
```

21.7 Modify the robot code so that it reverses if the obstacle is closer than 10 cm and turns if...

Add a distance test in FORWARD: if $d < 10 \rightarrow$ BACKING immediately; else if $d < 25 \rightarrow$ TURNING; otherwise keep forward.

21.8 Why does the Arduino often reset the instant a motor starts, and what are the fixes?

Starting a motor draws a large inrush current that drops the shared supply voltage below the microcontroller's brown-out level, and also injects noise. Fix: separate motor supply, thicker wires, capacitors at the motor and driver, keep grounds common.

21.9 Explain shoot-through and how motor-driver ICs avoid it.

If both the top and bottom switches on one side conduct together the supply is short-circuited. Driver ICs use a single direction input per side and dead-time logic to make sure one switch is off before the other turns on.

21.10 Complete Experiment 16. Draw a table of PWM value, motor speed (estimated rpm or by...

Typically 25–40 % duty for the motor to start; current rises with load; stall current is several times higher. Actual values depend on the motor.

Chapter 22

22.1 How long does it take to send the 12-character string "Hello, world" (with newline = 13...

13 bytes $\times 10\text{ bits} = 130\text{ bits}$. At 9600: 13.5 ms. At 115200: 1.13 ms.

22.2 You send one 200-byte packet per second over serial. What is the lowest standard baud rate...

$200 \times 10 = 2000\text{ bit/s}$, so 2400 baud is enough (standard rates: 1200, 2400, 4800, 9600...).

22.3 A 16-bit ADC reading is sent over I2C at 100 kHz. Roughly how long does the transaction...

4 bytes $\times 9 = 36\text{ clocks}$ (+ start/stop) $\approx 380\ \mu\text{s}$ at 100 kHz.

22.4 An I2C bus has 4 modules, each with 10 k Ω pull-ups. What is the effective pull-up...

$10\text{ k}\Omega/4 = 2.5\text{ k}\Omega$: fine (typical recommendation 1 – 10 k Ω).

22.5 An SPI display is updated at 8 MHz. How long does it take to send a 320×240 frame of...

Bits = $320 \times 240 \times 16 = 1,228,800$; time = $1,228,800/8\text{ MHz} = \mathbf{0.154\text{ s}}$ (6.5 frames/s): SPI speed limits the frame rate.

22.6 Wiring: an Arduino Uno with a BME280 (I2C, 3.3 V module with regulator) and an SSD1306 OLED...

Both to A4/A5 (SDA/SCL), VCC to 3.3 V/5 V as their boards allow. Addresses: BME280 0x76 (or 0x77), OLED 0x3C. The modules usually have pull-ups; no extras are needed.

22.7 Write code that reads an integer from the Serial Monitor and turns an LED on for that many...

Store `ledOffTime = millis() + value*1000UL`; `digitalWrite(LED,HIGH)`; and in `loop()` turn it off when `millis() >= ledOffTime`.

22.8 Why do you cross TX and RX in a UART connection but not SDA and SCL in I2C?

In UART each device has a transmit pin and a receive pin; one device's transmitter must feed the other's receiver. In I2C the data line is bidirectional and shared by all devices; there is no transmitter/receiver distinction.

22.9 What happens if two I2C devices with the same address are connected to the same bus?

Both respond to every message, corrupting each other's replies; the bus becomes unusable. Change one address, use a multiplexer, or use separate buses.

22.10 Do Experiment 17 and produce a table of all I2C addresses found with a description of each...

For example: 0x27 (LCD backpack), 0x68 (DS3231), 0x57 (AT24C32 EEPROM on the RTC module), 0x3C (OLED), 0x76 (BME280).

Chapter 23

23.1 A Timer1 in CTC mode uses prescaler 256 and OCR1A = 6249. What is the interrupt frequency?

$f = 16,000,000 / (256 \times 6250) = 10 \text{ Hz}$.

23.2 Choose a prescaler and OCR1A value to generate a 50 Hz interrupt. (Timer1, 16-bit)

Prescaler 64: timer clock 250 kHz $\rightarrow 250,000 / 50 = 5000$ counts $\rightarrow$ OCR1A = 4999. (Prescaler 8 would need 40,000 counts: also fits.)

23.3 A tachometer has 2 pulses per revolution. In 1 s you count 130 pulses. What is the RPM?

Pulses per rev = 2: rev/s = 65: **3900 rpm**.

23.4 Using period measurement: the time between pulses from a 1-pulse-per-revolution sensor is...

rev/s = $1 / 0.025 = 40$; RPM = **2400**.

23.5 A flow sensor produces 7.5 pulses per second for each L/min. You count 255 pulses in 10 s....

25.5 pulses/s / 7.5 = 3.4 L/min. Volume = $3.4 / 60 \times 10 = 0.567 \text{ L}$.

23.6 A node draws 20 mA for 100 ms every 10 s and sleeps at 5 μA . Find the average current and...

Average = $(0.1 \times 20 + 9.9 \times 0.005) / 10 = 0.2050 \text{ mA}$. Life = $1000 / 0.205 = 4878 \text{ h} \approx 203 \text{ days}$.

23.7 Why is Serial.println() inside an ISR a bad idea?

Serial output uses interrupts and a buffer; inside an ISR interrupts are disabled, so if the buffer is full the ISR can block forever (deadlock), and printing takes very long compared with the ISR's budget.

23.8 Find the bug: `int count; void isr() { count++; } loop() { Serial.println(count); }` What is...

`count` is shared between ISR and `loop()` but not `volatile`; the compiler may cache it. Declare `volatile int count`; and copy it with interrupts disabled before printing (int is 2 bytes).

23.9 Why does millis() stop advancing inside a very long ISR and what are the consequences?

millis() is updated by the Timer0 overflow interrupt, which cannot run while another ISR is executing (interrupts disabled). A long ISR therefore makes millis()/delay() lose time.

23.10 When should you prefer polling to an interrupt?

For slow, non-critical events (buttons pressed by humans, slow sensors) where simplicity and debuggability matter more than microsecond response; for events that occur only when you are checking anyway.

23.11 Do Experiment 18 step 4: print with TimerOne at 100 Hz and in loop() add delay(50). Count...

The timer interrupt keeps perfect timing; as long as the flag is checked before the next interrupt you should count about 1000 samples in 10 s, but the delay(50) means flags are missed (only ~200 are processed): showing why a longer loop() needs a buffer or a faster check.

Chapter 24

24.1 Write five testable requirements for "a doorbell with a button, a melody and a light"...

Example: (1) pressing the button starts the melody within 100 ms; (2) melody lasts $5 \text{ s} \pm 0.5 \text{ s}$; (3) LED lights during the melody; (4) runs on 4 $\times$ AA for ≥ 6 months with 20 presses/day; (5) button is debounced: one press = one melody.

24.2 You need to read a 24 V battery with the Arduino's 5 V ADC. Choose R1 (top) and R2 (bottom)...

Ratio $\leq 5/30 = 0.1667$: $R2 / (R1 + R2)$. Take $R1 = 47 \text{ k}\Omega$, $R2 = 10 \text{ k}\Omega \rightarrow$ ratio 0.1754: $V_{\text{max}} = 5.26 \text{ V} \rightarrow$ too high for 30 V; use $R1 = 56 \text{ k}\Omega$, $R2 = 10 \text{ k}\Omega \rightarrow 0.1515$, $V_{\text{out}}(30 \text{ V}) = 4.55 \text{ V} \checkmark$. Current at 24 V = $24 / 66 \text{ k}\Omega = 0.36 \text{ mA}$.

24.3 A pump draws 1.2 A from 12 V. You switch it with an IRLZ44N ($R_{\text{DS(on)}} = 0.035 \Omega$). Compute...

$P = 1.2^2 \times 0.035 = 0.05 \text{ W}$ (no heatsink). $R_{\text{G}} = 5 \text{ V} / 0.02 \text{ A} = 250 \Omega \rightarrow 270 \Omega$; (typically 220 Ω is fine).

24.4 Compute the 5 V current budget of: Arduino (45 mA), OLED (20 mA), two DS18B20 (1.5 mA)...

Total = $45 + 20 + 3 + 10 + 70 = 148 \text{ mA}$. USB: yes. 7805: $P = (9 - 5) \times 0.148 = 0.59 \text{ W}$; TO-220 without heatsink rises $\approx 38^\circ\text{C}$: acceptable but warm.

24.5 Draw a block diagram and pin table for a motion-activated light with a manual override...

Blocks: PIR (D2), LDR divider (A0), button (D3, INPUT_PULLUP) $\rightarrow$ Arduino $\rightarrow$ MOSFET (D5) $\rightarrow$ LED strip; 12 V supply with buck to 5 V. The code uses millis() for the 2-minute timeout and an OR between PIR-and-dark and the override button.

24.6 List which parts of your greenhouse design (Section 24.4) would make it unsafe if the DHT22...

If the DHT22 returns NaN the old temperature value remains (fan stays as it was); a hardware thermostat (bimetal) could be added in series with the fan for safety. The pump has an independent max run time and lock-out, so sensor failure cannot keep it running.

24.7 Read the datasheet of a part in your kit (2N2222, LM35, or IRLZ44N) and record: absolute...

For a 2N2222: $V_{\text{CEO}} 40 \text{ V}$, $I_{\text{C}} 600 \text{ mA}$, pinout E-B-C (TO-92 PN2222A) or C-B-E (metal can), $h_{\text{FE}} 100\text{--}300$, typical switch circuit with base resistor.

24.8 Why should you test each block separately before integrating the whole system?

If the integrated system fails, the number of possible causes is huge. Testing blocks separately isolates faults to the most recently added part and proves that each building block works alone.

24.9 Why is a hardware safety (fuse, thermal cut-out) important even if your code already limits...

Software can crash, hang or contain bugs; a hardware protection is independent of the code and acts even when the microcontroller misbehaves.

24.10 Carry out Experiment 19. Submit (to yourself) the requirements, block diagram, pin table,...

You should have a full set of documents for one small project and a pass/fail test table; repeat the loop for any failures.

Chapter 25

25.1 An LED connected to pin 9 never lights. List six possible causes in order of how quickly...

1) LED reversed 2) code pin $\neq$ wired pin 3) missing pinMode / code not uploaded 4) broken jumper or breadboard contact (continuity) 5) missing GND connection or resistor 6) dead LED (test with a battery) or pin damaged.

25.2 Your robot works on the bench connected to USB, but resets when you run it from a 9 V...

The battery's internal resistance and wiring cause the supply to sag when the motors draw current, dropping below the brown-out level. Test: measure 5 V at the Arduino while the motors start (min-hold or scope); fix with a separate supply/regulator and capacitors.

25.3 A 5 V Arduino circuit draws 350 mA through 1 m of thin jumper wire (two wires, total 0.5...

$\Delta V = 0.35 \times 0.5 = 0.175$ V; load sees 4.83 V.

25.4 Your ADC reading on A0 is 512 when it should be 768. The sensor is a divider with $R_{\text{top}} = \dots$

512 $\rightarrow$ 2.5 V: with $R_{\text{top}} = 10$ k Ω , the bottom is 10 k Ω rather than 30 k Ω (which gives 3.75 V = 768). Likely the sensor is 10 k Ω or a resistor is in parallel with it, or the wrong resistor was used.

25.5 Write a tiny sketch to test whether pin 6 can drive an LED, printing a message to Serial...

pinMode(6,OUTPUT); loop: digitalWrite(6,HIGH); Serial.println("HIGH"); delay(500); digitalWrite(6,LOW); Serial.println("LOW"); delay(500). It separates hardware from your main program.

25.6 Explain how to use the multimeter to find a short circuit between 5 V and GND on an...

Continuity mode between the 5 V rail and GND rail. If it beeps, disconnect sections one at a time (or remove parts) until the beep stops: the last removed section contains the short.

25.7 What information would you include when asking for help on a forum about an LCD that stays...

Goal, Arduino and LCD model (I2C backpack?), wiring photo, the complete sketch, library used, I2C scanner result (address), contrast trimmer tested, supply voltage measured.

25.8 Why should you fix one thing at a time and retest?

Otherwise you cannot tell which change fixed (or caused) the problem, and you may introduce new faults while the old one is hidden.

25.9 Do the debugging drill (Experiment 20). Record the time to locate each fault and the clue...

After practice, most of these faults can be found within two or three minutes; the clues are listed in the top-12 table.

Chapter 26

26.1 Modify Project 1 so that the fan speed uses a proportional band of 3 °C instead of 5 °C and...

FULL_SPEED_ERR = 3.0; MIN_DUTY = 128. At 1.5 °C: frac = 0.5 $\rightarrow$ duty = 128 + 0.5 $\times$ 127 = 191 (75 %).

26.2 In Project 1, how much power does the MOSFET dissipate at full PWM duty if $R_{\text{DS(on)}} = 0.035 \dots$

$P = 0.25^2 \times 0.035 = 2.2$ mW. The gate draws only pulses of charge (nanocoulombs): negligible average power (μ W).

26.3 Explain in your own words what hysteresis does in Project 1 and what would happen without it.

The fan turns on only above target + 1 °C and off below target + 0.3 °C. Without hysteresis, noise around the threshold would make the fan switch on and off rapidly (chatter), wearing it out and being annoying.

26.4 Project 2: if the game continues to level 10 and each step lasts 350 ms + 60 ms pause, how...

10×0.41 s = **4.1 s**.

26.5 Change Project 2 so the LED durations shorten as the level increases: from 350 ms at level...

`int d = max(120, 350 - (level - 1) * 15);` then `playStep(sequence[i], d);`

26.6 Project 3: what happens if the user presses the button during ALARM? And if nobody does for...

Button $\rightarrow$ noTone, state DISARMED (green LED on, red off). No press $\rightarrow$ after 30 s noTone and state ARMED (red LED steady), ready to detect motion again.

26.7 Add an “arming confirmation beep” to Project 3 when entering ARMED, and a double beep when...

Inside `setState()`: after setting `state`, `if (s == ARMED) tone(BUZZ, 1000, 200); if (s == DISARMED) { tone(BUZZ, 600, 100); }`, or use a small beep function with `millis()`.

26.8 Choose one project idea from the table in 26.4. Write five testable requirements, a block...

Your deliverable: the complete design documents described in Chapter 24. Check each part’s voltage/current/interface against the checklist.

26.9 Build Project 1, 2 or 3 (your choice) and complete the test plan. Record any deviations...

Keep your notes and photographs. You should be able to demonstrate each requirement.

26.10 Build your own design from the previous assignment. Document it with a schematic, photos,...

A complete, working, documented design is the proof that you can now design by yourself.

Chapter 27

27.1 List the five steps of making a good through-hole solder joint, in order.

1) clean and tin the tip; 2) heat the pad and the lead together; 3) feed solder to the joint; 4) remove the solder, then the iron; 5) let the joint cool without moving it.

27.2 You see a dull, lumpy solder joint on a resistor lead. What is it called, what probably...

A cold joint: insufficient heat or movement while cooling. Reheat with a touch of fresh solder/flux until it flows and becomes shiny.

27.3 A PCB track must carry 2 A. Using the rule of thumb of 0.3 mm per ampere as an absolute...

Choose 1 mm or wider. $100 \text{ mm} / 1 \text{ mm} = 100 \text{ squares} \times 0.5 \text{ m}\Omega = 50 \text{ m}\Omega$; $V = 2 \times 0.05 = 0.1 \text{ V}$ (0.2 W).

27.4 You solder 40 joints at 3 s each, and also spend 10 minutes preparing. How long is that in...

$40 \times 3 \text{ s} = 120 \text{ s} = 2 \text{ min} + 10 \text{ min} = \mathbf{12 \text{ minutes}}$.

27.5 Which is better for 5 identical copies of a finished project: perfboard or a custom PCB?...

A custom PCB: repeatable and consistent quality; faster assembly; fewer wiring errors; neater; cost per board is low in small batches.

27.6 Why should you place a decoupling capacitor as close as possible to each IC’s power pins on...

Track inductance and resistance delay the current delivery; a nearby capacitor supplies the fast current spikes locally (Chapter 6).

27.7 Why do professionals use IC sockets for expensive or experimental chips but not for...

Sockets allow replacement during development but cost more, take space and are less reliable (contact resistance, vibration); in production, direct soldering is cheaper and more reliable.

27.8 Create a plan to turn your best Arduino project into a permanent device: list the steps...

A typical plan: finalise the schematic; choose perfboard/shield or order a PCB; list the connectors, switch, fuse, enclosure; solder in stages with continuity checks; test with the supply; mount it in the box; document it.

27.9 Practise soldering: on a piece of perfboard solder 20 resistors, an LED and a header and...

You should produce shiny, concave joints with no bridges; a magnifier helps for inspection.

Formula cheat sheet

Topic	Formula	Notes
Ohm's law	$V = I \times R$ $I = V / R$ $R = V / I$	V in volts, I in amperes, R in ohms
Power	$P = V I = I^2 R = V^2 / R$	Watts; resistor heat
Energy	$E = P \times t$; 1 kWh = 3.6 MJ	Battery: Wh = V × Ah
Charge and current	$Q = I \times t$	Coulombs
Resistance of a wire	$R = \rho L / A$	ρ copper = $1.68 \times 10^{-8} \Omega \cdot m$
Resistors in series	$R = R_1 + R_2 + \dots$	Same current; voltages add
Resistors in parallel	$1/R = 1/R_1 + 1/R_2 + \dots$; two: $R_1 R_2 / (R_1 + R_2)$	Same voltage; currents add
Voltage divider	$V_{out} = V_{in} R_2 / (R_1 + R_2)$	Load $\geq 10 \times R_2$
Current divider	$I_1 = I \times R_2 / (R_1 + R_2)$	Two parallel resistors
KCL / KVL	$\sum I_{in} = \sum I_{out}$; $\sum V_{loop} = 0$	
Capacitor charge / energy	$Q = C V$; $E = \frac{1}{2} C V^2$	
Capacitors	Parallel: $C = \sum C$; series: $1/C = \sum 1/C$	Opposite of resistors
RC time constant	$\tau = R C$; $V_C = V_S (1 - e^{-t/\tau})$	63 % at 1τ ; 99 % at 5τ
RC cut-off frequency	$f_c = 1 / (2\pi R C)$	Low-pass / high-pass
Capacitive reactance	$X_C = 1 / (2\pi f C)$	Falls with frequency
Inductor	$V = L \Delta I / \Delta t$; $E = \frac{1}{2} L I^2$; $\tau = L/R$; $X_L = 2\pi f L$	
Transformer	$V_s / V_p = N_s / N_p$	AC only
Frequency / period	$f = 1 / T$	Hz and seconds
LED resistor	$R = (V_S - V_F) / I_F$	Round up to E12; check power
BJT switch	$I_B = I_C / 10$; $R_B = (V_{drive} - 0.7) / I_B$	Forced $\beta = 10$
BJT gain	$I_C = \beta I_B$	$\beta \approx 100 - 300$
MOSFET loss	$P = I^2 R_{DS(on)}$	Logic-level type for 5 V
Linear regulator loss	$P = (V_{in} - V_{out}) \times I$	Heat
Ripple (full-wave)	$V_r \approx I / (2 f C)$	f = mains frequency
Non-inverting op-amp	$A = 1 + R_2 / R_1$	
Inverting op-amp	$A = - R_2 / R_1$	
555 monostable	$t = 1.1 R C$	
555 astable	$f = 1.44 / ((R_1 + 2R_2) C)$; $t_H = 0.693 (R_1 + R_2) C$; $t_L = 0.693 R_2 C$	
ADC (10-bit)	value = $V_{in} / V_{ref} \times 1023$; step = $V_{ref} / 1024$	4.88 mV at 5 V
PWM average	$V_{avg} = V_{supply} \times \text{duty}$; duty = analogWrite value / 255	
Timer interrupt	$f = 16 \text{ MHz} / (\text{prescaler} \times (\text{OCR} + 1))$	
Ultrasonic distance	$d \text{ (cm)} = t(\mu s) \times 0.0343 / 2$	Speed of sound 343 m/s

Topic	Formula	Notes
Thermistor (beta)	$1/T = 1/T_0 + \ln(R/R_0)/\beta$	T in kelvin
Battery run time	$t \approx \text{capacity (mAh)} / \text{load (mA)} \times 0.7$	Derated
Baud to bytes	$\text{bytes/s} \approx \text{baud} / 10$	8N1

Standard values, codes and tables

SI prefixes

Prefix	Symbol	Power	Prefix	Symbol	Power
giga	G	10^9	milli	m	10^{-3}
mega	M	10^6	micro	μ	10^{-6}
kilo	k	10^3	nano	n	10^{-9}
(unit)	—	10^0	pico	p	10^{-12}

Resistor colour code

Colour	Digit	Multiplier	Tolerance	Colour	Digit	Multiplier	Tolerance
Black	0	$\times 1$	—	Violet	7	$\times 10$ M	± 0.1 %
Brown	1	$\times 10$	± 1 %	Grey	8	$\times 100$ M	± 0.05 %
Red	2	$\times 100$	± 2 %	White	9	$\times 1$ G	—
Orange	3	$\times 1$ k	—	Gold	—	$\times 0.1$	± 5 %
Yellow	4	$\times 10$ k	—	Silver	—	$\times 0.01$	± 10 %
Green	5	$\times 100$ k	± 0.5 %	(none)	—	—	± 20 %
Blue	6	$\times 1$ M	± 0.25 %				

Standard resistor values (E12 and E24), multiply by 1, 10, 100 ... 1 M

Series	Values
E12 (± 10 %)	1.0 1.2 1.5 1.8 2.2 2.7 3.3 3.9 4.7 5.6 6.8 8.2
E24 (± 5 %)	1.0 1.1 1.2 1.3 1.5 1.6 1.8 2.0 2.2 2.4 2.7 3.0 3.3 3.6 3.9 4.3 4.7 5.1 5.6 6.2 6.8 7.5 8.2 9.1
E6 (± 20 %) capacitors	1.0 1.5 2.2 3.3 4.7 6.8

Common values worth keeping in your kit

Use	Values
LED resistors (5 V)	220 Ω , 330 Ω , 470 Ω
Pull-ups / pull-downs / dividers	4.7 k Ω , 10 k Ω , 47 k Ω , 100 k Ω
Transistor base / MOSFET gate	220 Ω , 560 Ω , 1 k Ω ; pull-down 10 k Ω
Ceramic capacitors	22 pF, 100 pF, 10 nF, 100 nF (decoupling), 1 μ F
Electrolytic capacitors	10 μ F, 100 μ F, 470 μ F, 1000 μ F (16 V or 25 V)

Capacitor codes

Code	Value	Code	Value	Code	Value
101	100 pF	102	1 nF	103	10 nF
104	100 nF	105	1 μ F	222	2.2 nF
223	22 nF	224	220 nF	473	47 nF

Wire gauge and current (copper, chassis wiring)

AWG	Diameter	Area (mm ²)	Max current (typical)	Use
28	0.32 mm	0.08	0.5 A	Fine hook-up
24	0.51 mm	0.20	2 A	Signals
22	0.64 mm	0.33	3 A	Breadboard jumpers, general
20	0.81 mm	0.52	5 A	Power leads
18	1.02 mm	0.82	7–10 A	Motors, battery packs
16	1.29 mm	1.31	10–13 A	High current

Greek letters used in electronics

Letter	Name	Typical use	Letter	Name	Typical use
Ω	omega	ohm; angular frequency ($\omega = 2\pi f$)	μ	mu	micro (10^{-6})
τ	tau	time constant	β	beta	transistor gain; thermistor constant
ρ	rho	resistivity	ϵ	epsilon	permittivity
Δ	delta	change in a quantity	π	pi	3.14159...

Arduino quick reference

Structure and syntax

Language skeleton

```
void setup() { /* runs once */ }
void loop() { /* runs forever */ }

// comment      /* multi-line comment */
const int PIN = 8;      #define N 4      int a[5] = {1,2,3,4,5};
if (c) {...} else if (d) {...} else {...}
for (int i = 0; i < n; i++) {...}      while (c) {...}      do {...} while (c);
switch (x) { case 1: ...; break; default: ...; }
return value;      break;      continue;
```

Core functions

Function	Description
<code>pinMode(pin, OUTPUT / INPUT / INPUT_PULLUP)</code>	Configure a pin
<code>digitalWrite(pin, HIGH / LOW)</code>	Set a digital output
<code>digitalRead(pin)</code>	Read a digital input (HIGH / LOW)
<code>analogRead(A0..A5)</code>	ADC reading 0-1023
<code>analogWrite(pin, 0..255)</code>	PWM on pins 3, 5, 6, 9, 10, 11
<code>analogReference(DEFAULT / INTERNAL / EXTERNAL)</code>	Choose ADC reference
<code>delay(ms), delayMicroseconds(us)</code>	Blocking pauses
<code>millis(), micros()</code>	Time since start (unsigned long)
<code>tone(pin, Hz[, ms]), noTone(pin)</code>	Square-wave tone
<code>pulseIn(pin, HIGH, timeout_us)</code>	Measure a pulse width
<code>shiftOut(data, clock, MSBFIRST, byte)</code>	Send a byte to a shift register
<code>map(x, inLo, inHi, outLo, outHi), constrain(x, lo, hi)</code>	Rescale; limit
<code>min, max, abs, pow, sqrt, sin, cos</code>	Maths
<code>random(max), random(min, max), randomSeed(n)</code>	Pseudo-random numbers
<code>attachInterrupt(digitalPinToInterrupt(pin), isr, RISING / FALLING / CHANGE)</code>	Interrupts on D2, D3
<code>noInterrupts(), interrupts()</code>	Disable/enable interrupts
<code>Serial.begin(9600), print, println, write</code>	Serial output
<code>Serial.available(), read(), parseInt(), readStringUntil(c)</code>	Serial input

Pin map (Uno)

Pin	Function / special role
D0 (RX), D1 (TX)	Hardware serial / USB: avoid for general use
D2, D3	External interrupts INT0, INT1; D3 PWM
D4, D7, D8, D12	General digital
D5, D6	PWM (Timer0; also millis())
D9, D10	PWM (Timer1; Servo library); D10 = SPI SS
D11	PWM (Timer2); SPI MOSI
D12	SPI MISO
D13	SPI SCK; built-in LED
A0 – A5	Analog inputs (also D14–D19); A4 = SDA, A5 = SCL
5V, 3V3, GND, VIN, AREF, RESET, IOREF	Power and reference pins

Common library snippets

Libraries at a glance	
<code>#include <Servo.h></code>	<code>Servo s; s.attach(9); s.write(90);</code>
<code>#include <Wire.h></code>	<code>Wire.begin(); Wire.beginTransmission(addr); Wire.write(b); Wire.endTransmission(); Wire.requestFrom(addr, n); Wire.read();</code>
<code>#include <SPI.h></code>	<code>SPI.begin(); SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0)); SPI.transfer(b); SPI.endTransaction();</code>
<code>#include <EEPROM.h></code>	<code>EEPROM.put(addr, value); EEPROM.get(addr, value); EEPROM.update(addr, byte);</code>
<code>#include <LiquidCrystal_I2C.h></code>	<code>LiquidCrystal_I2C lcd(0x27, 16, 2); lcd.init(); lcd.backlight(); lcd.setCursor(c, r); lcd.print(x);</code>
<code>#include <DHT.h></code>	<code>DHT dht(pin, DHT22); dht.begin(); dht.readTemperature(); dht.readHumidity();</code>
<code>#include <Stepper.h></code>	<code>Stepper st(2048, 8, 10, 9, 11); st.setSpeed(10); st.step(2048);</code>
<code>#include <SoftwareSerial.h></code>	<code>SoftwareSerial ss(rx, tx); ss.begin(9600);</code>
<code>#include <avr/wdt.h></code>	<code>wdt_enable(WDTO_2S); wdt_reset();</code>

Data types

Type	Bytes	Range
bool / boolean	1	false, true
byte / uint8_t	1	0 – 255
char / int8_t	1	–128 – 127
int / int16_t	2	–32,768 – 32,767
unsigned int / uint16_t	2	0 – 65,535
long / int32_t	4	$\pm 2.1 \times 10^9$
unsigned long / uint32_t	4	$0 - 4.29 \times 10^9$
float / double (Uno)	4	$\pm 3.4 \times 10^{38}$, 6–7 digits

Absolute limits to remember

Quantity	Limit (ATmega328P / Uno)
I/O pin current	20 mA recommended, 40 mA absolute; 200 mA total
Pin voltage	–0.5 V to 5.5 V
5V pin from USB	≈ 400 mA total; 3.3V pin 50 mA

Quantity	Limit (ATmega328P / Uno)
VIN / barrel jack	7 - 12 V recommended

Component pinouts

Always confirm with the datasheet for your exact part and package. “Front” means the flat or printed face towards you with leads pointing down.

Part	Pins left → right (front view)	Notes
2N2222A / PN2222A (TO-92)	E - B - C (PN2222A); some brands C - B - E	NPN, 600 mA
BC547 / BC548 (TO-92)	C - B - E	NPN, 100 mA
2N3904 (TO-92)	E - B - C	NPN, 200 mA
2N3906 (TO-92)	E - B - C	PNP, 200 mA
2N7000 / BS170 (TO-92)	2N7000: S - G - D; BS170: D - G - S	N-MOSFET, small; check the datasheet
IRLZ44N (TO-220)	G - D - S (tab = drain)	Logic-level N-MOSFET
TIP120 (TO-220)	B - C - E (tab = collector)	Darlington NPN
LM7805 (TO-220)	IN - GND - OUT	Add 0.33 μ F in, 0.1 μ F out
AMS1117-3.3 (SOT-223)	GND/ADJ - OUT - IN (tab = OUT)	Check the board
LM35 (TO-92)	+Vs - Vout - GND	10 mV/°C
TMP36 (TO-92)	+Vs - Vout - GND	10 mV/°C, 0.5 V offset
LED (5 mm)	Long = anode (+), short = cathode (–, flat side)	
Diode (1N4007 / 1N4148)	Stripe = cathode	
Electrolytic capacitor	Stripe with “–” = negative (shorter lead)	
Tactile button (4 pin)	Pins on the same side are joined	Press connects the two sides

Integrated circuits (DIP, top view, notch up; pin 1 top-left, counter-clockwise)

IC	Pins (1 → 8 / 16)
NE555 (DIP-8)	1 GND · 2 TRIG · 3 OUT · 4 RESET · 5 CTRL · 6 THR · 7 DIS · 8 Vcc
LM358 (DIP-8, dual op-amp)	1 OUT A · 2 IN– A · 3 IN+ A · 4 GND · 5 IN+ B · 6 IN– B · 7 OUT B · 8 Vcc
74HC595 (DIP-16)	1–7 Q1–Q7 · 8 GND · 9 Q7' · 10 MR · 11 SH_CP · 12 ST_CP · 13 OE · 14 DS · 15 Q0 · 16 Vcc
L293D (DIP-16)	1 EN1,2 · 2 IN1 · 3 OUT1 · 4/5 GND · 6 OUT2 · 7 IN2 · 8 Vs (motor) · 9 EN3,4 · 10 IN3 · 11 OUT3 · 12/13 GND · 14 OUT4 · 15 IN4 · 16 Vss (5 V logic)
ULN2003A (DIP-16)	1–7 inputs 1B–7B · 8 GND · 9 COM (flyback, to + supply) · 10–16 outputs 7C–1C
ATmega328P (DIP-28)	1 RESET · 7 Vcc · 8 GND · 9/10 XTAL · 20 AVcc · 21 AREF · 22 GND · pins 2–6, 11–19 are the I/O (PD0–PD7, PB0–PB5, PC0–PC5)

Modules

Module	Connections
HC-SR04	VCC · TRIG · ECHO · GND
HC-SR501 PIR	VCC (5–20 V) · OUT · GND
DHT22 / DHT11 (4 pin)	1 VCC · 2 DATA · 3 NC · 4 GND (with 10 kΩ pull-up between 1 and 2)
I2C LCD backpack	GND · VCC · SDA · SCL
SG90 servo	Brown/black GND · red +5 V · orange signal
Relay module	VCC · GND · IN (check active level); NO · COM · NC screw terminals
L298N module	ENA IN1 IN2 IN3 IN4 ENB; +12 V (motor supply) · GND · +5 V; OUT1–4 to motors
DS3231 RTC	GND · VCC · SDA · SCL (· SQW · 32K)
28BYJ-48 + ULN2003 board	IN1–IN4 · + · – (5 V)
Rotary encoder KY-040	GND · + · SW · DT · CLK

Glossary

Term	Meaning
ADC	Analog-to-digital converter: turns a voltage into a number (Arduino: 10 bit, 0-1023).
Alternating current (AC)	Current whose direction reverses periodically (mains: 50 or 60 Hz).
Ampere (A)	Unit of current: one coulomb per second.
Analog	A signal that can take any value in a range.
Anode	The positive terminal of a diode/LED (current flows in).
Baud	Symbols (bits) per second on a serial link.
Bias	A steady DC voltage or current that sets a device's operating point.
Bit / byte	Binary digit; eight bits.
Bootloader	Small program on the Arduino that receives new sketches over USB.
Bounce	Rapid on/off chatter of a mechanical contact when it closes or opens.
Breadboard	Solderless board for building prototype circuits.
Capacitance	Ability to store charge per volt; unit farad (F).
Cathode	The negative terminal of a diode/LED (marked by a bar or flat side).
Common ground	The shared 0 V reference of all parts in a system.
Comparator	Circuit that outputs HIGH or LOW depending on which of two inputs is larger.
Conductor / insulator	Material that passes / blocks electric current.
Continuity	A complete low-resistance electrical path (tested with a beeping meter).
Coulomb (C)	Unit of charge: about 6.24×10^{18} electrons.
Current (I)	Flow of electric charge, measured in amperes.
Datasheet	Manufacturer's specification document for a component.
DC	Direct current: flows in one direction.
Debounce	Removing the effect of contact bounce in hardware or software.
Decoupling capacitor	Capacitor near an IC's supply pins that supplies fast current and removes noise.
Diode	Component that conducts in one direction only.
Duty cycle	Fraction of time a PWM signal is HIGH.
EEPROM	Memory that keeps data without power and can be rewritten ($\approx 100,000$ writes).
Flash memory	Non-volatile memory holding the program.
Flyback diode	Diode across an inductive load that absorbs the voltage spike at turn-off.
Frequency (f)	Cycles per second, in hertz.
Ground (GND)	The 0 V reference point of a circuit.
H-bridge	Four-switch circuit that drives a motor in either direction.
Hysteresis	Different thresholds for switching on and off, preventing chatter.
I2C	Two-wire serial bus (SDA, SCL) with addressed devices.
IC (integrated circuit)	Many components on one silicon chip.

Term	Meaning
Inductor	Coil that stores energy in a magnetic field and resists changes in current.
Interrupt	Hardware signal that makes the processor run a special routine immediately.
ISR	Interrupt service routine: the function run by an interrupt.
Kirchhoff's laws	KCL: currents at a node sum to zero; KVL: voltages round a loop sum to zero.
LDR	Light-dependent resistor; resistance falls with light.
LED	Light-emitting diode.
Load	The part of a circuit that uses power (lamp, motor, etc.).
Logic level	The voltage ranges interpreted as HIGH and LOW (5 V or 3.3 V).
Microcontroller (MCU)	A computer on a chip with memory and I/O.
MOSFET	Voltage-controlled transistor used as a switch or amplifier.
Multiplexing	Sharing lines among many devices by activating them one at a time very quickly.
Ohm (Ω)	Unit of resistance: one volt per ampere.
Ohm's law	$V = I \times R$.
Op-amp	High-gain differential amplifier used with feedback.
Open circuit / short circuit	A break in the path (no current) / an unintended low-resistance path (excess current).
Parallel / series	Components sharing the same two nodes / connected end to end.
PCB	Printed circuit board.
Polling	Repeatedly checking an input in the main loop.
Potentiometer	Variable resistor with a sliding wiper contact.
Power (P)	Rate of energy use: watts = volts $\times$ amperes.
Prescaler	Divider that slows the clock feeding a timer.
Pull-up / pull-down	Resistor holding an input at HIGH / LOW when nothing else drives it.
PWM	Pulse-width modulation: varying the duty cycle of a fast on/off signal.
Rectifier	Circuit converting AC to pulsating DC (diodes).
Regulator	Circuit that holds an output voltage constant (linear or switching).
Relay	Electrically operated switch with a coil and mechanical contacts.
Resistance (R)	Opposition to current flow, in ohms.
Ripple	Small AC variation left on a DC supply.
Servo	Motor with feedback that moves to a commanded angle.
Shift register	Chip that converts serial data to parallel outputs (e.g. 74HC595).
Sketch	An Arduino program.
SPI	Fast serial bus with clock, MOSI, MISO and chip select.
Stepper motor	Motor that rotates in discrete steps.
Thermistor	Resistor whose resistance varies strongly with temperature (NTC/PTC).
Time constant (τ)	Characteristic time of an RC ($R \times C$) or RL (L/R) circuit.
Transistor	Semiconductor device in which a small signal controls a larger current.
UART	Asynchronous serial interface (TX/RX).
Voltage (V)	Electrical "pressure": energy per unit charge, in volts.
Voltage divider	Two resistors in series that produce a fraction of the input voltage.
Watchdog	Timer that resets the microcontroller if the program stops responding.

Term	Meaning
Zener diode	Diode designed to conduct in reverse at a set voltage; used as a reference.

Keep building, keep measuring, keep asking “why does it work?”